

The Ship of Theseus — v0.60

One document, three parts. Part I is the specification: what Theseus is meant to be. Part II is the build plan: the order it is built in, with the test that gates each step. Part III is the record of what was actually built, milestone by milestone, and where it diverged from Parts I and II. The document is therefore both spec and documentation; when the code and Part I disagree, Part III says so and one of them gets fixed.

*Consistency pass by Tabitha, 2026-09-24, folding three deep dives (sessions, shells, memory) and all of Eddie's answers into one coherent document. Earlier provisional text that the deep dives superseded has been removed rather than annotated. **[D]** marks a provisional decision Tabitha made to keep the document whole; overturn freely. v0.5 incorporated an external review (GPT Astra, 2026-09-24; Appendix A). v0.6 closed the last open question. v0.7 added the hooks surface (§3.17). v0.8 adopted the event-driven execution model from Eddie's all-webhook proposal as reviewed in `notes/event-design-review.md` (§3.3, §3.16, §6, §7, §8). v0.9 incorporated the second external review (Appendix D) and renamed the document at Eddie's request. v0.10 records Eddie's decisions on the three questions it left open (storage kernel, shell default, control-plane separation) and adds the turn-lock durability model. v0.11 answers the concurrency question (what runs "simultaneously": one turn per **session**, where a session is a task or a conversation, §3.2a) and folded the build plan into this document as Part II. v0.12 settles when a session is recompiled (§4.4a): append by default, recompile only on need, with Jev owning the judgment; estimates are removed from Part II; the repository is `github.com/zeroaltitude/theseus`. v0.13 states how sessions persist across runtime restarts as lineages in a multi-parent graph (§4.4b). v0.14 defines **loop**, **turn**, and the **Advancer** (§3.3a), the wire protocol and client isolation (§3.18), the secrets posture (§3.19), and replaces the first milestone with **M0 First light**, a vertical slice Eddie specified. v0.15 added Part III (as built) and the standing rule that actuals are recorded as work lands (Eddie, 2026-09-25).*

Ariadne held the thread. Theseus walks the labyrinth on it.

Part I — Specification

0. Thesis

Theseus is a **durable agent runtime with a provenance-aware context compiler**: work has an identity, authority has a boundary, actions have recoverable histories, and context is compiled from one logical history with a manifest, appended to by default and recompiled only when something real demands it. Concretely, it is a single statically linked Rust binary that runs thousands of concurrent conversations, talks to humans through Discord in text and voice, acts on the world through AWS-native tools and graded shells, and owns its own turn loop against the Anthropic Messages API. The loop does not stop on a counter; it stops when Jev judges that the work has reached a definable

stopping point, and otherwise keeps going. Context is one multi-rooted, typed-edge graph held in memory, from which every prompt is assembled fresh. Tasks, memory, judgments, and self-tuning are intrinsic to the runtime. Every capability is a plugin against a small core, but the shipped binary is opinionated: Discord, AWS, Anthropic, and Jev are always present, and MCP is a first-class capability in both client and server roles.

It runs on one large node. That node may be an EC2 instance or Eddie's desktop. "AWS native" means the AWS tool surface is deep and default, not that the harness must live in AWS.

1. Settled decisions

Topic	Decision
Language, artifact	Rust. One statically linked binary per target. No dynamic linking, no sidecars; helper processes are the same binary launched with a role flag.
Licence	Open source, dual-licensed MIT OR Apache-2.0 in the Rust convention (decided 2026-09-25): Apache's patent grant and contribution clause for those who want them, MIT's brevity for those who do not. Permissive-only dependencies enforced by <code>cargo deny</code> from the first commit. No AGPL (rules out linking Vestige).
Comms	Discord only, text and voice. One Discord application invited to many guilds.
Model path	Direct Anthropic Messages API. Bedrock is a possible later provider, not the default.
Hands	AWS tool surface is deep and default. Shells are graded: local host, local native sandbox, and AWS classes, chosen per job by Jev within policy. L0 (native host shell) is the default for BigHat's deployment; agent-authored code and package installs go to L1; the open-source distribution ships L1 as default with L0 as documented opt-in. Committed 2026-09-24, to be revisited on evidence.
Execution model	Event-driven. No in-flight state lives only in harness memory; every dispatched thing is a WAL record with a harness-minted correlation id; completion arrives as an event (in-process, Unix socket spool, SQS pull, loopback HTTP as the off-by-default exception); the harness is quiescent between events; the one-minute heartbeat is the level-triggered reconciler. Adopted 2026-09-24 from Eddie's all-webhook proposal, with "webhook" generalized to "completion event" and "unkillable" replaced by "detached, durable, cancellable" (§3.3, §3.16).
Deployment	One large node: Theseus, source trees, and sandboxes together. Must also run on a home desktop with every AWS dependency optional at runtime.
Durability	Local fsync to a persistent SSD is the floor, before any action is dispatched. Off-node durability is eventual, 5–60 s (measured, not asserted), produced by asynchronous durability work the core performs in the time a turn has surrendered to a remote (model call, shell, judge, human). Single node; no replication.
Storage kernel	Decided by measurement (M1, 2026-09-26): the WAL is the truth; a pure-Rust embedded store is a rebuildable index over it, and that store is <code>redb</code>. On identical fsync-bound workloads the two candidates append at the same rate (the disk decides); with fsync removed <code>fjall</code> appends 1.7× faster but <code>redb</code> reopens 2× faster and answers "latest of every key" 10–17× faster, is one file, and has no background compaction. (<i>Amended 2026-09-29, theseus-0g4: <code>fjall</code> and the <code>Index</code> trait were removed in batch C. The index is <code>redb</code>, and a store manifest or a config naming any other engine is refused.</i>)

Topic	Decision
Unit of concurrency	One turn per session. A session is a task or a conversation; each has exactly one execution and one turn lock. Thousands of sessions exist durably; those with runnable work run concurrently, bounded by an admission scheduler; the rest are parked at zero cost. Channels order deliveries, not work. (Eddie, 2026-09-25.)
Graph shape	The context graph is a DAG with multiple parents , not a tree. A node belongs to a channel, to a session, to any number of compilations, and to derived nodes at once, each by its own typed edge. Order within any lineage comes from the node's WAL position, never from a single <code>next</code> chain.
Session persistence	A session persists as a small record pointing at its current <code>Compilation</code> and its tail range; it is durable by reference and resolves to a lineage of compilations linked by <code>derived_from</code> . Nothing is copied per session; rendered prefixes are a rebuildable cache.
Secrets	All secrets live in 1Password , in the deployment's vault, read at startup through a service account. The only secret the process may receive any other way is the service-account token itself. Config references secrets as <code>op://vault/item/field</code> ; no secret value is ever written to disk, config, log, or ledger.
Wire protocol	The core is a server speaking JSON-RPC 2.0 over newline-delimited JSON , on stdio when spawned and on a Unix domain socket as a daemon. Every client, including the in-binary CLI and later Discord and the web UI, talks to the core only through this protocol.
Loop, turn	A loop is one pass through the toolchain manager to the provider and back. A turn is the sequence of loops run under one acquisition of a session's turn lock, ended by the Advancer.
Recompilation	A session's compiled context is appended to by default and recompiled only on need . Deterministic triggers (audience, policy, schema, window) force it; otherwise Jev judges whether a real-world change warrants it. Long single threads therefore evolve exactly like a plain transcript, and cache their prefix. (Eddie, 2026-09-25.)
Repository	<code>~/projects/theseus</code> , <code>github.com/zeroaltitude/theseus</code> .
Turn lock	Per execution, exactly one turn advances at a time (the "GIL"). The lock is held only during local work and released at every offload boundary; the freed time is spent on durability, indexing, and maintenance.
Shell default	L0 and L1 both ship in the first useful agent. L0 is the early operator default, explicitly subject to change once L1 has real mileage.
Control-plane separation	Running the runtime and its storage under a separate OS identity from L0 jobs is an option, strongly recommended in the documentation and the installer, not a default.
Agents	One agent, many roles. No multi-agent, no inter-agent conversations.
Conversations	A conversation is the subset of the node graph that happened on a particular channel, in temporal order. Nothing more: it is a derived view, not a stored object, and it never ends because a channel never stops accumulating nodes. Many participants. Each execution (§3.15) acts in exactly one channel at a time; many executions run concurrently across channels.
Tasks	Fluid and conversational. The agent sees the whole task graph and reshapes it with the operator or alone.
Memory	Not a separate store. The graph is append-only, always : compaction <i>adds</i> summary nodes and later views may not include what was trimmed, but no node is ever lost. "Memory" is any node that retains enough strength to be selected into a prompt; decay lowers selection weight and moves payload to cold storage, it never removes anything. Implemented natively behind a <code>MemoryScience</code> trait; Vestige not used.

Topic	Decision
MCP	Full mode: Theseus is both client and server. Prompts and elicitation in; sampling in, budgeted and Jev-judged.
Proactivity	The agent may open a conversation with a human unprompted. Safety rests on the Jev security classifier plus a default-safe operator environment.
Destructive confirm	Goes to the person who issued the request, as an approval dialogue in a trusted channel . A trusted channel is any surface the static config lists as trusted (a Discord channel or DM, the web UI, the CLI), provided every member is a trusted user, also listed in static config (Eddie, 2026-09-27; §3.9). When there is no requester (proactive or scheduled work), it goes to the owner . Timeout means no action. <i>As built (2026-09-29, theseus-sgh; §3.9 "Approval"): without an [approval] section any surface answers, as before. With one, the CLI and the web UI are trusted channels when listed, since their member is this machine's operator. A Discord card for a place that is not a trusted channel goes to the requester's DM when that DM is trusted, else to another trusted user's DM, and the place gets a note. When nothing qualifies, the place gets the note alone.</i>
Owner vs operators	One owner per deployment: whoever runs the Theseus runtime, whether that is Eddie, a company's CTO, or a single person on their own laptop. The owner holds final authority over policy, budgets, and unrequested destructive actions. Many operators may converse with and configure the system within what the owner allows.
Owner's property	There is never a proposed action the owner cannot give an overriding approval for when it concerns the owner's own property (a repository, computer, or instance the owner owns), so long as it is in concordance with the model's terms of use and safety policy (Eddie, 2026-09-27). Property is declared statically in config. The override is owner-only and given in a trusted channel, and the floor takes a stronger ceremony (§3.9). <i>Since 2026-09-28 the gate refuses nothing, so the principle holds by construction: the owner can approve anything that waits, and the floor's ceremony is moot (§3.9; Part III A3b).</i>
Gate	Notify over block (Eddie, 2026-09-28). Every tool and every MCP inherits one posture, <code>open</code> , <code>notify</code> , or <code>approve</code> , with per-tool and per-MCP overrides. The gate never refuses a call, and the floor always asks (§3.9).
Unprompted actions	All allowed without confirm: DM a human, open a thread, post in a channel, speak in voice. The agent must be invited to a channel first, voice or text; it never joins uninvited.
Task terminal/stalled states	Judged by Jev like every other loop state; escalated to a human only when Jev's confidence is in question. No separate verification tier. Deterministic control paths (<code>/stop</code> , revocation, budget exhaustion) bypass Jev entirely (§3.15).
Provider outage	Fail closed and say so in Discord. No secondary provider.
Voice mode	PTT vs VAD is a human Discord preference, not an agent concern.
MCP server auth	Deferred; a single static API key for now.
Budgets	A first-class, flexible notion attachable to parts of the system by design; semantics deliberately unspecified for now. <i>Since 2026-09-29 the first budget is built: dollars, a \$100 limit per session, and a reset the operator approves at the limit (Eddie; §3.13).</i>
Reachability	Localhost only, outbound OK, no inbound. External proxies handle exposure.
Encryption at rest	Required, provided by the platform (EBS volume encryption, LUKS on desktop), not by Theseus.
Roles	Hints to the model, never enforced policy. Policy lives in §3.9 only. Upgradable later if it fails.

Topic	Decision
Redaction	Append-only is the logical history model. Payload erasure is an allowed, receipted exception for secrets and other must-not-exist content, with lineage-aware invalidation (§5.6). Suppression alone is not sufficient.
Deferred	GDPR/right-to-be-forgotten, MCP token scopes (one static key = one shared principal under the binding ceiling, accepted for now). Backup <i>restore path</i> is not deferred (§6); backup <i>drills</i> remain deferred.
Policy administration	The operator administers the Discord-role-to-policy mapping.
Observability	CloudWatch for historical search. An in-binary web UI, in the spirit of the OpenClaw gateway UI, for immediate and local: conversation snooping, RL feedback, category management and scoring nudges.
Embeddings	Local Nomic Embed v1.5 in the index tender, shared by all nodes, model id stamped on every vector.
Name	Theseus.

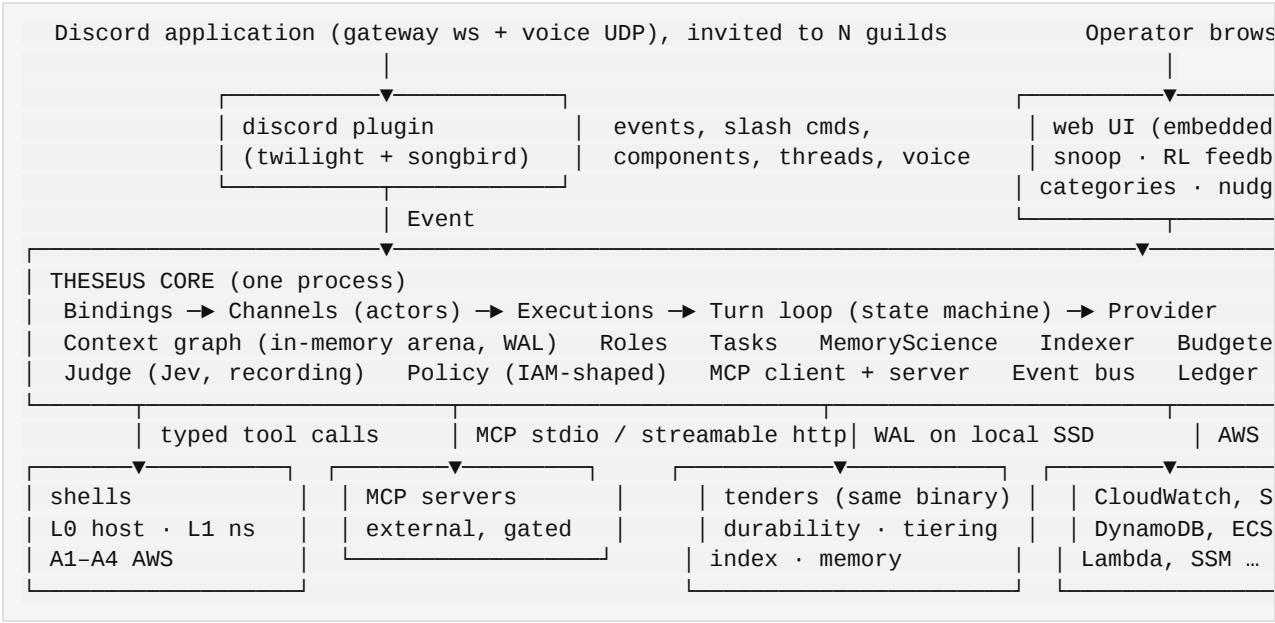
2. Principles as constraints

Principle	Constraint it imposes
FAST	Speed is a goal, not a tiebreaker. Every lifecycle edge has a budget in §9: cold start, clean shutdown, crash recovery, binary upgrade, migration, restore. The simulator measures each one on every commit, and missing a budget fails the gate the way a failing test does. Serving comes first. Nothing on the path to answering the socket waits on the network, a model, an index rebuild, or work that grows with history. Secrets, credential checks, warm-up, and integrity verification finish after the socket answers, and a request that needs one of them waits for that one alone. A migration never runs before serving: a release reads every format its supported predecessors wrote, and a tender rewrites old data afterwards. Shutdown never waits on work in flight, because in-flight work is already a durable record. A backup is a snapshot, never a copy on the start path. Between two otherwise equal designs, take the faster; when the faster one costs complexity, measure before refusing it. (Eddie, 2026-09-27, after an OpenClaw restart spent about six of its seven minutes before serving copying, integrity-checking, and migrating 9 GB of databases, one after another.)
EFFICIENT	An idle conversation costs kilobytes. Ten thousand on one node is the target; hundreds already beats everything but a hyperscaler's managed agents at a fraction of the cost.
TASKS	A persisted task graph, read whole every turn, edited through structured actions the harness executes. No MCP, no free-text tool for tasks.
MEMORY	One graph. Jev labels kind, durability, and trust. Every node eligible every turn; the indexer and budgeter make that cheap.

Principle	Constraint it imposes
LOOP FOREVER	Default is to continue while authorized, runnable work exists within reserved resources . Stopping requires a positive, logged reason: a Jev judgment or a deterministic control path. Budget exhaustion is correct operation, never a judge failure, even when the last judgment said <code>progressing</code> : a task can be progressing honestly when its allotment runs out. The ledger records <code>budget_exhausted</code> as its own outcome class so the stopping policy is never trained against honest progress. <i>(Since 2026-09-29 reaching the limit does not end the execution: it waits for the operator's reset, and the ledger records the stop as <code>budget . asked</code> ; §3.13.)</i>
OPINIONATED	One blessed path, few knobs, strong defaults. Discord's model is the session model. AWS is the tool model.
JEV IN THE LOOP	Continue/stop, role, continuation strategy, shell class, memory labels, security risk, and MCP sampling proportionality are all typed judgments over bounded state.
LEARNING	Every judgment is recorded with inputs, action, and later outcome. Question packs, memory-science parameters, and shell mappings are versioned data tuned by that record. Precisely: feedback-driven policy and parameter optimization with holdouts and canaries (§3.10), not reinforcement learning in the technical sense.
SPEED, DURABILITY, COST	In that order, for storage and for every runtime trade.
QUIET BY CONSTRUCTION	The harness has no busy loop. It parks on its event sources and the heartbeat. Work in flight is a durable record, not a waiting thread.
NATIVE FIRST	The default way Theseus does anything is a small, typed, in-process Rust toollet. Shelling out is the escape hatch, ledgered as such; the ratio of shell calls to native calls is a health metric, and the most frequent shell patterns are the queue for the next toollet. Typed arguments are what let policy read intent, provenance ride on outputs, and the learning loop see what the agent actually does. (Eddie, 2026-09-26.)
EXQUISITE VISIBILITY	Every turn, loop, provider call, judgment, and completion is timed and attributed as it happens, in the record, before anyone asks <i>(hook sites left this list with the hook system, 2026-09-28; §3.17)</i> . Statistics and visualization are built with the feature, not after it. Nothing that matters is sampled away, and any OpenTelemetry backend can be pointed at the running system for service-level stats without code changes. We move carefully because we can see. (Eddie, 2026-09-25.)
APPEND-ONLY	The event record only grows. Compaction, supersession, suppression, and forgetting are new records; nothing in the record is rewritten. Projections (retention, heat, task state, trust annotations, indexes) are mutable and rebuildable from the record. Payload erasure under §5.6 is the single, receipted exception.
NOTIFY OVER BLOCK	The gate tells the operator what ran; it does not stand in the way. "Theseus should notify over block -- the operator should /know/ when something bad is going to happen", and the operator should be "asked, sure, but a hard no, almost never." The finite lists of tools and MCP servers are what the operator controls, and the gate never tries to infer what a command's contents do. Safety rests on a default-safe environment and on the operator knowing what ran (§3.9). (Eddie, 2026-09-28; replaces IRREVERSIBLE WAITS.)

Principle	Constraint it imposes
IRREVERSIBLE WAITS	<i>Superseded 2026-09-28 by NOTIFY OVER BLOCK: telling which calls are irreversible meant detecting what a command does, and that detection was removed (theseus-8az; Part III A3b). The principle as it stood: The gate protects the owner's options. A call is irreversible when, after it, no option the owner has restores what was there: a history rewrite, a destroyed remote, lost uncommitted work, a publication, a post outside Theseus. An irreversible call waits for approval at every enforcement level. Everything else may run with a notice when the owner chooses <code>notify</code> or <code>open</code>, because a notice is enough while the owner can still undo (§3.9). (Eddie, 2026-09-27.)</i>
FUNGIBLE ONTOLOGY	The kinds of context (channel, guild, person, topic, culture, expertise, and any kind added later) are data, not code. They live in a versioned table that the owner, operators, and Jev extend, and whose memberships are trained, re-associated, and indexed by embedding. Interpretations route context; they never grant access (§4.1a). (Eddie, 2026-09-27.)

3. Architecture



3.1 Bindings (channel configuration)

Configuration is a list of bindings, resolved most-specific-first (thread over channel over guild). Each selects a Discord scope and attaches behaviour:

```

binding:
  scope:    { guild, channel | thread | dm | voice_channel, optional user/role filter }
  persona:  system prompt, name, default voice
  roles:    allowed role set and default weights (see §3.4)
  policy:   tool tags allowed, confirm thresholds, spend ceilings, shell classes allowed, p
  model:    provider/model, thinking effort, optional Jev cheap-first cascade
  memory:   namespaces read and written
  loop:     judge pack versions, ceilings
  mcp:      attached servers, which may sample, which may elicit
  listen_only: bool    # ingest to memory, never reply

```

Bindings live in a versioned config in the store, editable by slash command and by the web UI with an audit trail. The Discord-role-to-policy mapping is a top-level table the operator administers through the same two surfaces.

3.2 Channels and conversations

Channel is a Discord place under a binding: a thread, channel, DM, or voice channel. Channels are cheap, unbounded, and carry presence, permissions, and Discord message ids. Channel is also a classification dimension: where something is happening, and where past things happened.

Conversation has a deliberately tight definition: **the subset of the node graph that happened on a particular channel, in temporal order**. It is a view derived from the graph, not a stored entity with its own lifecycle. Its identity *is* the channel. It never ends because a channel never stops accumulating nodes; it can only be continued, re-rooted by compaction, or left dormant. The `next` chain is therefore per channel, and `in_conversation` is not a separate edge type; `in_channel` plus temporal order defines it.

Consequences:

- Many participants, each a `Person` node with `by` edges; no per-human forks.
- A voice channel and its paired text channel under one binding are two channels and therefore two conversations that share participants, tasks, and memory scope; the agent borrows across them freely (below), which is what makes them feel like one.
- Each execution (§3.15) reports into exactly one channel at a time; many executions run concurrently. A channel orders what is *delivered* to it, never what is *computed* for it (§3.2a). "Gliding" is an execution moving between channels, and pulling other channels' history into the current one.
- A reference to another channel's history ("what we did in the deploy thread") makes the agent pick it up: **borrow** by default (that channel's summary or relevant nodes enter the current context via `mentions_conversation`), **switch** on explicit ask (subsequent turns are posted and appended in that channel instead).

The runtime object behind a channel is a `tokio` actor with a mailbox and a few kilobytes of hot state. Parked, it costs no CPU and no thread. Turns are serialized per channel; messages arriving mid-turn are coalesced into the next context build **with each message's author preserved**, so authority never

blurs across a coalesced batch. Under LOOP FOREVER, an inbound message during autonomous work is a nudge, not an interrupt, unless it is a deterministic control (/stop, /cancel, revocation) or Jev judges it a new ask.

Delivery (theseus-q4v; built 2026-09-30). What must reach a channel is written when it becomes true, whether or not anything can deliver it: a turn's reply, a confirm card and how its question closed, a notice the operator must see, and a task's report. Each is an outbox post (§3.16), addressed to the channel its session reports into. The binding only delivers: it sends a channel's posts in order and once, and records the message ids in each post's completion. Live progress is not a post. Typing, and the edits of a reply while its turn runs, are the binding's alone: best-effort, never replayed, and only a message's latest state is ever sent. Nothing waits for a binding to start. A binding that was away sends what waited when it is back, before anything live.

Humans are `Person` nodes keyed by Discord user id with their own memory namespace that follows them across every guild and channel.

3.2a Sessions: what runs "simultaneously"

The word *session* is used deliberately and narrowly. A **session is a compiler scope**: the thing that decides which roots the context compiler starts from, which audience it compiles for, which budget it spends, and which continuation strategy it uses. It is not a store and not a transcript. There are exactly two kinds:

- A **conversation session**, one per channel: root is the channel's temporal view; audience is the channel's participants; cadence is human (a reply is expected soon).
- A **task session**, one per task that has been *promoted* to autonomous work: root is the `Task` node, with its evidence, its subtasks, and the conversation that created it as secondary roots; audience is whoever it reports to; cadence is autonomous (no one is waiting on the next token).

One turn per session. Every session has exactly one execution (§3.15) and that execution has one turn lock. "Hundreds or thousands of sessions running simultaneously" therefore means: thousands of task and conversation sessions exist as durable nodes; at any instant those with runnable model work hold a turn, the rest are `waiting` and cost nothing. An **admission scheduler** bounds how many hold a turn at once by budget, provider rate limits, and a configured concurrency ceiling; it is a queue, not a policy, and it never reorders deterministic control paths (/stop, /cancel, revocation).

Promotion. A conversation accumulates intent; at some point it becomes a structured, trackable, goal-oriented piece of work. That moment is `task.create` with `autonomous: true` (proposed by the model, judged by Jev under `CLASSIFY`, or asked for by a human). Promotion **forks an execution**: the new task execution inherits the requesting principal's authority and delegation limits (never broader, §3.9), receives its own budget allotment carved from the requester's, records `origin_channel`, and gets a `reports_to` edge to that channel. The conversation session returns to its human cadence immediately; it is not blocked by the task and never was the task.

Promotion requires an arrangement (Eddie, 2026-09-27; openrig's mission install, Appendix F). The requesting conversation's agent holds the discussion, so it writes an `Arrangement` node. The node names the pieces the task needs (objective, acceptance criteria, design nodes, all by id), what to trust, and what supersedes what. The task's first compilation admits the pieces by reference, never paraphrased, so a later correction still reaches them. The arrangement comes right after the objective, rendered as testimony with its origin and as-of. Promotion is refused without an arrangement, and the refusal gives the reason. No arrangement is ever generated at install time, because a generated summary is a second source that drifts. If a one-line objective is drawn from a long discussion, promotion flags it and asks for the design to be attached.

(As built 2026-09-30, theseus-qn2 (DD7), a first form of promotion. `task.create { brief, budget_usd? }` opens a task session whose first node is the brief, and returns at once. The child inherits the parent's authority, persona, context files, postures, and model, and, when the parent has read external text, its hold (§3.9, theseus-9bp). Its approvals go where the parent's go, and its notices name it. Its budget is carved from the parent's: `budget_usd`, or a quarter of what the parent has left, capped at all of it, as a reservation in the parent, with the child's spend counted in the parent's. Depth is one: a task cannot start tasks. A task is done when its turn would wait on input, and its last message is its report. The report goes once to the place through the outbox (§3.16), and once into the parent's session as a node at the parent's next turn. By default nothing starts a parent turn. A task opened with `wake_parent: true` starts that turn when it finishes or fails (theseus-lji, W1): the frame that ends it queues the parent, as a due wake does (§3.15), and the turn runs in the parent's session, under its authority and budget, with the report as its input and `task a1b2c3 reported` above its reply. A busy parent runs it when it is free, reports that land together start one turn, and a cancelled task wakes nothing, since whoever cancelled it is there. The option is for a chain, where the parent reviews each result and starts the next. `task.list` and `task.cancel`, theseus tasks and theseus cancel, and Discord's `/tasks` and `/cancel <id>` see and stop tasks, and `/stop` does not stop them, and the web UI shows each session's tasks as a tree. The arrangement, `autonomous: true`, sub-tasks, and §3.5's task graph are not built: the requesting model writes the brief. Part III A4, item 7.)

How they stay connected. Only through the graph, never through shared in-memory state:

- The task execution writes `Progress`, `Question`, and `Result` nodes with `reports_to` edges. Delivery to the channel is an *action* (§3.16), ordered by the channel like any other post. The channel never waits for the task to compute; it only orders what the task says.
- The conversation session's compiler admits a bounded summary of the channel's live task executions (state, last progress, open questions) every turn, so the agent talking to humans always knows what its background hands are doing.
- A human message in the channel is classified: a nudge or new ask for the conversation; a **control** (`/stop`, `/cancel <task>`); or **addressed to a task**, in which case it is written as a `Message` node with an `addressed_to` edge and *wakes* the task execution, whose next turn coalesces it with the author preserved. Ambiguity resolves toward the conversation, which can always hand it on.
- A task that needs a human (a confirm, a question, a blocked gate) enters `waiting` with a `Question` node delivered to its channel and its requester; the answer wakes it. This and terminal states (`complete`, `failed`, `budget_exhausted`) are the only points at which a task is

synchronous with anyone.

- A task session may **glide** like a conversation (report into a different channel; borrow another channel's history), under the intersected-ceiling rule.

Sub-tasks. By default a task's subtasks run inside its one execution, sequentially, under its one turn lock; the task is the unit of autonomy, not the subtask. A subtask is promoted to its own session only by an explicit `task.create {autonomous: true}` from inside the task, bounded by the parent's remaining budget and a per-task fan-out cap; the child's `reports_to` is the parent task, and its terminal state is a `Completion` for the parent. Fan-out is therefore a tree of sessions with budgets that sum, never a swarm.

Resources. Sessions do not serialize resources. Two task sessions can touch the same checkout or the same task node; §3.5's CAS versions, claim leases, and workspace locks are what arbitrate, and a lost lease is a `blocked` state with a reason, never a silent retry.

What this buys: the conversation stays responsive while work runs; a task survives its conversation going quiet for a week; a crash loses no task because the execution is durable and its session is recomputed from the graph on the next turn; and "what is running right now" is a query over executions, answerable in the web UI and to the model.

3.3 The harness loop and the model loop

There are two loops, and keeping them distinct is what makes LOOP FOREVER cheap and safe. v0.8 sharpens the first one into an **event-driven execution model** (review: [notes/event-design-review.md](#)): the harness holds no in-flight state that a restart would lose, and it is quiescent between events by construction.

The harness loop is deterministic, never calls a model, and has no busy loop. It is parked on `select()` over its event sources: Discord, the completion sources (§3.16), the wake queue, tender health, budget thresholds, and a one-minute heartbeat timer. It has two very common states, and they differ only in how many open records the heartbeat has to reconcile:

1. **Quiescent.** No open executions, no in-flight work, no scheduled wakes. Zero cost between heartbeats.
2. **Tending.** No agent output to send and no human input to add, but work is in flight: a build in a shell, an ECS task, a wake due later, a confirmation outstanding. "Tending" is not a running loop; it is **the set of open completion records** in the WAL. The harness is just as parked as when quiescent. It stays fully responsive to humans, and **no model is called** until an event makes a model turn necessary: a completion arrives, a human speaks, a wake fires, a confirmation lands, a budget nears its ceiling, or the heartbeat reconciler finds something.

The point of tending is that ongoing work is exactly when responses must not be dropped. A task marked `running` is not a reason to rest; it is a reason for its completion to have a durable home that does not depend on anything in memory. The harness decides *when a model turn is warranted*; the model decides *what to do* in that turn.

The heartbeat is the level-triggered reconciler. Events are the fast path and can be lost (a receiver restart between dispatch and completion, a wrapper whose delivery failed and exited, a callback during a network flap, a cron that fired while the node was down). Every minute, without a model call, the heartbeat walks the open records: is there a spooled result, a queued message, a finished job scope, or an external status that says a record completed without us hearing? Is any record past its deadline? Is any wake due? Is any tender unhealthy, any budget window rolling, the Discord gateway alive? Findings become ordinary events; a completion the reconciler cannot establish becomes `outcome_unknown` and is surfaced to the principal. If nothing is found, it appends one `Heartbeat` record and returns to `select()`. This is the Step Functions task-token model and the Kubernetes resync lesson applied together: edge for latency, level for correctness.

Startup is reconciliation first, in this order: (1) restore the last checkpoint and replay the WAL tail, so every action record exists; (2) **stage** the completion inbox and spool without consuming anything; (3) for each staged completion with a matching record, commit settlement **and** the execution's continuation state in one WAL transaction; (4) only then acknowledge or remove the completion entries; (5) reconcile remaining open records. Nothing in flight at crash time is lost or silently forgotten; it is either completed from durable evidence or reported as unknown.

The model loop is one execution taking one or more turns against the stateless Messages API. It runs only when the harness loop has an event for it. Its state machine:

```

IDLE -(inbound | wake)→ CLASSIFY → BUILD_CONTEXT → CALL_MODEL ┘ end_turn → JUDGE_STOP
                                     └ tool_use → GATE → E
JUDGE_STOP:   TERMINAL → reply, IDLE (human regains control)
               NOT_DONE → NUDGE (specific user-role message) → CALL_MODEL
JUDGE_CONTINUE: HEALTHY → CALL_MODEL
               THRASHING → INTERVENE (course-correction) → CALL_MODEL
               ESCALATE → ask in Discord, park until answered

```

- **CLASSIFY** is one Jev fan-out over the inbound turn: role called for, topics and people mentioned, security risk, whether it references another conversation, whether it is a follow-up fragment, continuation strategy hint.
- **BUILD_CONTEXT** assembles the prompt from the context graph (§4) under the budgeter, in cache-stable order (§4.4).
- **GATE** is the policy gate (§3.9) on each tool call.
- Each **JUDGE** is one Jev request over bounded state: the original ask, task-graph delta, last N tool calls and results truncated, turn count, spend, wall clock. Pack `loop.v1: work_state` (Choice: `complete` · `progressing` · `blocked_needs_human` · `thrashing` · `off_task` · `other`), `stopping_point_defined`, `same_action_repeating`, `cost_out_of_proportion`, `wants_human_input` (Nouls).
- **Execution outcomes** are distinct and deterministic where they can be: `complete` (accepted objective satisfied: `Jev complete` $\geq \tau$ and the execution's task scope has no open accepted tasks), `waiting` (no runnable work now; a wake condition exists, e.g. a due time, an external job id, a task blocked on another execution), `blocked` (progress needs a human), `cancelled`

(authority or intent withdrawn via a deterministic control path, never via Jev), `failed` (recovery exhausted), `budget_exhausted` (reservation consumed; correct operation). `LOOP FOREVER` means: continue while authorized, runnable work exists within reserved resources.

- **Jev unavailable or malformed:** the loop treats the judgment as `abstain`; the execution finishes its current tool call, then parks as `waiting` on Jev recovery with a bounded retry, and tells the channel. It never continues autonomously without a judge and never invents an answer.
- **Wake** covers scheduled and self-scheduled continuation: the agent may ask to be woken ("check the build in twenty minutes"), which is a `Task` with a due time in the harness loop's wake queue, not a separate cron system. (*Built 2026-09-30, theseus-cff (DD8): the self-scheduled wake is `wake.at { at | after, note }`, one-shot, into the current session. It is not a `Task`: a pending wake is a field of the session's execution, and at its time the session gets a continuation turn whose input is the note (§3.15).*) Running work is also a wake source: every shell job and external operation registers a completion event, so "the build finished" reaches the model loop as a turn, not as something a human has to notice and relay.

3.3a Loop, turn, and the Advancer

Three words the rest of the document leans on, fixed here.

Loop. One pass of the lifecycle: an input goes through the **toolchain manager** (which compiles or appends the context, §4.4a, and decides which tools are offered), a request is sent to the provider, and the model's response comes back, with text, tool calls, or both. A loop is the unit the ledger prices and the unit Jev sees. In a normal working turn there are many loops as the harness and the model chatter back and forth, executing tools and feeding results.

Turn. The sequence of loops run under **one acquisition of a session's turn lock** (§3.2a), from the stimulus that woke the execution to the moment the execution releases the lock. A turn ends when, and only when, the Advancer says so. This definition is chosen over "stimulus to reply to a human" because it lines up with everything else the harness already counts: the lock hold, the WAL transaction boundary (§4.6), the budget reservation, the "one turn per session" concurrency unit, and the append-or-recompile decision (one per turn). Under the event-driven model a long shell job splits what a human would call one exchange into two turns, one that dispatches and releases, and one that wakes on the completion; that is a feature, because each is a durable, resumable, separately priced record. When the human-perceived unit is needed (for the UI, or for the learning label "did this exchange succeed"), it is called an **exchange**: the run of turns in one session from a human stimulus to the next delivery addressed to a human. Anthropic's own "user turn / assistant turn" are called **provider messages** here, never turns, to keep the collision out of the code.

Advancer. The modular component that, after every loop, decides `continue` (execute the proposed tool calls, feed results into the next loop) or `end_turn(reason)`. It is a trait with pluggable policies:

- `stop_after_one_loop`: the first version's policy, and the permanent baseline. One loop, then the turn ends and the model's response is the turn's output.
- `until_no_tool_calls(max_loops)`: the conventional agent loop with a hard cap.

- `judged`: the spec's `JUDGE_STOP` (§3.5, §3.7), Jev deciding progress, stall, or done, with the deterministic controls (`/stop`, `/cancel`, budget, mechanical acceptance checks) always outranking it.

The Advancer never widens authority and never bypasses the gate: it decides whether to loop again, not what a loop may do. Its decision and reason are ledgered per loop, which is what makes the stopping policy learnable.

Turn trace. Every turn records a nested tree of timed spans as it runs: `turn > loop[n] > { compile, provider.call > { first_byte, first_token }, advancer } > ... > session.write`, with the wait for the session's turn lock as the first child. Each span has a start and end in microseconds from the turn's start, a kind (turn, loop, provider, mark, advancer, compile, store, lock), and attributes (request id and usage and stop reason for a provider call, decision for the advancer). (*The hook spans and the hook kind went with the hook system on 2026-09-28, in 11d2f43. Old traces keep them, and they still render.*) The finished tree rides on the turn result, is written as a `turn.trace` ledger row, and on failure rides in the error payload up to the point of failure. This is the structure that tools, thinking, judgments, and completions will fill in as they arrive: a loop with three tool calls is three more spans under it, not a new mechanism. Rendering: a waterfall with a per-kind time summary in the web UI behind the timing link, and an indented tree with bars from `theseus ask --trace`. It is not sampling and it is not optional: the cost is a few microseconds per span, and the payoff is that every slow or strange turn can be read after the fact in exquisite detail.

3.4 Roles

One agent, many roles. The role table is a **living, versioned table** seeded before the ontology is fully known; rows are added as Jev or an operator discovers a new role, and the table's schema is `{id, stance, weights by node kind, hints, tools and shell classes favoured, verbosity, stop strictness, announce, added_by, added_at, version}`. The seed rows, drawn from six months of OpenClaw operation:

Role	Stance	Weights and hints
planner	goal-directed	tasks and decisions up; long horizon; asks clarifying questions before acting
coder	goal-directed	code, tool results, repo resources up; L0/L1 shells; terse; tests as evidence
reviewer	critical	diffs, prior decisions, conventions up; never edits; produces findings with evidence
operator (infra)	cautious	resources, events, runbooks up; destructive-tag awareness; prefers read tools first
researcher	exploratory	external resources, topics, citations up; longer outputs; flags uncertainty
thought partner	exploratory	people, preferences, prior conversations up; asks back; no tools by default
expresser	creative	persona and voice up; prose quality; minimal tools
triager	fast, decisive	events, tasks, people up; short outputs; routes rather than solves
secretary	goal-directed	calendar, people, events up; scheduling, briefings, follow-ups

Role	Stance	Weights and hints
security analyst	sceptical	provenance, trust labels, policy up; treats content as evidence not instruction
teacher	patient	deep knowledge and resources up; explains structure; checks understanding
archivist	curatorial	memory nodes, contradictions, supersessions up; proposes merges and forgets
(new)	(proposed by Jev or operator)	rows appended here; every addition carries <code>added_by</code> and a ledger reference to the classification cluster or human request that motivated it

Operators may define roles by hand in the web UI **and** Jev proposes new ones through the learning channel; both land in the same versioned role table. Role changes are **announced by default**: a short, in-channel note ("switching to reviewer") that is also a `Judgment`-linked event in the graph, so humans see the shift and can react to it, and the ledger can learn from those reactions. A binding may set `announce: false` to make role changes silent.

Beyond this set the table grows: Jev classifies the role each turn and scores the candidates. A role is a bundle of **values**, never filters: per-node-kind budget weights, personality guidance, stance (goal-directed, exploratory, learning), preferred tools and shell classes, verbosity, stop-criteria strictness. The role ontology is Jev-owned and grows through the learning loop: poorly covered classification clusters propose new roles; scored classifications flow to the operator on the learning channel (§3.10) and the web UI.

Jev's ontology, minimum: roles, people, topics, events, resources, memories, capabilities, deep knowledge. Expanding and honing it is a permanent activity. Since v0.37 the ontology is one mechanism (§4.1a), and roles are one kind in its table, beside channel, guild, person, topic, culture, and expertise.

3.5 Tasks (fluid)

The task graph is a core persisted structure whose interface is conversational. The agent sees the task graph **for its execution's scope** every turn (ids, titles, states, dependencies, owners, one-line acceptance), bounded by the budgeter: beyond the budget, the view collapses to open tasks plus one-line summaries of closed subtrees, with a count of what was elided and edits it through a reserved action namespace the harness executes, in the same tool-call grammar as everything else:

```
task.create {title, parent?, deps?, acceptance?, owner?, due?}
task.update {id, patch}      task.move {id, new_parent}      task.split {id, into[]}
task.merge {ids, into}       task.close {id, done|abandoned, evidence}
task.claim {id}              task.handoff {id, to: human|conversation}
```

Authoritative task state machine (the only one in this document): `proposed` → `accepted` → `in_progress` → `{blocked, waiting_human, suspended}` → `in_progress` → `done` | `failed` | `abandoned`. `suspended` is an execution-level pause (cancelled or budget-exhausted execution) that leaves the task recoverable. Human edits in conversation become the same actions. Discord shows the graph as a living pinned

message per thread with components for common moves; slash commands cover work outside a conversation. Every mutation is a bus event and a ledger row. Three layers of the task graph have different mutability so the agent cannot redefine success:

- **Accepted objective and acceptance criteria:** authority-controlled. Changing them, or abandoning an accepted requirement, is a ledgered action that requires the requesting principal (or owner) to accept; the agent may propose it, never apply it alone. Abandoning is not completing.
- **Working plan and decomposition:** freely editable by the agent within the objective (create, split, merge, move, claim).
- **Evidence and outcomes:** append-only, tied to specific artifacts, test runs, or external states with their identity (commit, snapshot id, job id).

Where an acceptance criterion is mechanical (tests must pass, an artifact must exist, a check must be green), the deterministic check is a **necessary condition**: a failing check vetoes `done` while the requirement stands, regardless of what Jev says. Jev assesses the remaining semantic adequacy. This is not a second verification tier; it is the deterministic control path applied to acceptance.

`JUDGE_STOP` reads the graph: `done` is no open accepted tasks, every mechanical criterion satisfied, **and** Jev agreeing. Terminal and stalled states are first-tier Jev calls like every other loop state, escalated to a human only when the call is in question.

Concurrency. Channel serialization is not resource serialization; two executions in different channels can touch the same task or checkout. Task mutations are **versioned with compare-and-swap** (a stale version is rejected and the execution re-reads). Tasks have a **claim lease** with expiry; workspaces have an advisory lock per execution with a documented conflict behaviour; test evidence records the exact snapshot identity it ran against.

3.6 Providers

Profiles. A profile names a provider, a model, an output limit, and a system prompt. Exactly one profile is **live**; it is chosen in config at startup and can be switched over the protocol at runtime, and the switch persists in the store so a restart keeps it. A turn may name a profile explicitly without changing the live one. Sessions and tasks will carry their own profile override in a later milestone; the resolution order is fixed now: raw overrides, then the turn's named profile, then the session or task override (future), then the live profile.

Providers are a table, not code (M0.5): every endpoint that speaks the Anthropic Messages API is a named `[providers.<name>]` entry with its base URL, key reference, and optional timeouts. The first-party API is the implicit `anthropic` entry; Z.ai's GLM series is `zai` at `https://api.z.ai/api/anthropic`. A turn selects provider and model explicitly or takes the configured default; both are ledgered on every call. A second wire protocol (OpenAI-compatible, Bedrock) would be a new `Provider` implementation behind the same trait and a new `kind` value.

Provider trait: `complete(request) -> Stream<Event>` with text, thinking, tool-use blocks, and usage. First and default implementation: Anthropic Messages API with streaming, adaptive thinking and effort, prompt caching (§4.4), and the Opus 5.5 / Fable 5.1 contract (thinking always on, no forced tool use, thinking blocks tied to model). Model per binding, with an optional Jev-driven cheap-first cascade (the OpenClaw jev-router design carried over). On provider outage Theseus fails closed: the conversation is told plainly that the model is unavailable, in-flight work is parked with its state intact, and nothing falls back to another provider.

When the provider does not answer (built in M0.5, 2026-09-25). Every call is bounded by four timeouts: connect, first byte (request sent to response headers), stream idle (longest silence between events), and total. Each failure is classified before anyone reasons about it: `timeout` (with its phase), `network`, `rate_limited` (with the provider's retry-after and rate-limit headers), `overloaded`, `server`, `auth`, `invalid_request`, `stream` (an error event mid-stream), `truncated` (the stream ended without `message_stop`). The class carries two facts the harness needs: whether a later identical call could plausibly succeed (`transient`), and whether the provider may have billed tokens we never saw (`usage_unknown`: true for idle and total timeouts, stream errors, and truncation; false for connect and first-byte timeouts, where nothing was generated). The turn fails, `provider.error` and `turn.failed` rows are ledgered, the error reaches the client with the class in `error.data`, and **nothing retries on its own**: a held reservation and a human or a later policy decide. The provider's request id, rate-limit headers, and timing (first byte, first token, total) are ledgered on every successful call as `provider.call`.

3.7 Judgments (Jev) as a core service

Judge trait, one implementation (TypeSafe Jev), always wrapped in a `Recording` decorator that writes every call to the ledger: pack id and version, state hash and size, answers with probabilities and confidence, latency, the action taken, and a slot for the later outcome label. Rules baked into the trait: atomic questions, mandatory no-match option, three-band confidence gate (act, confirm, escalate), state built by a `StateBuilder` that truncates to the model's limit by construction. Jev is never the sole gate for anything security-relevant.

Jev is a substantial dependency with correlated-error and latency exposure: the ~350 ms figure is one call, and a turn may make several, some serially dependent. The critical path is measured per workload class (simple reply, coding tool iteration, long-job completion, cross-channel recall, voice turn, large candidate set) and the budgeter batches fan-outs so serial depth, not call count, bounds latency. Judge probabilities are treated as **uncalibrated until the ledger shows otherwise**; thresholds start conservative and are tuned only on labelled outcomes.

Question packs in the design so far: `classify.v1`, `loop.v1`, `continuation.v1`, `shell.v1`, `memory.v1` (kind, durability, about, trust), `security.v1`, `sampling.v1`, `role.v1`. All versioned, all tuned by §3.10.

3.8 MCP, both roles

As client. Transports: stdio for servers Theseus launches (in an L1 sandbox or an AWS class) and streamable HTTP for remote servers; no legacy SSE. Primitives: `tools`, `resources`, `prompts`, `elicitation`, `sampling`.

- Tools join the typed tool surface tagged `mcp:<server>` and pass the policy gate. Resources are readable via `resource.read` and pinnable into a binding's context.
- **Prompts** surface as slash commands scoped to the bindings the server is attached to; returned messages enter the turn as user-role content with a provenance label; templates are cached and diffed, and a change triggers an operator notice before reuse.
- **Elicitation** renders as Discord components (modal, select, buttons) to the human who owns the conversation; schema-validated answers only, a timeout that fails the call cleanly, only from servers the binding marks `interactive`, always recorded. Elicitation never substitutes for the policy gate.
- **Sampling** becomes a Theseus provider call under the conversation's model, spend ceiling, and ledger, judged by `sampling.v1` for proportionality and steering and passed through the three-band gate. Only servers the binding marks `samples` may sample.
- Credentials from Secrets Manager or SSM; OAuth for remote servers completes in Discord via a link component.

As server. Streamable HTTP bound to localhost only; Theseus never accepts inbound connections directly. Exposure, when wanted, is an external proxy's job. Auth is a single static API key for now; per-client scoped tokens are deferred. Exposed tools: `task.*`, `memory.search`, `memory.get`, `memory.propose` (goes through Jev ingest, never raw write), `conversation.open/send/status`, and any downstream tool the token's policy allows, re-exported through the gate. Exposed resources: task graphs, policy-gated transcripts, read-only memory namespaces. Exposed prompts: persona-authored templates. Theseus-initiated sampling and elicitation toward its clients are supported for the case where the client is a human's front end. Every inbound call is a conversation event and a ledger row; rate and spend limits per token.

Excluded: `roots` from external servers (Theseus decides filesystem views), and any capability that lets a server modify bindings, policy, or credentials.

3.9 Policy, authority, and safety

Authority context. Every execution carries an explicit authority context, fixed at creation and re-validated at each tool call: the **principal** (the requesting human, the owner for scheduled or proactive work, or an MCP client identity for inbound MCP calls), any **delegation** (a human may delegate a bounded capability set for a bounded time), the **binding ceiling**, and the **resource ceilings** of the channel and guild. Effective permission is the intersection: no broader than what the principal holds, capped by the binding and resource ceilings, with explicit deny taking precedence over any allow. Authority is never derived from whoever happens to be present in a channel; an administrator and an ordinary user sharing a channel do not pool their powers.

The gate: notify over block (Eddie, 2026-09-28; rebuilt in theseus-8az, Part III A3b). Theseus tells the operator what it does, and it waits only where the operator or the floor says to wait. In Eddie's words, "Theseus should notify over block -- the operator should /know/ when something bad is going to happen", and the operator should be "asked, sure, but a hard no, almost never." What the operator controls is the finite list of tools and, when MCP lands, of MCP servers. The gate decides per tool, from the tool's name and what its plan names: the resources, the argv, and the path arguments. It does not try to infer what a command's contents will do, since "it's literally hopeless to try to figure out all the different ways to hide a command." Safety rests on running Theseus in a default-safe environment (below) and on the operator knowing what ran.

Postures. One setting, `[policy].enforcement`, is the posture every tool and every MCP inherits:

posture	the call
open	runs
notify	runs, and a notice is posted
approve	waits for the operator's approval

The built-in default is `open`, and the template sets `notify`. The gate has no deny anywhere: no posture, list, or reason refuses a call. The strongest answer the gate gives is to wait, and the operator approves or declines. A `deny` in the config fails to load, and the error names the three postures.

Overrides.

- `[policy.tools]` sets one tool's posture. The template lists every tool, one line each: the read-only tools are `open`, and the writers and `proc.run` are commented, so they inherit. `proc.run`, the universal shell, is the one worth pinning stricter. A test keeps the list in step with the tool registry, and an unknown tool name fails to load.
- `[policy.mcp]` does the same for MCP servers: `"server"` covers every tool from a server, and `"server/tool"` covers one tool. An MCP tool takes its `[policy.tools]` line first, then `"server/tool"`, then `"server"`, then `enforcement`.

The operator's lists are stated will, not inference.

- `allow_argv` names commands that `proc.run` runs without asking, when every path argument is inside the workspace roots.
- `approve_argv` names commands, and `[tools].approve_paths` names paths, that wait for approval.
- A path outside the workspace roots waits for approval too. The workspace itself is configuration (`[tools].projects_dir`, plus any more `roots`); nothing assumes where an operator keeps projects.

Private addresses (theseus-yd6; built 2026-09-30).

- A network call's URL is judged at step 2 of the order below, as a path outside the roots is. It waits for approval when its host is one of these, and the confirm names which:

- a loopback, private, link-local (the cloud metadata service's 169.254.169.254 included), unique-local, shared (100.64/10), unspecified, multicast, or reserved address;
- `localhost`.

An IPv4 address inside IPv6 is judged as itself.

- A public name that resolves to such an address is refused at connect. The client's resolver checks the very answer the connection uses, so a rebinding name has no second answer to give. The result says so, and names the address to ask with. This refusal is the tool's, not the gate's: the gate never saw the address, so it could not ask.
- Each redirect hop is judged the same way before it is followed.
- An approval reaches only the private origin its URL named.
- A proxy is never used, since it would resolve names past the check.

The floor is Theseus's own state (store, spool, bindings file) and binary, and the 1Password CLI and its credentials, including the token file the daemon was given. The kernel is off limits to the agent (§3.21), and the vault holds every secret. A call that touches the floor waits for approval at every posture, `open` included, and its confirm says it is the floor. It is never refused and never silent. The floor is deterministic: no model judgment or Jev pack decides it. *(Until 2026-09-28 this also said that no hook decides it, and that the floor judges what a transform hook made of a proposal. The hook system is gone, §3.17.)*

Order. The first match wins:

1. the floor;
2. the approve lists, a path outside the roots, and a URL whose host is a private address;
3. the allow list;
4. the tool's posture.

A granted secret's posture (theseus-dcy; built 2026-09-30). Each secret the broker hands out (§3.19) carries a posture: `[broker.secrets.<name>] posture, notify` by default. A call that the broker gives a secret runs at no looser a posture than that secret's. After the order above, the stricter wins, as a tightening does:

- an `allow_argv` call that gets a `notify` secret is notified;
- a secret whose posture is `approve` makes every call that gets it wait, and the confirm names the secret's setting.

The call's tool line and its notice say what it was given ("gh got GH_TOKEN"). A spawn gives no secret whose posture is stricter than the one the call ran at.

External text (theseus-9bp; built 2026-09-30). This is the interim, deterministic floor for web text, until provenance labels ("Exposure", below) and Jev (M5) arrive, and it stays the floor after them.

- **When a session holds it.** A session reads external text when a result node marked `external` enters its context: `http.fetch` and `web.search` today, and MCP results and untrusted attachments when they come. The first such read since the operator last trusted the session is its **hold**, kept on the session's record, with a `session.external_read` row (the node, the tool, the URL). Both are written in the frame that writes the node, so no crash leaves the text in the context without the hold. A later read writes nothing.
- **From another session.** A task that a holding session starts holds the text from its brief, which that session's model wrote. A session that reads a report from a holding task holds it too, from the frame that writes the report.
- **What waits.** After the order above and a granted secret's posture, every call whose class is not `read` waits for approval: writes, edits, patches, `proc.run`, `task.create`, and `wake.at`, including the allow list's calls. The stricter posture wins, as a tightening's does. A `read` call keeps its posture, fetches and searches included, so research goes on. A call in the same response as the fetch keeps its posture too, since the model wrote it before it saw the page.
- A wake's turn, and a turn that a task's report started, are the session's own turns, so the hold covers them.
- **The confirm says why:** "this session read external text (`http.fetch` , at 13:05), and a call that acts waits for approval after that (§3.9)". Discord, the web UI, and the CLI show it.
- **Trusting it again.** Only the operator clears a hold, with the trusted answer an approval takes ("Approval", below), so a Theseus job's process is refused. There are two ways:
 - `policy.trust: theseus policy trust <session>`, or the Observatory's "trust again";
 - an approval that trusts the session as well: the card's **Approve + trust session** button on Discord and in the web UI, or `theseus confirm --trust`. The button appears only when the hold is why the call waits.

A trust is ledgered as `session.trusted` (who, how, and the hold it cleared). A trust accepts the text already in the context; a later read holds the session again.

- Health, `theseus health`, and the Observatory list the sessions that hold external text, since when, and from what. The hold is on the session's own record, so it survives a restart.
- `[policy] external_text = "ask" | "notify"`, ask by default. `notify` runs a call that acts with at least a notice.
- The rule judges what the session has read, not what the text says. It does not stop a page from sending data out through a fetch's URL, since a read keeps its posture; each fetch's notice names its URL. It also does not follow a job's own process: a job the operator approves can open a clean session through the socket (theseus-d64).

Notices and records. Every call, under any posture, is recorded on its tool-call node with the gate's decision, the posture, and the reason. A call that runs under `notify` is also ledgered (`tool.notified`) and shown on every surface as a notice: what ran, the setting that made it a notice, and the outcome. In the web UI and the CLI it is its own line. On Discord it is the call's line in its loop's tool message, `notified (<setting>)` and then its outcome, and a loop that overflows one message counts the notices on the line that folds its oldest calls. (*Amended 2026-09-29, theseus-w4f: a separate Discord embed per call is `[discord] notice_embeds`, off by default, because the DM's roughly 160 shell calls a day would each post one; Part III A4, item 3.*) A call that waits is a confirm on every surface. A declined call is recorded as declined and never runs. Only a toollet's own input validation stops a call at the gate, and it does so as an error, not a refusal.

What the gate is not. It judges what a call names, not what a program does once it runs, so it is not a sandbox. For arbitrary commands, the operator's control is `proc.run`'s posture, and the boundary is the environment (§7; L1 in M4).

Jev (M5; not built). Eddie's direction is one classifier (`security.v1`) that says "this is risky: 0-100%" and, based on the posture, lets the operator know. As everywhere, Jev may make a call's treatment stricter, never looser, and it is never the sole gate (§3.7), because adversarial content can move its score.

Approval (Eddie, 2026-09-27; built 2026-09-29, theseus-sgh). An approval is a dialogue in a **trusted channel**: a surface listed in `[approval].channels` whose members are all **trusted users** (`[approval].trusted_users`). Both lists live in the vault-held config, which agents cannot write. The rule covers every answer: a tool call that waits (posture `approve`, the `approve` lists, a path outside the roots, the floor, a session's hold on external text), the budget question, and the trust of a session that read external text.

- **Where it is judged.** In one place, where an answer becomes a decision (`Core::confirm_action`), which every surface reaches through `action.confirm`. An answer counts only from a trusted user through a trusted channel. One that does not is refused with the reason, ledgered as `approval.refused` (who, through what, and why), and narrated; the call keeps waiting, and nothing is lost.
- **Who answered is known from the connection, never from what a client says.** The listener that accepts a connection names its surface: the Unix socket and `--stdio` are `cli`, the loopback bridge is `web`, and the in-process Discord binding is `discord`. Only the binding may name a Discord channel and user. An answer's `author` is a label; it names and proves nothing.
- **Not a job's process** (theseus-6qy; accepted by Eddie on 2026-09-29, and built the same day).
 - At L0 a job runs as the operator's own user, so it can reach the CLI socket and the loopback web UI just as the operator does.
 - So an answer (an approval, a decline, or the spend reset), the undo of a tightening, and the trust of a session that read external text are refused when the process that asks, or any ancestor of it up to pid 1, is a live Theseus job wrapper. This holds with or without an `[approval]` section, and is checked before its rules.

- *Who asks.* The socket reads its peer's pid (`SO_PEERCRED`) and start time when it accepts, and `--stdio` names its parent. The web UI finds the process that holds the client's end of its loopback connection, and only when a judged act arrives. A process that cannot be traced counts as a job's: one that has exited, a pid reused since, or another account's.
- *A double fork stays under its job.* The job wrapper is a child subreaper. After its command exits, it lingers until the last descendant has, and health counts the wrappers that linger.
- *A job that kills its own wrapper* (theseus-z4b). Its processes then go to the daemon, which is a child subreaper. Nothing that answers an approval descends from the daemon: the operator's CLI, the browser, and Discord never do. So an answer from a process under `theseusd` itself, with no live wrapper between, is refused too. The reason is from a process under `theseusd` itself (`pid <n>, <argv0>`), which is a job's orphan, and the asker names the daemon (`under_daemon`).
- *Any serving daemon's descendants* (theseus-6uo; built 2026-09-30). An asker under any other serving `theseusd` is refused too: a scratch daemon's orphan, or a `--stdio` daemon's, which is a child subreaper as well. A serving daemon is recognized by its command line: `theseusd` with no subcommand after its options. The reason is from a process under another serving `theseusd` (`pid <n>, <argv0>; the daemon is pid <d>`), which counts as that daemon's job, and the asker names it (`under_other_daemon`).
- *A refusal is loud.* It is ledgered as `approval.refused`, with the reason from a Theseus job's process (`job <id>, pid <n>, <argv0>`). It is narrated, and sent to the operator where approvals go. The CLI prints the reason and exits 1.
- A "should have asked" press is not checked, since it only makes things stricter. Discord has no process to check: an answer there is a trusted user's press.
- This is a speed bump before L1, not a boundary (M4). A job can still drive a process that is not its descendant: a user systemd unit, a tmux server already running, cron, or anything else the operator runs that takes commands. *Since theseus-z4b, killing its own wrapper no longer takes a job's processes out of reach.*
- **The channels.**
 - `cli` and `web` are the operator's own machine: the socket is mode 0600, and the web UI is loopback-only, so anyone with an account on the machine can reach it. Their only member is this machine's operator, so listing one is the whole rule for it, with no trusted-user entry. *Since theseus-6qy, an answer through either counts only from a process of this account that is not a Theseus job's. Another account's connection to the web UI cannot be traced, so it is refused.*
 - `discord:dm` is a DM between the bot and a trusted user.
 - `discord:<channel id>` is a guild channel. It is trusted only while nobody outside the trusted users can view it; another bot counts like anyone else, and Theseus itself does not.
- **Who can view a guild channel** is worked out with Discord's permission rules (roles, the channel's overwrites, the owner). It is checked when the binding starts, when a card is about to post, and again when an answer arrives. The check needs the member list, which Discord gives only when the bot's Server Members intent is on in the developer portal. The binding reads that

setting from the application's flags and leaves the gateway intents as they are. Without it, a listed guild channel cannot be verified, so it is not trusted, and health says why. DMs, the web UI, and the CLI need no check.

- **Where the dialogue is posted.** On Discord a card goes only to a trusted channel.
 - If the session's place is not a trusted channel, the card goes to a trusted user's DM (the requester's, when theirs is bound), and the place gets a one-line note that approval was asked there.
 - With no trusted DM, the place gets only the note, which says where to answer.
 - A card names only the trusted local surfaces as other places to answer.
- **Without an [approval] section there is no rule:** the CLI, the local web UI, and a place's listed Discord users answer, as before. Once the section exists, `channels` defaults to the CLI and the web UI, and `trusted_users` to nobody.
- Health, `theseus health`, and the Observatory list the trusted users and each listed channel's state (trusted, or not trusted and why).
- The approver must also hold the capability (confirmation is not authorization, below). *Not built: until roles and delegation land (M4), every execution's principal is the operator, and a trusted user answers with the operator's authority.*

Should have asked (theseus-sgh, 2026-09-29). A notice can be answered with one press: "should have asked". That tool then asks first, on every surface, until the press is undone.

- **Where it lives.** A tightening lives in the store (a `meta` record written in the same frame as its `policy.tightened` row), never in the vault config, and survives a restart.
- **Stricter wins.** The gate applies the config's posture, then the tightening, and the stricter of the two wins. A press can therefore never loosen anything, and one on a tool the config already asks for changes nothing.
- **Undo.** Undo returns the tool to the config's posture. It loosens, so it needs the same trusted answer as an approval (§3.9 "Approval"; `approval.refused` with `act: policy.untighten`). A press only adds asking, so it is accepted from any surface that can answer an approval.
- **Surfaces.** On Discord, one select menu on the loop's tool message lists its distinct notified tools; with `notice_embeds`, each card has a button instead. The web UI has a button by each notice and an Undo in the Tools view. The CLI has `theseus policy tighten|untighten|list`.
- **Records.** The `policy.tightened` row keeps the call's correlation id and proposal digest, a labeled example for later judgment work (M5).
- **The allow list** still runs its entries outright, as it does under a config `approve`.

Owner override (Eddie, 2026-09-27; §1 "Owner's property"). *Moot in the gate since 2026-09-28: the gate refuses nothing, so there is no refusal for an override to lift, and the owner can approve anything that waits. Held for Eddie: close theseus-qc4, or keep these rules for a later layer that refuses (Part III A3b).* The rules as decided:

- An override is possible only when every target the call touches is the owner's declared property. That property is `[owner]` in static config: repository patterns, hosts, cloud accounts. A call that touches anyone else's property keeps its refusal.

- The override is its own dialogue, in a trusted channel, and only the owner can give it. It names the rule being overridden and why that rule refused, is bound to the exact action's digest, and is ledgered as `policy.overridden`.
- The floor can be overridden too, at every enforcement level, but only with a **stronger ceremony**: the owner alone, the exact command shown, and a typed confirmation instead of a button. (*Moot since 2026-09-28: the floor asks instead of refusing.*)
- An override approves what the model proposed. Theseus never turns a model's refusal into an action, and no override reaches property that is not the owner's.

Consequences (Eddie, 2026-09-27; Appendix F). *Superseded 2026-09-28 (theseus-8az)*. The consequence kinds under one `irreversible` property, the rule table over `argv` with its shell parser, `opaque` for what the rules could not read, and `replay (theseus policy replay)` were built in steps 2a through 2a.3 (theseus-770 and three rounds of hardening), then removed, along with the "should have asked" flow planned for 2b. Hand-written detection of hidden commands can never be complete, and it had become the two largest files and the densest branching in the tree. Part III A3b records what was built and why it went.

Exposure (M4; Appendix F).

- Integrity labels (`untrusted`, `quarantined`) inherit only along transmission edges, as an `effective_trust` projection, so they do not saturate.
- A tool call proposed from a context that holds a quarantined node, or untrusted text shaped like instructions, is judged one posture stricter: `open` behaves as `notify`, and `notify` as `approve`. This holds only while that node is in the compiled context. (Decided on the old four-level ladder; restated on the three postures, 2026-09-28.)

Beads: theseus-3vu.

Revocation. If a principal loses a Discord role mid-execution, the execution is re-validated on its next tool call and, if it no longer holds the needed permission, transitions to `blocked` with a clear message.

Derived work keeps its authority. A due-time wake, a task handoff, an execution restart, or a scheduled continuation **retains the initiating authority context and its delegation limits**. It never broadens. A user who cannot perform a privileged action now cannot obtain it by asking Theseus to do it tomorrow; the scheduled execution runs as that user and is `blocked` at the gate exactly as the live one would be. Owner-originated proactive work (binding-configured automations, heartbeat findings, consolidation) runs under a **separately declared owner grant** recorded on the binding, and an authorized person may explicitly **adopt** or reauthorize derived work to change its authority, which is itself a ledgered action.

Coalescing does not merge authority. When messages from several people are coalesced into one context build, authorship is preserved and, additionally, a message from a principal other than the execution's own is treated as a **new ask**: it either spawns its own execution under that principal's authority or, if it amends the current execution's accepted objective, requires an authorized scope amendment. A lower-privilege participant cannot steer an administrator's execution by typing into the same channel.

Channel switching intersects ceilings. An execution that moves to another channel keeps its original grant **and** must satisfy the destination binding's policy, disclosure rules, and resource ceilings; the effective permission is the intersection of both. It never carries a privileged origin binding into a less privileged destination.

Confirmation is not authorization. A confirm click proves intent for one exact action; it grants no capability the confirmer does not already hold. Confirmations are bound to the exact tool, arguments, target resource, policy context, and an expiry; a changed argument invalidates the confirm.

Gate. Every tool call passes the gate in one order: the toollet's plan (which validates the input), the policy's decision, a confirm bound to the proposal's digest, the kernel's revalidation of that confirm, then dispatch. *(Amended 2026-09-28. This said "the kernel's gate in the §3.17 order", starting with a transform. The transform went with the hooks (11d2f43), and bb3ce56 replaced the kernel's Policy trait with plan, then decide; Part III A3b.)* The policy gives one of two bands: run, or wait for a confirm. A tool's class (read, write, run) describes the tool; its posture comes from its name, not its class. Confirm goes to the requesting principal as a component on the message that would perform the action; for owner-authority executions it goes to the owner. On timeout nothing happens and there is no other fallback.

Information flow. Cross-channel and cross-namespace recall is two decisions, not one: a **read** decision (may this execution's principal see nodes from that namespace or channel?) and a **disclosure** decision (may the result be shown in this channel to these participants?). Identity continuity for a `Person` namespace is not permission to disclose that person's data in a different guild or to other people. Both decisions are policy, evaluated deterministically, with Jev able to tighten but not loosen.

Enforcement is at compile time, not at output time. Once private material is in the model's context there is no reliable deterministic test of whether generated prose reveals it. Theseus therefore enforces disclosure **before generation** with **audience-safe context compilation**: the compiler (§4.4) receives the destination audience (channel, participants, external target) and admits only nodes whose confidentiality labels permit disclosure to that audience. Nodes carry a **confidentiality label** derived from their namespace and origin; generated nodes (`Message`, `Summary`, `Synthesis`, tool arguments sent outward, MCP responses and sampling requests) **inherit the most restrictive label of their inputs**, so an agent-authored summary of private or untrusted material is not public or trusted merely because its `origin` is `agent`. Declassification is an explicit, ledgered action by an authorized principal. The same labels drive the learning channel, web UI views, and logs, and they are what redaction lineage (§5.6) walks.

Provenance vs trust. `origin` on a node (operator, agent, tool, external, MCP server) is immutable provenance. `trust` is an inferred, mutable projection Jev may adjust. Jev never relabels origin.

Secrets. From AWS Secrets Manager or SSM Parameter Store when AWS is configured; from a local encrypted credential file with a passphrase or OS keyring in desktop mode. Never in argv; injected as environment at spawn; stdout scrubbed for secret shapes before it reaches the model or Discord. All actions land on the bus and in the ledger.

Default-safe is the operator's job, and the spec is honest about where the boundary really is: once an execution holds L0 with the operator's SSH agent and instance role, the effective security boundary is everything reachable with those credentials, not the harness's per-action tags. L0 is therefore `privileged`, opt-in per binding, and its use is loudly visible in the web UI.

3.10 Learning ledger and the learning channel

Feedback-driven optimization of our questions and parameters, not of the models, and not reinforcement learning in the technical sense: replaying a candidate against the incumbent's recorded outcome shows decision agreement, not what would have happened had the candidate acted. Every judgment that drove an action gets an outcome label from the human (reaction or slash command: wrong stop, should have stopped, wrong role, memory was useful or wrong), the system (a `complete` judgment followed by the same task reopening; a re-ask; a rehydration miss), or an offline audit by a stronger model over a sample. A nightly job computes per-question precision and calibration, proposes wording and category changes as versioned packs, and evaluates them on **frozen, time-separated holdouts** with minimum sample sizes. Decision-quality evaluation (does the candidate agree with labels?) is kept separate from trajectory evaluation (did acting on it go well?), which only canaries can answer: a promoted pack runs on a small canary share with automatic rollback on safety regression. Changes touching `security.v1`, the privileged shell mapping, or authority decisions require human approval. The same machinery tunes `MemoryScience` parameters and the shell mapping table.

The **learning channel** is a Discord channel per guild (bound `listen_only` for memory, write-enabled for the ledger) where scored classifications, proposed new roles or categories, and pack promotions are posted for the operator to accept, reject, or nudge with a reaction. The web UI shows the same stream with richer controls.

3.11 Voice

Voice is a Discord voice channel via `songbird`, receive and transmit. STT and TTS are `Speech` plugins; Deepgram and Cartesia first, since ariadne carries the integration knowledge. Push-to-talk versus voice activity detection is a human Discord preference, not an agent concern. The agent joins a voice channel only when invited. Core behaviours lifted from ariadne: barge-in cancels TTS mid-sentence, proactive speech when background work reports back, deferred reports queued for the next pause, short verbal acknowledgements when a turn will take more than a couple of seconds. A voice channel is a channel; its conversation is shared with the paired text channel under the same binding; its transcript is text in the same graph. TTS voice is a persona attribute that roles may modulate.

3.12 Plugins

Two kinds. **Compiled-in feature crates** behind Cargo features: Discord, AWS, Anthropic, Jev, Deepgram, Cartesia, the native `MemoryScience`. One static binary; adding one is a rebuild and a pull request. **Runtime extensions** are MCP servers running in an L1 sandbox as children of the daemon's process tree (§3.8, §7): any language, isolated at the OS level, started in about a hundred milliseconds, hot-loaded and revocable without a restart. Contract surfaces: `Transport`, `Tool`, `Judge`,

Provider, MemoryScience, Speech, Shell, Store (Hook left the list on 2026-09-28, §3.17). Rejected: dynamic shared libraries (no isolation, break the static binary), deployment sidecars and Docker (tenders and sandboxed MCP servers are children of the same binary, not sidecars), embedded scripting engines (one language, weak isolation). **WASM was in earlier drafts and is not a commitment** (2026-09-26): it would buy microsecond starts and per-function capability grants at the cost of roughly doubling the dependency tree; it returns only as a measured experiment if per-tool process cost is shown to matter.

One tool contract, three backends (decided 2026-09-26, Appendix E). Every tool the model can call presents through one `Tool` contract: name, JSON schema, description, retry class (§3.16), the authority and capabilities it needs, and `invoke`. Two backends implement it and the model never learns which: compiled-in Rust, and MCP servers (out of process, in an L1 sandbox, §3.8). The gate, the ledger, the trace, and telemetry therefore see one shape and are written once. Vocabulary, so the layers do not blur: a **plugin** is a compiled-in unit that registers tools, providers, or channel adapters (*hooks left both lists on 2026-09-28, §3.17*); **MCP** is the transport for tools that live out of process, and the only runtime extension path. **Channels are adapters, never tools**: Discord is where authority comes from (who spoke, where, with which roles), and that context is trusted kernel data. Routed through a tool result it would become untrusted content under §3.9. An MCP server may expose Discord *actions* to the model (post, react); the inbound path and identity stay in the kernel. Likewise the kernel itself (executions, completions, the WAL, the turn lock), memory recall (compiler-selected, never model-invoked; an explicit lookup tool may exist alongside), and thinking (native to the model) are not tools.

3.13 Budgets

Budgets are a first-class notion: a `Budget` is a named ceiling with a unit (money, tokens, judgment calls, wall clock, tool invocations) and a window, attachable to any part of the system a binding, role, person, guild, conversation, task, MCP server, or shell class. The budgeter **reserves** against every budget in scope before a turn or tool call and settles actual consumption afterwards, atomically per execution, so concurrent executions cannot jointly overrun a shared ceiling. Admission control is global: new executions queue with bounded depth when node-wide reservations (model concurrency, Jev calls in flight, sandbox slots, arena headroom) are exhausted, and the channel is told it is queued. Overload sheds proactive and scheduled work first, human requests last. Budgets distinguish **enforceable limits** from **estimated exposure**. Reservations prevent concurrent executions from jointly admitting more work than permitted; they cannot by themselves guarantee a strict dollar ceiling when usage is unknown after a provider timeout, an external resource keeps billing while the harness is down, or cancellation is delayed or unsupported. Therefore: unknown consumption is treated conservatively (the reservation is held until reconciled, never released on timeout); Jev calls, memory and consolidation work, STT/TTS, retries, and compaction are accounted, not free (*hook handlers left this list with the hooks, 2026-09-28*); backend-enforced runtime limits (Lambda timeouts, ECS task limits, cgroup limits, wrapper deadlines) are set from the budget where available; a **reserved control and cleanup budget** exists so that reaching a ceiling never prevents cancelling work, recording outcomes, or telling a human (*retired 2026-09-29, theseus-Osg: nothing at the limit ends work or refuses a cancel, so nothing needs holding back; see "As built" below*); and elapsed lifetime, active

compute time, and monetary spend are separate units. Disk-full handling likewise reserves capacity for control records and completion metadata so a running job can still write its result. Visibility exists from the first version: every turn result carries tokens in, out, cache read and cache write, first-token and total latency, and the provider request id; every session record accumulates its tokens; health reports totals and provider-error counts; the ledger is readable over the protocol (`ledger.tail`) and the CLI. Budget defaults will be ascertained as the system runs and recorded here as they are learned; until then, the only default is that a ceiling stops new work and says so. (*Superseded 2026-09-29: the default is a \$100 limit per session, and reaching it asks the operator to reset the spend instead of stopping; see "As built" below.*) The owner sets budgets; operators may tighten them within their scope.

As built: dollars, a limit, and a reset (Eddie, 2026-09-29, 00:09; theseus-0sg, cb824c7, fdf4813, d6183d1). In Eddie's words: "Right -- dollars is good. Let's record all model costs in the config. Let's set the spend limit in the config to 100\$. Let's make it so when you hit the limit, the gateway asks the trusted operator whether the current cost can be reset to 0 to continue". The general design above stands: reservations before each call, and unknown consumption held until reconciled. What is built:

- **Money, in micro-dollars.** An execution's budget is its limit, its spend, its reservations, and its held-unknown amounts, all in micro-dollars priced from the catalog. The arithmetic is integer, with one rounding up per call. Token counts stay in usage, the ledger, and telemetry.
- **The limit** is `[kernel] spend_limit_usd`, \$100 by default, per session. An open session follows it when it changes (below).
- **The reservation.** A provider call reserves its output cap at the output price plus its input estimate at the input price. It settles at the real cost: input, output, cache reads, and cache writes, each at its own price.
- **At the limit, Theseus asks.** A reservation that does not fit writes nothing. The turn asks the operator, "This session has spent \$X of its \$100 limit. Reset its spend to \$0 and continue?", and the session waits, the way a call waits for approval. The question goes to the session's Discord place (only its listed users can answer), the web UI, and `theseus confirm`. When trusted-channel approval lands (\$3.9, Approval), the question follows its rules like any approval.
 - An approved reset sets the spend to \$0, and the waiting call goes ahead. Reservations and held amounts stay. The reset is ledgered as `budget.reset`, with who approved it, the spend before, and the limit.
 - The session's lifetime cost keeps counting (`cost_usd`, and the total in health), so a reset never hides money already spent.
 - A decline keeps the session waiting, and new input asks again. The operator can also cancel the session or start a new one.
 - A reset is the only way spend goes down, and an execution has one open question at a time.
- **No control reserve.** Nothing at the limit ends work or refuses a cancel, so nothing needs holding back for cleanup.
- **Unit budgets are retired.** `[kernel] default_budget` and `control_reserve` still load, are ignored, and warn. A versioned reader serves executions stored with unit budgets: each gets the configured limit, its spend comes from its session's recorded `cost_usd`, and the unit figures are

kept as `units_before`. Startup rewrites each one once, with a `budget.migrated` row. An execution that already ended `budget_exhausted` stays ended.

- **An open session follows the config's limit** (Eddie, 2026-09-29, 09:21; theseus-3pj, 430d291). A session's limit is what `[kernel] spend_limit_usd` says now, not what it said when the session opened. Since a changed config note restarts the daemon onto it (\$3.19), "I changed the limit and restarted" means what it says, for the long-lived Discord place too.
 - *When*. Once, at the moment the config becomes the vault's word. For a config that may act at once (a file, or a vault read before serving), that is in startup. For a start served from the copy, it is on the vault's confirmation, before anything may act. So nothing acts on an old limit once the vault has confirmed a new one, and startup under a copy writes no limit the copy decided.
 - *What*. Each open execution whose limit is the config's takes the new limit, all in one frame, with a `budget.limit_changed` row (from, to, the spend, and what is left). Spend, reservations, held amounts, and resets are untouched. So spend still goes down only through an approved reset, and the lifetime cost never goes down.
 - *A raise* lets a session waiting at its old limit go on. Its question is withdrawn, with the reason ("withdrawn: the spend limit was raised from \$A to \$B"). It becomes the result the next turn consumes, as an approved reset's does, and the driver makes the waiting call. If the raise is still not enough, that call asks again, with the new figures. A session whose question was declined goes on too.
 - *A lower limit* changes nothing else. The next reservation that does not fit is refused, and the turn asks, as usual.
 - *What keeps its limit*: an ended execution, and an execution opened with a limit of its own (`pinned`). DD7's carved task budgets will be the first of those.
- **A model with no price is not called**. A model priced neither in the config nor in the built-in table is refused, with the class `unpriced` (Part III A4).

Prices live in the config (cb824c7). The template lists every built-in model as a `[catalog."<id>"]` table with its four prices per million tokens (input, output, cache read, cache write), one table per model. The text is generated from the built-in table, and a test fails if the template and the built-in table ever disagree. A table over a built-in model names only what it changes, and a model the built-in table lacks must name every figure a call uses. At startup, one warning names every built-in model that has no table in the config; such a model is priced from the built-in table.

3.14 Web UI

Embedded in the binary, served on the node, authenticated by Discord OAuth against the operator role table. **First form (M0.5)**: a Vite + React app embedded in `theseusd` and served on `127.0.0.1:7433`, loopback only, no auth yet; the browser is a protocol client over a WebSocket where each text frame is one JSON-RPC line, so it has no privileged path into the kernel. It shows the prompt, the streamed reply, tokens in and out per exchange and per session, totals, timing, the classified error when a turn fails, and the notification stream behind each turn, where thinking and tool calls will render later. Purpose: immediate and local observability. Conversation snooping (live view of any conversation's transcript, assembled context manifest, and loop state), the ledger stream with RL feedback controls,

category and role management with scoring nudges, binding and policy editing with audit trail, tender health, arena occupancy, and budget burn. Historical search is CloudWatch: the ledger, bus events, and structured logs ship there through the durability tender when AWS is configured.

The Narrative (Eddie, 2026-09-28, 20:38; theseus-5fy, e3ba8d6, 810aa6d, 12a4805). In Eddie's words: "I want every architectural part of the session/turn/loop/model call structure to have a narrative output that goes straight into an output channel that shows up in the web interface in a new pane called 'the narrative.' If `narrative: true` is in the config, the pane exists as a new tab in the web UI and the narrative output populates it. If false, the outputs never happen, and the pane never displays." The narrative is the fourth window onto a turn, beside the trace (§3.3a), the ledger (§3.10), and telemetry (§3.20). It is written for a person watching live, not for a tool, and unlike the other three it is never stored.

- **What it says.** One plain sentence per step, as it happens: a session opened, woken, parked, or cancelled; a turn started, ended, or failed, with what its loops spent; each loop and the Advancer's decision; the context appended to or recompiled; each model call's reservation, answer, refusal, or failure; each tool call's posture and why, its approval, job, result, and late result; the driver resuming an execution.
- **How it is written.** Every sentence is a fixed template filled from values the structure already holds. No model writes a word, so it costs no tokens. A sentence names a tool and its main resource (a path, or the program and its subcommand), and gives counts, sizes, times, and costs. It never carries tool input or output, secrets, or message text beyond a character count.
- **The channel.** `narrative.watch` answers with the recent tail (the last 500 lines, held in memory) and then streams `narrative.line`, so a tab opened late still shows recent history. Nothing is written to the store. Unlike the other notifications, `narrative.line` is not a ledger row.
- **The pane.** With `narrative = true` at the top of the config, the side pane has two tabs, the Observatory and **The Narrative**. Each line shows its time, its part, its session (a link, by short id, that opens it), and its sentence. The pane follows the newest line unless the operator scrolls up, and it can filter to the open session.
- **Off means off.** Each step tests one flag before it formats anything, so when the key is absent or false nothing is formatted, sent, or kept. `health` says `narrative: false`, the tab does not exist, and `narrative.watch` answers DISABLED.
- **Cost.** On, it adds no frame: a plain turn still writes 17. A line costs about 1.4 μ s with a watcher, so a plain turn of about 117 ms spends about 13 μ s on it. Lines are serializable, so storing them later is a small step.

3.15 Executions

A **conversation** is a derived view (§3.2). An **execution** is the durable object that represents one piece of work being carried out. It is what the loop runs, what budgets are reserved against, what can be cancelled, and what survives a crash.

```

Execution {
  id, created_at, state: queued | running | waiting | blocked | cancelled | failed | budget
  session: { kind: conversation | task, root: channel id | task id },          // §3.2a
  channel: current channel it reports into (may change on switch), origin_channel, reports_
  authority: { principal, delegation?, binding, ceilings },                  // §3.9
  task_scope: [task ids accepted for this execution],
  role: current role, role_history,
  config_versions: { packs, model, memory params, shell mapping, binding revision },
  wake: { due_at? | external_op_id? | blocked_on_execution? | on_jev_recovery? },
  budget_reservations: [...],
  outstanding: [ confirmations, tool calls with idempotency keys, external ops ], // §3.1
  attempt: retry/recovery counters,
}

```

Rules: one execution per session, one turn at a time per execution; an execution reports into one channel at a time; a channel orders deliveries, not turns, so a conversation execution and any number of task executions reporting into the same channel run concurrently; the admission scheduler bounds how many hold a turn; a channel switch is an execution moving, not a new execution. An execution is `waiting` when it has no runnable model work but a wake condition exists (a due time, a running shell or external operation, a blocking execution, a pending confirmation, Jev recovery, and since 2026-09-29 a budget question, §3.13). `waiting` executions are tended by the harness loop at near-zero cost and never consume model or Jev calls until their wake fires. Deterministic control paths, `/stop`, `/cancel <execution>`, permission revocation, budget exhaustion, act on executions directly and never route through Jev. Executions are nodes in the graph (`Execution` kind) with `in_channel`, by (principal), `evidence_for` (tasks) edges, so the context assembler can show the model what it is currently executing and why.

One writer at a time.

- Every kernel transition that reads an execution, or one of its actions, and writes it back holds that execution's lock from the read until its frame is indexed: the append, its fsync, and the index update.
- Actions belong to their execution, and its lock covers them.
- Writers of different executions never wait for each other.
- A reader that writes nothing takes no lock, so its view can be one frame stale. A decision read from such a view (the reconciler's scan, startup's) is read again under the lock before anything is written.
- A transition that touches two executions, such as a child's spend counted against its parent's, takes both locks in id order.
- So a cancel is never lost to a commit read before it, and no commit is lost to a cancel.

Pending wakes (theseus-cff; built 2026-09-30). A conversation asks for a turn at a time with `wake.at { at | after, note }`. The wake goes on a list on the execution (`wakes`), beside its one `wake`, which keeps saying what the last turn parked on. One field cannot say both: a conversation waits on its next input and on its due times at once, and one that waits on its own job must still be woken ("check the build in ten minutes" while the build runs).

- **Firing.** A wake fires once it is due and its execution is free: waiting on input, a due time, a job, or another execution, where new input would start a turn too. It never fires over an approval or a budget question, since only the operator answers those. The due scan runs on the driver's half-second tick, and in the heartbeat's reconciler as a backstop, and queues the execution for the driver (`resume_pending`). A busy execution keeps its wakes, and the frame that ends its turn queues it again.
- **Once.** The next turn takes every due wake in one frame, which removes them and writes each as a user-role node from the harness (`wake (set 13:05): <note>`), before any new input. A crash before that frame leaves them pending, and a crash after it leaves them taken. A wake's id comes from the call that set it, so a call run again after a restart finds its wake.
- **Late.** Startup writes nothing for a due wake: the driver's first tick queues it, after serving and after the vault confirms the config. A wake that runs more than 5 s late says when it was due and how late, and, when it fell due before this process started, that the daemon was not running. The ledger's `wake.fired` row always carries `late_ms` and `while_down`.
- **Bounds.** At most 5 per session, 1 s to 30 days ahead, and notes up to 2,000 characters. A task cannot set one. A cancel of the execution, or its end, drops its wakes; a stop keeps them (W1). A wake's turn is an ordinary turn, with the session's authority, budget, and model. Its reply goes where the session posts, or, if the place has moved on to a new session, to where the wake was set.

Stopping (theseus-lji; built 2026-09-30). `/stop` halts a conversation's work and keeps the conversation (`execution.stop`, `Kernel::stop_execution`). It is not a cancel, which is terminal.

- **What stops.** Running jobs and tool calls are told to stop, and their backends terminated, as for a cancel. Planned calls, approvals, and the budget question are declined, so nothing the work asked for runs later. A turn running then gets a stop mark: its next step is refused, it runs none of its answer's calls, and it posts no reply and no failure notice. Its model call, if one is in flight, runs to its end, and its cost is booked.
- **What stays.** The execution waits on its next input, and the next message continues the same session and execution. Its history, budget and spend, pending wakes, and tasks are kept, and `/cancel <id>` stops a task or cancels a wake. A stopped job's result reaches the next turn as a late result, `[cancelled: stopped by <who>]`.
- **Once.** A stop is one frame. After a crash, startup does not resume a turn that a stop had stopped. A task cannot be stopped; its cancel ends it.
- `/new` alone starts a fresh session.

(theseus-id9; Part III A3c.)

3.16 External actions and completions

Local durability cannot make an external side effect atomic with the log, and holding a pending result in harness memory makes it die with the harness. Both problems have one answer: every tool call that leaves the process is a durable record whose completion arrives as an event from outside the harness's own stack.

Lifecycle, written to the WAL at each transition:

planned → authorized → dispatched → succeeded | failed | outcome_unknown

The `planned` record mints the **correlation id** and is committed before anything is dispatched; the `dispatched` record is committed before the call is made (transactional outbox). A crash between dispatch and result therefore leaves an open record for the reconciler, never a silent gap.

The Completion envelope is one type across all sources: `{correlation_id, outcome: succeeded|failed|unknown, result_ref (payload on SSD or S3, never inline beyond a cap), external_op_id?, started_at, finished_at, producer, signature}`. Handling is **idempotent**: a completion whose record is already settled is a logged no-op; a completion with no matching record is quarantined and surfaced, never inferred into a channel.

Transports, chosen per source, cheapest that preserves durability:

Source	Transport	Why
Native in-process tools that finish fast (reads, task and memory actions, most AWS reads)	synchronous return inside the tool loop	no ceremony for sub-second work; still a <code>planned / settled</code> pair in the WAL when the tool has side effects
On-node jobs (L0/L1 shells, PTY sessions, local tenders)	the job wrapper writes its result to the completion spool on the SSD, then signals over a Unix domain socket	survives harness restart; no port; no auth beyond filesystem permissions
AWS-side work (Lambda, ECS/Fargate, SSM, scheduled jobs, MCP servers running in AWS)	SQS long-poll fed by EventBridge and by the wrapper inside the job	pull, so "no inbound" holds; at-least-once with dedupe by correlation id
Sources that can do nothing but POST	loopback-only HTTP receiver, per-job HMAC, size-capped, off by default	the documented exception, never the default

Native in-process calls of one response that only read run concurrently (§4.6). Each is still a `planned / settled` pair in the WAL, dispatched before it runs. (*theseus-a60*.)

The job wrapper is part of every shell class's contract (§7): it is **detached** (its lifetime does not depend on the harness; on the node it runs as its own systemd scope or L1 process tree), **durable** (result spooled to disk before any delivery attempt), and **cancellable** (the harness terminates it by correlation id through the execution's cancel path: kill the scope, stop the task, cancel the command). It is deliberately not "unkillable"; a runaway job must remain stoppable.

Who reaps a job (*theseus-z4b*; built 2026-09-29).

- The daemon starts each job's wrapper and is its parent, so it reaps it. Each wrapper is registered with its job as it is spawned, and a sweep reaps it by its own pid once it has exited. The sweep is woken by SIGCHLD, and runs every 10 s besides.

- The daemon's other children are the `op` processes, which `tokio` waits for itself, each by its pid. They are registered as `tokio`'s, and the sweep never reaps one. Nothing in the daemon waits for "any child" (`waitpid(-1)`), which would take `tokio`'s.
- The socket daemon is a child subreaper. A job can kill its own wrapper, and the job's processes are then reparented to the daemon rather than to `init`. The sweep reaps them once they exit. `--stdio` is a child subreaper too (`theseus-6uo`), so its jobs' orphans stay under it, where the answer rule finds them.
- **A job that kills its own wrapper is a security event** (`theseus-6uo`; built 2026-09-30). The sweep may take a wrapper that a signal ended while its action is still `dispatched`, with no completion in the spool and no cancel state. Then:
 - the action is marked `outcome_unknown` at once, instead of at its deadline;
 - `job.wrapper_lost` is ledgered with the correlation id, the pid, and the signal;
 - the narrative says so.

A cancel's own kill carries its cancel state, and is expected.

- Once reaped, a wrapper's pid can belong to another process. So a wrapper is alive only while its pid's command line names its job, and a cancel never signals a pid that has become another process.
- An exec restart keeps the pid and every child. The new image sets the subreaper flag again and learns its children: a live child whose command line is a wrapper's is that job's wrapper, and any other is an orphan.
- Health's `children` counts the wrappers running and lingering, the orphans adopted, and the zombies, which are 0 in steady state.

Deadlines and reconciliation. Every record carries a deadline from the tool's class and the execution's budget. The heartbeat reconciler (§3.3) checks open records against the spool, the queue, job-scope state, and, for AWS classes past their deadline, the service API. Reconciliation is event-first (EventBridge task state changes flow into the same queue) and polls only overdue records, so its cost scales with stuck work, not with total work.

Settlement is atomic with continuation. Accepting a completion writes, in one WAL transaction, the action's settled state **and** the owning execution's next durable state (runnable with the result queued, or waiting on something else). An in-memory mailbox notification is never the only link between a settled action and a waiting execution; if the process dies after the transaction, replay reconstructs the pending continuation. A duplicate completion is a no-op only because the transaction already happened, never because the first one was "handled" in memory. The frame is written under the execution's lock (§3.15), so nothing else changes the execution between the settlement's read and its frame.

`outcome_unknown` **is knowledge, not a terminal fact**. A record marked unknown stays **resolvable**: later authoritative evidence (a late completion, a reconciler finding) settles it to succeeded or failed and is ledgered as a resolution. Resolution never revives a cancelled execution and never authorizes new work by itself; it updates the record and, if the execution is still waiting on it, delivers the result.

Recovery never blindly retries a non-idempotent action. Correlation ids are not idempotency keys: an MCP or JSON-RPC request id correlates a request with its response and guarantees nothing about repeated effects; Discord nonces and AWS client tokens have their own validity windows. Every tool adapter therefore declares a **retry class** for each operation: `safe_to_repeat`, `idempotent_with_key` (naming the downstream key), `recoverable_by_external_id`, or `non_repeatable` (requires a human to resolve uncertainty). Generic SDK retry behaviour is disabled or overridden so it cannot silently contradict the class. Where the outcome cannot be established, `outcome_unknown` goes to the principal with the exact action, and the model gets it as a tool result so it can reason about it.

The outbox (theseus-q4v; built 2026-09-30). A post is a kernel action of its own record kind (`OUTBOX`), so no execution waits on it. It is never outstanding, queues no result, and wakes nothing, and neither a cancel nor the end of its execution touches it.

- The core stages it planned and authorized in one record, since nothing gates what the core itself says. It rides in the frame of the transition that made it true: a reply in the frame that ends its turn, and a card in the frame that plans its question.
- The binding dispatches it before its first call, and settles it with the channel's answer, its message ids, as its completion. A settle is idempotent.
- A post that creates messages is `idempotent_with_key`: each create carries Discord's nonce, derived from the message's key, with `enforce_nonce`. A retry after a crash between the send and the settle returns the first message, and a first message that holds an earlier send's content is edited to the post's state. An edit is `safe_to_repeat`, and targets the recorded id.
- Past the nonce's window (120 s, ours; Discord says only "a few minutes"), a create that may have landed goes again with a line that says it may be a copy. So the worst case is a visible duplicate, never a lost reply. A refusal that no retry changes (a 4xx other than 429) settles the post as failed, and health counts it.
- A card whose question closed is settled by a post of its own: at the close, when the core closed it, and otherwise by a level-triggered pass on the binding's connect and on every heartbeat.

Cancellation is a lifecycle, not a flag. `cancel_requested` → `cancel_acknowledged` → `termination_verified`, or `cancel_unsupported` / `cancel_outcome_uncertain` where the backend offers no external termination (a running Lambda invocation, for example). Executions report which state they reached. Every job wrapper carries its **own deadline** enforced locally, so a harness outage never removes the only limit on a job's lifetime.

Provider and judge requests follow the same contract. An interrupted Messages API call may still be billed and its usage unknown; the record is settled as `outcome_unknown` for cost purposes and its reservation is held, not released, until reconciled. A partially streamed tool call is never dispatched: dispatch requires a complete, validated tool-use block.

Confirmations are bound to tool, arguments, resource, policy context, and expiry (§3.9); a confirm answered after a crash is re-validated against the record before the resumed call runs.

The simulator (§8) crashes at every transition, drops and duplicates completions, and restarts the harness mid-job as standing scenarios.

References, not payloads, between tools. Every tool result is a node with an id (§4.1). A tool that produces something large returns the node reference; the compiler decides how much of its text enters the model's context; another tool accepts the reference as input and reads the node directly. Raw bytes never round-trip through the prompt to get from one tool to the next, provenance and confidentiality labels ride with the node, and the ledger records the hand-off. Node ids are the pointers; no second URI scheme.

3.17 Hooks

There is no hook system (Eddie, 2026-09-28, 20:38; deleted in 11d2f43, theseus-hco). In Eddie's words: "I used to believe in the plugins, now I believe in one, tight, focused, monolithic single-function server and a similarly tight client. Let's remove the hooks." What was built had 25 events and four kinds, and no handler that could do anything: its `Blocked` and `Claimed` outcomes were never built, and 10 of the 25 events were never dispatched. Every turn still paid for it: 10 of a plain turn's 27 WAL frames were `hook.site` rows, each recording that a hook with no handlers proceeded.

How Theseus is extended instead.

- **Through the protocol.** A client watches the sessions (`session.watch`), the ledger (`ledger.tail`), and the narrative (`narrative.watch`, §3.14). It acts through the requests every client uses, such as `turn.submit`, `action.confirm`, and `execution.cancel` (§3.18).
- **Through tools.** A new capability is a toollet (§3.23), or later a tool on an MCP server (§3.8, M7). Either one goes through the gate like every tool (§3.9).
- **One small hook point, deferred** (theseus-bdn). Before a tool call, a handler would see the tool's name and its plan, and answer proceed or ask. It could never deny, matching the gate (§3.9), and a handler's error would mean proceed, with a notice. It is built only when a real handler exists, and none does.

History. From v0.7 to v0.42 this section specified a hook system after the Claude Agent SDK's: an event catalogue by cadence, typed results with `deny > defer > ask > allow` precedence, `defer` for answers given out of band, and handlers registered in code, over the protocol, and by MCP servers. v0.42 of this document keeps that text, and `notes/theseus-hooks-design.md` holds the full design. Part III records what was built and why it went (A3's hook-site row; A3b, the complexity cuts).

3.18 Wire protocol and client isolation

The core is a **server**. Nothing else in the system, not the CLI, not Discord, not the web UI, not the simulator, reaches into it except through one protocol. That is the isolation boundary Eddie asked for, and it is what keeps the kernel testable without a front end.

Protocol. JSON-RPC 2.0, newline-delimited JSON, one message per line, UTF-8. Requests, responses, and server-initiated notifications; requests are correlated by id, notifications carry the session id. Chosen over gRPC because it is what MCP, LSP, and ACP already speak, so every tool in the ecosystem can debug it with a terminal, and over a bespoke binary framing because message volume is token-bounded and the serialization cost is noise next to a provider call. All message types live in one dependency-free crate, `theseus-protocol` (serde types only), with a generated JSON Schema for non-Rust clients; if a binary encoding is ever needed, `MessagePack` over the same types is a framing change, not a protocol change.

Transports, same protocol on each.

1. **stdio**, when a client spawns the core as a child. This is the developer and test mode, and the way an editor or another agent harness drives Theseus (it is the shape of the Agent Client Protocol, and Theseus should be able to present as an ACP agent with a thin adapter).
2. **Unix domain socket**, the daemon mode and the real deployment: `theseus serve` listens on a socket in the state directory; many clients attach and detach while the core runs forever. Localhost only, by the settled reachability rule; file permissions are the authentication.
3. **In-process**, for adapters compiled into the binary (Discord, the web UI, tenders): the identical message types over a `tokio` channel. An in-binary adapter is still a client; it has no privileged path into the kernel.

Surface, first version. `session.open`, `session.list`, `turn.submit {session, input}`; notifications `turn.started`, `loop.started`, `model.delta` (streamed text), `tool.proposed`, `loop.ended`, `turn.ended {reason, output}`; `health`. (Wakes, `theseus-cff`: `wake.list { session_id?, target? }` reads, `wake.cancel { wake, author? }` acts and waits at the config gate, and `health's` `wakes` lists every pending wake. A cancel's author is the request's author, else the surface's name, the CLI or the web UI, for executions, tasks, and wakes. The CLI has `theseus wakes` and `theseus cancel <id>`, for a wake or a task. Discord has `/wakes` and `/cancel id:<id>`, for a task or a wake; DD7 named the option `task`.) It grows with the milestones (executions, tasks, ledger, confirmations, the narrative), but the shape is set: requests change state, notifications report it, and every notification is also a ledger row, except `narrative.line` (§3.14). (Amended 2026-09-29. `hooks.list` and `hooks.register` went with the hook system in 11d2f43 and now answer "method not found"; `narrative.watch` arrived in e3ba8d6.) (Amended 2026-09-29: `turn.submit` takes attachments, and its `input` may be empty when there are any; `theseus-9g2`. `confirm.list` (`theseus-0g4`) returns every question waiting for the operator, the most recently active session first, so `theseus confirm` with no id makes one request instead of one per waiting session. `policy.tighten` and `policy.untighten`, with the `policy.tightened` and `policy.untightened` notifications, arrived with "should have asked" (`theseus-sgh`).)

Two binaries, one protocol (revised in M0 at Eddie's request: a server binary paired with a CLI binary). `theseusd` is the server: the daemon on a Unix socket, or `--stdio` when a client spawns it, plus `check` and `example-config`; `tenders` and `restore` join it later. `theseus` is the CLI: `ask`, `health`, `sessions`, `rpc`, `shutdown` (`hooks list|watch` went with the hook system on 2026-09-28; `theseus`

watch follows a session), with `--json`, `--spawn`, stdin prompts, and shell exit codes (0 ok, 1 server or provider error, 2 usage, 3 cannot connect). The CLI links only `theseus-protocol`, never the core, so it cannot cheat. Both are static musl binaries.

3.19 Configuration and secrets

Opinionated, and simple. **Every secret lives in 1Password**, in the deployment's vault, and Theseus reads it at startup through a **service account**. The only secret the process may receive by any other path is the service-account token itself, from the environment or from a mode-0600 file whose path is configured.

- **Config** is a TOML document stored as a 1Password item (`theseus/config`) so the whole deployment is reconstructible from the vault. It may also be a local file for development; the schema is identical. Secret-valued fields are `op://vault/item/field` references, never values.
 - **The last-known-good copy** (`theseus-2fo`).
 - After every read of the vault's note that loads, the daemon keeps the note's exact text as `<state dir>/config.last-good.toml`: mode 0600, under a first line naming the reference it came from. It is written after serving, never on the start path.
 - The note holds only references, so the copy holds no secret. A note whose URLs could carry a credential (a user, a password, or a query in `api_base` or `otlp_endpoint`) is never copied.
 - The copy is found in `--state-dir`, else `~/.theseus`, before any config is read. The tool floor keeps it.
 - **A start serves from the copy, and acts only on the vault's word.**
 - A start whose config is `op://`, and that finds its copy, serves from it at once. It reads the vault behind the socket, beside the secrets.
 - Until the vault confirms the copy, the daemon answers only what reads. Every method that changes anything or starts work waits, bounded at 30 s like the secrets, then fails with `config_unconfirmed`.
 - The harness loop, the driver, telemetry, the web UI, Discord, and the GitHub check start only then.
 - Why: the copy is a file the operator's user can write, and at L0 a job runs as that user. The vault is the one thing an agent cannot write, so it stays the only authority.
 - The wait costs nothing in practice: the secrets come from the same vault at the same moment.
 - **The vault's answer.**
 - The same text, or one that differs only in comments or formatting, confirms. In the second case the copy is rewritten.
 - A different note that loads is ledgered as `config.changed`: the reference, both sha256 digests, and the tables that differ, never values. The copy is rewritten, and the daemon restarts onto the vault's version: the clean shutdown path, then an `exec` of its own image with its own arguments, so the pid, the terminal, and any supervisor stay the same.

- A process that began as such a restart and finds the note changed again holds, and says so, rather than restart again.
 - A note that does not load, or a vault that does not answer, holds too, with the reason in health and the ledger. The vault is read again at 5 s, doubling to a minute.
- **The rest.**
 - A first start, with no copy, reads the vault before serving. That is the one slow start.
 - `theseusd check`, `theseusd config` (which also says whether the copy matches), and `restore` read the vault directly.
 - A `--config` file has no copy, and needs no confirmation.
- **Resolution** starts at startup, in the background. `config.reload` was never built, and is not planned: **a restart is the reload** (§3.22 makes it routine). A changed config note is applied by the restart that the vault's answer triggers, or by the operator's own restart. (*Amended 2026-09-29, theseus-2fo: until then the note was read before serving, and "an explicit `config.reload`" was planned.*)
 - The daemon serves first (§2 FAST). It opens the store, runs the kernel's startup, and answers its socket while the secrets resolve.
 - One `op inject` fetches every reference: one process and one vault session. Measured against concurrent `op reads`, it is about as fast and costs a sixth of the CPU (theseus-qa0). If it fails, one `op read` per reference names each bad one.
 - A secret that fails is fetched again after 5 s, then at doubling intervals up to a minute.
 - Resolved values are held in memory in zeroizing containers. They are never written to disk, config, logs, the ledger, or a provider request, except where they belong (an `Authorization` header).
- **Each consumer waits for its own secret, and fails closed.**
 - A turn waits for its provider's key, and for the first round of every secret, because the scrubber must know each value before a tool result passes through it. The wait is bounded at 30 s. Then the turn runs, or it is refused with the class `secret_failed` or `secret_resolving`.
 - The Discord binding waits for its token, and never connects without it.
 - The GitHub token check and the telemetry exporter wait for theirs.
 - Health reports `secrets: resolving | ready | failed <names>`, with each failure's reason. The ledger records `secrets.resolved` and `secrets.failed`. (*Amended 2026-09-29, theseus-qa0: until then resolution ran before serving, and the process refused to start on a missing secret; Part III A3c.*)
- **The secret broker** (theseus-dcy; built 2026-09-30). It is the one place that hands a resolved value to anything beyond the daemon's own consumers, and it never calls `op`.
 - `[broker.programs.<program>] env = { VAR = "<secret>" }` gives a program a `[secrets]` value in an environment variable, for example `[broker.programs.gh] env = { GH_TOKEN = "github_token" }`.
 - **Direct argv only.** A grant applies when a job runs the program itself: `argv[0]`, resolved as `exec` resolves it, is the file that the program's name resolves to on the daemon's PATH. A shell, an interpreter, `env`, or `timeout` that runs the program gets nothing, since it

would hand the variable to every program it runs. The tool result says why, so the model learns to call the program directly.

- **At spawn**, the job's environment is `[tools].proc_env`, then the call's own `env`, then the grant. The values come from the board. A secret that has not settled is waited for, bounded as a turn waits. One that still has not, or that failed, is withheld, never replaced by a placeholder, and the result says so.
- **A native toollet** reads a granted secret through its context (`ToolCtx::secret`), bound to the call. The wiring grants each one (`grant_tool`): `web.search` its key (DD5). M4's run-time credential requests will come here too, at the requesting tool's posture, and a fetch from 1Password itself always waits (decision 15).
- A secret with no grant is never handed out, so the AWS keys stay behind the floor.
- **Never written**. A value goes to the job as its environment only. It is not in the wrapper's arguments, the job record, the WAL, a node, the ledger, or a log.
 - The ledger gets `secret.granted` (the program, the variable, the secret's name, and the call's correlation id), and `secret.withheld` (the same, and why).
 - Health and the Observatory list each grant with its uses.
 - Output is scrubbed as every tool's is. The raw output file in the spool keeps what a program prints, as for any job (`gh auth token` would put the token there); the node, the model, and every surface get the scrubbed text.
- At L0 a job runs as the operator's user. So the broker keeps a value out of every record and every other program's environment, but it is not a boundary against a hostile job of the same user, which can read another job's `/proc/<pid>/environ`, or a program's own stored login (`gh's hosts.yml`). L1 is that boundary (M4).
- **Mechanism**. The first version shells out to the `op CLI` (`op read op://...`) under the service-account token, because 1Password publishes no first-party Rust SDK; the community FFI wrappers around its C core exist and are the candidate for removing the `op` dependency later, once they are shown to build statically. The config note is read with one `op read`, beside the secrets' one `op inject`. The service account is read-only, so the config item is created by a human once; Theseus never writes to the vault.
- **Configuration is documented by its template, and the template is tested**. `theseusd example-config` prints a hand-written annotated TOML in which every parameter the code reads appears exactly once, set to its default or commented out with its default shown, with a line saying what it does. Three tests keep it honest: it parses and validates; a copy with every comment un-commented also parses under `deny_unknown_fields`, so no stale or not-yet-honored key can survive in it; and every key the loader can read appears in it, so no field can be added without documenting it. The consequence is a rule: config keys are not defined before code honors them; work not yet built is recorded in Part III, never as inert config. `theseusd config` prints the config actually loaded and its source, references only.
- **Token hygiene**. At startup Theseus checks the GitHub token against the API, logs its login, expiry, and days remaining, and warns when fewer than a configurable number of days remain (default 30). Never fatal.
- **Starting set** (vault `Eddie-Tabitha`, item names as they exist): `anthropic openclaw key`, `TypeSafe Jev key`, `zeroaltitude github PAT` (a fine-grained token with push on the owner's repositories, expiring 2027-02-18; chosen over the all-scopes classic token until Theseus is on

rails), `z.ai` key (line `api` key value), and `strata-jam-aws-key`, a `label: value` note whose `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` lines are referenced separately (verified 2026-09-25 against STS: IAM user `stratajam`, account 560512680793). Discord and any others are added as their milestones arrive. The same posture applies to them all: GitHub and AWS credentials are read from 1Password too, never from `~/.aws` or `~/.config/gh`, and a secret that cannot be resolved never lets its consumer run without it, and the daemon says which one failed and why; the process itself serves meanwhile. *(Fail closed per consumer since `theseus-qa0`, 2026-09-29; until then the process refused to start.)*

3.20 Telemetry

OpenTelemetry is a **projection of the record**, never a second instrumentation.

- The turn trace (§3.3a) is already a span tree, with absolute start and end times.
- When a turn ends, Theseus records its metrics in the process, and hands the finished tree to one sender task. The sender walks it into OTEL spans with those exact timestamps.
- So the hot path pays only the hand-over (tens of microseconds), and the exported picture is the ledger's.

The mapping:

Theseus	OpenTelemetry
turn	root span, with the trace root's attributes as recorded: <code>turn_id</code> , <code>session_id</code> , <code>profile</code> , <code>provider</code> , <code>model</code> , <code>continuation</code> , <code>outcome</code> , <code>loops</code> , <code>stop_reason</code> , <code>usage.*</code>
loop <i>n</i> , a group of calls run together (<code>tools</code>), tool <code><name></code> , <code>continuation</code>	child spans
provider.call	client span with the GenAI semantic conventions: <code>gen_ai.operation.name</code> , <code>gen_ai.system</code> , <code>gen_ai.provider.name</code> , <code>gen_ai.request.model</code> , <code>gen_ai.response.model</code> , <code>gen_ai.response.id</code> , <code>gen_ai.response.finish_reasons</code> , <code>gen_ai.usage.input_tokens</code> , <code>gen_ai.usage.output_tokens</code>
first_byte, first_token	events on the provider span
compile, advancer, store, lock, marks	events on their parent span <i>(since 2026-09-28 always: <code>hook</code> sites are gone, and <code>telemetry.hook_spans</code> no longer loads)</i>
provider.error, turn.failed	error status, with the message or the class; the class and transient as attributes
turns, tokens, provider errors, dollars, tool calls, durations	cumulative metrics: <code>theseus.turns</code> (<code>profile</code> , <code>provider</code> , <code>model</code> , <code>outcome</code>), <code>theseus.tokens</code> (the same, and direction), <code>theseus.provider.errors</code> (<code>provider</code> , <code>model</code> , <code>class</code> , <code>transient</code>), <code>theseus.cost.usd</code> , <code>theseus.tool.calls</code> (the turn's attributes, and tool); histograms <code>theseus.turn.duration_ms</code> , <code>theseus.provider.call.duration_ms</code> , <code>theseus.provider.first_token_ms</code>

Transport. OTLP/HTTP with the JSON encoding, posted to `<otlp_endpoint>/v1/traces` and `/v1/metrics` over the workspace's `request` and `rustls`, with no gRPC and no protobuf library.

- **The exporter is Theseus's own, and it is in every build.** It adds no crate to the daemon. (theseus-hee. Until then it was the `otel` cargo feature, off by default, over the OpenTelemetry SDK and 19 crates, `aws-lc` among them.)
- Headers (a Honeycomb key, a Datadog key) come from the vault like every other secret. Resource attributes carry `service.name`, `service.version`, and `service.instance.id`.
- **Nothing leaves the process until three things hold:** `[telemetry].otlp_endpoint` is set, the vault has confirmed the config (§3.19), and the headers secret has resolved.

A turn never waits for the network.

- The sender posts each turn's trace as it comes, from a queue of at most 64 whose oldest is dropped when it is full. It posts the metrics every `metrics_interval_secs`.
- A failure, a 429, or a 5xx is retried once after a second. Then the batch is dropped and counted.
- Health says what was sent and dropped, and the last error (`telemetry: exporting to ... · sent N · dropped M · last error ...`), or why nothing is sent yet.
- A stopping daemon flushes, bounded at a second.

Point it at a local Collector, Grafana Tempo, Honeycomb, Datadog, or the AWS Distro for OpenTelemetry. That is how "CloudWatch for historical search" (§1) is satisfied, with no CloudWatch-specific code. The trace, the ledger, and `turn.trace` are the record: the web UI and CLI read the trace directly, and OTel is for the fleet view. (*Amended 2026-09-29, theseus-hee: the transport was HTTP/protobuf through the SDK, a build feature since theseus-0g4; Part III A3c, Step O1.*)

3.21 Self-extension: planks, never the keel

Theseus may build its ship on the open ocean: add tools to itself while running. It may not touch the keel.

- **Planks** are tools. An `extend.propose` action lets the agent write a tool as an MCP server in any language, run it in an L1 sandbox, test it there, and produce a **manifest**: schema, capabilities requested, authority it needs, the tests it passed. Loading it is gated **deterministically** by an operator acknowledgement delivered as a Discord confirm (§3.9); Jev may advise, never approve. On ack the tool is hot-loaded with no restart, scoped to exactly the capabilities in its manifest, recorded as a versioned node with `derived_from` its proposal, ledgered, exported to telemetry, and revocable with one command. `extend.propose`, the ack, the load, and the revocation are each ledgered (*they were hook sites until the hook system was deleted, 2026-09-28*). A tool built by the agent is `trust: agent` and cannot request more authority than the execution that proposed it holds (§3.9 never widens).
- **Promotion to native** (`extend.promote`). A plank that has earned in-process speed or deep integration with the shell classes and the completion spool is ported to Rust inside the repository by the agent and opened as a **pull request**: implementation, tests, a Part III note. CI builds the static binary and runs everything; the operator reads the diff and merges; deployment is the routine graceful upgrade (§3.22), an operator action. Promotion can be one conversational

request that yields a PR link, but the merge is human by default: native tools run in-process with kernel trust, and a bug or an injected malicious tool at that trust level owns the WAL, the secrets, and the policy store. A pull request is the only review artifact that can be read, tested, and reverted; an ack button for a compiled binary would approve something the operator cannot inspect. The repository *may* be set to auto-merge on green CI, which makes promotion fully agentic; that is a deliberate operator choice, off by default, to be made with evidence. Which planks earn promotion, and whether promoted tools are ever demoted, are ledger questions for the learning loop.

- **The keel** is the kernel binary: the gate, the WAL, the policy engine, the store, the trace, the compiler. The agent may propose changes to it only as pull requests to the repository, which pass CI and a human review and are deployed by the operator. It never self-applies, compiles, or restarts its own binary; "compile and restart yourself" is the control-plane tampering §6 warns about, and an ack dialog for it would ask the operator to approve code they have not read. A pull request gives them a diff.
- **When to extend** is a judgment, not a comprehension problem. The model can write a tool; deciding that a new tool is warranted rather than composing existing ones is a Jev pack (`EXTEND_WARRANTED`) plus the operator gate, and it is ledgered so the learning loop can see which extensions earned their keep.

3.22 Lifecycle: restart is routine

A restart of `theseusd` is designed to be cheap, because nothing that matters lives only in process memory (§3.16, §4.4b). What a restart costs, exactly: any provider stream in flight at that instant, which becomes a classified `usage_unknown` failure with its trace, reservation held, turn resumable. Everything else is reconciled by the five-step startup (§3.3): finished jobs are picked up from the spool, running jobs are still running because their wrappers never depended on the harness, sessions and executions are records, the live profile is in the store, Discord resumes its gateway session.

Graceful upgrade is a first-class operation (`theseusd upgrade`, or a signal): stop admitting new turns; let in-flight provider calls finish within a bounded window or fail them cleanly; flush telemetry; checkpoint the store; exec the new binary and hand it the listening sockets so no client sees a refused connection. Target: no lost work, no lost client connections, sub-second gap in turn admission.

Upgrade under load is a standing simulator scenario beside crash-at-every-boundary: a hundred sessions mid-turn, upgrade, every one settles correctly. Deploying a new keel is an operator action made cheap enough to do without ceremony; the agent never triggers it (§3.21).

This is a deliberate contrast with the gateway Theseus replaces, where in-flight tool calls, subagent handles, and session state live in process memory and a restart loses them. The M0 binary is not yet restart-safe in this sense; M1 Keel and M2 Kernel are where the construction happens, and the simulator is what proves it.

3.23 Toollets: native first

A **toollet** is a small, typed, in-process Rust tool behind the tool contract (§3.12): a few arguments with a JSON schema, structured output that becomes a node with provenance, microsecond to millisecond latency, no shell parsing, no PATH or environment dependence, a unit test. Theseus ships many of them and prefers them to shelling out everywhere.

Why this is a principle and not a taste. A `bash` string is opaque: the gate cannot tell `rm -rf` from `ls`, quoting is a permanent tax, the output is text that must be parsed back, and the ledger records only that "a shell ran." A native `fs.edit { path, find, replace }` has arguments policy can reason about, output that is already a node, and a trace span with real attributes. Every principle in §2 is stronger on a typed surface.

Families, each a crate:

Family	Replaces	Built on
<code>fs.*</code> read, write, edit, glob, grep, stat, tree	cat, sed, find, rg	the <code>ignore</code> and <code>grep</code> crates <code>ripgrep</code> is built from
<code>git.*</code> status, diff, log, blame, commit, branch, worktree	the git CLI	gitoxide
<code>text.*</code> diff, patch, json, yaml, toml, regex, hash, count	jq, diff, sha256sum, wc	serde, similar
<code>http.*</code> fetch, post	curl	reqwest; as built 2026-09-30, <code>fetch</code> only, as an async tool (theseus-yd6)
<code>gh.*</code> issues, pull requests, checks, reviews	the gh CLI	the GitHub REST API
<code>aws.*</code> per service	the aws CLI	the official Rust SDK
<code>task.*</code> , <code>memory.*</code> , <code>extend.*</code>	—	the kernel
<code>proc.run</code> typed argv, no shell	bash	tokio process; the escape hatch

Rules.

- Native does not mean unbounded. A toollet that does I/O or exceeds the synchronous bound (§3.16) is an action with a completion record like anything else; durability is unchanged.
- Native does not mean unscoped. A toollet runs with kernel trust, so it lands through the pull-request path (§3.21) and declares the authority it needs; the gate checks typed arguments, not strings.
- Tool count is the toolchain manager's problem, not the model's: toollets are offered by family and by the turn's needs (roles, Jev), and searched, so a hundred of them do not bloat every prompt.
- **A toollet computes on one of the daemon's cores.**
 - One semaphore, with a permit per core (`available_parallelism`), is held by every in-process toollet while it runs on tokio's blocking pool.
 - So calls across every session never run more CPU work at once than there are cores, and never use a thread per call.

- A call that waits for a permit is waiting, not failing: its deadline starts with its run. (*theseus-a60*.)
- **A toollet whose work splits borrows free cores, and never waits for one.**
 - `fs.grep` searches the files of a big tree in chunks, on every core that is free at that moment, and merges them in walk order, so its output is the same with or without them.
 - Waiting is async and computing takes a core, so a pool full of greps cannot deadlock.
- Every tool call is a trace span and a metric with family, tool, backend, and outcome. The **shell-fallback ratio** (`proc.run` over all calls) is watched; the most frequent `proc.run` argv patterns are the promotion queue for the next toollet (§3.21 `extend.promote`).
- A new capability arrives as a toollet unless there is a written reason it cannot; Part III records where this slipped.

3.24 The tool surface

Reviewed against Claude Code (about twenty tools, six of which do nearly all the work), Codex (seven native tools plus MCP; its `apply_patch` and `write_stdin` are the two ideas worth taking), and OpenClaw (seventy-odd top-level tools in a typical session, several with thirty-verb action enums, two memory systems, and operator controls exposed as model tools). Full review with verdicts in `docs/notes/tool-surface-review.md`.

Principles of the trim. One tool, one verb: no action enums. Typed in, node out: every result is a node with provenance, and composition is by node reference (§3.16). The kernel is not a tool: sessions, executions, cancellation, budgets, policy, config, secrets, and operator controls are protocol requests or deterministic commands. Memory is compiled, not called (§5), with one explicit lookup and one explicit note. The shell is reachable only through a typed argv and is counted (§3.23). Everything else is a plank (§3.12).

The selected set, thirty-five tools in twelve families, offered by family per turn so a coding turn sees perhaps fifteen schemas:

Family	Tools
<code>fs</code>	<code>read</code> (text, image, PDF as typed nodes), <code>write</code> , <code>edit</code> (exact-string replace with occurrence control), <code>patch</code> (unified diff), <code>glob</code> , <code>grep</code> , <code>list</code> (stat and tree)
<code>proc</code>	<code>run</code> (one-shot typed argv: cwd, env allowlist, timeout, shell class), <code>session.open</code> / <code>session.send</code> / <code>session.close</code> (a persistent interactive process: REPL, debugger)
<code>text</code>	<code>diff</code> , <code>query</code> (a jq subset over JSON, YAML, TOML nodes)
<code>http</code>	<code>fetch</code>
<code>web</code>	<code>search</code>
<code>git</code>	<code>diff</code> , <code>log</code> , <code>commit</code> ; the rest is <code>proc.run git ...</code> until the fallback ratio says otherwise
<code>task</code>	<code>create</code> , <code>update</code> , <code>move</code> , <code>split</code> , <code>merge</code> , <code>close</code> , <code>claim</code> , <code>handoff</code> (§3.5)
<code>memory</code>	<code>recall</code> , <code>note</code>
<code>channel</code>	<code>post</code> , <code>react</code> , <code>ask</code> (confirm, choose, or free text, to the requester or the owner)

Family	Tools
node	<code>read</code> (any graph node by id, with a range: the reference-passing primitive)
wake	<code>at</code>
extend	<code>propose</code> , <code>promote</code> (§3.21)

(As built 2026-09-29: `fs.read` returns text, and an image as an image block for a model with vision (theseus-9g2). A PDF is still reported as a binary file.)

(As built 2026-09-30, theseus-yd6: `http.fetch { url, max_bytes? }` and `web.search { query, count? }` are async tools (Backend::Async), not toollets on a core. Each call is a future on the daemon's runtime, and holds no core while it waits. Only turning an HTML page of 16 KiB or more into text takes a core from the pool (§3.23). `http.fetch` is GET only. It follows up to 5 redirects, within a total timeout and a byte cap. It returns an HTML page as text, through a hand-written converter with no parser crate, and text, JSON, and XML as they are. Any other type is named with its size and not read, so a PDF is named, not read (decision 10). `web.search` is the Brave Search API, and its key is a broker grant (§3.19). Both are class `Read`, so the fetches and searches of one response run together. Each result node is marked external, with its URL (§5.2).)

Rejected, with the need's new home: free-form `bash` (→ `proc.run`); subagents, spawn, workflows (→ task sessions); todo and plan-mode tools (→ `task.*`); multi-action `message`, `browser`, `nodes` (→ `channel.*`; browser and device control are planks); fourteen memory tools (→ two); session, subagent, and automation tools (→ protocol requests, `wake.at`, `/cancel`); secrets, gateway, config, plugin tools (→ the operator's CLI and web UI, never the model); media generation, TTS, PDF, image viewing as tools (→ node types for input, planks for generation); skills as tools (→ roles and MCP prompts); notebook and worktree tools (→ `fs.edit`, `git.*`, snapshots).

The distinguishing claim is not fewer tools. It is that each tool is the whole of one idea, is typed enough for policy to read intent, and composes with every other through the graph.

4. The context graph

One master graph per deployment. Everything the agent could put in front of a model is a node; everything relating nodes is a typed edge. This is the session model, the transcript, and the memory system at once.

4.1 Nodes, edges, roots

- **Nodes:** `Message`, `ToolCall`, `ToolResult`, `Judgment`, `Task`, `Execution`, `Heartbeat`, `Person`, `Topic`, `Event`, `Resource`, `Capability`, `Role`, `Summary`, `Synthesis`, `Suppression`, `Redaction`, `Root`. Every node has an immutable **record** (`kind`, `origin`, payload reference, timestamps, authored edges) and a mutable **projection** (`retention` and FSRS state, `heat`, `trust`, task state, index membership) that is rebuildable from the record and the ledger. Append-only applies to the record; projections change freely.

- **Edges (typed, directed, timestamped):** `next`, `replies_to`, `in_channel`, `by`, `about`, `derived_from`, `summarizes`, `supersedes`, `contradicts`, `same_entity`, `mentions_conversation`, `in_role`, `evidence_for`, `judged_by`, `part_of`, `in_session`, `includes` (compilation → node, ranked), `reports_to`, `addressed_to`.
- **Multi-parent by design.** The graph is a DAG, not a tree. One `ToolResult` written by a task execution is `in_session` of that task, `in_channel` of the channel it was reported to, `includes-d` by any number of later compilations, and `summarizes-d` by a compaction. Each membership is its own edge; none is privileged. **Order is positional:** every record carries its monotonic WAL position, and any lineage (a channel's transcript, a session's tail, a compilation's contents) is a filter over membership edges sorted by position. `next` is a convenience edge for the per-channel fast path, never the definition of order.
- **Roots:** a node a lineage starts from. A channel's first node is its first root. A `Compilation` (§4.4a) is a root for the session that made it; a compaction's `Summary` is a compilation whose `summarizes` edges point at the range it stands in for. The range is not removed; it is simply outside the default view. The graph never loses a node.
- **A context is a path selection.** The guaranteed default: follow `next` back to the nearest root within the channel. That is a plain transcript, served as a sequential read of an append-only per-channel log, never a graph traversal, and it must work with no Jev and no index.
- **Classificatory kinds are ontology.** `Person`, `Topic`, `Event`, `Resource`, `Capability`, and `Role` are seed kinds of the ontology (§4.1a), and their instances are categories. A new kind is a row in the ontology's table, not new code. The machinery kinds (messages, tool calls, tasks, executions, compilations, summaries, judgments, suppressions, redactions) stay code.

4.1a Ontology: kinds, categories, memberships, guidance (Eddie, 2026-09-27)

*The ontology is fungible (§2). This section began as openrig's pods, which it calls "context domains" (Appendix F), and was generalized at Eddie's direction so that nothing about which kinds of context exist is hard-coded. The first new kind is the **topic**. Beads: theseus-8kk.*

What is code, and what is data. The code is small and fixed:

- the machinery kinds (§4.1);
- one generic `Category` node;
- one membership edge, `member_of`, carrying `origin` (`transport` | `operator` | `jev` | `sweep` | `dream`), `confidence`, and `as_of`;
- `Guidance` nodes attached to categories;
- a closed set of **composition rules** the compiler knows:
 - `chain`: walk up the parents, nearer overriding farther (for rules);
 - `intent_line`: one line per category (for intent);
 - `ranked`: admitted within the budget by relevance (for lessons);
 - `recall_only`: never admitted automatically.

The data is a versioned **kinds table**, like the role table (§3.4). Its seed rows are `channel`, `guild`, and `person` (given), and `topic`, `culture`, and `expertise` (interpreted). Each row declares:

- what may be a member, and how many categories of the kind a session may hold;
- its parent kind (topics nest);
- its precedence against other kinds, so that clashes resolve the same way every time;
- which origins may assign membership;
- its composition rule;
- a description, which is embedded.

The owner or an operator adds rows in the web UI. Jev or a dream may propose rows, which are accepted like new roles. Every row carries `added_by`.

Four verbs.

- *Added*, as above.
- *Trained*: a Jev `categorize.v1` judgment assigns memberships, in shadow first, and operator corrections are its signal.
- *Re-associated*: a membership is an append-only record. A sweep, a dream, or a live check writes a newer one that supersedes the old.
- *Embedded*: each category keeps its description and a centroid of its members in the index tender. They are used to propose memberships for new sessions, to find near-duplicate categories to merge, and to find drifted ones to split.

Two guardrails.

- *Given versus interpreted*. Channel, guild, and person memberships come from the transport. They are facts, and they are never re-associated. Topic, culture, and expertise memberships are interpretations, and they may be.
- *Interpretations route context but never grant access*. Which nodes a principal may read, and which audiences may see them, is decided by §3.9's labels and the bindings the operator declares. An interpreted membership never decides it. It is the same line §1 draws for roles.

The compiler.

- A compile looks up the session's current memberships, one read per kind, all precomputed. FAST forbids an embedding search or a Jev call on this path.
- It walks each category's parents and admits guidance by its kind's rule and precedence.
- The manifest records every membership used, with its origin and as-of, so "why did it know that?" always has an answer.
- An interpreted membership change takes effect at the session's next recompile, so the prompt cache survives it. A declared change that alters access forces a recompile (§4.4a).

Consequences (§3.9) were also a kind in this table, on the authority side, with memberships from deterministic detection only. *Superseded 2026-09-28: the consequence kinds were removed with the detection that assigned them (§3.9; Part III A3b).*

Phasing.

- **M4**: the kinds table, declared memberships, guidance, and the compile walk, beside the labels.

- **M5:** `categorize.v1` in shadow.
- **M6:** embeddings, sweeps and dreams, lessons as guidance, and §5.5's namespaces as kinds.

Every kind picks one of the closed composition rules, so no kind can exist without a reader (P0's standing rules).

4.2 Continuation strategies

1. **Transcript:** `root` → `current`. Always correct, always cheap.
2. **Ring:** transcript truncated from the front at the token boundary. Zero model cost.
3. **Compaction root:** summarize the range the ring would leave out into a `Summary` and re-root. One cheap call; the raw range stays in the graph and reachable.
4. **Assembled:** transcript tail plus a budgeted, Jev-selected neighbourhood: durable nodes, the task subgraph in play, borrowed summaries, participant `Person` nodes, the active role's hints.

Strategy 1 is what *appending* means; strategies 2 to 4 are what a *recompile* produces (§4.4a). Jev's `continue.v1` answers both questions at once: append, or recompile with which strategy, and whether a stronger model performs the compaction or synthesis. Strategies 1 to 3 need neither Jev nor the index.

4.3 Indexer and budgeter

The **indexer** turns "everything is eligible" into a few hundred candidates: exact indexes by channel, person, topic, and time; hybrid similarity through the index tender (§6). Incremental on every append. The **budgeter** allocates tokens across prompt sections and money across judgment calls per turn, weighted by the active role, aware of cache-read versus cache-write pricing, and logs every allocation for the ledger.

4.4 The context compiler

Graph selection yields node ids; the **compiler** turns them into a valid provider request. Its contract: preserve tool-call/tool-result pairing (never emit one without the other), preserve message ordering within a channel, honour model-specific constraints (thinking blocks tied to the model that produced them; no forced tool use on models that reject it), truncate only at message boundaries (the ring never cuts inside a message or a tool pair), and emit a **manifest** that records node ids, compiler version, renderer version, binding revision, model, and pack versions, so a turn can be reproduced faithfully from the ledger. The compiler takes the **session** (§3.2a) as its first input: the session's roots decide where traversal starts, so a task session compiles from its `Task` node outward (evidence, subtasks, the originating conversation as a secondary root, recalled memory) while a conversation session compiles from the channel's temporal view; both then pass through the same budgeter, labels, and manifest. The compiler is deterministic and unit-tested in the simulator against every strategy in §4.2. **Reproducibility** requires more than node ids because projections mutate: the manifest records an **as-of WAL position**, tool-schema versions, role versions (and hook versions, until the hook system was deleted on 2026-09-28), pack versions, binding revision, model, and every prompt-affecting

transformation, plus a **canonical request digest** so a reconstruction can be verified byte-for-byte. Judgment records likewise store the bounded state itself (or enough versioned references to rebuild it), not only its hash and size.

Context files (2026-09-29; theseus-58a, 76e7d35, a29e9f7; two levels since theseus-c48, 68cb127). The config names files that the compiler puts into the system block, in two levels. In Eddie's words: "Context files by persona with a system/default level", and "the Jev will classify which persona is in play, based on a dynamic, growing ontology of personas. Pre-jev, it will always pick the system/default persona."

- **The system level**, `[context] files`: the files every session gets.
- **A persona's files**, `[personas.<name>] files`: added after the system level's while that persona is in play.
- **Which persona**. `[context] default_persona` is the only choice until Jev chooses one from the ontology (§4.1a; theseus-8kk, theseus-0j2).
 - An unknown name is refused at load, and the error names the known ones.
 - With none set, the system level alone is used, and personas defined without a default warn once at load.
- **Placement**. The system block is the built-in persona, the tools note, and the profile's `system`, then the system level's files, then the persona's. Each file is in list order, under a header that names it and its level: `# Context file (system): <path>` Or `# Context file (persona theseus): <path>`.
- **No gate**. The compiler reads the files itself, not through a tool, so no posture or approval applies.
- **Digest**. Their text is part of the system block, so the system digest covers it. An edit to a file at either level is one `system_changed` recompile, and an unchanged file appends.
- **Timing**. A turn's system block is fixed for all its loops. A file edited during a turn takes effect at the next turn, never between a tool call and its result.
- **FAST**. Nothing is read at startup. A turn stats each file and rereads it only when its size, mtime, or inode changed, or when it had changed less than two seconds before it was last read. A file named at both levels is read once.
- **Failure**. A missing or unreadable file never stops a turn. The block says it is missing, and the daemon warns once per file per run (a log line and a `context.file_missing` row).
- **Cap**. A file over 64 KB is cut at a character boundary and marked as cut.
- **Records**. The manifest and every `context.compiled` row record each file's path, the digest of the text included, its bytes, whether it was cut or missing, and its persona (absent at the system level). The row names the persona in play. Health, `theseus health`, and the Observatory show the persona in play and each level's files.
- *(Amended 2026-09-29, theseus-c48: until then a profile carried `context_files`, defaulting to `[model].context_files`; both are gone.)*

Attachments and images (2026-09-29; theseus-9g2, 7bcfd9a, 48fd2c8). A user message carries the files that came with it, and a tool result may carry an image.

- **Where they come from.** `turn.submit` takes attachments: `[{name, media_type, size, text? | data? (base64) | not_read?}]`.
 - The Discord binding downloads text files, known by type or extension, up to `[tools].max_read_bytes`, and PNG, JPEG, GIF, and WebP images up to 5 MiB. It downloads them beside its gateway loop, and the message's turn waits for them. Anything else is listed with its type, size, and reason.
 - `theseus ask --attach <file>` sends a file the same way.
- **What the node keeps.**
 - Text is capped at `[tools].max_read_bytes` on a character boundary and marked if it was cut.
 - An image is a reference (digest, type, size, and dimensions) to its bytes. The bytes are stored once in `<store dir>/blobs/<sha256>` and never in a WAL frame, so the frame budget and recovery are unchanged.
 - A file that was not read keeps the reason.
 - Nothing about an attachment fails a turn.
- **Placement.** Each attachment is its own block before the typed text, under a header that names the file and its sender (`[Attachment message.txt from discord:eddie, 5,012 bytes]`). Until the labels of §3.9 exist, the header is what marks the text as the file's.
- **Vision.** The catalog entry of the compilation's model decides how an image shows. A model with vision gets an image block. Any other model gets one line: `[Image photo.png from discord:eddie, 1.2 MB: not shown, this model has no vision].fs.read` of an image does the same inside its `tool_result`.
- **Byte-stable.** The render reads only the node, the model's vision flag, and the blob, through a bounded cache. The same node renders to the same bytes, and a message without attachments renders exactly as before.
- **Budget.** The input estimate leaves out base64 data and adds each image's estimated tokens. The image's long edge is scaled to 1,568 px (Haiku 4.5) or 2,576 px (the rest), then it counts $\lceil w/28 \rceil \times \lceil h/28 \rceil$, capped at 1,568 or 4,784.
- **Refused.** An image over 5 MiB or 8,000 px on a side, or whose bytes are not one of the four types, is listed with the reason.

4.4a When a session is recompiled

Compiling every turn from scratch would be wrong twice over: it burns a prompt-cache miss on every turn, and it discards the plain fact that most turns in a live thread are simply the next thing said. So a session's context has two parts:

- a **compilation**: the prefix, produced by the compiler from the session's roots at some as-of position, persisted as a `Compilation` node (root kind) with its manifest. A compaction root (§4.2) is one kind of compilation; an assembled context is another.
- an **append tail**: everything the session has added since, in order, with no selection. This is the transcript strategy, and it is the default motion of every session.

Each turn the harness asks one question before the model runs: **append, or recompile?** The answer is layered so that the expensive judge is consulted only when something has actually changed.

1. **Deterministic triggers force a recompile** and never consult Jev: the audience or a confidentiality label in play changed (disclosure, §3.9); a declared ontology membership changed in a way that alters access (§4.1a; an interpreted membership change waits for the next recompile, so the prompt cache survives it); policy, tool schemas, role, or binding revision changed; the model changed; the tail would overflow the window or the configured tail budget; the session is new (a promoted task's first turn is always a compile); the execution glided to another channel; a redaction touched a node inside the current compilation.
2. **Candidate signals arm the judge**, cheaply and deterministically: a reference to another channel or an old topic (`mentions_conversation`), a recall hit outside the tail), a material task-state change in the session's scope, a long dormancy gap, a role hint change, a human asking for a fresh look, the tail crossing a soft length band, a cache-state change reported by the provider. If no signal fired, the turn **appends** and Jev is not called.
3. **Jev decides when a signal fired**: `continue.v1` receives the signals, the tail length, the cache state, the current compilation's manifest summary, and the budget, and answers `append` or `recompile(strategy)`. Jev owns this judgment as a core responsibility: it is deciding whether the world has changed enough that the model needs a rebuilt view rather than one more message.

Consequences. A long single-thread conversation appends turn after turn; its prefix is stable and cached; it behaves exactly like a normal transcript, because it is one. A task promoted from that conversation gets its own session and therefore its own first compilation, built from the task outward, then appends as it works. Diverse work is thereby compelled into distinct sessions, each compiled when it was needed and not again until something real changes. Recompiles are ledgered with their trigger, their cost (the cache miss, the compaction call), and the trajectory that followed, so the learning loop can tune the soft bands and Jev's threshold against outcomes: a recompile that did not change what the model did next was waste, and a stale append that preceded an error was a missed recompile.

Recompilation never loses anything. The old compilation, the tail, and the new compilation all remain in the graph with `derived_from` edges; the manifest of every turn names which compilation and which tail range it used, so any turn is reproducible.

4.4b How sessions persist across runtime restarts

A session is **durable by reference**, and that is the whole plan. Suppose a thousand sessions each hold a custom-selected context. Nothing about those thousand contexts is held in runtime memory that matters; each resolves in the store to a persisted **lineage**.

What is stored per session. One small record, updated in the same WAL transaction as the turn that changes it (§4.6):

```

Session {
  id, kind: conversation | task, roots: [channel id | task id, ...],
  execution_id,                               // §3.15, itself durable
  compilation_id,                             // current prefix
  tail: { from_position, last_position },      // append tail = in_session nodes in this
  strategy, budget_ref, labels_in_play, config_versions
}

```

Finding a session's own records. Positions are global across every session, so a `tail` position range is full of other sessions' records on a busy runtime. The index therefore carries a per-session ordered table, `session_id || position → ()`, and the tail walk is a range scan over exactly this session's records. The table is the same shape as the per-kind table and is rebuilt from the WAL like the rest of the index. (*Added v0.24, before M2 began; Session records are M2 work.*)

What a compilation is. A `Compilation` node whose record is the **selection**: ranked `includes` edges to the nodes it admitted, plus the manifest (§4.4: compiler and renderer versions, as-of position, model, binding revision, pack versions, request digest). Its rendered form, the provider-message bytes, is a **cache** stored alongside it and rebuildable byte-for-byte because the compiler is deterministic given the manifest. A thousand custom contexts are a thousand `Compilation` nodes that share the underlying nodes; no content is copied per session. At a few hundred references each, that is a few hundred thousand edges, which is nothing.

What a lineage is. `Session → Compilation → derived_from → Compilation → ... → first Compilation`, each carrying the tail range it was compiled from, so the full history of *how this session saw the world* is a chain of selections over one shared history. Promotion (§3.2a) is a branch in that DAG: a task's first compilation has `derived_from` the conversation's current compilation as well as `includes` edges into the conversation's nodes, which is why the graph must be multi-parent and why it is one.

Restart. The runtime reads Session and Execution records back from the store; nothing is "resumed" in memory. On a session's next turn the compiler loads the current compilation (the manifest, and the rendered cache if it is still resident on SSD), scans the tail by position, and proceeds with the append-or-recompile question exactly as if no restart had happened. The rendered prefix is reproducible, so a restart inside the provider's prompt-cache window still hits the provider cache. Sessions that are `waiting` cost a record each and nothing more; the resident graph (§6) rehydrates what a turn touches and only that.

What compile touches, and what it never touches. On the common turn the compiler loads the current `Compilation` by key (one index lookup), takes its rendered cache if resident or re-renders from its `includes` list (positioned reads, mostly arena hits), walks the session's tail through the per-session table (a handful of records since the last compile), and decides append or recompile (§4.4a). A recompile builds its candidate set from the retrieval indexes (§6.1) and adjacency scans bounded by the session's neighbourhood, then writes a new `Compilation` with `derived_from` the old one. Every step is a key lookup, a range scan, or a positioned read. The WAL is walked from a checkpoint in exactly two situations, crash recovery and index rebuild, and neither is on a turn.

Consequences for storage. Rendered compilation caches are the first thing tiering demotes, since they are rebuildable; selections and manifests are records and stay durable like everything else. Redaction (§5.6) of a node included by a compilation marks that compilation dirty, which is a deterministic recompile trigger (§4.4a), and restore applies tombstones before any rendered cache is trusted.

4.5 Prompt caching layout

Most stable first: persona and role hints; tool schemas; frozen transcript prefix from root to the last compaction; cache breakpoint; dynamic assembly; recent tail and new message. Compaction roots keep the prefix small and stable by construction. `continue.v1` receives cache state so it can prefer appending on hot conversations, and a recompile is scheduled at a natural boundary (after a tool loop closes, not mid-loop) whenever the trigger allows deferral. The system block leads the prefix: the built-in persona, the tools note, the profile's `system`, the system level's context files, then the persona's (§4.4). A persona that Jev switches, or an edited file, changes the block, and so the whole prefix is written to the cache again. Until the files get a breakpoint of their own (`theseus-ev1`), putting the system level first keeps the block's longest stable run at its front.

One header across sessions (Eddie, 2026-09-26; `theseus-ev1`). The header is kept as static as possible and reused across sessions and the parts of sessions (derived tasks), so one provider cache entry serves many of them instead of each session warming its own. This is the **provider-safe caching** work, scheduled after M3; M3 built only a breakpoint on the system block plus automatic caching of each session's growing prefix (Part III A3). It respects each provider's rules (a header shorter than the model's `cache_min_tokens` in the catalog never caches) and never rewrites earlier history to win hits, since that breaks preserved thinking signatures. It lands together with money budgets (`theseus-0sg`): under a token budget a cache read counts as much as fresh input, so better caching would lower the bill without stretching the budget. (*Money budgets landed first, on 2026-09-29, after Eddie's DM hit its unit limit; a cache read now counts at its own price, §3.13. The caching work stays scheduled.*)

4.6 What a turn writes

Inbound `Message` nodes; the turn trace (§3.3a); the `Compilation` node when the turn recompiled (selection plus manifest, and its rendered cache), otherwise a manifest reference to the compilation and tail range used; model output as `Message`; `ToolCall/ToolResult` with `in_session` and, when delivered, `in_channel` edges; loop `Judgment`s; `Task` mutations; and the updated `Session` record. Each is written in the WAL frame of the kernel transition that produced it (the model's message with its provider completion, a tool call with its planned action, an in-process result with its completion, a late result when it is absorbed), not in one transaction at the end of the turn, so a crash mid-turn leaves exactly the nodes whose effects are durable and a continuation resumes from them (as built in M3, Part III A3). The memory pass (§5) runs over these afterwards.

The frame rule. Every frame is an `fdasync`, so a turn writes no more frames than its durability needs.

- A state transition, a node, the session record, and a compilation are each in a frame that is synced before the call that wrote it returns, as above.
- An observability row that is no state transition waits in the turn's store handle. It rides at the front of the next frame that the turn, or the kernel, commits for it, in the order it was written. These rows are `turn.started`, `context.compiled`, `loop.started`, `loop.ended`, `turn.ended`, `turn.trace`, and the tool layer's rows. `provider.call` rides in its call's completion frame, and `tool.notified` in its call's plan frame. A crash in the middle of a turn can lose some of these rows, but never a transition or a node.
- An action that never waits for a confirm is planned, authorized, and dispatched in one frame: the provider call, and any tool call the policy runs (`open` or `notify`). Each transition keeps its own record and row, so the record still reads as three transitions.
- A plain one-loop turn writes 8 frames, and each further loop with one in-process tool call writes 4. `tests_m3::a_plain_turn_stays_within_its_frame_budget` holds the first number.
- A turn reads its session's transcript once, at its first reader, and every node its frames write joins that view. No reader in the turn decodes the transcript again: not resume, not absorb, not the check for anything new, and not each loop's compile.

(Amended 2026-09-29, theseus-qa0 step F2: until then a plain turn wrote 17 frames, and decoded its transcript at every reader; Part III A3c.)

Calls that run together (theseus-a60).

- A response's tool calls are gated in order. The first call that waits for the operator ends the gating, and the calls after it are left for the continuation, as before.
- The calls the policy runs go in groups. A run of consecutive reads is one group. Each write or program is a group of its own, a barrier: it starts after every call before it has finished, and the calls after it start after it finishes.
- An unknown tool or invalid input is answered at once, wherever it is.
- A group's calls run at once, in the turn's own task, and each keeps its own frames, plan and completion, written as they happen. So within a group, the WAL holds the calls' plan frames first, then their completions as they finish. Nowhere else does the order change.
- The next request carries the results in the order the model asked, because the compiler places each result after its call, whatever its position.
- A question for the operator is asked after every call before it has finished, so it keeps its sequential place.

(Amended 2026-09-29, theseus-a60 step F3; Part III A3c.)

5. Memory

The graph is append-only. Compaction adds `summary` nodes and re-roots the default view; the trimmed range stays in the graph, reachable through `summarizes`, merely absent from that view. Decay lowers a node's retention and heat until the tiering tender moves its payload to cold storage, but the node, its edges, and its stub remain. Nothing is ever lost.

"Memory", then, is not a kind of node and not a survival test. It is **any node whose retention is high enough to be selected into a prompt**. Ingest is a durability decision over the nodes a turn wrote: how much retention each starts with, and which edges tie it into the graph. Not extraction of a separate object, and never a decision to discard.

5.1 MemoryScience trait, native implementation

```
trait MemoryScience {  
  fn gate(&self, candidate: &Node, neighbours: &[NodeRef]) -> GateDecision;    // store |  
  fn schedule(&self, node: &Node, event: AccessEvent) -> Retention;             // FSRS-6 u  
  fn activate(&self, seeds: &[NodeRef], budget: usize) -> Vec<(NodeRef, f32)>; // spreadi  
  fn decay_sweep(&self, now: Time) -> Vec<Demotion>;                          // candida  
}
```

Native v1: gate by embedding cosine plus exact about overlap for duplicates, Jev kind/durability for store-vs-drop, and a Jev Noul for contradiction between a candidate and its top neighbour; FSRS-6 stability and difficulty per node with dual-strength kept as two numbers; weighted BFS over typed edges with per-edge-type weights the role adjusts; decay by retention plus heat. Parameters are versioned data tuned by the ledger. A remote scorer is possible as a plugin; none is planned. Vestige's science informed this design; its code is not used (AGPL, SQLite-first).

5.2 The memory pass

1. Jev memory.v1 labels each new node: kind (preference · fact · decision · procedure · episode · transient · other), durability, about targets, trust.
2. Policy: transient and low-durability nodes stay heat-managed; secrets and external untrusted text never become durable without operator confirmation.
3. gate() assigns initial retention, merges by adding same_entity edges (the duplicate node remains; the edge routes recall to the canonical one), and flags contradictions with contradicts plus supersedes when the human is the source. "Drop" means starting retention at the floor so the node is cold from birth; it does not mean deletion.
4. Compaction Summary nodes take the same pass, so a summary can be durable while its range is not.
5. **Recursion exclusion:** Judgment and Heartbeat nodes, context manifests, and ledger records are never themselves candidates for the memory pass or for memory judgments. Only human and agent Message, ToolResult, Task, Summary, and Synthesis nodes are. Routine bookkeeping is deterministic, not judged.

5.3 Recall

BUILD_CONTEXT gathers candidates from the exact indexes and the hybrid similarity index, adds activate() neighbours of the seeds, dedups by id, fans out one Jev relevance Noul per candidate in a single request, then lets the budgeter allocate by node kind under the role's weights. Every recall records what was shown so outcome labels can flow back to both Jev tuning and schedule().

5.4 Consolidation (what remains of dreaming)

Re-scoring offline is unnecessary; selection is recomputed live. Synthesis is kept, as a memory-tender job: cluster nodes the ledger shows are recalled together, have a cheap model propose one synthesis per cluster, have Jev citation-check it against its sources, store accepted ones as `Synthesis` nodes with `derived_from` edges, `trust: agent`, and the most restrictive confidentiality label of their sources. Shadow first, with a precise meaning: a shadow synthesis is *scored* (would the relevance judge have selected it; does an independent assessment rate it as supported and non-redundant) but its live *utility* cannot be shown while it is withheld. Promotion to live recall is a **bounded canary** measured on trajectory outcomes, exactly as question packs are (§3.10). No main-thread work, no unbounded replay.

5.5a Memory science must earn its place

FSRS models human recall; agent context selection has a different objective, selecting what improves the current task. The transfer is a hypothesis. There is also a known self-reinforcing loop: selection strengthens retention, strength increases selection, and repetition is mistaken for usefulness. Being shown is not being useful. Theseus therefore ships and measures a **baseline first**: transcript tail + task graph + summaries + BM25/embedding retrieval + deterministic freshness and provenance rules. Graph spreading activation, FSRS-style retention, Jev relevance reranking, learned role weights, and synthesized memories are each **ablated independently** against that baseline at a fixed total budget that includes judgment cost. Metrics: task success, false completion, unnecessary continuation, stale or contradictory recall, disclosure violations, latency, total cost. Retrieval agreement alone is not a success metric.

5.5 Namespaces, trust, forgetting

Namespaces are kinds in the ontology (§4.1a): `person:<discord_user>`, `guild:<id>`, `channel:<id>`, `topic:<id>`, `global`, and any kind added to the table later. Bindings declare read and write sets; personal preferences always write to the person namespace. Trust labels on every durable node; external-derived nodes are recalled with their label and never gate policy. Forgetting is always an append: FSRS decay by disuse lowers retention; `supersedes` chains leave the superseded node in place so backward reach still works; operator `forget` appends a `Suppression` node that excludes its target from every view, every index, and tiering rehydration, with a receipt. Payload erasure is the single exception, specified in §5.6.

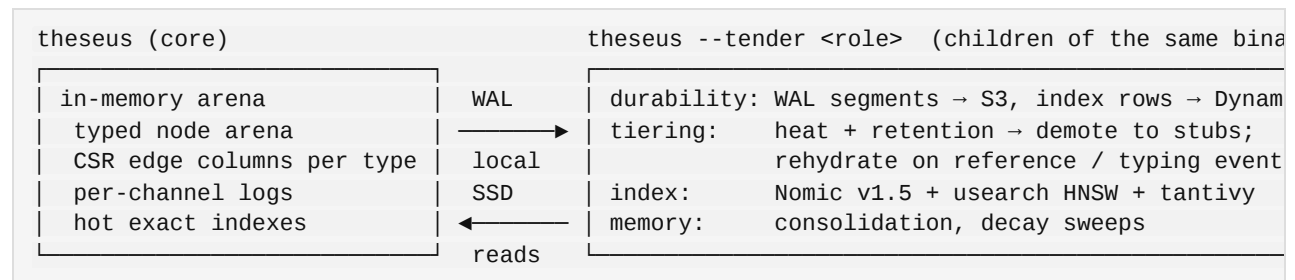
5.6 Redaction (the exception to append-only)

Suppression hides a node from views; it does not remove information that has already spread into summaries, syntheses, tool-result copies, ledger snapshots, indexes, embeddings, backups, and previously assembled contexts. For secrets and other must-not-exist payloads, Theseus permits **payload erasure with preserved structure**: the node record keeps its id, kind, origin, timestamps, and edges; the payload is replaced by an erasure marker (not a plain hash, which leaks low-entropy values), and a `Redaction` node is appended with a receipt naming who ordered it and why. Redaction

is **lineage-aware**: it walks `summarizes`, `derived_from`, `part_of`, and `same_entity` edges to find descendants that may carry the content, flags each for re-summarization or erasure, removes the vectors and index entries, invalidates cached contexts that included the node, and records which backup segments contain the original so a backup policy (retention window, or targeted rewrite) can act. Low retention is not low persistence; a secret that slipped past ingest policy is redacted, never merely cooled.

6. Storage and tenders

Speed first, durability second, cost last.



- **The logical graph is permanent; the resident graph is a bounded cache over durable history.** Nothing about append-only requires anything to stay in RAM. Resident memory scales with the active working set, not lifetime traffic.
- **Arena.** Nodes by monotonic id; edge storage as immutable sorted segments per edge type with an in-memory delta, compacted in the background (compressed-sparse-row columns are a benchmark candidate for cold segments, not a commitment); per-channel logs as segments in a shared append file with an allocation index. Cold nodes leave RAM entirely, metadata and adjacency included, represented only by their id in a compact presence filter, and rehydrate from the SSD index on demand. Memory targets (§9) cover the whole process tree including tenders and loaded embedding weights.
- **Storage kernel (built in M1; Part III A1).** Two layers, one contract. The **WAL** is the truth: segment files of checksummed frames, one frame per append, one `fdatasync` per frame; a frame holding several records is atomic, which is how `settle(completion, continuation)` commits both or neither. Recovery verifies every frame, truncates a torn tail in the last segment, and refuses to guess at corruption anywhere else. The **index** is a rebuildable projection of the WAL in a pure-Rust embedded store, `redb` (the M1 benchmark's pick; `fjall`, the other candidate, was removed in batch C, `theseus-0g4`): position → location, (kind, key) → latest position, (kind, position) for per-kind scans, and a checkpoint position. Index writes are non-durable; a checkpoint makes them durable; open replays the WAL past the checkpoint, so deleting the index entirely loses nothing. The arena remains the cache over this, never a second source of truth. Pure Rust keeps the static `musl` build honest.
- **The turn lock and eventual durability.** "Speed first" is preserved by *where* the time goes, not by skipping durability. Within a channel exactly one turn advances at a time; that lock is held only while the core is doing local work. A turn is mostly waiting: a Messages API call is seconds, a shell job is seconds to hours, a judge call is hundreds of milliseconds, a human is minutes. At

every such offload boundary the turn releases the lock and the core spends the surrendered time on **asynchronous durability work**: sealing the current WAL segment and handing it to the durability tender, taking checkpoints, flushing index updates, compacting edge segments, running the memory pass, uploading. The floor remains unchanged (intent is fsynced locally before dispatch); what changes is the off-node recovery point, which becomes **eventual: 5–60 s** rather than 1–2 minutes, achieved for free from time the loop was not using anyway. The tender scheduler prioritizes by staleness: the oldest unshipped committed record bounds the current recovery-point exposure, and that number is exported as a metric and alarmed on.

- **WAL.** Every record appends to the local SSD and is fsynced on a short group-commit interval before the turn proceeds. Records carry a length prefix and checksum; a torn tail is truncated on recovery. Periodic **checkpoints** snapshot the arena so recovery is checkpoint plus tail, not full-history replay. Schema versions are stamped on every segment and migrations are forward-only transforms run by the durability tender. Disk-full is handled by refusing new turns with a clear message while tenders continue to drain. The SSD is a persistent volume that survives instance death and is encrypted at rest by the platform (EBS encryption, LUKS on a desktop), not by Theseus.
- **Tenders** consume the WAL and answer rehydration over a local socket; they never touch the arena directly. Durability ships to S3 and DynamoDB when configured, with a 5–60 s target measured as "age of the oldest unshipped committed record." Tiering demotes payloads by heat and retention (stub stays in the arena, payload on SSD and S3; nothing is removed from the graph), with a Jev backup opinion for lower heat bands; rehydration misses are logged. Index owns embeddings: 768-d stored, 256-d indexed, 768-d rerank; usearch memory-mapped from SSD; tantivy BM25; reciprocal-rank fusion then Jev relevance; asynchronous after commit; long nodes chunked with `part_of`. Memory runs consolidation and decay sweeps.
- **Completion spool.** A directory on the same SSD as the WAL where job wrappers write results before attempting delivery. Startup drains it before accepting events; the heartbeat reconciler reads it every minute. It is the reason a harness restart never loses a finished job.
- **Restore.** Rebuilding a node from S3 segments plus the DynamoDB index is a first-class, tested path from the first release (`theseus restore --from s3://...`), because S3 is presented as disk-failure recovery. Periodic automated drills remain deferred.
- **Embedding weights** are a versioned artifact fetched to the SSD on first run and pinned by hash, distributed separately from the static executable.
- **Desktop mode.** Same binary; tenders write to a local directory and SQLite; every AWS-side dependency is absent without error.

6.1 Graph state

Written 2026-09-26 in answer to Eddie's questions before M2. The durable layer and the structural index are conclusions implied by §4.4b and the kinds reserved in M1; the arena layout is a hypothesis to be benchmarked in M3 and is written here so the benchmark has something to confirm or overturn.

No graph database. The WAL is the truth for the graph as for everything else, and the graph is a set of projections over it. An embedded graph store (CozoDB, IndraDB) or SQLite would add a second durability story and a second recovery path beside the one proven in M1; a graph query language buys

nothing when every traversal needed is "neighbours of X by edge type" or "lineage of session Y".

Durable layer. Nodes, edges, and compilations are ordinary WAL records (kinds `NODE`, `EDGE`, `COMPILATION`, `JUDGMENT`, reserved in M1). A node is keyed by its id, a UUIDv7 so ids sort by creation time. An edge is keyed `type || from || to`; its payload carries rank and weight. Both are append-only: a redaction or re-ranking is a new record carrying a tombstone or superseding flag, and `latest_by_key` gives current state. A `Compilation` record holds the manifest and the ordered `includes` list for fast rendering; the reverse `includes` edges, needed by redaction to ask "which compilations include this node", are written in the same frame. A turn that creates a node and a compilation with three hundred edges appends one frame; record count inside a frame costs microseconds and the frame pays the disk's one fsync.

Four projections, all rebuildable.

Projection	Store	Answers	Lag behind commit
Structural index	redb (§6 storage kernel)	position → location; latest by key; per-kind, per-session, and adjacency range scans (<code>type from to</code> , plus a reverse column <code>type to from</code> for <code>includes</code> , <code>mentions</code> , and the edges §5.6's lineage walk follows: <code>derived_from</code> , <code>summarizes</code> , <code>part_of</code> , <code>same_entity</code> ; Appendix F)	none (same call, non-durable until checkpoint)
Resident arena	process memory	the working set: metadata columns, payloads, adjacency segments	none
Retrieval index	Index tender: usearch (256-d indexed, 768-d stored and reranked) + tantivy BM25, memory-mapped from SSD	"things about E" by paraphrase and by literal mention; reciprocal-rank fusion, then Jev relevance	seconds
Entity edges	Memory tender writing <code>mentions</code> and related edges back through the WAL	"things about E" as one adjacency scan once E is an entity node	seconds to minutes

Live traffic is arena first, structural index on a miss, and the WAL only as a byte source at a known offset. Nothing populates the whole graph into memory at startup; rehydration is lazy and per node, which is why a thousand waiting sessions cost a record each and no RAM. Writes go to WAL, index, and arena in one call, and only the WAL fsync is on the critical path.

Resident arena. A struct-of-arrays arena with a dense process-local handle per node and edges as sorted columns per edge type:

```

type Handle = u32;                                // dense, process-local, never persisted

struct Arena {
  ids:      Vec<NodeId>,                          // handle → uuid v7
  by_id:    HashMap<NodeId, Handle>,
  kind:     Vec<NodeKind>,
  position: Vec<u64>,                             // WAL position of the latest record
  loc:      Vec<RecordLocation>,                  // where the payload lives
  heat:     Vec<Heat>,                            // last-touch tick + count, for eviction
  payload:  Vec<Option<Arc<Bytes>>>,              // None = stub (evicted)
  free:     Vec<Handle>,
  edges:    [EdgeColumn; EdgeType::COUNT],
}

struct EdgeColumn {
  sealed: Vec<Arc<Segment>>,                      // immutable, sorted by (from, rank, to)
  delta:  Vec<(Handle, Handle, u32)>,              // unsorted appends since last compaction
  rev:    Vec<Arc<Segment>>,                      // sorted by (to, from); only for types that ne
}

```

Handles index every column directly, so a node's metadata is a few contiguous reads with no pointers or lifetimes; handles are never persisted and the uuid is the durable identity. Columns keep traversals cache-friendly and let a payload be evicted without disturbing the row. A neighbours query binary-searches each sealed segment and scans the small delta; compaction merges the delta into a new segment during turn-lock gaps, and readers hold `Arc<Segment>` so they never block the writer (one writer, the turn holding the lock; `arc-swap` on the segment lists, a short `RwLock` around the delta). An evicted node keeps its row and drops the payload; a node never touched in this process has no handle and is answered by the index until first touch. Not chosen: `petgraph` as the store (its value is algorithms; a throwaway view over a subgraph is fine for those), per-node `Vec<Edge>` lists (fragment, defeat eviction), and CSR in the first cut (the natural format for a sealed cold segment, and still a benchmark candidate). The arena sits behind a trait so the segment format can change under a benchmark without touching the compiler.

What does not scale, and where it is handled. WAL bytes grow forever by design; compaction of live-latest records into fresh segments and shipping sealed segments off-node are M5 Tenders, and until then `max_total_bytes` refuses appends rather than filling the disk. Index rebuild from scratch is linear in WAL size and is a recovery path, never a startup path. Edge payloads stay tiny (a large graph is millions of forty-byte edges, a WAL of a few hundred megabytes); node payloads carry the bulk and are what tiering demotes.

Durability boundary, stated plainly:

Failure	Guarantee
Harness process crash, SSD intact	Committed intent, results, and pending continuations recover exactly (WAL + spool)
Node restart, persistent disk intact	Same, plus job reconciliation; uncertainty reported where execution evidence is unavailable

Failure	Guarantee
SSD loss	Recover to the backed-up prefix with the stated 5–60 s recovery-point loss; actions in the gap may have happened externally without an intent record, so restore runs reconciliation of surviving external work with a conservative policy near the gap, and never reuses action identities after rollback
External side effect without recoverable evidence	Report uncertainty; never invent success, never blindly repeat

The DynamoDB index is rebuildable from S3 segments and is never a source of truth whose consistency recovery depends on. **Redaction receipts** distinguish completed local erasure, pending backup expiry or rewrite, and copies outside Theseus's control (already sent to Discord, a provider, or another service); restore applies redaction tombstones before restored content becomes visible.

Control-plane tampering: with L0 as default, code running under the operator's credentials could write the WAL, policy store, or spool. The runtime and its storage can run under a separate OS identity from L0 jobs (a dedicated `theseus` user owning the WAL, store, spool, and policy files, with L0 jobs running as the operator). This is an option, not a default, and the installer and documentation recommend it in strong terms: without it, L0 offers no protection of the record against the code it runs.

7. Shells

One tool family, many backends. Policy first determines the set of classes an execution's authority permits; `Jev shell.v1` then picks among that permitted set. Jev never widens the security boundary, only chooses within it. The model never names an instance.

```
shell.run {cmd, cwd?, env?, timeout_s?, workspace?, class_hint?}  shell.open/send/close (p
workspace.{create, attach_repo, snapshot, list}
```

Class	Where	Speed	Cost	Durable access	Isolation	Default for
L0 host	the node, its SSD	ms	included	full: source trees, SSH agent, caches, instance role	none; <code>privileged tag</code>	coding in known repos
L1 native sandbox	the node; Linux namespaces + cgroups + overlayfs, no daemon; contract below	~100 ms	small	read-only binds of chosen trees, scratch overlay	moderate, by contract	code the agent just wrote; package installs
A1 Fargate	ECS task per job, EFS or S3 workspace	20–60 s	per second, zero idle	EFS durable	strong	purpose-built one-offs; long batch

Class	Where	Speed	Cost	Durable access	Isolation	Default for
A2 Lambda	container image	~1 s warm	per invocation, 15-min cap	none durable	strong	short stateless jobs
A3 SSM	tagged fleet	seconds	fleet's	the host's	the host's	operating a specific host
A4 dev box	named EC2	seconds	its own	full, curated	the box's	curated environments

L1 contract (what "sandbox" means here, so it is not called strong by assertion): user, pid, mount, uts, ipc, and **net** namespaces; no network by default, with an explicit per-job egress allowlist and **no access to the instance metadata service or to localhost services**, including Theseus's own MCP server and web UI; capabilities dropped to none; a default seccomp profile; no device nodes beyond null/zero/random; masked `/proc` and `/sys`; cgroup limits on CPU, memory, pids, and disk with output size caps; the whole process tree killed on timeout or cancel. Anything the contract does not grant is denied. L0 grants everything the operator's user can do, and the spec says so plainly.

The consequence boundary (M4; §3.9 Consequences). *Superseded in part, 2026-09-28: the consequence kinds are gone (§3.9; Part III A3b), so the boundary has no consequence to name. What stands is the default-safe environment: an L1 job starts with no ambient credentials and reaches the network only through its allowlist. Whether a credential request or a recognized request shape should notify or wait is held for Eddie, with M4.* The design as it stood, where L1 is where consequences stop depending on spelling:

- **Credential brokering.** A job starts with no ambient credentials. To push, publish, or post it must ask Theseus for a scoped credential, and that request is the consequence, judged by the gate.
- **Egress recognition.** Allowlisted egress passes a local proxy that recognizes request shapes: a git receive-pack, a PR merge, a registry publish. So `./deploy.sh` is caught when it pushes, however the command was written.

L0 has neither: its job environment keeps `HOME`, so the operator's credential helpers are ambient. Under L0 the gate is all there is: `proc.run`'s posture, the operator's lists, and the floor (§3.9). (Until 2026-09-28 this read "the argv rules and `opaque` are the whole of detection"; both were removed.)

Default (decided 2026-09-25). Both L0 and L1 ship in the first useful agent. L0 is the early operator default; it is explicitly provisional and expected to be revisited once L1 has run real work for a while. Roles and Jev may steer a job to L1 within policy at any time; the default only decides what happens when nothing else has an opinion.

Every class launches through the **job wrapper** (§3.16): detached from the harness, result spooled before delivery, cancellable by correlation id. `shell.run` for a fast command still returns synchronously to the tool loop, but the record and the spool exist from the first millisecond, so a harness restart mid-command finds the result waiting rather than lost.

Workspaces point at existing directories on the node by default (the operator's real checkouts); cloning is an explicit attach for A1/A2. Snapshots everywhere. Credentials: L0 has the operator's agent and role; L1 gets nothing unless policy grants short-lived STS tokens or agent forwarding per job; AWS classes use scoped task roles. Persistent PTY sessions are `Resource` nodes owned by a conversation, reaped by an idle timeout the role may extend. A1–A4 light up only when AWS credentials exist.

8. Verification: the simulator and the replay harness

Two test targets are first-class from the first commit.

Replay harness. The ledger records every Jev call's state and answers, every context manifest, and every tool call. Replay re-runs recorded states through a candidate question pack, memory-science parameter set, or shell mapping and reports how decisions would have changed against the recorded outcome labels. It is how packs get promoted (§3.10) and how a bad judgment is reproduced from production without touching production.

Deterministic simulator. One process, no network, fully scripted, fast enough for CI:

- *Fake Discord*: a gateway that emits scripted events (messages, reactions, component clicks, voice joins) from a scenario file and records every outbound action Theseus takes, with a virtual clock so timeouts and wakes run in milliseconds.
- *Fake provider*: answers keyed by prompt hash from a recorded fixture set, or a small scripted policy ("call tool X then end_turn"), or, in a separate mode, a real cheap model for fuzzier scenarios.
- *Fake Jev*: recorded answers, or rule-based answers, or a real Jev call in a separate mode.
- *Real everything else*: the arena, WAL to a temp directory, tenders, indexer, budgeter, policy gate, MCP client against an in-process fake server.
- *Scenarios* are files: a binding, a cast of people, a script of inbound events, and assertions over the graph, the ledger, and the recorded outbound actions.

What it buys: invariants as property tests (a destructive tool never runs without a recorded confirm from the right person; the loop never exceeds a ceiling without a logged judge failure; a user `/model` pin is never overwritten; a compaction root is always reachable back to its range); regression tests for every incident, by turning a redacted production ledger slice into a scenario; and a way to develop the loop, roles, and memory pass for weeks before the Discord plugin is finished. It also gives the learning loop a dry-run: promote a pack in the simulator first, then in shadow, then canary, then live.

Standing scenario sets: crash at every external-action boundary (§3.16); lost completion (event never delivered, reconciler must find the spooled result); duplicate completion (second delivery is a logged no-op); completion arriving during harness restart; `/cancel` of a detached job and proof the scope died; completion with no matching record quarantined; a low-privilege user schedules a privileged action; crash after settlement but before continuation delivery; `outcome_unknown` followed by a genuine success; a late completion after cancellation; arguments changed after confirmation (a hook mutation, until the hooks were deleted on 2026-09-28); two executions on the same task or

workspace; interrupted provider streaming with unknown usage; restore from a stale backup containing later-redacted content; a task promoted from a conversation runs while the conversation continues and a human message is routed to each correctly; a task waits a week for an answer and resumes with intact context; a hot thread of two hundred turns appends throughout with a stable cached prefix and a disclosure change forces exactly one recompile; a thousand sessions with distinct compilations survive a restart and each next turn reproduces its prefix byte-for-byte from the manifest; the admission ceiling is hit with a `/cancel` still honored immediately; authority edge cases (role revoked mid-execution, coalesced messages from two principals, confirm with changed arguments); redaction lineage; disk-full and Jev-outage behaviour. **First vertical slice:** one human request, one constrained typed tool action with a confirm, a task that goes `waiting` on a due time, a `/cancel`, and a crash during a dispatched action, all green in the simulator before any Discord code is written.

9. Efficiency targets (to measure, not assert)

Under FAST (§2), the lifecycle rows are budgets. `theseus-sim bench lifecycle` measures them (M3.5, P5b):

- cold start to the first `health` answer;
- clean shutdown with executions waiting and a job running;
- SIGKILL, then restart to the first answer.

Each phase runs N times, with p50 and p95, per phase, per start-path phase, and per kernel step. It runs on an empty store, on the owner's store, or on a synthetic store of parked sessions.

`scripts/gate.sh` runs it on every commit: ten runs of each phase on an empty store, with debug binaries, which are never faster than release. Each p95 is checked against its budget plus that phase's noise margin, measured as the spread of p95 over repeated runs on the build machine (in 2026-09: 7 ms cold, 4 ms shutdown, 25 ms kill). A miss fails the gate. Between today's store sizes and 10,000 parked sessions, the cold-start budget is the line from 50 ms to 250 ms. Measured values are in Part III (A3, lifecycle timings, and the M3.5 entry).

Metric	Target
Process start to answering the protocol socket (config parsed, store open, WAL tail replayed, spool drained; secrets, credential checks, and the Discord gateway may still be connecting)	under 50 ms at today's store sizes; under 250 ms with 10,000 parked sessions. It grows with the WAL tail since the last checkpoint, never with history (<i>With the config in the vault, a start serves from the note's last-known-good copy and reads the vault behind the socket. Only a first start, with no copy, reads it before serving, which takes about 1 s. theseus-2fo, step F1b. The bench's <code>vault</code> phase holds this to the budget in the gate.</i>)
Clean shutdown, request to process exit, with work in flight	under 100 ms; nothing in flight is waited for
Crash to serving again (SIGKILL, then restart)	the cold-start budget plus tail replay, with the checkpoint interval keeping replay under 100 ms (<i>The bench's budget: the cold-start budget plus 100 ms.</i>)

Metric	Target
Binary upgrade (swap, stop, start; job wrappers keep running)	under 200 ms without a protocol answer
Store or schema migration	adds nothing before serving: old formats are read in place, and a tender rewrites them in the background using at most 5 % of one core
Restore from a local WAL	at the disk's sequential read speed; to measure
Binary size, static	under 60 MB with Wasmtime, AWS SDK, voice, search, and web UI; embedding weights are a separate artifact
RSS at 10,000 parked channels, 50 active executions	under 1 GB including arena metadata for the active set
Per-turn harness overhead (context compile + gate + WAL commit, warm arena; excludes model, Jev, tokenization, rehydration)	under 5 ms
Jev per turn	to be measured: calls, questions, input tokens, p50/p95/p99, per representative turn class; the ~350 ms / sub-cent figure is one call, and a turn makes several
Process start to accepting Discord events (checkpoint loaded, WAL tail replayed; excludes model/index warm-up, which proceeds in tenders)	under 2 s
WAL commit latency	under 5 ms at group-commit interval

10. Open questions

The three questions v0.9 held for Eddie are decided (§1: storage kernel, turn lock, shell default, control-plane separation). What remains open is deliberately the kind of question that only running code answers:

1. **Which embedded store.** `redb` or `fjall`, decided by the M0 benchmark in the plan, not by taste.
2. **The real recovery-point number.** 5–60 s is a target; the measured value under load, and whether the turn-lock model delivers it without stealing latency from turns, is an M3 result.
3. **Whether memory science transfers.** §5.5a's ablations are the answer; until then FSRS, spreading activation, and synthesis are experiments, not features.
4. **When L0 stops being the default.** Revisit after L1 has carried real work; the trigger is evidence, not a date.

Appendix A — Response to external review (GPT Astra, 2026-09-24)

Accepted and incorporated: an `Execution` record separate from the conversation (§3.15); concurrency defined per execution; an explicit authority context with intersection semantics and deny precedence, revocation handling, confirmation-is-not-authorization, read vs disclosure decisions, immutable origin vs inferred trust (§3.9); deterministic control paths and the six execution outcomes, budget exhaustion as correct operation, Jev-outage behaviour (§3.3, §3.15); the external-action lifecycle with outbox, idempotency, reconciliation, and bound confirmations (§3.16); record/projection separation, cold metadata leaving RAM, checkpoints, torn records, schema versions, disk-full, a tested restore path, embedding weights as a separate artifact (§4.1, §6); payload redaction as the receipted exception with lineage-aware invalidation (§5.6); the memory-pass recursion exclusion (§5.2); the context compiler contract and full manifests (§4.4); the L1 contract and "Jev chooses within policy, never widens it" (§7); the narrower learning claim with holdouts, canaries, rollback, and human approval for security-adjacent changes (§3.10); atomic budget reservation, global admission control, bounded queues, overload shedding (§3.13); physical storage details demoted to benchmark candidates (§6); workload-defined performance targets (§9); the unified task state machine, `Suppression` and `Redaction` in the schema, task-graph scope limits, coalesced-message authorship, voice/text as linked separate channels, desktop-mode credentials, and the removal of the "no open questions" claim.

Pushed back or held for Eddie: L1-by-default (conflicts with a settled decision; §10.1); backup drills (compromise: restore path now, drills deferred; §10.2); role hints as enforcement (§10.3); per-client MCP credentials (Eddie chose a static key for now; §10.4).

Not adopted: nothing else in the review was rejected outright. The review's bottom line is taken as the sequencing constraint: the five contracts (execution, authority, external actions, storage and retention, resource accounting) and the first vertical slice in the simulator come before the custom storage and memory machinery.

Appendix C — Response to the all-webhook proposal (Eddie, 2026-09-24)

Adopted: the principle that nothing in flight lives only in harness memory; completion as an externally delivered event against a durable record; the harness quiescent between events; the heartbeat as reconciler; the detached job wrapper for every shell class; startup as reconciliation. Evidence: 24 lost subagent-completion wakes and hundreds of blocked-tool-call stalls in three days of our own gateway logs; Step Functions task tokens, Temporal activities, Kubernetes resync, Erlang mailboxes.

Changed in the folding: "webhook for everything" became "one `Completion` envelope over the cheapest durable transport per source" (in-process, Unix socket spool, SQS pull, loopback HTTP as the exception), preserving the no-inbound posture and avoiding a receiver that must be up for anything

to finish. "Unkillable wrapper" became "detached, durable, cancellable." Fast in-process tools stay synchronous inside the tool loop.

Appendix D — Response to the second external review (GPT Astra on v0.8, 2026-09-25)

Adopted and incorporated: the thesis reframed as a durable agent runtime with a provenance-aware context compiler (§0); derived work retains initiating authority, owner grants are separately declared, coalesced asks from other principals get their own authority, channel switches intersect ceilings (§3.9); audience-safe context compilation with inherited confidentiality labels and explicit declassification (§3.9, §4.4); atomic settlement-plus-continuation, resolvable `outcome_unknown`, the five-step startup order, per-adapter retry classes, cancellation as a lifecycle with wrapper deadlines, provider and judge interruption accounting (§3.3, §3.16); hook ordering before final authorization and typed observer results (§3.17); the three-layer task graph with mechanical acceptance checks as necessary conditions and CAS/lease concurrency (§3.5); enforceable limits versus estimated exposure, held reservations on unknown usage, reserved control budget and disk capacity (§3.13); the budget-exhaustion label fix (§2); the memory baseline-and-ablation programme and the precise meaning of shadow (§5.4, §5.5a); the resident-graph-as-bounded-cache contract, reproducibility via as-of position and request digest, the durability boundary table, redaction receipt classes, and the tamper note (§4.4, §6); Jev critical-path measurement and uncalibrated-until-shown (§3.7); the eight additional simulator scenarios (§8); and retiring the "no open questions" claim (§10).

Held for Eddie, then decided 2026-09-25 (v0.10): embedded store as the day-one kernel with the arena as cache, plus the turn-lock model for eventual 5–60 s off-node durability; L0 and L1 both in the first agent with L0 the provisional default; control-plane separation as a strongly recommended option.

Taken as sequencing input, not spec text: the five-phase build order (prove the kernel in the simulator; ship a deliberately narrow agent; validate real failure boundaries; add intelligence features as measured experiments; expand the integration surface). It matches Appendix A's constraint and will seed the sequencing document.

Appendix E — Response to the "Omni-MCP" proposal (2026-09-26)

Eddie forwarded an essay arguing that a harness should treat everything as an MCP node: filesystem, memory, shell, sub-agents, thinking, channels, all speaking one RPC contract through a stateless router.

Adopted. The essay's fifth point is right and is the one worth keeping: one tool shape means authorization, approval, budgets, and tracing are written once. That became the tool contract with two backends (§3.12). Its fourth point, references instead of payloads between tools, is adopted natively with node ids as the pointers (§3.16).

Rejected, with reasons. Channels as MCP servers: Discord is the source of authority context, which must stay trusted kernel data (§3.9); MCP's request-response session model also does not fit a long-lived event source. The kernel as a stateless MCP router: MCP has no durable completion model, and a stateless router loses in-flight work on restart, which is the failure the execution kernel exists to prevent (§3.16). Memory as a tool: recall is compiler-selected every turn, never something the model must remember to ask for (§5). Thinking as a tool: native extended thinking exists; a tool adds latency and tokens. Sub-agents as nested MCP servers: the design has no multi-agent (§1); task sessions cover the need (§3.2a).

Where it led. Self-extension of tools at runtime under operator ack, with the kernel binary off limits to the agent (§3.21), and restart as a routine, tested operation (§3.22). v0.20 removes WASM as a commitment: the one runtime extension path is an MCP server in an L1 sandbox (§3.12, §3.21). v0.21 adds **NATIVE FIRST** (§2, §3.23): many small typed Rust toollets; the shell is the escape hatch and every shell call is a data point. v0.22 selects the tool surface itself (§3.24) after a review of Claude Code, Codex, and OpenClaw ([notes/tool-surface-review.md](#)). v0.23 closes **M1 Keel**: the WAL store exists, the crash test passes, and the engine is decided (Part III A1). v0.24 answers Eddie's questions on graph state (§4.4b, §6.1): nodes, edges, and compilations are WAL records; the graph is four rebuildable projections over them; compile never scans the WAL; the resident arena's layout is written down as an M3 hypothesis. v0.25 closes **M2 Kernel** (Part III A2): executions, actions, completions, the spool and job wrapper, budgets, admission, the five-step startup, group commit, and the deterministic kernel simulator. v0.26 adds the **Observatory** to the web UI (A2 addendum): every M2 object observable live. v0.27 records **M3 First hands, parts 1 and 2** (Part III A3): the session graph as nodes, the context compiler, the model catalog, eleven native toollets through the gate with durable confirmations, background jobs that survive a restart, and the web UI's sessions, transcript, and Observatory panels for all of it; Discord (part 3) waits on its bot token. On whether the model would "get" it: the tool half, yes, deeply; the risks are tool-count bloat (dynamic tool search in the toolchain manager) and judgment about when to extend (a Jev pack plus the gate), not comprehension.

Appendix F — Response to the openrig and herdr research (2026-09-27)

*Tabitha, 2026-09-27 (theseus-s3m). Eddie paused development to ask what Theseus should take from two open-source projects: **openrig** (github.com/mvschwarz/openrig), which runs many agent harnesses as a team, the opposite of §1's "one agent, many roles", but has thought hard about context domains and segregation; and **herdr** (github.com/herdrdev/herdr), a terminal multiplexer for agents. He also asked how Theseus should think about "context epidemiology", the spread of good and bad ideas between contexts. The three reports and the synthesis are in `~/reports/theseus-research/`, and the citations they rely on were checked against the source. **[D]** marks a decision Tabitha made in Eddie's absence; overturn freely.*

What it found. Openrig's own failure record (recipients never woken, phantom occupant generations, one seat's restore state injected into another, per-seat role text colliding in a shared instruction file) is the cost of opaque harness windows, which this design does not have. On context mechanics, it

supports the thesis. The one thing a team buys that one harness must build on purpose is independent judgment. Openrig's lasting contribution is its doctrine about where knowledge lives: at the narrowest scope that needs it; intent composes and bodies do not; knowledge learned in a position graduates only by re-authoring; handed-over material is testimony; a volatile fact is the command that derives it. Openrig can hold these only as files and culture; here they can be compiler rules. Context epidemiology is best treated as truth maintenance over the context graph, with information-flow labels and recompilation as the containment primitive. The graph already records exposure exactly (each compilation's `includes`), but not use, and nothing can be looked up in reverse. herdr's machinery solves problems Theseus doesn't have, but its attention model transfers, and studying it showed that the protocol pushes no execution-state changes.

Adopted [D]. Each item has a follow-up issue waiting for Eddie to schedule it; none changes a milestone's exit test.

- *Now* (theseus-n4m, theseus-in3):
 - reverse compilation membership written at `persist_compilation`, as intervals for a session's own runs;
 - a reverse `derived_from` column;
 - `node.reach`, with the Observatory showing a node's reach; §6.1 now names the reverse columns that §5.6's lineage walk follows;
 - `execution.changed`, a watch over all sessions, and a daemon-owned `session.wait` in the protocol;
 - one `attention()` mapping in `theseus-protocol` that every surface uses: *needs you* for a pending confirm, a failure, or an exhausted budget; then *done* until seen, *working*, and *idle*. Notifications fire on transitions only, debounced, and never for the session in focus.
- *M4* (theseus-3vu):
 - content refused by label class wherever it leaves (commit, push, PR body, public post, MCP response), deterministically and before any content scanner;
 - widening an audience writes a new authored node with `graduated_from` and a warrant, and relabelling in place is not an operation;
 - origin `external {source}` before any fetching tool lands;
 - hashes on file reads and writes;
 - an append-only `Advisory` (annotate, quarantine, redact) with an operator correction control, appended now and folded in at the next recompile, with the §5.6 walk generalized into the advisory walk.
- *M5* (theseus-vug):
 - independence as a compiler property: a check task compiles without the maker session's messages, tool calls, and thinking; its `Judgment` records that basis; evidence sharing an ancestor or a method counts once;
 - promoted tasks admit their pieces by reference;
 - obligation invariants: a task in progress with no live execution, `Question`, or wake is *parked*;
 - delivery recorded as `posted` | `transport_failed` | `never_posted`, never seen;
 - a reply that references a `Question` resolves without inference;
 - `relies_on` attribution in shadow.

- *M6 (theseus-3nk):*
 - a compilation is never silently thinner: the manifest lists what the budget dropped, and a core coverage is a named outcome;
 - material from prior contexts renders as testimony, with its as-of and origin, in a fixed precedence;
 - volatile values render as of their position;
 - lessons carry a stage, a scope, a warrant, and a re-verification rule; they are admitted by scope and by reference, as ablation arms, and kept out of the shared cache header (§4.5, theseus-ev1).
- *Tools and license:*
 - listing toollets default to narrow results and say what they left out (§3.24, theseus-8ye);
 - herdr is Apache-2.0, so its ideas are re-implemented, never copied, which keeps Theseus's MIT option.

Decided by Eddie (walkthrough, 2026-09-27 13:53–22:21; theseus-vmh, now closed). Each decision is written into Part I in v0.37, and the build order is P5c.

1. **The context domain becomes the topic, in a fungible ontology.** The kinds of context are data (§2 FUNGIBLE ONTOLOGY; §4.1a). They can be added, trained by Jev, re-associated by sweep, dream, or live check, and indexed by embedding. Tabitha's two guardrails were confirmed: given memberships are facts and are never re-associated; interpretations route context but never grant access (theseus-8kk).
2. **Irreversible consequences wait for approval.** Eddie generalized the consequence tags into one idea, irreversibility, and the kinds remain as its reasons (§2 IRREVERSIBLE WAITS; §3.9).
 - *Enforcement:* under `notify`, dangers that aren't irreversible get a notice, and irreversible ones wait for approval. So do they under `open`.
 - *Approval* is a dialogue in a statically configured trusted channel, among trusted users only. Any surface can be configured as one.
 - *A new principle:* the owner can override any refusal on the owner's own property, within the model's terms of use and safety policy. Overriding the floor takes the stronger ceremony (owner only, exact command shown, typed confirmation).
 - *Also adopted:* integrity labels by transmission, and the exposure rule as a one-step-stricter enforcement level. `opaque` stays at needs-approval, and Jev learns to classify related calls (theseus-770, theseus-sgh, theseus-qc4, theseus-3vu).
 - *Reversed 2026-09-28 (theseus-8az):* the consequence kinds, the irreversible column, and the detection behind them were built and then removed, and the gate became notify over block (§2, §3.9; Part III A3b). Trusted-channel approval stands. The owner override is moot in the gate, which refuses nothing.
3. **The standing rule is adopted with a gate test:** nothing is declared without its reader (P0; theseus-wjy).
4. **The herdr adapter:** build it (theseus-l1l).
5. `theseus tui`: build it; "first we try everything" (theseus-7yx).
6. **Promotion requires an authored arrangement** (§3.2a; theseus-vug).

Side findings, decided the same night.

- The hook sites are to be fixed as proposed (§3.17 amended; theseus-0dp). (*Superseded 2026-09-28: the hook system was deleted instead, and theseus-0dp is moot; §3.17, Part III A3b.*)
- Three OpenClaw bugs that cost this research its replies went to Tank: the 8 MiB stream cap (openclaw-v1et), a rebuilt `dist/` under a running gateway (openclaw-lzca), and a watchdog false positive (openclaw-jygw).

Not adopted:

- multi-harness orchestration (§1 stands);
- the terminal as the wire, instruction files as the context channel, identity by environment variable, permission config translated by an agent, role authority written as prose, byte proxies for tokens, and receipts that print their own answer;
- minimizing reach as a goal, pre-loaded warnings, and integrity labels inherited by exposure;
- herdr's screen scraping, PTYS, terminal core, tiling, and federation.

Also recorded. A3 gains a divergence row: ten of the 24 declared hook events have no dispatch site, seven of them on code paths that exist (theseus-0dp).

Appendix B — What is at stake in the default shell class

The question is only about the **default**: which class a coding-role shell command lands in when nothing else decides. L0 stays available either way, and policy can force either class for any binding.

What L0 as default buys. Speed and fidelity. Commands run at native disk speed on the operator's real checkouts with the operator's real toolchain, caches, SSH agent, and instance role. Nothing has to be bind-mounted, no overlay diverges from the real tree, git and package managers behave exactly as they do for a human at that machine, and interactive PTY sessions are trivially persistent. This is the OpenClaw experience today, and it is why the coding role is fast.

What L0 as default costs. The harness's per-action safety tags stop meaning what they say. `shell.run "npm test"` is tagged as a read-ish action, but what actually runs is whatever the repository's scripts, hooks, and dependencies do, with the operator's SSH keys and AWS role in the environment. A malicious or merely careless dependency, a prompt-injected file the agent was asked to read, or a plain model mistake can push to any repo the agent can push to, call any AWS API the instance role allows, or exfiltrate anything on the disk. The three-band gate still exists but it is gating the *command*, not the *effects*, and for arbitrary code those are not the same thing. Under L0-default the true security boundary is "everything reachable with the operator's credentials", and the reviewer's point is that the spec should not imply otherwise.

What L1 as default buys. The tags mean what they say again, within a written contract: no network unless allowlisted per job, no metadata service, no localhost (so no reaching Theseus's own MCP server or web UI), no SSH agent, no instance role, scratch writes in an overlay that can be discarded or promoted, the process tree killed on cancel. A repository hook that tries to phone home fails. A model mistake that runs `rm -rf` in the wrong place hits an overlay. The per-action gate becomes a real second layer rather than the only layer.

What L1 as default costs. Roughly 100 ms per command and real friction for the hot loop: source trees are read-only binds, so writes go to an overlay that must be promoted back to the real tree as an explicit step; credentials for git push or AWS calls must be granted per job as short-lived tokens, which is more machinery and more prompts; some tooling misbehaves in namespaces (anything that wants the real user id, dockerd, some debuggers); and interactive sessions are less natural. Every one of these is solvable, and every one is a place where the agent will occasionally get stuck and ask.

The middle path the spec could take. Make the default depend on what the job needs, not on a global switch: L1 for anything Jev classifies as running code the agent wrote or installing packages (already the mapping); L0 for read-only exploration of known repos; and require an explicit, per-binding grant for L0 with write or credentials, defaulting the coding role's *writes* to L1-with-promotion. That keeps the fast path fast for reading and thinking, and puts the sandbox exactly where arbitrary code runs.

Decision (Eddie, 2026-09-24): L0 is the default; commit and see how it goes. Tabitha's recommendation, adopted: For BigHat, where the operator is the owner and the node is already trusted with these credentials, L0-default with the middle path's two exceptions (agent-authored code and package installs in L1, which is already the mapping) is the honest choice: it matches how we actually work and the spec now says plainly where the boundary is. For the open-source distribution, ship L1 as the default and make L0 the documented opt-in, because a stranger's default should be the safe one. The binding config already supports both; this is a defaults question, not an architecture question.

Part II — Build Plan

Tabitha, 2026-09-25. Part II says in what order the spec gets built, what each stage must prove before the next begins, and what is deliberately left out of the early stages. It follows the reviewer's five-phase shape (kernel → narrow agent → failure boundaries → intelligence as experiments → surface) and Appendix A's constraint: the execution kernel and simulator are the critical path, and everything else is a feature that must earn its place.

P0. How to read the plan

Each milestone below says what it will **build**, what must be **proved** before the next begins, and what is deliberately **not yet** done. What actually happened is recorded in Part III, one section per milestone, so a plan section is never edited to match reality after the fact; the divergence is written down instead.

There are no duration estimates. Milestones are ordered by what each must prove before the next can begin, and two things will dominate the pace: how much of the kernel the simulator forces us to rewrite (it always forces some), and how much time the Discord and Anthropic integration steals from the kernel if started too early. The plan defends against the second by refusing to start them until M2 is green.

Every milestone has three parts: **build** (what exists at the end), **prove** (the test that gates the next milestone, always executable, never a judgment call), and **not yet** (what a reasonable person would want to add here and must not). Milestones are Beads epics under `openc1aw-ph78`; each "prove" line becomes a closing criterion.

Standing rules for every milestone. Part III records where any of them slipped.

1. **Visibility** (M0.6). A new kind of work lands with its span, attributes, and metric on the same commit. A new capability lands as a native toollet unless a written reason says it cannot.
2. **Speed** (M3.5, §2 FAST). New startup work lands with its bench row. A new on-disk format lands with the reader for the format it replaces.
3. **Nothing declared without its reader** (Eddie, 2026-09-27; theseus-wjy).
 - Every new route by which content reaches another context lands with its edge and a reverse-index entry. Examples are a summary admitted into another session, a task report into a channel, borrowing, gliding, and MCP.
 - Every new edge type, label, or hook event lands with at least one reader, on the same commit.
 - Anything declared ahead of its reader carries a "reserved for Mx" marker, and Part III lists it.

The gate enforces rule 3 with a registry test: it enumerates every hook event, edge type, and label, and fails unless each one has a reader or a reserved marker.

P1. Milestones at a glance

#	Name	One-line exit test
M0	First light	The binary starts with config and secrets from 1Password, all hook events registered with no handlers, and one <code>turn.submit</code> over the protocol returns the model's reply from exactly one loop
M1	Keel	<code>kill -9</code> at any point during a simulated workload; restart recovers every committed record byte-for-byte
M2	Kernel	All kernel scenarios in spec §8 pass under randomized fault injection, including crash inside each of the five startup steps
M3	First hands	Eddie completes a real coding task in a known repo from Discord; the harness is killed mid-job; the job finishes and its result lands in the channel
M3.5	Fast	The lifecycle bench meets every §9 lifecycle budget on Eddie's store and on a synthetic 10,000-session store, the gate fails a commit that misses one, and a store in the previous format serves at once under the new binary
M3.6	Daily Driver	Eddie carries his daily Discord work on Theseus end to end, on one channel beside OpenClaw, and each item's prove passes

#	Name	One-line exit test
M4	Boundaries	Every row of the durability table (§6) is demonstrated, the measured off-node recovery point is under 60 s, and L1's contract tests pass
M5	Judgment	Jev-driven stopping and classification beat the deterministic baseline on a held-out trajectory set at equal total budget
M6	Memory as experiment	An ablation report over the §5.5a metrics says which of FSRS, spreading activation, reranking, and synthesis stay
M7	Surface	Theseus carries Eddie's daily Discord work end to end; OpenClaw is no longer in the loop for that channel

A usable agent exists at M3; the shape of the whole exists at M0. The back half is where the plan is least certain, and that is fine: by then the measurements exist to re-plan.

P2. M0 — First light

The very first version, specified by Eddie: a vertical slice through every layer, each at its thinnest, so the shape of the whole is real before any part is deep.

Build.

- Toolchain pinned: `rust-toolchain.toml` at stable (1.98.1 on 2026-09-25), target `x86_64-unknown-linux-musl`, static release profile, `cargo deny` with the permissive allowlist, `cargo nextest`, CI building the static binary on every push.
- Workspace crates: `theseus-protocol` (types only), `theseus-core` (kernel library), `theseus` (the binary: `serve`, `chat`, `--tender`).
- **Config and secrets (§3.19)**: TOML config, `op://` references, resolution through the `op` CLI under the service-account token, zeroizing in-memory secrets, fail-closed startup. Starting set: Anthropic, Jev, GitHub, AWS (`strata-jam-aws-key`).
- **Hooks (§3.17)**: every hook event defined as a typed enum with its kind (Gate, Transform, Claim, Observe), a registry that accepts handlers over the protocol (`hooks.register`) and in code, the run-hooks path wired at each event site, and zero handlers installed. The turn runs through every hook site and nothing fires.
- **Turn runner (§3.3a)**: session with a turn lock; a toolchain manager that compiles the context (the user prompt, nothing else), offers the tool list (empty), sends one provider request to the Anthropic Messages API with streaming, and returns the response.
- **Advancer**: the trait, with `stop_after_one_loop` as the only policy, ledgering its decision.
- **Protocol server (§3.18)**: JSON-RPC over NDJSON on stdio and a Unix socket; `session.open`, `turn.submit`, streamed `model.delta`, `turn.ended`, `hooks.list`, `hooks.register`, `health`. `theseus chat` as the thin client.
- A first `Store`: the session record and a per-turn ledger row in an embedded store, so even the hello slice persists what it did. No WAL discipline yet; that is M1.

Prove. From a clean shell with only the service-account token in the environment: `theseus serve` starts, resolves every referenced secret from the vault, and refuses to start if one is missing. `theseus chat` sends a prompt; the core runs one loop against the Anthropic API and returns the reply; the ledger shows one turn, one loop, `stop_after_one_loop`, and every hook site visited with zero handlers. The same conversation works over stdio and over the socket. The binary is static.

Not yet. No tools. No Discord. No Jev. No store durability guarantees. No context beyond the prompt.

P2b. M0.6 — Exquisite visibility (added 2026-09-25)

Not in the original plan. Eddie's principle, adopted as work before Keel because every later milestone is judged through it.

Build.

- The turn trace (§3.3a): nested spans on every turn, on the result, in the ledger, in failure payloads; waterfall in the web UI; `ask --trace`. (*Done, theseus-8af.*)
- OpenTelemetry as a default projection (§3.20): spans from the trace with exact timestamps, GenAI conventions on provider calls, metrics, OTLP/HTTP with vault-sourced headers, no-op until an endpoint is configured. (*theseus-vng.*)
- A standing rule for every later milestone: a new kind of work (tool call, judgment, completion, compaction, memory pass) lands with its span kind, its attributes, and its metric on the same commit; and a new capability lands as a native toollet unless there is a written reason it cannot (§3.23). Part III records where either slipped.

Prove. With a collector listening, one turn produces one trace whose spans match the ledger's `turn.trace` row exactly in count, names, nesting, and durations; the metrics for that turn arrive; with no endpoint configured, nothing is sent and the turn is no slower. Tested against an in-memory exporter; verified live against a receiver.

Not yet. Logs as OTel log records (the ledger is the log; it can be exported later). Prometheus scrape endpoint (optional, small).

P3. M1 — Keel

Build.

- CI extended to aarch64; the `--tender <role>` entry point that does nothing yet. (Toolchain, `cargo deny`, and the `x86_64` static build arrived in M0.)
- The event record types: `Node`, `Edge`, `Execution`, `Session`, `Compilation`, `Action`, `Completion`, `JudgmentRecord`, `LedgerRow`, with schema version stamps and forward-only migration hooks.
- The `store` trait: append, read-by-id, range-scan-by-position, checkpoint, and a transactional `settle(completion, continuation)` primitive.

- Two store implementations behind a feature flag: `redb` and `fjall`. A benchmark harness with our shape of workload: append-heavy small records with group commit, recent-window scans, id lookups, edge-segment reads, concurrent readers during writes.
- The WAL and spool layout on disk, with checksummed, length-prefixed records and torn-tail truncation.
- The simulator skeleton: virtual clock, deterministic scheduler, scripted fault injection (kill at record boundary, kill mid-fsync, disk full, torn write), and a replay checker that compares the recovered state against the oracle.

Prove. Under a randomized simulated workload, `kill -9` at any point followed by restart recovers every committed record exactly and no uncommitted record; the benchmark picks the store, and the number that picked it is recorded in the spec's §1.

Not yet. No Discord, no Anthropic, no tokio actors per channel, no arena optimization. The arena at this stage is a `HashMap`.

P4. M2 — Kernel

Build.

- Executions as durable objects with the state machine from §3.15 and an authority context (principal, grant, delegation limits, channel ceiling) that derived work inherits.
- Actions with harness-minted correlation ids, `ActionPlanned` → `Dispatched` → `Settled` | `OutcomeUnknown`, per-adapter **retry class**, and resolvable `OutcomeUnknown`.
- The `Completion` envelope and two transports: in-process channel and Unix-socket spool. SQS and loopback HTTP are stubs with the same interface.
- The job wrapper: detached systemd scope (or plain double-fork on the desktop), spooled result, own deadline, cancellable by id, with the cancellation lifecycle `requested` → `acknowledged` → `verified` | `unsupported`.
- The harness loop parked on `select()`, the one-minute heartbeat reconciler, the five-step startup order, at-least-once delivery with idempotent settlement.
- The turn lock per channel with explicit release points at offload boundaries, and a first durability job that runs in released time (checkpointing), so the model is exercised before it matters.
- Sessions as compiler scopes (§3.2a): one execution per session, per-execution turn locks, the admission scheduler, promotion by fork with inherited authority and carved budget, `reports_to` delivery as ordered channel actions.
- Budgets as **hard limits** with reservations, held reservations on unknown usage, and the reserved control-and-cleanup budget. No estimation yet.
- Deterministic policy gate with the ordering from §3.17 (`transform` → `validate` → `policy` → `confirm` bound to the final action → `revalidate` → `dispatch`), without hooks yet; the ordering is what is being tested.
- Ledger rows for every state transition.

- *Added 2026-09-26, before M2 began (v0.24):* Session and Execution records in the store with the per-session position table (§4.4b); the `derived_from` edge written on promotion; the deterministic kernel simulator with virtual clock and fault injection, moved here from M1 (Part III A1); group commit, moved here from M1.

Prove. All kernel scenarios in §8 pass under randomized fault injection: lost completion, duplicate completion, completion during restart, cancel of a detached job, unknown then success, late completion after cancel, crash after settlement before continuation delivery, crash inside each of the five startup steps, two executions on the same task, wrapper deadline with harness down, a promoted task running concurrently with its conversation with messages routed to each, admission ceiling hit while `/cancel` is honored, graceful upgrade with a hundred sessions mid-turn. Reproducible from a seed.

Not yet. No model. The "tool" in M2 is a fake that sleeps and sometimes fails; the "channel" is a simulated mailbox.

P5. M3 — First hands

The narrow agent. One channel binding, one shell class, no intelligence beyond the model.

Build.

- Discord via `twilight`: one application, one guild, DM and one text channel, message send and edit, one component (the confirm button). Bindings as a file.
- Direct Anthropic Messages API with streaming, tool use, prompt-cache layout from §4.5, complete-block-only dispatch, and usage accounting into the ledger. Interrupted-call reservations held as unknown.
- A **model catalog** (`[catalog."<model id>"]`): per model, the serving provider, context window, maximum output tokens, prices per million tokens for input, output, cache read, and cache write, and capabilities (tools, vision, reasoning). A built-in catalog ships in the binary for the Anthropic family (`claude-opus-5-5`, `claude-opus-5`, `claude-sonnet-5`, `claude-fable-5-1`, `claude-haiku-4-5`) and the GLM 5.x models; config entries override or add. Three consumers: a profile that omits `max_output_tokens` defaults to the model's real ceiling; the budgeter uses the context window to decide what fits and when a recompile is forced; the ledger and telemetry turn tokens into dollars. An unknown model id still runs, with cost marked unknown and a startup warning. Prices and limits cannot self-update (the provider's models endpoint lists ids, not limits or prices), so the catalog is a versioned table and every priced ledger row names the catalog version that priced it. Decided with Eddie 2026-09-26; it is config the moment code reads it, and not before (§3.19 rule). Until then `max_output_tokens` on a profile is the only token limit in config, and it is an output cap, never an input one.
- The context compiler in its simplest form: one compilation per session then transcript append; recompile only on the deterministic triggers of §4.4a (no Jev yet); a manifest that records the compilation, the tail range, the as-of position, and the request digest.

- **Toollets, native first (§3.23), from the selected set (§3.24):** the `fs` family (read, write, edit, patch, glob, grep, list), `proc.run`, `text.diff`, and `git.diff/git.log` on the operator's real checkouts, with `proc.run` as the typed, shell-free escape hatch through the L0 job wrapper; `proc.session.*`, `git.commit`, `text.query`, `http.fetch`, `web.search`, `channel.*`, `memory.*`, `node.read`, `wake.at`, and `extend.*` follow in their milestones. No `bash` tool: a shell is `proc.run { argv: ["bash", "-c", ...] }`, visible as such in the ledger. Fast in-process toollets stay synchronous; anything doing I/O past the bound or crossing the process boundary is an action with a completion. `theseus.tool.calls` and the shell-fallback ratio from the first turn.
- The model loop with deterministic control only: `/stop`, `/cancel`, budget exhaustion, confirm.
- The in-binary web UI in its first form: list executions, actions, and ledger rows; tail a channel. Read-only.
- `theseus.restore` from a local WAL directory (S3 comes in M4), because the restore path exists from the first release.

Prove. Eddie completes a real coding task in a known repository from Discord. During a long shell job the harness is killed and restarted; the job finishes, its completion is settled from the spool, the execution continues, and the result lands in the channel. The web UI shows the whole history. A request Eddie is not permitted to make is blocked at the gate with a clear message.

Not yet. No Jev, so promotion to an autonomous task is by explicit human command (`/task`) only. No roles. No memory beyond transcript. No MCP. No voice. No compaction (long conversations simply get a fresh transcript root by hand). This is the discipline Appendix A demanded and the first place we will be tempted to break it.

P5b. M3.5 — Fast (added 2026-09-27)

Not in the original plan. This is Eddie's principle (§2 FAST), adopted after an OpenClaw restart took most of ten minutes. It comes before Boundaries **[D]** because every later milestone adds startup work, and the gate should refuse regressions before they pile up. Beads: `theseus-qa0`.

Build.

- **A lifecycle bench,** `theseus-sim bench lifecycle`, over five phases:
 - cold start to the first `health` answer;
 - clean shutdown with executions waiting and a job running;
 - `SIGKILL`, then restart;
 - a binary swap under load;
 - restore.

Each phase runs N times, with p50 and p95 reported per phase and per startup step.

`scripts/gate.sh` compares the result with §9 and fails on a miss, allowing a noise margin that is itself measured.

- **Serve first.**

- Secrets resolve in the background. Today six concurrent `op read` processes take 1.0 s between them (A3, lifecycle timings); one `op inject` for every reference is the first candidate, and measurement picks the winner.
- The provider, the Discord binding, and the GitHub client each wait for their own secret, and health reports `secrets: resolving | ready | failed <name>`.
- The GitHub token check (0.2 s) runs after serving.
- A failure is loud (health, the ledger, the Observatory) but never delays the socket.
- **Startup phases in the Observatory**, from the kernel's existing step timings plus the new background phases, so a slow start names its cause the first time it happens (EXQUISITE VISIBILITY).
- **Versioned readers for WAL record layouts and manifest formats.** The next format change migrates in a tender, and the M2 practice of moving an old-format store aside (A2) is retired.
- **A standing rule for every later milestone:** new startup work lands with its bench row, and a new on-disk format lands with the reader for the format it replaces, on the same commit.
- *Added 2026-09-29. **Parallel tool calls** (theseus-a60). Eddie, 2026-09-28, 23:57: "we're writing all our core tools in Rust right? I mean I know that sometimes using shells and other expected binaries like git is good practice, but I do want as many things native and performant as possible. Also, I imagine that you often can run things in parallel. My thought is that the most performant way to run trivially parallelizable tasks is to avoid os threads and use truly async code." Today a response's calls run one after another, and in-process toollets run on tokio's blocking pool, whose default ceiling is 512 threads.*
 - Gate every call in the response first. A call under `approve parks`, as today.
 - Run the `open` and `notify` calls of one response concurrently, as async tasks, and answer them in the model's order.
 - Calls on the same path keep their order.
 - CPU work goes to a fixed pool the size of the cores, never a thread per task.
 - `fs.grep` and `fs.glob` use the parallel walker on big trees.
- *Added 2026-09-29. **Frame batching and one transcript read per turn** (the complexity review's findings 3 and 8; theseus-hco, Part III A3b).*
 - A ledger row that is not a state transition rides in the next frame the kernel or the runner already commits, and an action that needs no confirm is planned, authorized, and dispatched in one frame. The review put a plain turn at about 8 frames.
 - A turn reads its session's nodes once, at its start, and keeps the nodes it writes. Today every turn decodes the whole transcript three or more times, plus once per loop.

Prove. The lifecycle bench meets every §9 lifecycle budget at p95, both on Eddie's store and on a synthetic store of 10,000 parked sessions. With 1Password unreachable, the socket still answers inside its budget and health names the missing secrets. A store written in the previous record layout serves immediately under the new binary, and a tender rewrites it while turns run, with no request

failing. The gate refuses a branch that adds a 100 ms sleep to cold start. *Added 2026-09-29*: a response of five reads and two greps takes the slowest call, not the sum; a plain turn writes fewer frames than 17, and the frame-budget test holds the new count.

Not yet. Turn-path latency beyond §9's 5 ms. The arena's memory layout (M6–M7). Anything that needs more than one node.

P5c. The build order after M3 (added 2026-09-27)

These are Eddie's decisions from the openrig and herdr walkthrough (Appendix F). Eddie approved the order on 2026-09-27; the umbrella issue is theseus-5r9. The items run strictly in sequence on `main`, each through the gate. Each is recorded in Part III as it lands.

Note, 2026-09-28 (theseus-8az). Item 1 began as steps 2a through 2a.3: consequence detection in the gate, then three rounds of hardening. They were built, then superseded by theseus-8az, and the gate is now notify over block (§3.9; Part III A3b, the reversal). Step 2b, trusted-channel approval (theseus-sgh), still stands. Step 2c's floor ceremony (theseus-qc4) is moot, since the floor now asks instead of refusing; with no refusal left in the gate, the rest of 2c waits on Eddie (§3.9, Owner override). The list below is the plan as approved.

Note, 2026-09-29 (theseus-5r9). The order now runs as the queue in the chain's working file, `theseus-5r9.md`, shows. Eddie asked for "a long stream of work" at 23:32 on 2026-09-28 (P5d), and the usage audit (theseus-p3k, reviewed 00:27 on 2026-09-29) set its order. The queue is: the complexity cuts (theseus-hco) and Narration (theseus-5fy), both done; M3.6 items 1 to 4, of which dollar budgets and context files are done; complexity batch C (theseus-0g4); item 1's step 2b (theseus-sgh); item 5, M3.5 Fast, with parallel tool calls (theseus-qa0, theseus-a60); M3.6 items 5 to 8; then Eddie's end-to-end testing. After it come item 2's reader-rule test, items 3, 4, 6, and 7, and M4, re-planned. Item 2's hook half went with the hooks: theseus-0dp is moot, and only the reader-rule gate test (theseus-wjy) remains. The list below is still the plan as approved.

1. Irreversible consequences, trusted approval, owner override (theseus-770, theseus-sgh, theseus-qc4; §3.9). *Prove:*

- `proc.run git push --force` in a workspace repository waits for approval under `notify` and under `open`.
- An answer from an untrusted channel or user does not count.
- An owner override of an against-policy call on declared property runs and is ledgered.
- A floor override requires the typed ceremony, and a call on undeclared property cannot be overridden.
- Every detection rule passes its examples.
- An existing config with no `[approval]` or `[owner]` section still loads.

2. Hooks and the reader rule (theseus-0dp, theseus-wjy; §3.17, P0). *Prove:*

- A blocking in-process handler on each gating event stops its action in a scenario.
- The registry test fails a branch that declares an event without a site.

3. **Protocol push** (theseus-in3): `execution.changed`, a watch over all sessions, `session.wait`, and one `attention()` mapping. *Prove:*
 - A client learns every execution transition without polling.
 - `session.wait` returns on `blocked`, `settled`, and `terminal`.
 - A subscriber that falls behind re-snapshots.
4. **The herdr adapter** (theseus-l1l). *Prove:*
 - In a herdr pane, a session waiting on a confirm shows `blocked` within a second.
 - Answering in the pane resumes the session, when the CLI is a trusted channel.
 - `theseus herdr sync` is idempotent.
5. **M3.5 Fast** (P5b, theseus-qa0).
6. **Epidemiology, step 1** (theseus-n4m): reverse compilation membership, a reverse `derived_from` column, `node.reach`, and `reach` shown in the Observatory. *Prove:* `node.reach` returns every compilation and session that included a node, in a core scenario.
7. `theseus tui` (theseus-7yx). *Prove:* from the TUI alone, the operator can see every session's state, jump to the next session that needs attention, approve or decline a confirm (as a trusted channel), and submit input.

P5d. M3.6 — Daily Driver (added 2026-09-29)

Not in the original plan. Eddie, 2026-09-28, 23:32: "I do want to wait until I have an end to end version before extensively testing. Please prepare a long stream of work". The back half of the plan (M4 to M7) was ordered to prove the kernel before features, and the kernel is proven: M0 to M2 are closed, and M3 is built. What keeps Theseus from carrying a day is a short list spread across M5 to M7. This milestone pulls the minimum of it forward, as simply as possible. Its exit test is M7's, started early. Beads: theseus-5jl.

Why these items. The usage audit (theseus-p3k; reviewed 2026-09-29, 00:27) read 30 days of the OpenClaw work Eddie does now.

- 13,719 tool calls: 48 % in his Discord DM, and 43 % in delegated sessions.
- The shell made 69 % of the calls, and the file tools 22 %.
- The DM's median turn took 90 s, and its p95 20 min.
- The DM used about 52 M budget units a day, and Theseus's template then gave a session 20 M, so a session would have ended within hours.
- No MCP server is configured anywhere the main agent runs.

Each item answers one need in those numbers, and the order follows the audit.

Order. Items 1 to 4 run early, before M3.5, because Eddie already uses Theseus from Discord and they fix first-day pain. Items 5 to 8 follow M3.5. Item 9 is optional.

Build.

1. **Budgets in dollars** (theseus-0sg): the limit, the question, and the reset of \$3.13. Eddie's design (2026-09-29, 00:09) replaced the audit's renewing daily allowance. *Prove*: a session at its limit waits and asks, an approved reset sets its spend to \$0 and the waiting call goes ahead with the lifetime cost unchanged, a decline keeps it waiting, and executions stored with unit budgets still serve.
2. **The workspace: context files and roots** (theseus-58a): `context_files` on a profile, compiled into the system block with their digests (§4.4), and `roots` shown in the template. *Prove*: a DM turn follows a rule that only a named file states, without a tool call; an edit to the file causes exactly one `system_changed` recompile; and with `~/.openclaw/workspace` and `~/reports` in `roots`, a spec session's reads and edits run without a confirm.
3. **Quiet notices** (theseus-w4f): under `notify`, the notice rides on the loop's tool message, and the separate embed per call becomes a `[discord]` setting that defaults off. *Prove*: a turn with 30 notified `proc.run` calls posts one tool message, edited in place, and no other message, while the ledger has 30 `tool.notified` rows and the web UI shows each notice.
4. **Attachments and images** (theseus-9g2): the binding reads each attachment up to `[tools].max_read_bytes`, text as user content marked external and an image as an image block for a model with vision, and `fs.read` returns images the same way. *Prove*: a 5,000-character paste (Discord sends it as `message.txt`) is answered from its content, a PNG screenshot is described, and a 20 MB archive is listed as not read, with the reason.
5. `http.fetch` **and** `web.search` (theseus-yd6): native toollets that inherit the posture; a loopback or private address waits for approval. The search backend is Eddie's choice. *Prove*: a fetch of a docs page quotes a fact from it, a search returns ranked results with URLs, and a fetch of `127.0.0.1:7433` waits for approval.
6. **Durable delivery** (theseus-q4v): every post and edit the binding makes is an outbox action whose completion records the Discord message id, replayed in order and once on reconnect; the driver's startup wait for bindings goes. *Prove*: with Discord unreachable during a long turn, the reply posts exactly once when Discord returns, a `kill -9` between send and settle does not duplicate it, and the M3.5 bench shows no driver wait at startup.
7. **Task sessions** (theseus-qn2): `task.create { brief }` opens a child session that inherits the place's authority, takes a carved budget, and reports to the place through the outbox. *Prove*: from the DM, "run the theseus gate and report" returns at once, a second question is answered while the task runs, and after a `kill -9` and restart the task finishes, its report posts once, the parent's next turn quotes it, and the child stays within its budget.
8. `wake.at` (theseus-cff): `wake.at { at | after, note }` sets the kernel's existing `Wake::DueAt` on the current session; one-shot only. *Prove*: "check the build in 10 minutes" gives a turn in the DM ten minutes later, across a daemon restart, and a wake that fell due while the daemon was down runs after startup, marked late.
9. **Optional: secrets in a command's environment** (theseus-dcy): `[tools].proc_secrets` maps an environment variable to a `[secrets]` name, injected at spawn and scrubbed from output. Taken only if the `op` waits prove tiresome in testing. *Prove*: `gh api user` runs under `notify` with `GH_TOKEN` from the vault, with no `op` call and no approval, and the token appears in no node.

Prove. Eddie carries his daily Discord work on Theseus end to end, on one channel beside OpenClaw, and each item's prove passes. The exit test's config needs no code: the roots, the context files, `allow_argv` for the frequent read-only programs, the DM's profile, and the limit from item 1. Theseus has its own bot, so its DM runs beside Tabitha's, and OpenClaw leaves the loop only when Eddie says so.

Not yet. Two items stay in M7, per the audit's "Challenged" list:

- The MCP client. No MCP server is configured anywhere the main agent runs, and vestige comes in through an HTTP plugin.
- Recurring schedules. Every recurring automation posts to Slack, reads Google Workspace, or maintains OpenClaw's or vestige's stores, and the DM's own scheduling was mostly one-shot (6 of 8).

Slack, the automations, and the heartbeat stay in M7 too, and on OpenClaw until then. Voice, AWS shells, multi-guild bindings, Jev packs, memory science, and self-extension stay in their milestones.

P6. M4 — Boundaries

Make the durability and safety claims true, and measure them.

Build.

- The durability tender: WAL segments to S3, index rows to DynamoDB, scheduled in released turn-lock time by staleness; the "oldest unshipped committed record" metric and alarm.
- `theseus restore --from s3://...` with reconciliation near the gap and redaction tombstones applied before restored content becomes visible. Redaction with receipts (`erased_local`, `pending_backup`, `external_copies`).
- L1 native sandbox with the §7 contract, and **contract tests** that prove each denial: no route to the metadata service, no route to localhost services including Theseus's own UI, capabilities empty, seccomp active, process tree killed on cancel.
- Confidentiality labels on nodes with inheritance through generated nodes; audience-safe compilation; disclosure tests in the simulator (private material never reaches a public audience's context).
- Control-plane separation as an installer option: dedicated `theseus` user owning store, WAL, spool, and policy; L0 jobs as the operator.
- Cancellation verification per backend (systemd scope, L1 process tree), and `cancel_unsupported` reporting.
- The ontology (§4.1a, `theseus-8kk`): the kinds table, declared memberships, guidance, and the compile walk, with topics as the first new kind.
- Integrity labels by transmission, and the one-step-stricter rule for exposed contexts (§3.9 Exposure). Also the `external` origin, file hashes, and the `Advisory` with its correction control (`theseus-3vu`).
- The consequence boundary under L1 (§7): credential brokering and egress recognition.

Prove. Every row of the durability table is demonstrated by a test: process crash, node restart with disk intact, SSD loss with restore from S3, external effect without evidence. The measured off-node recovery point under a synthetic load is under 60 s at p99 and the turn-latency cost of the durability work is reported. L1 contract tests pass. Disclosure tests pass.

Not yet. No AWS shell classes. No hook handlers beyond tests; the hook points themselves are wired before M4 (P5c). No Jev.

P7. M5 — Judgment

Jev enters, in shadow first, and hooks arrive because Jev packs are the first real hook handlers.

Build.

- Jev client with the confidence gate, state capping, batching by the budgeter, latency measurement per workload class, and every call accounted as spend.
- Question packs: `CLASSIFY` (new ask vs nudge vs control vs addressed-to-task, and when a conversation should promote to an autonomous task), `JUDGE_STOP`, `ROLE_GUESS`, `CONTINUE` (append or recompile, and how). All in **shadow**: recorded, compared with the deterministic baseline, never acting.
- The learning ledger with correct labels (`budget_exhausted` its own class), holdout split, and canary promotion of a pack from shadow to live when it beats the baseline.
- Hooks: `Gate`, `Transform`, `Claim`, `Observe` kinds with fail-closed gates, typed observer results, ordering before final authorization, ledger rows per invocation. Compiled-in handlers first; remote handlers over the protocol may observe.
- Roles table with the twelve seed rows, announced role changes, roles as hints in the compiler.
- Executions gain `waiting` on Jev recovery and provider outage (fail closed, say so).
- Task sessions and promotion, with the required arrangement (§3.2a) and pieces admitted by reference.
- Independence as a compiler property, obligation invariants, honest delivery receipts, and `relies_on` in shadow (theseus-vug).
- `categorize.v1` in shadow (§4.1a).

Prove. On a held-out set of recorded trajectories, Jev-driven stopping and classification beat the deterministic-only baseline at equal total budget (judge cost included) on task success, false completion, and unnecessary continuation. Any pack that does not beat baseline stays in shadow and the plan says so.

Not yet. No memory science. No MCP.

P8. M6 — Memory as experiment

Build.

- The context graph beyond transcript: typed edges, roots, compaction roots (append-only, with `derived_from`), rotating ring, assembled continuation. `CONTINUE` goes live for recompile-strategy choice if M5 said it could.
- The index tender: Nomic v1.5 embeddings (768 stored, 256 indexed), usearch, tantivy, reciprocal-rank fusion. Baseline retrieval: transcript tail + task graph + summaries + BM25/embedding + freshness and provenance rules.
- `MemoryScience` trait with a **baseline** implementation (no retention model, no activation) and a native FSRS-6 + prediction-error + spreading-activation implementation behind it.
- The memory pass and recall as specified, consolidation as a tender job producing shadow syntheses with citation checks.
- Tiering tender: demote by heat, rehydrate on reference; arena as a bounded cache with the presence filter.
- The ablation harness: each feature toggled independently at fixed total budget, scored on §5.5a's metrics over recorded trajectories and a live canary.
- The ontology's learning half (§4.1a): category embeddings, re-association by sweep and dream, lessons as guidance, and §5.5's namespaces as kinds. Also compilations that are never silently thinner, testimony and precedence, and volatile values rendered as-of (theseus-3nk).

Prove. An ablation report exists and is honest. Features that do not move task success, false completion, stale recall, or disclosure violations at equal cost are disabled by default and marked experimental in the spec.

Not yet. Voice, MCP, AWS shells, multi-channel gliding.

P9. M7 — Surface

Build.

- Multi-guild, multi-channel bindings; per-channel ceilings; gliding with intersected ceilings; coalescing with per-author authority; proactive and scheduled work under derived authority and owner grants; `wake` nodes.
- Tasks fluid in chat with the three mutability layers, CAS, claim leases, workspace locks.
- MCP client (tools, prompts, elicitation in), then MCP server on localhost with the static key, sampling budgeted and Jev-judged.
- Voice: `songbird` receive and send, STT and TTS as accounted spend, the voice turn as a workload class in the Jev latency budget.
- AWS shell classes A1–A4 with scoped task roles, SQS completion transport live, EventBridge task-state changes into the queue, reconciler API polling only past deadline.
- Web UI grows: in-thread observability, policy mapping administration, budgets, ledger views, learning channel.
- Self-extension (§3.21): `extend.propose`, the operator ack, hot-loading a sandboxed MCP server as a tool, revocation.

Prove. Theseus carries Eddie's daily Discord work end to end for two weeks with OpenClaw out of the loop for that channel, with the ledger showing budgets, judgments, and hook runs, and no disclosure or authority violation in the record.

P10. What is cut from the first useful agent, on purpose

Jev, roles, memory science, compaction, MCP, voice, AWS shells, hooks, multi-channel. M3 is a Discord front end on a durable execution kernel with a bash tool. If that is not already useful for coding in a known repo, the intelligence features will not rescue it; if it is, every later feature has a baseline to beat.

P11. Risks the plan is built around

Risk	Where it bites	Mitigation in the plan
Kernel rewrite after simulator findings	M2	Simulator exists before the kernel does (M1); the fake tool and mailbox keep the rewrite cheap
Integration work starves the kernel	M3	Discord and Anthropic are not started until M2 is green
Durability work steals turn latency	M4	Measured explicitly; the turn lock releases only at offload boundaries, and the tender is a separate process
Jev does not beat the baseline	M5	Shadow first; a pack that loses stays in shadow and the spec says so
Memory science does not transfer	M6	Baseline first, ablations gate every feature
L0 default proves unsafe in practice	M4 onward	L1 ships with contract tests in M4 so switching the default is a config change, not a project
Embedded store becomes the bottleneck	M6–M7	The <code>Store</code> trait and the M1 benchmark harness make the swap a bounded project

P12. Immediate next steps

1. Beads epics `theseus-9w9` (M0) through `theseus-ext` (M7) exist in the theseus repo, chained by dependency, each carrying its "prove" line.
2. M0 first steps: workspace crates, `rust-toolchain.toml`, `cargo deny`, the 1Password config loader, the hook registry, the turn runner, the protocol server, `theseus chat`.
3. Repository: `~/projects/theseus`, `github.com/zeroaltitude/theseus` (decided). This document lives there as `docs/the-ship-of-theseus.md` alongside the design notes.

Part III — As Built

The record. Each milestone gets a section when it closes: what exists, how it is proven, and a divergence table against Parts I and II. Entries are dated. Nothing here is aspirational; if it is not running, it is not in this part.

A0. M0 First light (closed 2026-09-25, theseus-9w9) and M0.5 Visibility (theseus-rbj, theseus-l32)

What exists. Repository `github.com/zeroaltitude/theseus`, dual-licensed MIT OR Apache-2.0, Rust 1.98.1 stable, static `x86_64-unknown-linux-musl` release builds (ring's C compiled by musl-gcc), `cargo deny` with a permissive-only allowlist, CI on every push (fmt, clippy `-D warnings`, tests, deny, web build with a dist diff, static build, artifact upload).

Workspace crates:

Crate	Role
<code>theseus-protocol</code>	Wire types only: JSON-RPC 2.0 over newline-delimited JSON; every payload struct. No runtime, no core dependency.
<code>theseus-core</code>	The kernel library: config, secrets, hooks, sessions, turn runner, Advancer, provider, store, ledger, RPC server.
<code>theseusd</code>	The server binary: daemon on a Unix socket, <code>--stdio</code> when spawned, embedded web UI, <code>check</code> , <code>example-config</code> .
<code>theseus</code>	The CLI binary: links only the protocol crate.

Protocol surface. Requests `health`, `session.open`, `session.list`, `turn.submit {session_id?, input, provider?, model?, hooks.list, hooks.register, hooks.unregister, ledger.tail {n?, kind?, session_id?}, shutdown`. Notifications `turn.started`, `loop.started`, `model.delta`, `loop.ended`, `turn.ended`, `hook.event`. Errors carry a JSON-RPC code and, for provider failures, `error.data {class, transient, usage_unknown, turn_id, session_id, elapsed_ms}`. Every connection has one ordered outbound queue, so a turn's notifications always precede its response.

Transports. Unix socket at `~/.theseus/theseus.sock` (mode 0600, stale-socket detection, refuses to steal a live one); stdio; WebSocket at `127.0.0.1:7433/ws` bridged through an in-memory duplex so the browser is an ordinary client.

Configuration and secrets. TOML, read by default from the 1Password item `op://Eddie-Tabitha/theseus-config/notesPlain`, or from a file via `--config/THESEUS_CONFIG`. Only the service-account token enters the process outside 1Password (env or a mode-0600 file). Every `[secrets]` reference resolves concurrently at startup through the `op` CLI or the process refuses to start, naming

the failing references. References accept a `#label` suffix selecting one `label: value` line of a multi-line note. Values live in zeroizing memory; `Debug` never prints them. The GitHub token is checked at startup (login, expiry, days left; warn under 30).

Providers. `[providers.<name>]` entries speaking the Anthropic Messages API, the implicit `anthropic` from `[model]`, `zai` at `https://api.z.ai/api/anthropic` in the example config. Streaming client with typed content blocks and stream events, tool input assembled at block stop, rate-limit headers, request id, first-byte/first-token/total timing, four timeouts (connect 10 s, first byte 60 s, stream idle 60 s, total 600 s), classified `ProviderError` with `transient` and `usage_unknown`, no automatic retry.

Kernel. Sessions as records with one turn lock each (re-read under the lock, so concurrent turns never lose updates); a toolchain manager that compiles the prompt alone and offers no tools; the `Advancer` trait with `stop_after_one_loop` active and `until_no_tool_calls` implemented but unused; 24 hook events with kinds Gate, Transform, Claim, Observe, every site visited per turn and ledgered, remote Observe handlers over the protocol; redb store with `sessions` and `ledger` tables; ledger rows for server start/stop, session open, turn start/end/fail, loop start/end, provider call/error, hook site visits, hook registration.

Accounting. Tokens in, out, cache read, cache write, first-token and total latency, provider request id per turn; cumulative per session; totals, provider-error count, and ledger size in health; `theseus ledger`, `theseus sessions list`, `theseus health`.

Web UI. Vite 8 + React 19, source in `web/`, built dist committed and embedded with `rust-embed`, loopback bind enforced, no auth. Prompt box and submit, streamed replies, per-exchange footer (provider/model, loops, stop reason, tokens, timing, request id), session and global totals, classified error display, collapsible event log per turn.

Proof. `scripts/smoke.sh` against the real API on debug and static binaries: check, health, hooks, streamed ask, piped `--json`, web served, ledger, stdio spawn mode, shutdown. 26 unit tests: SSE assembly, tool-input assembly, error classification, timeout phases, hook registry, secret reference parsing and line selection, Advancer policies, and RPC over an in-memory duplex (ordering, streamed deltas, every hook site ledgered, error codes, remote observer, same-session serialization, provider failure classification, usage accumulation, ledger tail, per-turn provider selection, parse errors). Live: provider connect and first-byte timeouts against a refused and a hanging endpoint; Z.ai reached and classified `rate_limited` (account unfunded).

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
One binary with subcommands (§3.18 originally)	Two binaries: <code>theseusd</code> and <code>theseus</code>	Eddie asked for a server paired with a shell-friendly CLI	Part I §3.18 updated; kept
Config item created in 1Password by Theseus	Created by Eddie by hand	The service account is read-only and cannot create items	Part I §3.19 updated: Theseus never writes to the vault

Planned	Actual	Why	Disposition
1Password read through an SDK	Shell out to the <code>op</code> CLI	No first-party Rust SDK; community FFI wrappers unproven on musl	Open: revisit when a wrapper builds statically
<code>#label</code> selection did not exist	Added to <code>op://</code> references	Real vault items are multi-line notes	Part I §3.19 updated; kept
M0 had no web UI, no provider table, no timeouts, no usage rollups	All four built as M0.5 the same day	Eddie wants visibility and a failure story from the first version	Part I §3.6, §3.13, §3.14 updated; plan not rewritten
One store per node	stdio mode uses <code>theseus-stdio.redb</code>	redb is single-process; a spawned server must not fight the daemon	Kept for now; M1 Keel decides the store layout
L1 shell not in M0	Still not built	As planned	none
Continuing a session carries context	Prompt only	As planned for M0; transcript continuation is M2	none
Web UI authenticated by Discord OAuth	No auth, loopback only	First form; auth arrives with bindings in M7	Part I §3.14 says so
musl build via musl-tools from the start	Host gcc first, musl-gcc after Eddie provided sudo	No passwordless sudo on the node	Resolved

Added after M0.5 (theseus-rfl, 2026-09-25): profiles. `[profiles.<name>]` with provider, model, `max_tokens`, system; the implicit `default` profile is built from `[model]`; `[model].live` names the startup profile; `profile.use` switches at runtime and persists in the store's `meta` table (a persisted switch wins over config on restart unless it names a profile that no longer exists, in which case config wins and a warning is logged); `profile.list` reports the live name and whether it came from config or runtime; `turn.submit.profile` runs one turn under another profile; the CLI has `theseus profile list|use` and `ask -P`; the web UI has a live-profile selector in the header. Every turn result and ledger row names its profile. Test: switch, route, explicit override, persistence across a fresh core over the same store.

Added the same evening (theseus-8af): the turn trace. `Trace` builder in the core (enter/exit/mark/record/finish), `span` in the protocol, every hook site visit recorded as a span with its handler count and outcome, provider calls as spans with first-byte and first-token marks and request id, usage, and stop reason as attributes, the advancer decision, the lock wait, and the session write. On the result, in a `turn.trace` ledger row, and in `error.data.trace` for failed turns. Web UI: the timing link opens a waterfall with a per-kind summary and click-for-attributes; CLI: `ask --trace`. Test asserts the tree's shape. Observed on the first real turn: 1.35 s total, of which 1.25 s was the provider (first byte at 704 ms, first token at 846 ms) and 4.6 ms the session write; every hook site under 15 µs.

Added the same night (theseus-vng): OpenTelemetry as a default projection. `core::telemetry:` OTLP/HTTP (protobuf, reqwest + rustls) span and metric exporters built only when `[telemetry].otlp_endpoint` is set, otherwise a no-op that costs nothing; the finished turn trace is walked into OTEL spans with the recorded timestamps (root placed by `origin_unix_ms`, which the trace

now carries); provider calls are `client` spans with the GenAI attributes (`gen_ai.provider.name`, `gen_ai.request.model`, `gen_ai.response.id`, `gen_ai.response.finish_reasons`, `gen_ai.usage.*`); hook sites, marks, compile, store, lock, and advancer are events on their parent unless `hook_spans = true`; failed turns export with error status and the partial trace; metrics `theseus.turns`, `theseus.tokens`, `theseus.provider.errors`, `theseus.turn.duration_ms`, `theseus.provider.call.duration_ms`, `theseus.provider.first_token_ms`; headers from a vault secret; resource `service.name/version/instance.id`; flushed on shutdown; health reports the endpoint. The upstream semantic-conventions crate deprecated its GenAI constants (they moved to a separate repository), so the attribute names are pinned locally. Five tests against in-memory exporters (nesting, parent ids, exact timestamps, events vs spans, error status, metric names, header formats, disabled no-op). Dependency cost: ~185 crates in the core against ~140 before.

Reversal, 2026-09-26: WASM out. Earlier drafts named WASM components (`wasmtime`) as the hot-loadable plugin form and even scheduled a "WASM plugin ABI" for M7. Eddie asked why; the honest answer was that it bought microsecond starts and fine-grained capability grants we have not shown we need, at the cost of a very large dependency and toolchain churn, while the L1 sandbox plus MCP, both already specified, give the same isolation and hot loading for free. The single runtime extension path is now an MCP server in an L1 sandbox; WASM returns only if measured per-tool process cost demands it. Recorded here so the reasoning survives.

Finding, same day: the vault's `z.ai` key item is not the key OpenClaw uses. Fingerprints differ; the vault key returns 429 code 1113 (insufficient balance) and the OpenClaw key (`models.providers.zai.apiKey`, shared by every agent including Tank) returns 200 with a GLM reply. Eddie to update the vault item; Theseus reads only from the vault by design, so no GLM reply has been observed through Theseus yet.

Known gaps carried forward. Continuing a session sends the new prompt only. Remote hook handlers can observe but not gate or transform. Cost in dollars is not computed (tokens only; a pricing table per model is a later addition). The web UI has no model selector yet (the CLI has `-p/-m`). The `Z.ai` account has no balance, so no GLM reply has been observed end to end.

A1. M1 Keel (closed 2026-09-26, theseus-cfl)

What exists. Crate `theseus-store`: `NewRecord/Record` with kind and schema version (kinds: session, ledger, meta, execution, action, completion, node, edge, compilation, judgment, checkpoint); `wal` with 64 MB segments, frames `MAGIC | len | crc32 | body{count, records}`, one `write_all` and one `fdatsync` per frame, a `max_total_bytes` cap that fails appends cleanly (the disk-full case), recovery that verifies every frame, truncates a torn tail only in the last segment, and refuses (`Corrupt`) if an earlier segment is damaged; `Index` trait with `redb` and `fjall` implementations over four tables; `walstore` composing them with atomic `append/settle`, `get`, `scan`, `latest_by_key`, `latest_of_kind`, `tail_of_kind`, `count_of_kind`, `checkpoint`, `stats`, a manifest that pins the engine a directory was created with, and index rebuild from the WAL past the checkpoint on open. Crate `theseus-sim`: `worker` (appends random 1–4-record frames of 16 B–32 KB, printing each committed position and payload `crc` only after the append returned), `crash-test` (spawns the worker, SIGKILLS it after 15–250 ms, in 70 %

of runs tears the WAL tail by truncation, junk, or a flipped byte, reopens, verifies every reported record byte-for-byte, contiguity, and that appends continue; three restarts per iteration), and `bench`. The M0 core now writes through `theseus_store ([server].store_engine, default redb)`, checkpoints on shutdown, and reports engine, last position, and WAL size at startup.

Proof. 47 workspace tests, including the WAL round trip, torn tail truncated with earlier frames intact, corrupt middle frame refused, two-record frame atomic (both or neither), full WAL refusing cleanly, both indexes, both stores, and index loss rebuilt from the WAL. The exit test: `theseus-sim crash-test --iterations 40 --restarts 3` on each engine, 120 kill-and-recover cycles per engine with tail tearing, **zero committed records lost**, 255 KB and 141 KB of torn bytes removed, contiguity held every time. Smoke test green on the migrated daemon. Static musl build green.

Benchmark (this node, WSL2, one writer, 20 k–50 k records):

	redb	fjall
append, fsync per frame	142 frames/s, 7.1 ms/frame	145 frames/s, 6.9 ms/frame
append, no fsync	17.8 k frames/s	30.5 k frames/s
get by position	374–381 k/s	279–351 k/s
latest by key	428–452 k/s	475–521 k/s
newest-50 of a kind	8–11 k/s	8.7–10.3 k/s
latest of every key (50 sessions)	0.12 ms	0.89–2.29 ms
reopen with 950 records to replay	68–138 ms	129–289 ms

Decision. `redb`. The fsync floor of this disk is about 7 ms per frame and both engines sit on it; where they differ, `redb` reopens twice as fast and answers the "current state of every entity" query the kernel asks constantly an order of magnitude faster, and it is one file with no background threads. `fjall`'s raw append advantage is real and would matter only once group commit removes the fsync-per-frame floor.

Divergence from Part II M1.

Planned	Actual	Why	Disposition
Simulator with virtual clock, deterministic scheduler, replay checker	A real-process crash harness: SIGKILL of a worker, tail tearing, byte-for-byte verification	The store's exit test needs real fsync and real kill semantics, which a virtual clock cannot give; the deterministic kernel simulator belongs with the kernel it simulates	Kernel simulator moves to M2; the crash harness stays as the store's standing test
Group commit on a short interval	One fsync per frame	One writer today; group commit needs concurrent appenders to batch, which arrive with executions in M2	M2
CI on aarch64	x86_64 only	Not needed yet; noted so it is not forgotten	M2 or later

Planned	Actual	Why	Disposition
Record types for Execution, Action, Completion, Compilation, Judgment as structs	Kinds reserved; structs not yet defined	They are the kernel's (M2) types; reserving the kind numbers now avoids a migration	M2
"The arena at this stage is a HashMap"	No arena at all; reads go index → WAL	Nothing needs a resident graph yet; latency is already sub-3 μs per get	M2/M5
Disk-full handling	<code>max_total_bytes</code> cap on the WAL, tested	A real ENOSPC test needs a small filesystem; the cap exercises the same code path	Real ENOSPC in the kernel simulator

Known gaps carried forward. Positions are read one index lookup at a time in `scan` (fine at this size; a sequential frame walk is the obvious optimization). `latest_of_kind` decodes every latest record; a count-only path exists. The WAL never compacts or archives segments (append-only is the model; tiering to S3 is M4 Boundaries).

A2. M2 Kernel (closed 2026-09-26, theseus-2p3)

What exists. A `theseus-kernel` crate, synchronous and deterministic (time from an injected `clock`; the only process state is the set of turn locks held), driving the M1 store through the `Store` trait, and wired into the daemon so that every live turn is a kernel turn.

- **Executions** (§3.15) as durable records keyed by id: `queued` | `running` | `waiting` | `blocked` | `cancelled` | `failed` | `budget_exhausted` | `complete`, with an `Authority` (principal, delegation, ceilings intersected on fork and never widened), a `Budget` (limit, spent, reserved, held-unknown, control reserve, open reservations), a `Wake` (`due_at`, `actions`, `execution`, `confirm`, `input`), `outstanding` and `queued_results` correlation ids, turn and interruption counters. One execution per session; the core's `SessionRecord` carries its id and sessions written before M2 get one on their next turn.
- **Actions** (§3.16) keyed by harness-minted correlation id: `planned` → `authorized` → `dispatched` → `succeeded` | `failed` | `outcome_unknown` | `cancelled`, each transition its own frame committed before the next step (the `dispatched` frame is the transactional outbox). Every action declares a **retry class** (`safe_to_repeat`, `idempotent_with_key`, `recoverable_by_external_id`, `non_repeatable`), a deadline, the sha256 digest of its canonical proposal, and an optional **confirmation binding** (confirm id, principal, bound digest, expiry). `outcome_unknown` is resolvable: later evidence settles it and is ledgered as a resolution.
- **The Completion envelope:** one type from every producer (`inproc`, `provider:<name>`, `wrapper:<pid>`, `reconciler`, `sim`). Accepting one is **idempotent and atomic with continuation**: the completion record, the settled action, the execution's next state (result queued; woken if it waited on this action), and the ledger row are one frame. A duplicate is a logged no-op; a completion for no action is quarantined under its own key and counted in health; a late completion after cancel is recorded as a resolution and revives nothing.

- **The gate** (§3.17 ordering): `transform` → `validate` → `policy` → `confirm` bound to the final digest → final revalidation at `authorize` → `dispatch`. The kernel enforces the ordering by digest: any change to the proposal after planning or after the confirm was bound fails `authorize` with `ConfirmInvalidated`; an expired or wrong-principal confirm likewise. Policies are a trait with a permissive default; M2 tests the structure, not a policy.
- **Budgets as hard limits.** A reservation is taken in the `planned` frame; a known result converts it to spent; unknown usage moves it to `held_unknown` and it is never released; a later resolution moves held to spent. Exhaustion is terminal and is written in the same frame as the refusal, so nothing keeps spending against a dry budget. A control reserve is kept back so a cancel and a final report are always affordable.
- **Admission and the turn lock.** `admit` takes a turn only for a `queued` execution, only under the ceiling (`[kernel].admission_ceiling`), only once per execution per process; it writes `running`. `end_turn` records the Advancer's decision (`complete`, `wait(wake)`, `requeue`, `fail`, `blocked`); a terminal state that landed during the turn (cancel, exhaustion) is never overwritten; ending with actions still outstanding requests their cancellation in the same frame. Waiting for admission in the daemon is a notify-and-retry, not a poll, and appears in the turn trace as `admission.wait`.
- `/cancel` acts directly on the execution and marks every dispatched action `cancel_requested`, returning the list the harness must terminate; the daemon kills wrapper process groups by pid from the spool and walks `acknowledged` → `terminated_verified` | `unsupported` | `outcome_uncertain`. Never routed through admission or a judge.
- **Promotion** (§3.2a): `promote(parent)` forks a task execution with derived authority (widening refused), a budget carved from the parent's available units, `reports_to` the parent, and the `derived_from` edge written in the same frame under the child session's scope; a parent waiting on `Wake::Execution` wakes when the child ends.
- **The spool** (`<state_dir>/spool`): completions as `tmp` → `fsync` → `rename` files, pid files, captured results, a `malformed/` quarantine; `drain` returns them oldest first and the kernel removes each file only after its frame committed, so a crash between the two redelivers a duplicate, which is a no-op.
- **The job wrapper** (`theseusd job-wrapper`): the daemon spawns itself detached in a new session (`setsid`), the wrapper runs the command, enforces its own deadline, captures output to the spool, writes the completion durably, then pokes the harness over `spool/notify.sock`. `WrapperEvidence` answers the reconciler from the spool and pid files (`completed` / `still running` / `gone`).
- **The harness loop** (`core::harness`): parked on `select()` over the heartbeat timer, the notify socket, and shutdown; store work runs off the reactor. **The reconciler** wakes due executions, checks only *overdue* dispatched actions against evidence (settling or marking unknown), re-probes unknowns, and writes one `reconcile` ledger row when anything changed.
- **Five-step startup**, each idempotent and ledgered with its duration: store recovered → executions found `running` requeued as interrupted → spool drained → reconciled → accepting. Events are refused before step 5. A test hook lets the simulator kill the process after any step.
- **The live turn.** The provider call inside every turn is now an action: `provider.messages` is planned (reservation = `max_output_tokens` + input estimate), authorized through the gate, dispatched, then settled with real token usage; an interrupted call settles as `outcome_unknown`

with its reservation held. New trace spans `admission.wait`, `action.outbox`, `action.settle`; new ledger kinds `execution.*`, `action.*`, `completion.*`, `budget.*`, `startup.step`, `reconcile`.

- **Store changes for the kernel.** Records gained a **scope** (record layout v2; manifest format 2, format-1 stores from the one day they existed are moved aside with a warning) and the index a per-scope position table, so a session's own records are one range scan (§4.4b). **Group commit** in the WAL: concurrent appenders write back to back under a short lock and one of them syncs for everyone whose bytes are written; counters for frames and syncs ride in `stats`.
- **Surface.** Protocol: `execution.list`, `execution.cancel`, `health.kernel` (accepting, ceiling, turns held, counts by state, quarantined, last startup report), `session.*.execution_state`. CLI: `theseus executions [list|cancel <id>]`; `theseus health` prints a kernel line. Config: `[kernel]` with six keys, all in the tested template. `theseusd` logs each startup step's microseconds and the store's frame/sync counts.

How it is proven.

Scenario tests (18, `theseus-kernel/src/tests.rs`, real `walstore` + `redb` in a temp dir, virtual clock): full lifecycle in one frame per transition; duplicate no-op and stray quarantine; crash mid-turn requeues as interrupted and a second startup is a no-op; completion during restart lands in the spool and startup drains it, a second copy is a duplicate; **crash inside each of the five startup steps** recovers on the next startup with the requeue counted once; cancel of a dispatched job then a late completion does not revive; unknown then genuine success resolves and moves held budget to spent; overdue action with spooled evidence settles from the reconciler and a not-yet-overdue one is not polled; a due wake fires only at its time; confirm binds the final action and any change, wrong principal, or expiry invalidates; budget is a hard limit and exhaustion is terminal; admission ceiling holds and `/cancel` is honored immediately; promotion derives authority, carves budget, writes the edge, wakes the parent; a wait on already-settled actions does not park forever; a hundred sessions mid-turn survive an upgrade (new kernel over the same store) and every one takes its next turn.

The deterministic kernel simulator (`theseus-sim kernel-sim`): virtual clock, real store and spool in a temp dir, a fake tool whose jobs finish after a scripted delay, never, twice, or after cancel; seeded fault injection of a crash **between any two kernel frames**, a crash inside a random startup step on 30 % of restarts, lost notifies (spool only), duplicate deliveries, lost jobs, and `/cancel`s; the kernel invariants checked after every step (running count equals turns held and is under the ceiling; waiting has a wake and queued has none; budget sums; every outstanding id is an unsettled action of that execution; no duplicate queued results; terminal with outstanding only as cancel-in-flight; dispatched actions are outstanding; quarantine holds only unminted ids), and at the end every durable completion settled its action or was recorded as late, every lost job ended unknown or cancelled, nothing left dispatched, nothing held.

Run	Result
40 seeds × 400 steps, redb	298 crashes (140 inside startup steps), 2 348 turns, 2 225 actions, 1 447 completions (203 duplicates, 335 notifies lost, 164 lost jobs, 72 late after cancel), 389 cancels, 197 unknown → 115 resolved, 16 040 invariant checks, all held , 45 s

Run	Result
1 seed × 2 000 steps, 30 sessions, ceiling 4, p(crash) 0.05	58 crashes, 190 turns, 174 actions, all invariants held
5 seeds × 400 steps, fjjall	26 crashes, all invariants held
1 seed × 300 steps, fsync on	all invariants held

The simulator found two real bugs before any test did: a turn ending `complete` with earlier actions still outstanding leaked them (now a cancel request in the same frame), and a completion for an action whose execution had already reached a terminal state settled the action but left it listed as outstanding (now always removed; only wake and result-queueing are skipped for terminal executions).

Group commit (`theseus-sim bench --writers N, redb, fsync on`): 1 writer 146 frames/s with one sync per frame (unchanged from M1); 4 writers 304 frames/s, one sync per 2 frames; 16 writers 1 019 frames/s, one sync per 7.6 frames; 16 writers with group commit off, 146 frames/s.

End to end (`scripts/smoke.sh`, real API): a turn produces `execution.running`, `action.planned/authorized/dispatched/succeeded`, `execution.waiting`; `theseus executions` shows the execution with its budget spent; a stray wrapper completion (`theseusd job-wrapper ... -- /bin/echo`) is quarantined and counted, never inferred into anything; the stdio server runs on its own store and spool; 67 workspace tests, clippy clean, licences clean.

Divergence from Part II M2.

Planned	Actual	Why	Disposition
Two completion transports (in-process channel, Unix-socket spool) plus SQS and loopback HTTP stubs	In-process (the provider call and the tests) and the spool with a Unix-socket notify; no SQS or HTTP stubs	Nothing would call a stub yet; the <code>Completion</code> type and <code>accept_completion</code> are the interface they will share	M4 Boundaries
Job wrapper as a systemd scope (or double-fork on the desktop)	<code>setsid</code> in a spawned child of the same binary, killed by process group	Same lifetime independence without a systemd dependency; a scope is a later hardening on hosts that have systemd	M4
Turn lock per channel with explicit release points at offload boundaries and a first durability job in released time	Turn lock per execution (§3.2a supersedes the per-channel wording); release points exist (the turn ends <code>waiting</code> on a dispatched action) but no durability job runs in released time yet	The checkpoint still happens every N records and at shutdown; the tender scheduler that uses released time is M5	M5 Tenders
Admission scheduler with budget and provider-rate-limit inputs	A concurrency ceiling only, FIFO by who asks	Rate-limit signals exist on the provider response but nothing feeds them back yet	M3/M4

Planned	Actual	Why	Disposition
Deterministic policy gate "without hooks yet"	As planned; policies are a trait, hooks do not participate in the kernel gate yet	The core's hook sites and the kernel gate meet when tools arrive	M3
reports_to delivery as ordered channel actions	reports_to recorded; no channel exists to deliver into	M4	
Sessions as compiler scopes with the per-session position table	Table built and used by session_tail; the compiler does not exist	M3 Context	
Kernel simulator: fake Discord, fake provider, fake Jev, scenario files	A fake tool, seeded random operations, in-code invariants; no scenario files	The kernel has no Discord, provider, or Jev to fake; scenario files arrive with those pieces	M3+
Group commit "on a short interval"	Leader-follower: the first appender past the write syncs for all bytes written so far; no timer	A timer adds latency for a single writer; the leader pattern gives one sync per frame alone and batching under load with no tuning	Keep
CI on aarch64 (carried from M1)	Not done	Still not needed	Later
Real ENOSPC in the simulator (carried from M1)	Not done; the WAL cap is still the disk-full path	Needs a small filesystem in CI	Later

Added the same day (theseus-92j): the Observatory. Eddie's rule, stated when M2 closed: everything a milestone writes must be transparently observable in the web UI before the next milestone starts. The web UI gained a panel beside the chat that is nothing but live windows onto the store, each a protocol query (`health`, `execution.list`, `action.list` (new), `ledger.tail`, `session.list`) re-run every 2.5 s and after every turn: **Kernel** (accepting, turns held over the ceiling, executions and actions by state, quarantined completions, the last startup's five steps with their microseconds and what each recovered); **Executions** (state, turns, interruptions, outstanding and queued counts, a budget bar of spent / reserved / held-unknown over the limit, a cancel button on the deterministic control path, click to filter everything below to that execution); **Actions** (tool, state, retry class, planned → dispatched → settled with the two intervals in ms, reserved units, completions seen with duplicates flagged, overdue flagged, the resolution note); **Ledger** (family chips `execution.*`, `action.*`, `completion.*`, `budget.*`, `turn.*`, `loop.*`, `provider.*`, `hook.*`, `startup.*`, `reconcile`, `session.*`, a this-session filter, a one-line summary per row and the full JSON on click); **Sessions** with each one's execution state. A restart reads as `server.stopping`, five `startup.step` rows, `server.started`, with the requeued and drained counts in the summary. Nothing in the panel is computed in the browser from events, so it shows what a restarted daemon would show. The web UI gap noted below is closed.

Known gaps carried forward. No tool other than the provider call goes through the kernel because there are no tools yet (M3). Budget units are tokens; dollars arrive with the model catalog (theseus-5xn). `Execution.authority.ceilings` are strings compared for equality; the partial order for shell

classes comes with the shell mapping. The notify socket is unauthenticated beyond directory permissions, which is the spec's stance for on-node transports.

A3. M3 First hands (2026-09-26, theseus-2rr)

M3 was built in three parts: **content** (the session graph, the context compiler, resume from any surface), **hands** (native toollets through the gate, confirmations, background jobs, and the Observatory for all of it), and **Discord** (theseus-9ko), plus **restore** from a local WAL (theseus-at8). All four are built. The milestone closes when the P5 prove passes live: Eddie completes a coding task from Discord, and a job interrupted by a daemon kill finishes and reports into the channel.

What exists.

- **The session graph as nodes** (§4.4b, §6.1). `NODE` records scoped to their session: `user_message`; `assistant_message` with the provider's content blocks kept byte for byte (thinking blocks and their signatures included), model, provider, stop reason, usage, dollars, the catalog version that priced them, request id, the action's correlation id, the compilation id, and the request digest; `tool_call` with the tool's canonical and wire names, the input, and the gate's record (validation, the policy decision and its reason, the plan, the proposal digest); `tool_result` with a status (`ok`, `error`, `denied`, `background`, `unknown`, `cancelled`), the scrubbed and capped content, bytes, truncation with a reference to the full output in the spool, duration, a `late` flag, and meta (exit code). Each node is written in the frame of the kernel transition that produced it (§4.6 now says so).
- **The context compiler** (§4.4, §4.4a, simplest form). A **compilation** is a frozen prefix (node ids) with a manifest (compiler and renderer versions, profile, provider, model, system digest, tool digest and names, catalog version, context window, whether prefix thinking is stripped, and, since 2026-09-29, the context files it carried, A4) and an as-of position; every node after it is the **tail**. Each loop answers *append or recompile*, deterministically: `new_session`, `model_changed`, `system_changed`, `tools_changed`, `overflow` (a ring that drops leading turns at a user-message boundary until the estimate is under 60 % of the window less output and headroom), `manual_fresh` (keep only the current exchange, from the latest operator message), `manual_transcript`. The renderer is byte-stable: a later request's messages begin with the earlier request's messages unchanged, and a test asserts it. Parallel tool results go in one user message after their assistant message; late results render as `[Background result for your earlier ... call]`; a tool use with no recorded result gets a synthetic error result and is listed in the decision's `repairs`. Thinking is replayed byte for byte and stripped only from the prefix at a recompile that edits history (system or tools change, ring, fresh, transcript); a model change keeps it. Caching: `cache_control` on the system block plus top-level automatic caching. A new compilation is persisted with its `derived_from` edge, the session update, and a `context.recompiled` row in one frame; every loop ledgers `context.compiled` (decision, trigger, prefix and tail sizes, messages, estimated tokens, digest, repairs).
- **The model catalog** (theseus-5xn, P5). Built-in version 2026-09-26.1, eleven models (Table 5.1, Table 5, Opus 5.5, Opus 5, Opus 4.8, Sonnet 5, Haiku 4.5, GLM 5.3, 5.2, 5.3-flash, 5.3-flashx), each with provider, context window, maximum output, prices per million tokens (input, output,

cache read, cache write), thinking mode (adaptive, always, budget, none), effort support, server-side refusal fallbacks, cache minimum, and the source it was taken from. [catalog, "<id>"] overrides or adds rows and the version becomes ...+config:N. Consumers: a profile without max_output_tokens gets the model's real ceiling; overflow uses the window; every assistant node, turn result, and provider.call row carries dollars and the catalog version; health reports the total. An unknown model runs with a startup warning and unknown cost.

- **Request building from the catalog.** Adaptive thinking with display summarized, omitted, or updates (with its beta header, on the models that have it), output_config.effort, refusal fallbacks (beta) on first-party calls to models that support them, eager_input_streaming on every tool. A tool input that arrives as invalid JSON goes back to the model as an INVALID_JSON error result instead of failing the call, and no tool runs on a max_tokens or refusal stop.
- **The toollets** (§3.23, §3.24), crate theseus-tools: fs.read (numbered lines, paging, binary detection), fs.write (atomic), fs.edit (exact match with occurrence control, returns the diff), fs.patch (multi-file unified diffs, atomic, create and delete), fs.glob and fs.list (gitignore-aware walker), fs.grep (ripgrep's searcher and regex crates; content, files, or counts, with context), text.diff, git.diff and git.log (native through gix; no git binary), and proc.run (typed argv, the only shell path). Each declares its class (read, write, run), backend (in-process or job), retry class, and a plan naming the resources and argv the gate judges.
- **The policy gate** (§3.9). Canonical workspace roots; protected paths that are always denied (~/.ssh, ~/.gnupg, ~/.aws, ~/.config/op, the 1Password token file, ~/.theseus); an argv deny list (sudo, su, doas, rm -rf /, mkfs, dd, shutdown, reboot, op, theseusd) and allow list (ls, pwd; git status|diff|log|show until 15aadce, below); a command's path-like arguments are judged as well as its working directory: a protected one is denied for any command, and an allow-listed command runs unconfirmed only if every path argument is inside the roots and no argument is one of --output, --no-index, --ext-diff, --textconv, --exec, --upload-pack, --receive-pack, --config-env, -c (15aadce); per-class modes (read allow, write confirm, run confirm) with per-tool overrides. The decision and its reason are on the tool_call node and in tool.denied or tool.confirm_requested rows.
- **Confirmations.** A confirm call is planned (its node written in the same frame) and the turn parks waiting(confirm). The request is a notification and a ledger row, and it is durable: pending confirmations are derived from planned actions and their nodes, so they survive a restart. Each is bound to the proposal digest and expires ([kernel].confirm_ttl_secs, 15 min). action.confirm approves (binds and wakes) or declines (cancels the action with a denied by ... resolution and wakes); the model reads a decline and any note. Calls after it in the same response wait their turn; a new operator message supersedes whatever is still pending, and the model is told so.
- **Background jobs.** proc.run goes through the M2 wrapper with the environment cleared to [tools].proc_env, a working directory, and a clamped timeout; the action is dispatched before launch. The turn waits up to [tools].proc_sync_secs, then writes a background placeholder and ends waiting on the action; the completion (spool plus notify) wakes the execution, and a continuation turn absorbs the late result and lets the model carry on.
- **The continuation driver** (core::harness::drive) runs a turn with no input for any queued execution that has resume_pending or queued results, one at a time per execution, on a 500 ms tick and on admission notifications. A continuation that fails before admission backs off per

execution (2ⁿ s up to 5 min) with a warning; one that fails after admission publishes `turn.failed`.

- **The session event bus.** `session.watch` and `session.unwatch`; a turn's notifications reach its requester and every watcher once, whoever started it (web UI, CLI, the driver); watchers are dropped on disconnect.
- **Metrics.** `theseus.tool.calls` (by tool, from the turn's `tool` spans) and `theseus.cost.usd` join the OTEL metrics; `tool.list` keeps durable per-tool counts (seeded from `tool_call` nodes at startup) and the shell-fallback ratio.
- **Scrubbing.** Tool output is scrubbed before it becomes a node: exact vault values become `[redacted:<name>]`, token shapes (`sk-ant-`, `ghp_`, `github_pat_`, `gho_`, `ops_`, `xoxb-`, `xoxp-`) become `[redacted:shape]`.
- **Protocol.** Methods `session.history`, `session.watch`, `session.unwatch`, `session.recompile`, `action.confirm` (with `watch`, which subscribes before the answer wakes anything), `catalog.list`, `compilation.list`, `node.list`, `tool.list`. Notifications `model.thinking`, `context.compiled`, `tool.proposed`, `tool.started`, `tool.ended`, `confirm.requested`, `confirm.resolved`, `node.written`, `turn.failed`. Turn results gain execution id, dollars, tool calls, the awaiting confirmation, stop details, and a continuation flag; sessions gain last activity, dollars, tool calls, profile and model, compilation, title, and pending confirmations; health gains total dollars and the catalog version.
- **CLI.** `theseus history`, `watch`, `confirm` (without an id: everything waiting; with one: answer it and follow the turn it resumes), `tools`, `catalog`, `sessions recompile`; `ask` shows tool activity, confirmation requests, and recompiles on stderr, `--thinking` shows thinking summaries, and the status line has tool calls and dollars. SIGPIPE is back to its default, so `theseus history | head` ends quietly.
- **Discord** (theseus-9ko, crate `theseus-discord`, twilight 0.17). A protocol client of the core, connected the way the web UI is (an in-process pipe into `serve_connection`), so a Discord turn, a button press, and `/stop run turn.submit`, `action.confirm`, and `execution.stop` (`execution.cancel` until W1, 2026-09-30). It watches the sessions behind its places, so continuation turns (a job's late result, a restart, an answer given in the web UI) reach Discord with nobody asking. **Places** come from the **bindings file** (P5): one guild, text channels, DMs, and the Discord users who may drive each; its SHA-256 prefix is the binding revision. The file lives in the state dir by default, beside the store that remembers each place's session, so a scratch instance with its own state dir never opens a second gateway connection on the same token; only the socket daemon binds (never `--stdio`). Each place is an actor with one conversation session (its id in store meta, replaced when that execution is cancelled or exhausted): turns are serialized, messages that arrive mid-turn are coalesced into the next turn with their authors, and a new place posts a short bind notice. **Rendering** is pure and tested: each loop's streamed text becomes messages edited in place (at most one edit per message per `edit_interval_ms`, split under Discord's 2000 characters with code fences closed and reopened across a split); each loop's tool calls are one message of lines updated as they run (proposed, running, waiting for approval, background, done with duration, denied with the reason, answered by whom); a confirmation is its own message with **Approve** and **Decline** buttons bound to the correlation id, settled (buttons removed, who answered) when it is answered anywhere; a footer carries the profile, model, loops, tool calls, dollars, and time; a

failed turn says its class and error. Mentions are never pinged. **Controls:** `/stop` and `/cancel` (cancel the session's execution, then bind the place to a fresh session), `/new`, `/status`, as slash commands and as plain text. Only listed users can drive a place or press its buttons; anyone else is ignored and counted. A channel binding is **mention-only** by default (9383bdc): only a message that @mentions the bot (as a user or through its managed role) or replies to one of its messages starts a turn, with the mention stripped, so a channel shared with people or other bots does not get a turn per message. Eddie bound `#openclaw` this way, beside five OpenClaw agents that answer only when mentioned.

- **Restore** (theseus-at8). `theseusd restore --from <dir>` takes a WAL directory or a store directory, refuses while a daemon serves the socket, copies the segments into a staging store beside the live one, opens it (every frame's checksum checked, a torn final frame cut and reported, the index rebuilt from the WAL), writes a `store.restored` ledger row, and swaps it in. The source is only read; a store already there needs `--force` and is moved aside, never deleted.
- **Web UI** (Eddie's observability rule). A sessions sidebar: any session can be resumed, the open one survives a reload, and a session opens with its first message rather than per page load. The transcript is rebuilt from nodes: tool cards with the gate's decision and reason, status, exit code, duration, bytes, colored diffs, and the full input and output on click; a late result both on the card of the call that started it and in the turn that received it; calls queued behind a confirmation. Confirmation cards inline, with a preview (the edit as a diff, the file content, the command line), an optional note, Approve and Decline, and the expiry. Thinking summaries collapse; dollars per turn, per session, and in total; a pulsing "N waiting for you" in the header and "needs you" in the sidebar; automatic reconnection that re-watches and reloads; timing trees for past turns from their `turn.trace` rows. The Observatory gained **Context** (compilations with trigger, strategy, as-of, prefix size, model, and thinking kept or stripped; each loop's decision with prefix and tail, messages, estimated tokens, repairs, and digest; recompile buttons), **Tools** (policy per tool, calls, roots, the shell-fallback ratio), **Nodes** (by kind, the JSON on click), **Model catalog**, and sessions with dollars, tool calls, and pending confirmations; the ledger gained `tool.*` and `context.*` chips. For M3c/d the Observatory gained a **Discord** panel (state and why, bot, guild, bindings file and revision, connection age and heartbeat, messages in and out, edits, button and command presses, ignored messages, errors and the last one; each place with its channel, its session as a link, who may drive it, and last activity; the latest `discord.*` rows), and `discord.*` and `store.*` ledger chips. Health carries `bindings[]`, and `theseus health` prints a line per binding. The protocol gained optional `author` labels on `turn.submit`, `action.confirm`, and `execution.cancel`, so the ledger and the confirm card say `discord:eddie`; they are labels, not authority.

How it is proven.

Tests. 94 in the workspace. Seven core scenarios (`theseus-core/src/tests_m3.rs`, a scripted provider over the real store and kernel): a tool loop reads a file, and the second request's messages begin byte for byte with the first's, over one compilation; a write parks for confirmation, is approved, and the continuation writes the file; a decline (the model sees it and the note) and a supersede by new input (the denial precedes the new text in one user message); a path outside the roots and a protected path are denied with their reasons, and an invalid input and an unknown tool are errors; `proc.run` returns

in-turn, and a slow one comes back as a background placeholder, then as a late result once the heartbeat drains its completion; a session keeps its memory across a restart (new core over the same store, still one compilation); a fresh recompile keeps only the current exchange and is recorded with `derived_from`. Compiler unit tests: append stability, prefix thinking stripped on a system change and kept on a model change, repair of a missing result, fresh and ring overflow at a user boundary. Toollet tests include native `git.diff` and `git.log` against a real repository. The scenarios found one real bug: a fresh recompile applied after the turn's own user node was written produced an empty request; fresh now keeps the current exchange. The M2 kernel sweep reran on the changed kernel (`kernel-sim`, 40 seeds × 400 steps): 318 crashes, 129 of them inside startup steps, 16 040 invariant checks, all held.

Live, real API (`claude-sonnet-5`, a scratch daemon with the scratch repository as its only root). A coding task: "a small Python module has a failing test; run it, find the bug, fix it, rerun". The model listed the directory, asked to run the tests (confirmed; exit 1), read the module, proposed a one-line edit shown as a diff (confirmed), reran the tests (confirmed; exit 0), and explained the fix: four turns, five tool calls, \$0.023, and 4.5–8 k cache-read tokens on each continuation.

Kill test. A confirmed `proc.run` of `bash -c 'sleep 20; echo build finished'`; the daemon was SIGKILLED 2.5 s into its 5 s sync window. The job survived (`setsid`). On restart the interrupted execution was requeued; its continuation found the job still running and wrote a placeholder ("the harness restarted meanwhile"), and the model said it would report. 20 014 ms after launch the completion was drained from the spool, the late result written, the execution woken, and the next continuation reported "build finished", exit 0.

Discord and restore. 108 tests. Nine in `theseus-discord`: a streamed turn renders as edits and ends with its footer; a confirm gets buttons and loses them when answered; denied calls and failed turns say why; long text splits under the limit with fences balanced, and appending never moves an earlier cut; tool summaries; the bindings example parses and bad files are refused with the reason; controls and button ids parse. Four in `restore`: a WAL restores into an empty state dir and the source is byte-identical afterwards; an occupied store needs `--force` and survives moved aside; a torn tail is cut and reported; nothing to restore is an error. Live against Discord (a scratch daemon, DM binding): the gateway reached ready, four slash commands registered, the DM channel opened, and the bind notice was sent, all in the ledger (`discord.bound`, `discord.message.out`, `discord.ready`). Live restore from a copy of that daemon's WAL: 13 frames, 14 records, the index rebuilt, a second restore refused without `--force`, and the restored store served.

Eddie's exit test from Discord (2026-09-26, 21:03–21:05, his daemon, DM). Two gate refusals rendered with their reasons (`fs.read /etc/hostname`: outside the workspace roots; `proc.run sudo ls`: the deny list), a `proc.run` of `bash -c 'sleep 45; echo done'` approved with the Discord button, then `pkill -9 -f 'theseusd$'` and a restart. The job survived, startup settled it from the spool (45 005 ms, exit 0), and the continuation answered. **The answer never reached Discord:** the driver resumed the execution 0.2 s after the kernel accepted, and the turn ended at 21:05:36.6, but the binding watched its sessions only at 21:05:37.8, and the bus delivers live events only. Fixed in 8a41519: the driver waits (at most 20 s) until every expected binding watches its sessions and ledgers `driver.started` with the wait; the renderer shows a turn it only saw end from the result's final text. (*Replaced 2026-09-30,*

theseus-q4v: the reply is a post, and the driver waits for nothing.) Eddie's rerun at 21:18 passed: the driver waited 1 687 ms for the binding (`driver.started`), the continuation reported the job still running into the DM, and 45 022 ms after launch the late result was settled and its answer posted there. What remains of the P5 prove is a real coding task from Discord. Walking Eddie through the policy then found a hole in the gate itself: a `proc.run` call's only resource was its working directory, so an allow-listed command could name any path in its arguments (`git diff --no-index /dev/null ~/.ssh/id_rsa` would have printed a protected file unconfirmed, and `git log --output=<path>` could write anywhere). Fixed in 15aadce as described under the policy gate above; git left the default allow list because its config and attributes can run programs, and the native `git.diff` and `git.log` read history without it. Then Eddie set the policy's shape (862f5c6, §3.9): `[tools].projects_dir` replaces the built-in `~/projects` (no default: without it every path is outside the workspace), and one `[policy].enforcement` ladder (`strict | ask | notify | open`, 866bd1b) decides what a confirm and a deny do, replacing a first cut of two independent settings (862f5c6) after Eddie asked that they be intrinsically compatible: two knobs allowed a call against the policy to run with no card while a merely sensitive one waited. Anything that runs without asking posts a structured notice in the channel, and a floor no level lifts. Scenario tests run a write with no card and a read outside the roots under `open` (proving `theseusd` stays floored) and turn a refusal into a marked confirm under `ask`.

Note, 2026-09-29. The coding task is still open. Eddie took one to Theseus from Discord on 2026-09-28 and wrote at 23:57: "Coding task from discord went very well (haven't written so far just analyzed)". The session analysed the code but did not yet change it, so this prove still waits on a task that writes. The prove's clause on a request Eddie is not permitted to make is now read under no deny, as the reversal records (A3b).

Web UI, driven by Playwright in headless Chrome: two sessions resumed; an edit's card opened to its diff; a new session whose write needed confirmation, approved in the page, and read back by the continuation; a reload that returned the same session; the daemon restarted under the open page ("reconnecting", then the same transcript reloaded), and a turn submitted from the CLI in that session streamed into the page. Screenshots: `specs/rust-harness/shots/m3-*.png`. Smoke green with new `tools`, `catalog`, and `history` steps.

Lifecycle timings (2026-09-27, measured for FAST, §2 and P5b). These use the installed release binaries at 524f535 on a scratch daemon: three runs on a copy of Eddie's store (603 WAL positions, 314 KB) and three on an empty store.

- Cold start to the first `health` answer took 1 281–1 340 ms, and 2 087 ms on the very first run.
- The daemon's log accounts for the time. Resolving six secrets through six concurrent `op read` processes took 1 023 ms, and the GitHub token check took another 195 ms. Both run before the store opens.
- The five kernel startup steps took 6.5–8.0 ms each, and the socket answered about 50 ms after the second network call returned.
- Clean shutdown, from the request to process exit, took 24–39 ms.

Local work is about 5 % of startup; the other 95 % is two network calls on the start path, which P5b moves off it.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
Discord (twilight) as M3's front end, the confirm as a Discord component	Built third, after the web UI and CLI, as a protocol client of the core	The confirm needed a surface first and Eddie's rule required the web UI to show everything first; a protocol client runs exactly the paths the other surfaces run	Keep
Delivery to a channel is an action (§3.2a)	Discord posts and edits are direct HTTP calls, counted in health and ledgered (<code>discord.message.out</code> , <code>discord.error</code>)	M3 has no task executions reporting into channels; a conversation's reply is already the turn's own output	Done in M3.6 item 6 (theseus-q4v, A4): what must be seen is an outbox post, and live progress stays direct
Messages coalesced with each author preserved (§3.2)	Coalesced into one input: joined when one author wrote them all, <code>[author]</code> tags otherwise; the node's author label is <code>discord:<name></code>	One operator in M3; separate nodes per author arrive with Person nodes	Revisit with roles and Persons
<code>/stop</code> stops the model loop	<code>/stop</code> cancels the session's execution and binds the place to a fresh session	The kernel has no soft stop for a turn; an execution is one per session and cancel is terminal	Done in M3.6 (theseus-lji, A4 item 9): the kernel has a turn-level stop, so <code>/stop</code> halts the work and keeps the session, and <code>/new</code> alone starts a fresh one
<code>theseus restore</code>	<code>theseusd restore --from <dir></code>	The daemon owns the store, and restore must run with the daemon stopped	Keep
Toollet families as separate crates (§3.23)	One <code>theseus-tools</code> crate, a module per family	Eleven small tools; a crate per family is packaging, not isolation	Split when a family brings a heavy dependency
<code>git.diff</code> staged and unstaged	HEAD against the worktree, or rev against rev	The index diff was the costlier path; the common question is "what changed"	Add staged when asked
A turn's writes in one WAL transaction (§4.6)	Per step, in the frame of the kernel transition	A crash mid-turn then leaves exactly the durable effects, which resume relies on	Part I §4.6 updated
Web UI read-only in M3 (P5)	Confirm, resume, recompile, and cancel from the web UI	The confirm needed a surface before Discord; the rest are the same deterministic control paths as the CLI	Keep

Planned	Actual	Why	Disposition
Manifest records the tail range	As-of position; the tail is everything after it	A range would duplicate the as-of and the session's position table	Keep
Four-part cache layout (§4.5)	<code>cache_control</code> on the system block plus automatic top-level caching	Automatic caching follows the growing prefix; continuations read 4.5–8 k cached tokens	Moved to provider-safe caching (§4.5, theseus-ev1)
Recovery from transient provider failures in continuations	A continuation that fails after admission parks the execution waiting for input, with <code>turn.failed</code>	No automatic retry by design (§3.13); the late result is in the graph and the next input sees it	Consider a retry wake with backoff
The run-hooks path wired at each event site (P2); the core's hook sites and the kernel gate meet when tools arrive (A2)	The same fourteen events as at M0 are dispatched. <code>tool.pre_call</code> , <code>tool.post_call</code> , <code>confirm.requested</code> , <code>action.planned</code> , <code>action.dispatched</code> , <code>completion.received</code> , and <code>ledger.row</code> sit on paths that exist and are never dispatched, and the confirm is decided in the policy gate, not through <code>PreToolCall</code> 's defer (§3.17). <code>memory.*</code> and <code>judgment.made</code> wait for their subsystems	Not recorded when M2 and M3 landed; found by the theseus-s3m review, 2026-09-27 (Appendix F). Re-verified that night, with a further finding: three dispatched sites discard their result. They are <code>tool.proposed</code> (gate), <code>context.built</code> (transform), and <code>reply.claim</code> (claim). With <code>tool.pre_call</code> unwired, no hook could stop a tool call. This is latent while <code>Hooks::dispatch</code> always returns <code>Proceed</code> (only remote observers exist)	Eddie, 2026-09-27: fix as proposed. Wire the seven, make the three honor their results, amend §3.17 (done in v0.37), add a blocking scenario per gating point, and let the P0 registry test prevent a recurrence (theseus-0dp, P5c item 2). <i>Moot since 2026-09-28: the hook system was deleted (A3b, the complexity cuts).</i>
Process start to accepting events in under 2 s, warm-up excluded (§9)	Met, at 1.3 s, but 1.2 s of it is network on the start path: secrets through <code>op</code> and the GitHub token check run before the store opens	The startup order dates from M0, and no bench timed startup end to end	Serve first; secrets and checks resolve in the background (P5b, M3.5)

Known gaps carried forward. The simulator's random operations do not yet include the kernel calls M3 added (decline, wake, the resume flag, records riding in the plan and settle frames); the core scenarios cover them, the fault injection does not. An allowed call interrupted between its plan and its authorization is treated as awaiting confirmation (the safe direction). In-process results over `[tools].result_max_chars` are truncated without a full-output reference (`fs.read` pages instead). `session.history` returns whole sessions, and `theseus.confirm` without an id asks each waiting session in turn. The web transcript renders plain text, not markdown. The `updates` thinking display is wired but not yet exercised live. A profile's `max_output_tokens` still wins over the catalog, so a config carrying the old `max_tokens = 1024` truncates tool inputs; regenerate it from `theseusd example-config`. Since 8c8a53a the template caps no profile, so every model runs at its catalog ceiling. Budgets count every token at full weight, cache reads included, and each provider call reserves its output cap plus the input estimate: about 130 k at Sonnet 5's ceiling, so the default million-token budget ends a session near 862 k minus its context (theseus-0sg). A session's limit is fixed when its

execution opens; changing `default_budget` affects new sessions only. (Closed 2026-09-29 by *theseus-0sg*: budgets are dollars, a session at its limit asks for a reset instead of ending, and `default_budget` is retired; A4, item 1.) Discord: attachments are listed in the input by name and size, not read; a DM place whose channel could not be opened at startup stays silent until that user writes; rendering remembers the last eight turns per place, so a confirm answered after a restart is settled by the button handler from the message itself (since 2026-09-30, *theseus-q4v*, a card is settled by its settle post, and only a card from before then settles by its press); slash commands are registered globally; there are no threads. Delivery is not yet durable: a turn that ends while Discord is unreachable, or after the 20 s startup wait gave up, is in the store and the web UI but is not re-posted when Discord returns (delivery becomes an action with its own completion in M5; since 2026-09-29 it is M3.6's item 6, *theseus-q4v*, P5d). (Closed 2026-09-30 by *theseus-q4v*: A4, item 6.)

A3b. The build chain from the 2026-09-27 decisions (theseus-5r9)

P5c's items, in the order Eddie approved on 2026-09-27, each recorded when its review ends. Step 1 is spec v0.37 (3877fe9). Steps 2a through 2a.3 were removed on 2026-09-28; they stay here as the record, and the reversal says what replaced them. The complexity cuts and Narration came after the reversal, on 2026-09-28 and 29, and close this section; M3.6's items are in A4.

Step 2a. Consequences in the gate (theseus-770; 2026-09-27, 22:30–23:24; 03521f0, a218d8b, a3dcb64, 43faa91)

Removed 2026-09-28 in dbd7567, except the `fs.patch` deletion fix; see the reversal below.

What exists.

- **Kinds.** Nine seed kinds (`theseus_tools::consequence::SEED_KINDS`), graded as §3.9 lists them, with `opaque` beside the three that need approval. `[consequences.kinds.<name>]` regrades a seed or adds a kind. Every key is optional (`irreversible`, `description`, `argv`), and Eddie's pre-2a template loads unchanged (tested from a fixture copy).
- **Detection, layers 1 to 3.** A toollet declares consequences in its plan (`Plan.consequences`). Today every native toollet declares none:
 - the read-only toollets only read;
 - `fs.edit` keeps the replaced text in its arguments;
 - `fs.patch` now refuses a deletion that does not show every line it removes, so a patch always records what it took.

For `proc.run`, `theseus_tools::shell` finds every command a call would start. It parses `bash`, `sh`, `dash`, `zsh`, and `ksh -c` strings (operators, pipes, subshells, `$(...)`, backticks, heredocs, redirections), sees through wrappers (`env`, `sudo`, `xargs`, `timeout`, `nohup`, `find -exec`, `ssh`, and more), and tracks every directory a command might run in. The rule table (`RULES_VERSION = 2026-09-27.1`, 33 rules) then names what each command does.

Whatever the parser cannot see through is `opaque`: inline code, scripts, task runners, `eval`, remote commands, unknown git and gh subcommands, and a program word built from an expansion.

- **The regenerable rule.** It decides when a recursive delete is not `bulk_delete`:
 - inside a work tree, the target must be at or under a directory with a known build-output name (25 of them), which the repository's own ignore files ignore and under which `HEAD` tracks nothing;
 - outside any work tree, the target must be under `/tmp`;
 - a target the parser cannot resolve never qualifies.
- **Examples are the tests.** Every rule carries examples it must and must not match (162 and 125). Each example runs three ways (as shell text, under `bash -c`, and as a direct argv) in a fixture repository with ignored outputs, tracked work, and a force-added `build/`. A planted wrong example fails the test.
- **The irreversible column** (`ToolPolicy::decide`; the table is on `Enforcement`):
 - The floor is decided first, and nothing after it changes that answer.
 - An irreversible call waits for approval at every level.
 - A call that is irreversible and also against the policy gets the stricter treatment: refused under `strict` and `notify`, a marked wait under `ask` and `open`.
 - A needs-approval kind raises an allow-listed call: it waits under `strict` and `ask`, and runs with a notice naming the kind under `notify` and `open`.
- **The kinds appear wherever the decision does:**
 - `Decision` and `ConfirmRequest` (the new fields are optional, so old clients keep working);
 - the notice text ("Irreversible: history_rewrite · needs approval: opaque");
 - the ledger rows `tool.confirm_requested`, `tool.notified`, and `tool.denied`;
 - the node's gate record, the tool span's attributes, and the counter `theseus.tool.consequences`;
 - Discord ("Irreversible. Approve?"), the web confirm card (an irreversible pill and a red border), and `theseus watch` and `theseus confirm`.
- **Replay.** `theseus policy replay [-n N]` (`policy.replay`, read-only) judges the newest N tool calls again with the current gate, and lists the ones it would treat or name differently. `theseus policy rules` (`policy.rules`) lists the kinds as graded, and every rule with its example counts.

How it is proven.

- **Tests.** 132; the gate passed at 43faa91, and again in review.
- **Seven scenarios** over the real store and kernel, with a local bare repository as `origin`:
 - Under `notify`, a force push parks, with `history_rewrite` named on the card, in the ledger row, and in the node. The remote does not move until the call is approved, and then the rewritten history lands.

- A plain push runs with one notice.
- `rm -rf somedir parks`, and `rm -rf target` runs.
- A force push inside `bash -c parks`.
- `open` still parks a force push.
- `strict` changes nothing else.
- **A 28-spelling test:** quoting, wrappers, compound commands, substitutions, and `find -exec`; no spelling hides a force push.
- **Live, on a scratch daemon** (GLM 5.3 flash, `notify`):
 - The model's force push waited, and the bare remote stayed put.
 - A decline reached the model.
 - A feature-branch push and `rm -rf target` each ran with a notice.
- **In review, on a copy of Eddie's store**, with the installed release binaries:
 - `policy rules` lists 9 kinds and 33 rules.
 - `policy replay -n 500` judged his 4 past tool calls and reported 2 changed. Both are his exit test's `bash -c 'sleep 45; echo done'`, which waited under the enforcement of the time and would now run with a notice under `notify`.
 - So replay compares against the whole current gate, settings included. The "should have asked" flow (2b) should hold the settings fixed and vary only the rule.

Found in review (2026-09-28, 00:05–00:14). A probe of the detector and the gate (not committed) confirmed that the plain spellings are caught, and these are not:

- **The floor**, a gap since M3b. It checks only the top-level program, and only the plan's resources.
 - Under both `notify` and `open`, `env op read ...`, `bash -c 'op read ...'`, and `bash -c 'theseusd config'` each run with an amber notice.
 - A floor path named as an argument or a redirection (`cat <store file>`, `cat < <store file>`) is judged only by the deny list, so under `open` it runs with a red notice.
 - The deny list is blind inside shell strings too (`bash -c 'sudo ls'`).
- **Detection.**
 - Brace expansion: `git push origin main --{force,}, rm -rf target/{,..}/src`.
 - Words the parser cannot resolve: `F=--force; git push origin main $F, git push origin main -$(printf f), bash -c 'git push origin main "$@"' _ -f, F=-rf; rm $F somedir`.
 - Unique-prefix long options, which git and GNU tools accept: `git push --mirro, git push --force-with-leas, git reset --har, rm --recursiv`.
 - A glob followed by `..`: `rm -rf target/*/../../src` resolves to `src`.

All of them are fixed in step 2a+ (theseus-xbg), before 2b.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
Detection rules are versioned data (§3.9)	Each row is a Rust matcher; its id, table version, and examples are data. The owner's <code>argv</code> prefixes are the data-only form	Shell semantics need code; the examples keep each matcher honest	Keep. A proposed rule (2b) lands as an owner prefix first
Owner-added kinds (§3.9)	They also take <code>argv</code> prefixes	A kind with no reader is inert (P0)	Part I §3.9 to say so
A needs-approval kind raises a call that needs confirmation	It also raises an allow-listed call	Allow lists match spelling; consequences are semantic	Held for Eddie
Layer 2 lives in <code>proc.run</code> 's plan	It lives in the gate	The plan stays the toollet's own statement	Keep
<code>external_post</code> excludes Theseus's own places	Every loopback post counts	An over-approximation, until an exception names Theseus's own addresses	Held for Eddie; with 2b or a later exception
An overwrite loses no work unnoticed	<code>fs.write</code> is not graded, and it keeps no before-image	It touches one file and is the everyday path; its class already stops it under <code>strict</code> and <code>ask</code>	Held for Eddie: a before-image, or a grade when the file has uncommitted changes
P5c goes from 2a straight to 2b	Step 2a+ added between them	The holes above	Recorded here
The web confirm card is seen	Checked through the bundle and the RPC, not screenshotted	The OpenClaw browser tool refuses localhost	A headless-Chrome screenshot in 2a+

Known gaps.

- Argv detection cannot see:
 - `git config (remote.*.push = +..., remote.*.mirror);`
 - `proc.run`'s environment (`GIT_CONFIG_*`);
 - escape hatches inside text tools (`awk`'s `system()`, `sed`'s `e`);
 - what runs behind `eval "$(...)"`, a heredoc fed to a shell, or `ssh`, beyond naming the call `opaque`.
- A floor check over `argv`, however careful, stops only the spellings it can read. Code the gate cannot parse is `opaque`, and under `notify` it runs with a notice. Only M4's boundary closes this for certain: L1 gives a job no ambient credentials and keeps the token file out of its reach.

Step 2a+. Gate hardening from the 2a review (theseus-xbg; 2026-09-28, 00:24–01:28; 0d0b769, 0093ec4, e57c48c)

Removed 2026-09-28 in dbd7567; see the reversal below.

What exists.

- The floor and the deny list judge every command a call starts**
(`ToolPolicy::floor_over_commands`, `deny_over_commands`), over what `shell::commands` returns:

- each command's words, and the wrappers it was reached through (`Cmd.via`);
- its path arguments and redirection targets (`Cmd.redirs`, including fd forms), resolved against every directory it might run in;
- those directories themselves.

`$HOME`, `${HOME}`, and `~` resolve to the daemon's HOME for path judgments. The floor still runs first and is refused at every level; the deny list follows the ladder.

- **A floor mention scan over code the gate cannot parse** (`floor_mention`). It covers unparsed commands, dynamic programs, inline code, `eval`, and remote commands, and it looks for:
 - a floor path, in its canonical, `~/`, `$HOME/`, or `${HOME}/` form, or a floor file's name;
 - `theseusd` as a word;
 - `op` followed by one of its subcommands.

The refusal says it came from a mention, so a model can rephrase an innocent call.

- **Detection reads every word, not just the program:**
 - brace expansion inside shell strings, capped at 64 results (past the cap, the word is dynamic);
 - `$1 ... $9`, `$@`, and `$*` substituted for `bash -c 'script' arg0 args...` when the args are literal;
 - an unquoted unresolved word counts as any option, and a dynamic positional as a dangerous operand;
 - unique-prefix long options count as triggers;
 - a glob followed by `..` is never regenerable.

Triggers and exemptions use separate helpers (`opt_detect` and `opt_exact`). An exemption never accepts a variable or an abbreviation.

How it is proven. 138 tests; the gate passed at each commit and again in review.

- **Unit tests.**
 - Seven floor spellings are refused with `floor: true` at all four levels.
 - `bash -c 'sudo ls'` follows the ladder.
 - Mentions inside `python3 -c` are refused.
 - The `$HOME` forms reach a HOME-based floor path.
- **Scenarios** over the real store and kernel:
 - Under `notify`, `env theseusd --version` is refused, and the model is told why.
 - Under `notify`, `bash -c 'git push origin main --{force,}'` parks as `history_rewrite`, and the bare remote does not move.
 - Under `open`, `cat` of a store file is refused as floor.
- **FAST.** In release, `ToolPolicy::decide` takes 510 μ s on a 2.9 KB `bash -c` string of 30 commands, and 9.8 μ s on a plain argv.
- **Live, on a scratch daemon** (GLM 5.3 flash, `notify`):

- `bash -c 'theseusd --version'` and `bash -c 'cat <state>/store/MANIFEST.json'` are refused as floor.
- A brace-spelled force push parks, the ledger row carries the expanded command, and the bare remote stays put through a decline.
- **The web confirm card has its first screenshot** (`shots/2a-plus-web-confirm.png`): the `IRREVERSIBLE` pill, the red border, the kind line, the exact command, and Approve and Decline.
- **In review:** installed at `e57c48c`. On a copy of Eddie's store, replay is unchanged (2 of 4, the same two calls).

Found in review (2026-09-28, 01:32–01:37). A second probe (not committed) confirmed that the floor spellings from the first review are refused and that all eleven detection misses are now caught. These still get through:

- **The floor:**
 - a program word built at run time: `x=op; $x read op://..., $(echo op) read op://...;`
 - inline code in its natural form: `python3 -c 'subprocess.run(["op", "read", "op://..."])',`
`perl -e 'system("op", "read", ...)'`. The scan wants `op read`, with a space;
 - a relative program path inside a string, which keeps its directory: `bash -c`
`'./target/release/theseusd config';`
 - globs through a floor path: `cat <state>/sta*/store/wal.`
- **Detection:**
 - A quoted variable is treated as never an option. Quoting stops word splitting, not option parsing, so `x=-rf; rm "$x" somedir, m=--hard; git reset "$m", f=--force; git push "$f",` and `x=-delete; find . "$x"` all get through.
 - `"$@"` with two or more literal arguments is joined into one word, so `bash -c 'rm "$@"' _`
`-rf somedir` gets through.
- **The record:** the rule table's version is still `2026-09-27.1`, although detection changed. Replay and the ledger therefore cannot tell old judgments from new ones.

All of these go to step 2a++ (theseus-0tv).

A correction to the first review's record: its probe expected `history_rewrite` for `git reset --har`, but that rule's kind is `bulk_delete`. The miss in 2a was still real (the rule matched only the exact option), and 2a+ catches it.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
The floor is refused at every level (§3.9)	True now for the command forms the gate can read: wrappers, shell strings, arguments, redirections, and working directories. Step 2a++ closes what the second probe found	Before 2a+, the floor saw only the top-level program and the plan's resources (a gap since M3b)	Keep
The floor judges what a proposal became	It also scans unparsable code for a mention of a floor path or program	The gate cannot see inside inline code; a mention is all it has	Keep; M4's boundary is the real close

Planned	Actual	Why	Disposition
The parser over-approximates the program word	It over-approximates every word a rule reads	§3.9's "over-approximates on purpose", applied consistently	Keep. Accepted friction: inside <code>bash -c, git push origin "\$BRANCH" and git reset \$SHA</code> now wait. Held for Eddie
P5c goes from 2a+ to 2b	Step 2a++ added between them	The second probe	Recorded here

Known gaps.

- A recursive read through an ancestor of a floor path (`find ~ -exec cat {} +, grep -r x ~`) is judged by the deny list, not by the floor. `~/theseus` and the token file are on the deny list by default. A floor rule for ancestors would be far too broad, because `~` is an ancestor of everything.
- Deliberate obfuscation in inline code (string building, base64) stays `opaque`.

Only M4's boundary closes these.

Step 2a++. Gate hardening, round 2 (theseus-0tv; 2026-09-28, 01:42–02:29; 1a58cfb, 995f830)

Removed 2026-09-28 in dbd7567; see the reversal below.

What exists.

- **The floor, round 2:**
 - A `$name` program word whose assignments in the script are all literal resolves to each candidate value (the union across branches). Any other dynamic program word is scanned by its own spelling for `op` or `theseusd`.
 - Inline code is refused when it names a floor program as a string literal (`"op"`, `'theseusd'`, `"/usr/bin/op"`), when it contains `op://` anywhere, or when `op` is followed by a subcommand across separators (so `["op","read"]` counts).
 - Parsed commands never get this scan: `grep -r "op://" crates` stays allowed.
 - A program word is matched by its file name, so `./target/release/theseusd` reaches the floor.
 - A glob argument or redirection target is matched against each floor and deny path, component by component (`*` and `?` also match a leading dot). A pattern shorter than the path names an ancestor and is left to the deny list.
- **Detection, round 2:**
 - Any dynamic word before `--` may be an option, because quoting stops word splitting, not option parsing.
 - `rm.recursive` keeps one guard: when `-r` is only possible, not certain, it needs a separate operand. So `for f in *.log; do rm "$f"; done` stays quiet.
 - `find` treats a dynamic word in its expression as a possible `-delete`.

- `$@` and `$*` expand with bash's word semantics.
- `RULES_VERSION` is 2026-09-28.1, and `policy rules` prints it.

How it is proven. 141 tests; the gate passed at each commit and again in review.

- **Unit tests** refuse every floor spelling from the second probe, with `floor: true` at all four levels, and catch every detection case. Precision cases stay quiet: `grep -r "op://", ls ~/*`, the `rm "$f"` loop, and `find -name "$p"`.
- **A scenario:** under `notify`, `python3 -c 'import subprocess; subprocess.run(["theseusd", "--version"])'` is refused, and the model is told why.
- **FAST:** 497 μ s on the 2.9 KB `bash -c string`, and 7.9 μ s on a plain `argv` (release).
- **Live, on a scratch daemon:** `x=theseusd; $x --version` and the inline-code call are both refused as floor. `x=-rf; rm "$x" somedir` parks as `bulk_delete`, and the files survive a decline.
- **In review:** installed at 995f830. On a copy of Eddie's store, `policy rules` prints 2026-09-28.1, and replay is unchanged.

Found in review (2026-09-28, 02:49–02:53). A third probe found every floor and detection case from the first two probes handled, and no over-refusals among the allowed cases. These still get through:

- **The floor.** A variable used as the program after a wrapper or a re-parse: `x=op; command $x, exec "$x", env $x, timeout 5 $x, eval "$x whoami", bash -c "$x whoami"`. The literal-assignment resolution applies only to the first word of the outer command.
- **Detection:**
 - `set -- -rf somedir; rm "$@"; set --` redefines the positionals;
 - `rm "${a[@]}"`: a quoted array still expands to several words;
 - `x="-rf somedir"; rm $x`: an unquoted variable can carry both the option and the operand;
 - `rm -r$(printf f) somedir`: the literal `-r` prefix of a dynamic word is ignored;
 - `x=push; git $x -f`: a dynamic git subcommand is only opaque.

Most share one cause. So step 2a.3 (theseus-905) resolves every simple `$name` from the script's literal assignments, everywhere. That also makes honest scripts exact: `BRANCH=main; git push origin "$BRANCH"` resolves to a plain push, and stops waiting.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
The floor is refused at every level (\$3.9)	True now for run-time program words, natural inline code, relative program paths, and globs. Step 2a.3 closes variables that reach the program through wrappers and re-parses	The second and third probes	Keep
Over-approximation reads every word (2a+)	A quoted dynamic word may be an option too, and <code>\$@</code> splits as bash splits it	Quoting stops word splitting, not option parsing	Keep. Accepted friction: <code>git checkout "\$branch"</code> and <code>git restore "\$x"</code> wait, since a <code>-f</code> needs no operand. Held for Eddie

Planned	Actual	Why	Disposition
P5c goes from 2a++ to 2b	Step 2a.3 is added, as the last argv-layer round	The third probe	Recorded here. After 2a.3, the remaining long tail goes to M4

Known gaps. The following stay `opaque`, and under `notify` they run with a notice:

- string building (`x=o`; `y=p`; `$x$y`);
- obfuscated inline code (`base64`, `chr()`, implicit concatenation);
- a script that is written and then run.

Honest calls are caught by their natural spellings. Against a model that is working to get around the gate, the argv layer is a speed bump, not a wall. The protection for sessions that read untrusted content is M4's exposure rule (one step stricter, so `notify` becomes `ask`) and its boundary. Until they land, such sessions should run under `ask`.

Step 2a.3. Gate hardening, round 3, the last argv-layer round (theseus-905; 2026-09-28, 02:57–03:46; dc3ed99)

Removed 2026-09-28 in dbd7567; see the reversal below.

What exists.

- **Every simple variable reference resolves from the script's literal assignments**, not only the program word. Resolution reaches:
 - words behind wrappers (`command`, `exec`, `env`, `timeout`, ...);
 - the text `eval` runs, a nested `bash -c "..."`, and `$(...)`, backtick, and subshell bodies, which inherit the enclosing scope;
 - arrays (`a=(...)`, `"${a[@]}"` word by word);
 - `for` lists, and `set --` positionals (`shift` and function bodies make the positionals unknown);
 - embedded references (`$x$y`, `--$flag`), as a product of candidates capped at 64.

The resolved value follows bash's rules. Unquoted, it is split; with a glob character, it gets the glob flag; if the script assigns `IFS`, it stays dynamic.

- **A dynamic word's literal option prefix is certain**, so `-r$(printf f)` counts as `-r`.
- **An unresolvable git subcommand** (`git $x -f`) is judged as each subcommand a git rule reads.
- **Notices and refusals show the resolved command and where each value came from:** `rm -rf somedir (`$DIR` = `somedir`)`.
- `RULES_VERSION` is 2026-09-28.2.

How it is proven.

- **The run did not finish.** OpenClaw's stuck-session watchdog aborted the subagent at 03:46:25 (openclaw-jygw). The gateway had stopped seeing its tool events at an `Edit` that started about 03:31, although the agent kept working: it made commit dc3ed99 at 03:37. So its report, live check, and FAST and friction numbers were never written. This record comes from the commit and from Tabitha's review.
- **Tests.** 147; the gate passed at dc3ed99 and again in review.
- **Review probe** (release, finished 08:25:46):
 - All 23 detection cases from all three rounds are caught. They include `set -- -rf somedir; rm "$@"`, `rm "${a[@]}"`, `x="-rf somedir"`; `rm $x`, `rm -r$(printf f) somedir`, `git $x -f`, `y=$x` chains for options, function arguments, and `shift`.
 - All 8 precision cases stay quiet. Among them is the friction round 2 added, which is now removed: `BRANCH=main; git push origin "$BRANCH"`, `SHA=abc123; git reset $SHA`, `DIR=target; rm -rf "$DIR"`, and `branch=feature; git checkout "$branch"`.
 - A wait remains only when a variable comes from outside the script: `git checkout "$branch"` and `git push origin "$BRANCH"`, both unassigned.
 - The floor refuses 20 of 23 spellings. Among the refused are every earlier case, the wrapper and re-parse cases, `$x$y` built from literals, `$'op'`, `"o"p"`, `${x:-op}`, and a function calling `op`.
- **FAST** (release): 488 μ s for a 1.7 KB script with 20 assignments and 30 references with substitutions, and 8.4 μ s for a plain argv.
- **Live, in review** (release build of dc3ed99; scratch daemon; GLM 5.3 flash; `notify`):
 - `x=theseusd; command $x --version` is refused as floor, with the reason `theseusd --version (`$x` = `theseusd`) is never run ... This is final for this request; tell the operator rather than trying another way around it.`
 - `BRANCH=main; git push origin "$BRANCH"` runs with an ordinary notice (`tool.notified`, `approval_skipped`, no kinds), and the bare remote moves from e06a233 to a41ba3e.
 - `DIR=somedir; rm -rf "$DIR"` parks as `bulk_delete`, with the detail `rm -rf somedir (`$DIR` = `somedir`)`. After a decline, the model says it won't route around the refusal, and the files remain.
- **Installed** at 08:32 from dc3ed99.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
The subagent writes the report and runs the live check	The run was aborted; Tabitha did the review probe, live check, and FAST numbers	openclaw-jygw (the gateway lost the tool stream; a single agent, no fan-out)	Recorded here; the evidence is on openclaw-jygw
Over-approximation on every word a rule reads (2a+, 2a++)	Literal values resolve exactly first; over-approximation is left for what stays unknown	Precision: honest scripts with variables no longer wait	Keep

Planned	Actual	Why	Disposition
P5c goes from 2a to 2b	2a+, 2a++, and 2a.3 came between them	Three review probes	Recorded here. After 2a.3, the argv-layer long tail goes to Jev <code>security.v1</code> (M5, or earlier in shadow) and M4's boundary

Known gaps (theseus-1eh, deferred; Eddie, 2026-09-28: "I don't particularly mind the possibility that strange content could be hijacked in as e.g. `base64 ... it would have to be unwrapped then run`"):

- The floor misses a program reached through a variable copied from another (`x=op; y=$x; $y`), and values set by `printf -v` or `read`.
- Obfuscated inline code and scripts that are written and then run stay `opaque`. Beyond the gate, the defenses are model judgment, the notices, Jev `security.v1` (tighten-only), and M4's boundary.
- Cosmetic: nested backticks in a resolved reason break markdown rendering.

The reversal (theseus-8az, 2026-09-28)

What had been built. Steps 2a through 2a.3 (theseus-770, theseus-xbg, theseus-0tv, theseus-905) ran from 22:30 on 2026-09-27 to their install at 08:32 on 2026-09-28. They taught the gate to name what a call would do to the world:

- nine consequence kinds under one `irreversible` property, and the irreversible column on the `strict | ask | notify | open ladder`;
- a shell parser that found every command a `proc.run` call would start, and a versioned rule table over argv, with examples as its tests;
- `opaque` for whatever the parser could not read, and a floor that also scanned code it could not parse;
- `theseus policy replay` and `theseus policy rules`.

Each review probe found spellings that still got through, and each round of hardening closed them and made the parser larger. By 2a.3 the two detection files, `shell.rs` (2,499 lines) and `consequence.rs` (2,568), were the largest in the codebase, and `policy.rs` had grown from 667 lines to 1,978.

Why it was removed. Eddie reversed the direction on the morning of 2026-09-28. Detection written by hand cannot be finished: "attempts to find embedded commands by hand -- that can **never** work" (emphasis his). The operator should be "asked, sure, but a hard no, almost never", because "Theseus should notify over block -- the operator should /know/ when something bad is going to happen", and because "we were responsible for running theseus on a default-safe environment." He added a general rule the same morning: "we should *ruthlessly remove complexity*" (emphasis his). The order he approved was a read-only scan of the vault's item titles and categories first, to anchor what "risky" means (its findings stay out of this document), then the teardown, then the replacement.

What replaced it. The commits, each behind the gate, signed, and pushed:

- **The teardown, dbd7567** (reviewed and installed 10:38). `shell.rs`, `consequence.rs`, their scenario tests, and a template fixture were deleted: 5,911 lines. Every other file the chain had touched went back to 3877fe9 byte for byte, except the `fs.patch` fix that refuses a deletion which does not show every line it removes. Rust went from 35,151 lines to 27,489, tests from 147 to 114, and `policy.rs` from 1,978 lines back to 667, before the ladder was added.
- **The ladder, 09bad8b and 78b7fd9** (reviewed and installed 11:31). One posture that every tool and MCP inherits, with `[policy.tools]` and `[policy.mcp]` overrides. The template lists every tool, and a test binds the list to the registry. The floor asks at every posture and sees path arguments. The posture shows in `theseus tools`, in history, and on every surface.
- **No deny anywhere, 2f4a285, 3c9d0c3, and a046f53** (Eddie's decisions at 11:36; reviewed and installed 13:26).
 - `deny` left the postures and the kernel's band, so the gate cannot express a refusal.
 - The deny lists became approve lists, and a path outside the roots waits.
 - The floor keeps the token file whether it came from the flag or the environment.
 - `op` and `theseusd` left the default approve list, since the floor covers them.
 - A declined call reads "not run".
 - The config is parsed once, unless an old key name appears.
- **The stored vocabulary, e49a363** (Eddie's approval at 16:00; reviewed and installed 16:21). A decline is stored as `declined`, the ledger kind is `action.declined`, and the resolution reads "declined by". Rows stored under the old names still decode and render. **Rolling back past e49a363 is unsafe once a decline has been stored this way**, because an older binary has no `declined` value. Upgrades from here are forward only.
- **The catalog, 274facd** (`theseus-px4`; not part of the gate, landed the same evening; installed). Claude Sonnet 5.5 joined the catalog, version 2026-09-28.1, and became the default model: the built-in default, and the template's `[model]` and `[profiles.sonnet]`.

After all of it, Rust is 28,523 lines, `policy.rs` is 899, and there are 135 tests. `decide` parses no shell. It takes about 4.5 μ s per call in a debug build; under 2a.3 it took 8.4 μ s on a plain argv and 488 μ s on a 1.7 KB script, in release.

How it is proven.

- The gate passed at every commit and again in each review: 114 tests after the teardown, 126 after the ladder, 133 after no deny, and 135 after the vocabulary.
- Live, on scratch daemons:
 - a `proc.run` under `notify` ran with a notice, and `"proc.run" = "approve"` made the same call wait;
 - under `open`, the floor made three calls wait, each marked as the floor: a read of the file named as the token file (a decoy), a listing of the store, and `theseusd --version`;
 - a read outside the roots waited, and ran once approved;
 - `sudo -n true` waited on the approve list, and its decline read "not run".
- Eddie's vault config loaded under the teardown, ladder, and no-deny binaries, in read-only checks. Until its old key names are renamed, every start logs one warning per old key.

- On a copy of Eddie's store, rows written by the chain builds decode under the restored types. An old "denied" decline renders as "not run", and `theseus ledger --kind action.declined` returns the old row.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
IRREVERSIBLE WAITS (§2); the consequence kinds, rule table, <code>opaque</code> , replay, and "should have asked" flow (§3.9; P5c item 1, steps 2a to 2a.3)	Built, hardened three times, then removed in dbd7567. Only the <code>fs.patch</code> deletion fix was kept	Detection of hidden commands can never be complete. Each review found spellings that got through, and the cost was the two largest files in the tree	Part I §2 and §3.9 marked superseded; §3.9 rewritten to the gate as built
theseus-8az as first stated: one Jev risk classifier (0–100) per call, feeding the ladder	Not built. The gate is a posture ladder over the finite tool and MCP lists	Eddie's direction at the ladder step: the tool and MCP lists are the surface the operator controls	Jev stays M5 design, stricter only (§3.9, Jev). When is held for Eddie
The ladder <code>strict ask notify open</code> (§3.9, v0.37)	Postures <code>open notify approve</code> , inherited by every tool and MCP, with <code>[policy.tools]</code> and <code>[policy.mcp]</code> overrides. The first cut (09bad8b) also had <code>deny</code> , and 2f4a285 removed it	Notify over block: the operator is asked, and almost never told no	Kept
The floor refused at every level (§3.9)	The floor waits for approval at every posture and says it is the floor. It sees path arguments, and it keeps the token file however the daemon was given it	Eddie's floor decision (09:30): never a hard block, never a silent notice	Kept
<code>deny_argv</code> , <code>deny_paths</code> , and a path outside the roots refused (A3)	<code>approve_argv</code> and <code>approve_paths</code> : a match waits, and so does a path outside the roots. The old key names load with a startup warning	No deny anywhere	Keep the aliases until the vault config is renamed
The class modes <code>read</code> , <code>write</code> , <code>run</code> (A3)	Retired: the old template's values load with a warning and are ignored; any other value fails to load	Eddie's live config still had them	Remove when his config is updated
A decline stored as <code>denied</code> (A3)	Stored as <code>declined</code> (<code>action.declined</code> , "declined by"); rows stored under the old names still decode and render	"deny" nowhere, for consistency (Eddie, 16:00)	Forward only: rolling back past e49a363 is unsafe once a decline is stored
A request Eddie is not permitted to make is blocked at the gate with a clear message (P5, the M3 prove)	It waits for approval with its reason, and a decline means it never runs	No deny anywhere	Recorded here; P5 is left as written

Planned	Actual	Why	Disposition
An owner override for any refusal, and the floor's typed ceremony (§1, §3.9; step 2c, theseus-qc4)	Nothing in the gate refuses, and the owner can approve whatever waits	Follows from no deny	The floor ceremony is moot. theseus-qc4 is held for Eddie
The exposure rule, one level stricter on the old ladder (§3.9, M4)	Restated on the three postures: <code>open</code> behaves as <code>notify</code> , and <code>notify</code> as <code>approve</code>	The ladder changed	Part I §3.9 updated; built in M4
The consequence boundary under L1 (§7, P6)	There is no consequence left for it to name	The kinds are gone	Held for Eddie, with M4

Open, held for Eddie.

- theseus-qc4 (the owner override): close it, or keep it for a later layer that refuses.
- Appendix F's M4 item "content refused by label class wherever it leaves" is a refusal. Under `notify` over block it may become a wait.
- §7's boundary: should a credential request, or a recognized request shape, `notify` or wait?
- The Jev risk classifier: when, and whether.
- Under `approve`, the allow list still runs `ls` and `pwd` inside the roots (the ladder step's Q2, unchanged).
- A hook may still block a call (§3.17; P5c item 2). A hook is not the gate, so it was left as designed. (*Resolved 2026-09-28: the hook system was deleted; see the complexity cuts, below.*)

The complexity cuts (theseus-hco; 2026-09-28, 19:16–23:53; 95ee929, bb3ce56, b242258, 11d2f43, 8a2b503, 54b0918)

Why. Eddie set the rule on the morning of 2026-09-28: "ruthlessly remove complexity ... If we reach a supremely exquisitely simple codebase to reach 80% security, that's so much better than a complex codebase that reaches more at the price of brittle patterns, brittle code." With the gate tower gone (the reversal, above), a read-only review looked for what else the tree carried that it did not need.

The review (19:16–19:48, at 8a1e41d; nothing changed). 21 ranked findings in 28,523 lines of Rust: 22,803 production and 5,720 test. Four things dominated:

- speed traps that grow with history: `session.list` re-decoded every action once per session, 2.87 s at 400 sessions on a debug build, polled every 2.5 s by the Observatory, and startup decoded every node in the store to seed a counter;
- 27 fsyncs for one plain turn, 10 of them `hook.site` rows;
- machinery with no production caller: a second kernel session model, the kernel's `Policy` trait wired through an adapter, and a simulator that ran in no test and no gate;
- giant functions: `TurnRunner::run_inner` at 785 lines, the CLI's `run()` at 652, and the RPC `dispatch` at 496.

In all, it proposed cutting about 2,600–3,200 production lines and 39 of 325 external crates, and bringing a plain turn from 27 fsyncs to about 8. Eddie approved its five do-first cuts at 20:38: "do all of your suggestions, including 2, and add that feature". At the same time he asked for Narration: "Let's remove the hooks. But let's add a new feature: Narration." Findings 3 and 8 went to M3.5 (P5b), and findings 10 to 16 and 18 to 21 are batch C (theseus-0g4).

What each cut did. Each commit went through the gate, signed and pushed, and each was reviewed and installed.

1. **Findings 1a and 7: the quadratic and the start-path scan** (95ee929; batch A). `session.list` scans the open actions once, not once per session. Startup no longer decodes every node to seed the tool-call counter, and the GitHub token check runs after the socket binds.
 - `session.list` at 400 sessions went from 3,130 ms to 45 ms, and it now grows linearly (debug build, tmpfs, fake provider).
 - Cold start to the first `health` went from 1,300–1,397 ms to 1,096–1,181 ms on an empty store, and from 1,521–1,655 ms to 1,281–1,331 ms on a 2,000-session store. About 1 s of what remains is secrets through `op` (P5b).
 - `tool.list` now counts calls since the daemon started.
2. **Findings 5 and 9: the rest of the gate teardown** (bb3ce56; batch A). The kernel keeps only `Proposal`, `digest_proposal`, and a new `digest_json`. The `Policy` trait, `run_gate`, `GateTrace`, `GatePolicy`, and `Mode` are gone: the toollet plans a call, then `ToolPolicy::decide` decides it. One serializer replaces the two canonical-JSON copies, and golden-digest tests prove every digest byte-identical. Production lines fell by 248, and `gate.rs` went from 151 lines to 36.
3. **Findings 6 and 17: dead kernel API, and the simulator in the gate** (b242258; batch A, finished in review). The kernel's second session model (`Session`, `open_session`, `promote`, `session_tail`) and `KernelError::PolicyDenied` are gone. The crash test and the kernel simulation run in the gate, the simulation on the product's own `open_execution` path. The crash test's `--tear` defaults off (theseus-4x6). On a copy of Eddie's store, the first `health` took 1,132 ms and `session.list` 3 ms.
4. **Finding 2: the hook system** (11d2f43). Deleted: `hooks.rs` (25 events, 4 kinds, the registry, and the outcomes), its 11 call sites in the turn and three in the RPC server, `hooks.*`, `theseus hooks`, and the `hook_spans` mode.
 - A plain one-loop turn went from 27 frames and fsyncs to 17, and a two-loop turn with one `fs.read` from 45 to 29.
 - The median of 20 plain turns went from 211–224 ms to 134–142 ms (debug build, before and after interleaved).
 - Production Rust went from 22,177 lines to 21,618, 559 fewer, and tests from 140 to 138.
 - A new test fails the gate if a plain turn writes more than 17 frames.
 - Eddie's config loads, with one warning for the retired `[telemetry] hook_spans`. Old `hook.site` rows and hook spans still decode and render, and `hooks.list` answers "method not found".
5. **Finding 4: the lost cost, then the split of `run_inner`** (8a2b503, 54b0918).
 - **The failing test came first.** A turn fails in its second loop, once on a provider error and once on the budget. On the old code it found 18 lost facts. After a provider failure, the session and health lost the finished loop's cost (\$0.30 in the test) and its tool call. After a

budget failure, the session recorded nothing. Neither exit put cost or tool calls in `turn.failed` or in the error.

- **The fix** (8a2b503). One function closes the books on success and on both failure exits. `turn.failed` gains `cost_usd` and `tool_calls` beside its `usage_so_far`, and the error's data gains `usage`, `cost_usd`, and `tool_calls`. A turn that fails on the budget now counts in the session's turns, as a provider failure always did.
- **The split** (54b0918). `run_inner` became 11 named steps, with one `fail()` for both failure exits. No function in `turn.rs` is over 113 lines, where one was 676, and clippy at the review's strict thresholds finds no cognitive-complexity or nesting warning in the file.
- **Behaviour is identical**. An output-diff probe over 8 scenarios dumped every frame, ledger row, notification, session total, result, and trace, on the fix and on the split. Each dump was 37,791 lines, and after masking ids, digests, and times the diff was empty.

How it is proven.

- The gate passed at every commit, and again in each review: 136 tests at 95ee929, 138 at bb3ce56, 140 at b242258, 138 at 11d2f43, and 139 at 8a2b503 and at 54b0918.
- Live checks on scratch daemons:
 - batch A, on a copy of Eddie's store, with a real Sonnet 5.5 turn;
 - cut 2, with rows the installed daemon wrote read by the new binaries, and again on a copy of Eddie's store;
 - cut 5, on a copy of Eddie's store: a GLM `fs.list` turn ran 2 loops and 1 tool for \$0.0008, and the session's books matched the ledger.

Eddie's vault config loaded under each new binary.

- Installed: batch A at 21:54, cut 2 at 23:04, and cut 5 at 23:52.
- Two runs since v0.42 were aborted by OpenClaw's stuck-session watchdog (`openclaw-jygw`) and finished in review. One is batch A, at 21:25:56 during its item 3, with 12 files edited and nothing committed. The other is the budget step (A4, item 1).

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
M0's hook registry: every event typed, handlers registered in code and over the protocol, and the run-hooks path at each site (P2; §3.17 as it stood). P0 rule 3's hook half: every hook event lands with a reader, and the registry test enumerates them	Deleted in 11d2f43, with <code>hooks.*</code> , <code>theseus hooks</code> , and the <code>hook_spans</code> mode	No handler could do anything: <code>Blocked</code> and <code>Claimed</code> were never built, and 10 of the 25 events were never dispatched. A plain turn still paid 10 of its 27 frames for it. Eddie, 20:38: "Let's remove the hooks."	Part I §3.17 rewritten, and its other mentions amended. Part II is left as written, including P6's hook points and M5's hooks bullet (P7). Rule 3's edge and label half stands (<code>theseus-wjy</code>)

Planned	Actual	Why	Disposition
Wire the seven unwired sites, make three honor their results, and prove a blocking handler per gating point (P5c item 2; A3's hook-site row; theseus-0dp)	Moot: no sites remain	Follows from the deletion	theseus-0dp closed as superseded
One small hook point to replace the system (theseus-bdn)	Deferred	"Add that feature" meant Narration; it was first read as the hook point and corrected at 23:44. The hook point was Tabitha's suggestion in the review, to bring one back "when a real handler exists", and none does	Deferred until a real handler exists (§3.17)
Cut 5 saves about 130 lines (the review)	turn.rs grew by 113 lines, from 1,134 to 1,247	The estimate assumed one payload per fact for the trace, the ledger, and the notification. Those payloads differ field by field, and the cut had to keep behaviour identical. Each of the 11 steps also pays a signature and a doc comment	Kept for its shape
tool.list keeps durable per-tool counts, seeded at startup (A3)	Counts since the daemon started, so the shell-fallback ratio (§3.23) covers the current run	The seed decoded every node in the store on the start path (finding 7)	Lifetime counts can return cheaply with an index (finding 1b, M3.5)

Known gaps.

- A fault after a paid loop still skips the books. A raw error inside the loop, such as a cancel between a provider settle and a tool's plan, is not a classified failure, so the session misses the loops that already ran.
- OpenTelemetry's `record_failure` takes no usage or cost, so the OTel counters miss a failed turn's finished loops. Eddie's config has telemetry off.
- The failure path ignores the result of its session write (`let _ = put_session`).
- The frame-budget test asserts at most 17 frames. It is tightened to the new count when batching lands (P5b).
- The crash test's `--tear` stays off until theseus-4x6 bounds it.

Batch C, part 1: the low-risk deletions (theseus-0g4; 2026-09-29, 04:22–05:11; cf258c7, ea06ff8, b407dd3, f8a68c2, 605979e)

Why. The review's remaining findings, queued before new features so they land on simpler code. Batch C runs in three parts: this one takes the five low-risk deletions (findings 21, 15, 16, 20, and 14), part 2 takes findings 10, 13, and 18, and findings 11, 12, and 19 wait until after the Daily Driver (M3.6).

What each cut did. One gated, signed commit each, in this order.

1. **Finding 21: dead code and lint cleanups** (cf258c7). Eight items no code called went, with `kinds::CHECKPOINT` (255) and the protocol's `BLOCKED` (-32001) kept as reserved numbers; two test-only constructors are `#[cfg(test)]`; 24 of the review's 25 lint sites were fixed (the 25th is `Core::new`, finding 10's); and a workspace `[lints]` table holds `unused_async`, `unused_self`, and `redundant_clone` from now on.
2. **Finding 15: one index engine** (ea06ff8). `fjall`, the `Index` trait, the engine parameter, the simulator's `--engine` flags, and the format-1 move-aside are gone, and the manifest is read once. `Engine` has one variant, so the manifest and `[server] store_engine` still name it and refuse anything else. Eddie's store is format 2, `redb`, so it needed no migration.
3. **Finding 16: config shims** (b407dd3). The `theseus-8az` renames (`deny_paths`, `deny_argv`), the retired policy class keys, `telemetry.hook_spans`, and the `max_tokens` alias fail to load as unknown keys; the retired `[kernel] default_budget` and `control_reserve` still load with their one warning, because Eddie's note sets them. `effort`, `thinking_display`, and a provider's `kind` are `serde` enums, and the implicit profile and provider are resolved once at load.
4. **Finding 20: policy leftovers** (f8a68c2). The allow list's flag blacklist and its near-miss text are gone. An allow entry is a prefix, documented, and still runs only when every path argument is inside the roots. `[policy.mcp]` and its overrides stay (Eddie, 2026-09-28).
5. **Finding 14: OpenTelemetry behind** `otel` (605979e). The exporter builds only with the cargo feature, off by default; `[telemetry]` parses the same, and a set endpoint warns once. The gate lints the feature (about 1.5 s).

measure	before (48fd2c8)	after (605979e)
crates in <code>theseusd</code> 's default build	331	292
external crates, workspace	325	286
<code>theseusd</code> release binary	24,281,704 B	19,021,104 B
clean release build, 8 jobs	253 s	218 s
production lines, total / compiled by default	25,815 / 25,815	25,474 / 25,013
gate tests	204	198

How it is proven. The gate passed at every commit (204, 204, 202, 202, 202, and 198 tests). Eddie's vault note loaded with the same single warning under the binaries after findings 16, 20, and 14 (`theseusd check` on its `op://` reference), and a copy of it with Discord and the web UI off did under those after 21 and 15; after 16, `theseusd config` printed the same bytes. On copies of Eddie's store

(only `store/`; Discord and the web UI off; never port 7433), the new binaries opened it, served it, and printed byte-identical `theseus sessions` and `theseus history`. The final check ran a GLM `fs.list` turn: 2 loops, 1 tool call, \$0.0013, with its `turn.trace` row in the ledger.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
Two <code>Store</code> implementations, <code>redb</code> and <code>fjall</code> , behind a trait, <code>fjall</code> selectable by config (M1; \$1, \$6)	<code>redb</code> only; the trait and <code>fjall</code> are gone	M1 chose <code>redb</code> , and a store never switches engines	\$1 and \$6 revised (drafts in the run's report)
OpenTelemetry on by default, always compiled in, a config change to enable (\$3.20; <code>theseus-vng</code>)	Compiled only with the <code>otel</code> feature, off by default	19 crates, <code>aws-lc</code> among them, for an export no deployment enabled	\$3.20 revised (draft)
Old config spellings load with a warning (<code>theseus-8az</code> , <code>theseus-hco</code>)	They fail to load as unknown keys, except the two Eddie's note still sets	No deployment uses them	Eddie's note loads unchanged
The allow list never covers nine git and curl flags (<code>theseus-8az</code> , 09bad8b)	Removed; the roots rule stays	It only second-guessed entries the operator added	Allow entries documented as prefixes

- **Review** (05:22). The gate reran at 198 tests. Tabitha's own check on the release build, over a store copy, listed Eddie's 5 sessions and ran a GLM `fs.glob` turn (2 loops, 1 tool call, \$0.0005), with no error in the log. Eddie's `allow_argv` is `[["ls"], ["pwd"]]`, so dropping the flag blacklist changes nothing for his config. Installed at 05:22.

Known gaps.

- The `otel` exporter's tests do not run in the gate, which only lints the feature.
- The default test build still compiles `opentelemetry` and `opentelemetry_sdk`, a dev-dependency cargo cannot gate on a feature.
- `core::new` still takes the resolved secrets by value, and the functions over six arguments stay for part 2 (findings 10 and 18) and after the Daily Driver (finding 19).

Batch C, part 2: the structural cuts (`theseus-0g4`; 2026-09-29, 05:23–06:21; 750ec12, 6e76a5f, 69851ad)

Why. These three findings shape the code the next Daily Driver items touch: durable delivery goes through the tool runtime and `core`, and task sessions go through `core` and the kernel. Eddie's rule is "ruthlessly remove complexity", and behaviour stays identical.

What each cut did. One gated, signed commit each, in this order.

1. **Finding 13: pending confirms, one way** (750ec12).

- Before, "what waits for the operator" was computed four ways: a node scan, an open-action scan, the CLI's fan-out, and the transcript re-read that `confirm_action` and `resume` each did to find the proposal.
- Now `Action::awaits_confirm` and `Kernel::pending_confirms` are the one derivation. The history, the session list, a new `confirm.list` (so `theseus confirm` is one request), the web UI, and `resume` all use it.
- An action that waits for the operator keeps its `Proposal`. That is a tool call the policy stopped (`plan_confirm_with`) or a budget question. Other actions keep none, so the transcript holds the arguments once.
- An action stored before this decodes without a proposal and still waits. Its proposal comes from its node's gate record, in one place (`confirm_proposal`).
- The simulator checks that a question carries its proposal.

2. Finding 18: `toolrun.rs` (6e76a5f).

- A `ResultNode` value replaces the 11-argument `result_node` and its `#[allow]`.
- `process` is a list of named gate steps, with one `plan_call` for both postures.
- `execute` splits into `run_inproc` and `run_job`.
- `resume` places each unanswered call (`Pending`), then acts in one flat match: 261 lines became 61.

3. Finding 10: `rpc.rs` (69851ad).

- `Core`'s six jobs have a file each under `rpc/`.
- `dispatch` is a 49-line table of named methods, where it was 462 lines. `route` parses, runs, and serializes, and `?` is `INTERNAL`.
- One constructor path: `Core::new(cfg, secrets, store)` still takes the secrets by value, so the daemon keeps no copy, and builds `Parts` for `Core::build`.
- `BindingBoard` counts starting bindings with a watch channel instead of a sleep loop.
- `health` reads the sessions once, the `turns` counter is gone, and `Config::profile` owns the unknown-profile message.

measure	before (605979e)	after (69851ad)
production lines (not cutting at a test-module declaration)	25,487	25,930
longest function in the rewritten files	457 (<code>dispatch</code>)	141 (<code>Core::build</code>)
functions over 60 lines there, and their total length	9, 1,583	10, 848
functions over 6 arguments there	5 (2 behind <code>#[allow]</code>)	2 (the kernel's planning pair)
gate tests	198	201
frames for a plain turn	17	17

How it is proven.

- The gate passed at every commit.
- An output-diff probe dumped every WAL record, frame count, notification, narrative line, and result for 26 scenarios. These include `confirm approved`, `declined`, `superseded`, `answered after a restart`, `pending across the upgrade`, and `a budget question approved and declined`. Its

masked diff was empty between runs, empty across findings 18 and 10, and across finding 13 showed only the proposal on the 56 records of actions that waited, and the new method.

- On copies of Eddie's store (only `store/`; Discord and the web UI off; never port 7433), the old and new binaries printed byte-identical sessions, histories, and confirm lists.
- A GLM turn's `fs.write` outside the roots waited, was listed by `theseus confirm`, was approved, and resumed and wrote the file.
- **Review** (06:48–06:51). The gate reran at 201 tests. Tabitha's own check on the release build, over a store copy, listed Eddie's 5 sessions and an empty `confirm.list`; a GLM `fs.write` outside the roots waited, `theseus confirm` listed it, the approval resumed the turn, and the file held the text. Installed at 06:51.

Divergence from Parts I and II, and from the review.

Planned	Actual	Why	Disposition
An action's arguments are a node and are not duplicated on the action (§3.16; Part II M2)	An action that waits for the operator also keeps its proposal	The confirm binds it and <code>authorize</code> re-checks it; every approve path went back to the transcript for it	Recorded here; the Part II text stays as the plan
The review: store the proposal on every action at plan time	Only on actions that wait	Each action record is rewritten at every transition, and <code>session.list</code> (polled every 2.5 s) decodes every action ever stored	Recorded here
Pending confirms derived from planned actions and their nodes (Part III M3)	From <code>Kernel::pending_confirms</code> ; the node is read only for a question's text	One derivation for every reader	Recorded here
The review's estimate: about –340 production lines for the three findings	+443	Named functions and types with their docs, per-file docs, and every payload kept identical	Accepted; the structure, not the count, was the target

Known gaps.

- `Kernel::cancel_execution` leaves a planned tool call planned (theseus-w98), though its comment says planned actions are cancelled. Such a call still counts in the history and the session list.
- A crash between a call's plan and its authorization leaves an action that still "awaits a confirm" that was never asked.
- `open_session` and `execution_for` still differ in their narrative and ledger rows.
- The open-action scan still decodes every action (finding 1's index).
- `run_job` (109 lines), `Core::build` (141), and `turn_submit` (88) are still long.

Narration (theseus-5fy; 2026-09-28, 23:53, to 2026-09-29, 00:55; e3ba8d6, 810aa6d, 12a4805)

Neither a P5c item nor a plan item. It is a request of Eddie's (2026-09-28, 20:38; his words are in §3.14), recorded as an addition to M0.6's visibility (P2b). It took the hook point's slot in the queue.

What exists.

- **Config.** A top-level `narrative = true`, the first field of the config, so `theseusd config` prints it before any table. Absent or false means off. Written under a table, it belongs to that table, and the config fails to load (unknown field `narrative`). The template sets it to true, just before `[model]`.
- **The narrator** (`theseus-core/src/narrative.rs`) holds a sequence counter, a 500-line tail in memory, the subscribers, and the sessions met since start (for "resumed"). Every emission is a macro that tests the flag first, so with narration off no argument is evaluated.
- **Eight parts:** session, turn, loop, context, model, tool, approval, and job. Each line carries `seq`, `at_unix_ms`, `part`, `session_id`, `turn_id` when there is a turn, and `text`. The report (`~/reports/theseus-5fy/narration.md`) lists every narrated point with its file and line. Startup, the Discord binding, profile switches, and `session.recompile` requests are not narrated; the recompile a request causes is.
- **What a sentence may carry.** A tool's subject is its first path, relative to the working directory. For `proc.run` it is the program's basename, plus `argv[1]` when that looks like a subcommand (a lowercase word of at most 24 characters), and never the rest of the `argv`. The gate's "why" is the rule from the gate's reason, with the `argv` replaced by "the command". Both pass through the secret scrubber. Input shows only as a character count, and a result as lines, bytes, and an exit code.
- **Protocol.** `narrative.watch` (the tail, then `narrative.line` notifications), `narrative.unwatch`, and `narrative` in health. With narration off, both requests answer `DISABLED` (-32004), and the message says how to turn it on. `seq` restarts at 1 with the daemon, and a client merges the tail and the live lines on it.
- **Web.** When `health` says `narrative`, the side pane has two tabs, the Observatory and The Narrative (`web/src/Narrative.tsx`). Otherwise the pane is the Observatory exactly as before, and nothing calls `narrative.watch`. A line's session link shows the short id, and its tooltip the title and the full ids (12a4805, from the live check).
- Since `theseus-0sg` the narrative speaks dollars, and since `theseus-58a` its context line counts context files (A4).

How it is proven.

- **Tests.** The failing tests came first: against the skeleton, 5 of the 13 failed. All pass now, and the workspace ran 151 tests, with the gate green before each commit. They cover:
 - off, which makes no line and refuses a watch;
 - a two-loop tool turn narrated part by part in exact order, with no input or file text in any line;

- a failed turn that narrates its class and what its finished loops spent;
 - a late subscriber that gets the bounded tail, then every line;
 - the frame budget (17) with narration on and a watcher;
 - the config key;
 - an approval and a background job, with no line carrying the command's script.
- **FAST** (release, interleaved over five rounds). The median of medians of a plain turn was 118.9 ms on the parent, 116.5 ms off, 117.2 ms on, and 117.3 ms on with a watcher, all within noise. A line costs 1.43 μ s with a watcher, 0.44 μ s on without one, and nothing off: about 13 μ s per plain turn with the tab open.
 - **Live** (2026-09-29, 00:41–00:46, a scratch daemon over a copy of Eddie's store).
 - On: a GLM turn that read a file showed its 17 lines live in the tab, in order, and a tab opened afterwards got the same 17 from the tail.
 - Off: health said `narrative: false`, both requests answered `DISABLED`, the same turn ran normally, and the page had no tab.

The screenshots are in `~/reports/theseus-5fy/`.

- **Review** (00:55). The gate reran at 151 tests. An independent check on the release build, over a store copy, got 17 narrative lines from a GLM glob turn through `narrative.watch`. Installed at 00:54.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
M0.6's windows onto a turn: the trace, the ledger, and telemetry (P2b)	A fourth, the narrative, written for a person watching live and never stored	Eddie's request, 2026-09-28, 20:38	Part I §3.14 records it
"a new pane called 'the narrative' ... a new tab in the web UI" (Eddie)	A tab in the side pane, beside the Observatory	The transcript stays in view beside it	Kept
The first build named each line's session by its title	The link shows the short id, and the title is in its tooltip	A title is the first words of a prompt	Kept (12a4805)

Known gaps.

- Subscriber queues are unbounded, as for every notification. A stalled tab grows its queue by about nine lines per turn.
- Two sentences print a kernel error's text. They are the only free text in the narrative that is not a fixed template.
- A `proc.run` subject may include `argv[1]`, so a secret that is a plain lowercase word, and is not in the vault, could show there.
- Eddie's daemon stays off until his vault note has `narrative = true` at its very top, before `[mode1]`, and the daemon restarts.

Step 2b, part 1. Approval from trusted channels and trusted users (theseus-sgh; 2026-09-29, 06:52–07:25; 8fe368f, 795356f)

What exists.

- `[approval]` with `trusted_users` (`discord:<user id>`) and `channels` (`cli`, `web`, `discord:dm`, `discord:<channel id>`). It is optional; the template has it commented, with Eddie's DM as the example. A bad entry fails to load and names the forms. There are two warnings, both only when the section exists: an empty `channels`, and Discord listed with nobody trusted.
- One judgment, in `Core::confirm_action`, before the budget branch, so tool calls and the budget question pass the same rule. A refusal is error `REFUSED` (-32005) with `{who, via, why}`, an `approval.refused` row, and a narrative line; the action and its execution do not move. `action.confirm_answered` rows gain `via`.
- A typed surface per connection: `serve_connection` takes a `Client {label, surface}`, and the socket, `--stdio`, the web bridge, and the binding each name theirs. `action.confirm` gains `discord: {user_id, channel_id, guild_id}`, taken only from the binding's connection.
- Discord:
 - The binding sends the ids with each button press.
 - A refused press keeps the card's buttons and gets an ephemeral reason.
 - Cards for an untrusted place go to a trusted DM (`Op::InDm`) with a one-line note in the place.
 - Listed guild channels are checked with twilight's `PermissionCalculator`. The Server Members intent is read from the application's flags, and health reports it as `bindings[].members_intent`.
- Health's approval `{configured, trusted_users, channels[]}`, a `theseus health` line, and the Observatory's Approval section and ledger summaries.

How it is proven.

- **Review** (07:48–07:52). The gate reran at 222 tests. Tabitha's own check on the release build over a store copy: with `channels = ["web"]` a CLI answer was refused with the reason and ledgered, and after a restart with `channels = ["cli"]` the same answer approved and the call ran. Installed at 07:52. The review found `theseus-6qy`: a `proc.run` job runs as the operator and can reach the CLI socket and the web UI, so with `cli` or `web` trusted it can answer its own session's approvals until M4's boundary; mitigations are held for Eddie.
- **Tests**. 222. They include:
 - Without `[approval]`, the existing confirm, decline, supersede, and budget-reset tests pass unchanged, and every surface approves over the protocol.
 - With it, a trusted user in a trusted channel approves. An untrusted user, a trusted user through an unlisted surface (the CLI, the web UI, an unlisted guild channel), and a forged Discord claim are each refused with the reason, and the call keeps waiting. The budget question follows the same rule.
 - The renderer routes an untrusted place's card to the DM with a note.
 - A guild channel without the intent is not trusted.
 - Eddie's config shape loads with no rule and no new warning.

- The frame budget still holds (a plain turn is at most 17 frames).
- **Live**, on a scratch daemon over a copy of Eddie's store, with Discord and the web UI off. Eddie's note loads under this build with only the known budget-units warning. With `channels = ["web"]`, theseus confirm on a waiting `fs.read` of `/etc/hostname` was refused with the reason (exit 1, `approval.refused` ledgered), and the call kept waiting. After a restart with `channels = ["cli"]`, the same answer approved and the read ran. Discord was not exercised, because one bot token means one daemon.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
Theseus checks who can see a guild channel when it posts, and again when an answer arrives (§3.9)	Checked at startup, before each card, and at each answer, for listed guild channels only, and only when the portal has the Server Members intent on. The bot's gateway intents lack <code>GUILD_MEMBERS</code> , so today a listed guild channel is not trusted	The member list needs the privileged intent. Asking for it on the gateway while the portal has it off closes the gateway (4014), so the binding reads the portal setting from the application's flags and fetches members over HTTP	Kept. Eddie decides whether to turn the intent on
Every trusted channel's members are all trusted users	<code>cli</code> and <code>web</code> need no trusted-user entry	Their member is the operator of this machine: the socket is 0600; the web UI is loopback-only, so any account on the machine can reach it, which makes listing it a choice	Proposed as the spec's reading (5a)
The approver must be a trusted user and hold the capability	A trusted user only	There is no capability model yet; every execution's principal is the operator	Open until M4
The confirm goes to the person who issued the request (§1)	A card for an untrusted place goes to the DM of the place's latest author when trusted and bound, else to the first trusted DM	A Discord place knows message authors, not a principal	Kept
—	<code>channels</code> defaults to <code>["cli", "web"]</code> and <code>trusted_users</code> to none once the section exists	Safe defaults: the section alone takes Discord out until Discord is listed, and leaves the local surfaces as they are	Held for Eddie (6)

Known gaps.

- Discord has not run live; the first run is Eddie's daemon on this build.
- The web UI still shows Approve and Decline when `web` is not trusted; pressing one shows the refusal on the card.
- Each check with the intent is at least three HTTP requests, which is fine for one guild.
- A thread under a listed channel has its own id, so it is not trusted unless that id is listed.

Step 2b, part 2. Should have asked (theseus-sgh; 2026-09-29, 07:52–08:53; 814ec6d)

What exists. One press on a notice makes that tool ask first until it is undone (§3.9 "Should have asked"):

- `tighten.rs`: the tightenings are one `meta` record, `policy.tightenings`, written in the same frame as each `policy.tightened` or `policy.untightened` row, and read once at startup.
- The gate's `posture_now` takes the stricter of the config's posture and the tightening. A tightened call's reason names who tightened it and what the config says.
- `Core::judge_act` is the one judgment for every approval-like act: an answer, a press, and an undo. An undo needs a trusted answer; a press only needs a surface that can answer an approval.
- `policy.tighten` and `policy.untighten`, with notifications to every watching connection. The notice (`tool.notified`, `policy.notified`) now carries its call's correlation id, since it is written after the plan.
- Discord: one "Should have asked..." select menu on each loop's tool message, or a button per card with `notice_embeds`. The web UI has a button by each notice and an Undo in the Tools view. The CLI has `theseus policy tighten|untighten|list`, and the notice line ends with the exact `tighten` command.

How it is proven.

- **Tests.** 236, 14 of them new. They cover: a press makes the next call wait and an undo notifies again; a tightening never loosens and its undo stops at the config; it survives a restart; only a trusted answer undoes it; without `[approval]` every surface can press and undo; the Discord menu and button; and the CLI lines. The frame budget holds (a plain turn is at most 17 frames).
- **Live** (the step's own check, 08:26–08:29, and Tabitha's, 08:51–08:52, on the release build over a store copy): a notified `proc.run` offered the `tighten` command; the press made the next `proc.run` wait with the tightened reason; the tightening survived a restart; the undo put it back to `notify`. With `[approval] channels = ["discord:dm"]`, a CLI undo was refused with the reason and ledgered.
- **The run.** The subagent's run was aborted at 08:31, after its commit and live check. Tabitha finished the report, reran the gate, checked the release build, and installed it at 08:52.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
The "should have asked" button turns a call into a labeled example and a <i>proposed rule</i> (theseus-770, the consequence gate)	It tightens the tool's posture to approve, stored and undoable; the call is kept as a labeled example	There are no rules left to propose since the reversal (2026-09-28); the posture is the one control surface	Kept

Planned	Actual	Why	Disposition
A button on every amber notice	One select menu per loop's tool message on Discord (quiet notices), a button per card with <code>notice_embeds</code> , and buttons in the web UI	Quiet notices put a loop's notices on one message	Kept

Known gaps. Discord was not exercised live, because one bot token means one daemon. The allow list still runs its entries outright under a tightening, as it does under a config `approve`. An undo from a job through the CLI or the web UI counts wherever those channels are trusted (theseus-6qy).

A3c. M3.5 Fast (theseus-qa0)

M3.5 (P5b) runs as these steps:

- F1: serve first, and the lifecycle bench.
- F1b: start from a last-known-good copy of the config note (theseus-2fo, accepted by Eddie 2026-09-29).
- F2: fewer frames and one transcript read per turn.
- F3: parallel tool calls (theseus-a60).
- F4: the swap and restore phases, with versioned readers, and F2's remaining frame merges (theseus-l6y).

Beside them, in the same chain:

- K1: the kernel never loses an update (theseus-id9), found by F3.
- O1: the native OTel exporter (theseus-hee), with theseus-gi7.
- J1: a job cannot answer approvals (theseus-6qy).
- Z1: the daemon reaps its job wrappers, and adopts a job's orphans (theseus-z4b).
- B1: the secret broker (theseus-dcy), with theseus-6uo.

Eddie's order (2026-09-29) is:

1. F1, F2, F1b, and F3;
2. then the native OTel exporter, the self-approval fix, and the secret broker;
3. then the Daily Driver's items 5 to 8, before his end-to-end testing.

F4 comes after that.

Step F1. Serve first, and the lifecycle bench in the gate (theseus-qa0; 2026-09-29, 08:54–09:18 and 09:49–10:52; 10c35c4, 269642f, 36e2173, 460a35b)

Why. Cold start took 1.3 s to the first `health` answer, and 1.0 s of it was `op read s` before the store opened (A3, lifecycle timings). Eddie made FAST a primary goal on 2026-09-27 (§2).

What exists.

- **Serve first.** Secrets resolve in the background into a board:
 - one `op inject` for every reference, `op read` per reference after a failed injection;
 - a retry of what failed at 5 s, doubling to 60 s.

Each consumer waits for its own secret, and none runs without it. A turn waits for its key, bounded at 30 s, or is refused with `secret_failed` or `secret_resolving`. The Discord binding, the GitHub check, and the telemetry exporter wait for theirs. Health says `secrets: resolving | ready | failed <names>`. The ledger gets `secrets.resolved` and `secrets.failed`, and the Observatory has a Startup section.

- **The start path pays one fsync of its own.** The kernel's five `startup.step` rows share a frame. `server.started` goes after serving, in one frame with `server.serving`. The harness loop and the driver start once the socket answers. `redb's open` adds its two.
- **The store's read path.** A read is one `pread` on a kept segment handle, and a batch of positions shares one index transaction. Startup reads every execution once, not twice.
- **The lifecycle bench,** `theseus-sim bench lifecycle`:
 - phases: cold start; clean shutdown with executions waiting and a job running; SIGKILL, then restart;
 - the job is a real `proc.run` from a real turn against a stand-in for the Messages API;
 - secrets come from a fake `op` that answers after 1 s;
 - stores: empty, a copy, or synthetic (`--sessions N`, 250 sessions to a frame);
 - `--config` runs the operator's own config with the real `op`.
- **The gate** runs it on every commit: debug, empty store, ten runs a phase, 4.7 s. Each p95 is held to \$9 plus a measured per-phase margin (7, 4, and 25 ms). A throwaway 100 ms sleep in cold start failed it.

How it is proven. 256 tests in the gate, 20 of them new:

- serve first under a vault that hangs, answers late, or fails;
- the binding without its token;
- the bench's arithmetic, budgets, and config;
- the synthetic store read back through the kernel and the core.

The frame-budget test holds at 17. Release, at 460a35b:

Store	Cold p50 / p95	Shutdown p50 / p95	Kill p50 / p95
empty	17.2 / 31.3 ms	33.5 / 44.9 ms	36.3 / 56.1 ms
Eddie's copy	18.4 / 21.7 ms	33.6 / 38.7 ms	35.3 / 45.4 ms
10,000 sessions	124.7 / 142.7 ms	22.5 / 34.4 ms	147.0 / 154.3 ms

With Eddie's config and real secrets, the first answer took 1 075–1 104 ms before and 18.3–18.7 ms after, with `secrets: resolving` at each. A GLM turn sent at once waited 1.32 s for its key and then ran.

Review (11:18–11:21). The gate reran at 256 tests, with LIFECYCLE OK (cold p95 24.6 ms of 50, shutdown 49.2 of 100, kill 45.1 of 150). Tabitha's own check on the release build, over a store copy, with Eddie's real secrets: the first `health` answer came at 20–21 ms warm (57 ms on the copy's first open), with `secrets: resolving`. A GLM turn sent right after the start showed `secrets.wait 1.02 s` in its trace, then answered, and `health` then said `ready · 7 ready 1071 ms after start (inject)`. Installed at 11:21. The first run (08:54–09:18) was killed when an abort in its parent DM session cascaded to it; its uncommitted work was backed up, and the second run continued from the tree.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
The process refuses to start if a referenced secret cannot be resolved (§3.19)	The process serves. Each consumer refuses to run without its secret, and <code>health</code> , the ledger, and the Observatory name the failure and why	FAST (§2): the socket never waits on the network	§3.19 amended
References resolve concurrently through <code>op read</code> (§3.19)	One <code>op inject</code> for every reference; <code>op read</code> per reference only after a failed injection	Measured: the same wall time, a sixth of the CPU, one process	Keep
Resolution happens once at startup (§3.19)	Once, then again for what failed, at 5 s doubling to 60 s	The process no longer exits on a failure, so it must fetch again	Keep
The bench has five phases (P5b)	Three; binary swap and restore are F4's	A swap under load needs F4's upgrade path	F4
Nothing on the path to serving waits on the network (§2)	When the config is <code>op://</code> (Eddie's), the note is read before serving: about 1 s	Everything after it needs the config	Built in step F1b (theseus-2fo): the daemon starts from a last-known-good copy of the note, and nothing acts until the vault confirms it
Store open grows with the WAL tail, never with history (§9)	<code>wal:open</code> reads and checks every segment: 47 ms at 10,000 sessions	Since M1	F4, with the versioned readers (theseus-8ni)
The gate measures §9 on every commit (P5b)	On debug binaries and an empty store; Eddie's store and 10,000 sessions are release bench runs	A release build takes 3 min, and debug is never faster than release	Keep; revisit if a debug-only slowdown fails the gate

Known gaps carried forward:

- `health`, the heartbeat's reconcile, and the driver's 500 ms tick each read every execution, and `health` also reads every session and action: $O(\text{all})$, where $O(\text{open})$ would do;
- `redb`'s open costs two fsyncs, and its repair after a SIGKILL about 26 ms;
- the Observatory re-lists every execution and session every 2.5 s.

Step F2. Fewer frames per turn, and one transcript read (theseus-qa0; 2026-09-29, 11:22–12:18; 6402e80, 49c4db4)

Why. A plain one-loop turn wrote 17 WAL frames, each an fdatsync of about 7 ms here. Every turn also decoded its whole transcript at least twice, plus once per loop (the complexity review, findings 3 and 8; Part III A3b).

What exists.

- **A turn's own handles.** `Store::for_turn` and `Kernel::view` give each turn a store handle, and a kernel that commits through it. The turn's observability rows wait there. They ride at the front of the next frame that either one commits for the turn, in order. `provider.call` is in its completion's frame. The turn flushes whatever still waits at its end.
- `Kernel::plan_and_dispatch`. The provider call, and every tool call the policy runs, are planned, authorized, and dispatched in one frame. The kernel simulator plans half its actions this way. It holds a new invariant: such an action is never found planned or authorized, crash or no crash.
- **One transcript read per turn.** The turn's handle keeps its session's transcript (`Vec<(u64, Arc<Node>)>`). It is read at the first reader, and extended with every node that the turn's frames write. A debug build compares it with the store at every read, and panics on a difference. Every node write in a turn goes through the turn's handles, so no writer can miss the list.

How it is proven.

- The gate passed at each commit: 260 tests, then 262 (6 new), with the lifecycle bench within its budgets both times.
- The batch C output-diff probe, with each frame's grouping added, ran across its 26 scenarios. After masking, every record, notification, and result is identical. Frames went from 1,116 to 624 (–44%). The second commit's dump is byte-identical to the first's.
- `kernel-sim` (18 seeds, 105 crashes, every invariant held) and `crash-test` (zero committed records lost).
- A copy of Eddie's store was read under the old and the new binaries: every session list, history, and confirm list identical. A GLM tool turn wrote 13 frames where it would have written 30, and the old binary reads its session identically.

Release, medians of three interleaved rounds:

Turn	Before	After	Frames	Transcript reads
plain	123.0 ms	58.0 ms	17 → 8	2 → 1
two loops, one tool	211.6 ms	90.0 ms	29 → 12	4 → 1
plain, 200-node session	135.7 ms	65.9 ms	17 → 8	2 → 1
two loops, 200-node session	235.8 ms	104.6 ms	29 → 12	4 → 1

Reviewed (Tabitha, 2026-09-29, 12:23 to 12:31).

- The gate rerun passed: 262 tests, with the bench's cold start at p95 35.4 ms against 50.

- On the release build, over a fresh copy of Eddie's store, the old and the new binaries gave identical output: the session list, and all five histories, as text and as JSON.
- A GLM turn with one `fs_read` wrote 13 frames, and a plain follow-up turn wrote 8. Each outbox (plan, authorize, dispatch) was one fsync of about 7 ms.
- The old binary reads the new session byte for byte.
- Installed at 12:30.
- Found in passing: `theseus shutdown` removes the socket before the process releases the store's lock. So a start that follows at once can fail with "Database already open", which F4's swap phase must handle.

Divergence from Parts I and II, and from the review.

Planned	Actual	Why	Disposition
Each node and row is written in the frame of the kernel transition that produced it (§4.6)	Nodes, yes; observability rows ride in the turn's next frame	FAST: each frame is an fdatsync	§4.6 amended
The review: <code>turn.ended</code> , <code>turn.trace</code> , and the session write in one frame	<code>turn.ended</code> and <code>turn.trace</code> ride with <code>end_turn</code> 's frame	The trace times the session write; the count is the same	Keep
The review: <code>take_results</code> returns early when nothing is queued (one frame)	It always did; the plain turn's <code>results_consumed</code> frame consumes the provider call's own queue entry	Dropping the entry changes what a fault leaves	Drop it, with an explicit wake on a fault (Tabitha at review): <code>theseus-l6y</code> , with F4
The review: <code>plan_and_dispatch</code> , with the provider call as its first user	Also every tool call the policy runs	A loop with one tool: 12 frames → 4	Keep
The review: push each node the turn writes onto the list	The turn's handle adds every node its frames write, from the bytes written	No writer can forget one; a debug build checks the list at each read	Keep
Per-turn harness overhead under 5 ms (§9)	About 58 ms for a plain turn: 8 fdatsyncs of about 7 ms	The disk	Recorded; <code>theseus-l6y</code> takes a plain turn to about 5 frames

Known gaps. These are `theseus-l6y`'s:

- the plain turn's `results_consumed` frame;
- `wake_input` and `admit`, and a confirmed call's `authorize` and `dispatch`, which are two frames each;
- the session write, which could ride in `end_turn`'s frame;
- `confirm.list`, which still reads a transcript for a question's reason.

Beyond those, each loop still renders the whole transcript into its request. That is §4.5's rendered cache, which is not built.

Step F1b. Start from a last-known-good copy of the config note (theseus-2fo; 2026-09-29, 12:33–13:39; 98bce09, 06b8d53, 2b654d4, 6299536)

Why. Eddie's daemon runs as a bare `theseusd`, so its config is the vault note, read with one `op read` (about 1.0 s) before anything else. F1 moved every secret behind the socket, which left the note as the last network wait on the start path. Everything after it needs the config, so it could not simply move. Eddie accepted the fix on 2026-09-29 at 11:52 ("That's perfect in practice"). Tabitha's refinements made the vault the only authority.

What exists.

- **The copy**, `<state dir>/config.last-good.toml`: the note's exact text under a line naming its reference, mode 0600, written after serving. It is never kept for a note whose URLs could carry a credential, and it is on the tool floor.
- **Serve from the copy; act only on the vault's word.** One check in the dispatcher:
 - 8 methods that act (`turn.submit`, `session.open`, `profile.use`, `session.recompile`, `action.confirm`, `policy.tighten`, `policy.untighten`, `execution.cancel`) wait for the vault, bounded at 30 s, then fail with `config_unconfirmed` (-32006);
 - the 16 reads, and `shutdown`, answer at once;
 - the actors start on the confirmation, and each also waits on the gate itself;
 - a turn's trace shows `config.wait` when it waited.
- **The vault's answer:**
 - the same text, or only comments, confirms (`config.confirmed`);
 - a changed note is ledgered (`config.changed`), rewrites the copy, and restarts the daemon in place: an `exec` of `/proc/self/exe`, marked so that it never restarts twice;
 - an invalid note or a silent vault holds (`config.invalid`, `config.unreachable`), and retries at 5 s, doubling to 60 s;
 - health, the Observatory, and the narrative (a new `config` part) say which.
- **The restart keeps the process:** the same pid, terminal, arguments, and name. The name, `theseusd`, is written back to `/proc/self/comm` after the `exec`, since the kernel would name the image `exe`.
- **Startup writes nothing a copy decides.** The kernel had one config-dependent startup write: the dollar limit given to executions stored with unit budgets. It is refused under an unconfirmed config, and that start reads the vault first.
- `[web] bind` must be a loopback address, or the config fails to load.
- **The bench's vault phase:** cold starts from the copy, with a fake `op` that answers after 1 s. Each first answer must say `confirming`. It adds about 0.6 s to the gate.

How it is proven. 280 tests in the gate, 18 of them new:

- the gate under a vault that hangs, answers late, answers the same text, only comments, a changed note, an invalid note, or nothing;
- no restart loop;
- the security test: a copy widened to let the CLI approve never judges an approval, and the vault's version refuses it;

- the copy on the tool floor;
- the kernel's refusal of unit budgets under an unconfirmed config;
- the real binary: the copy kept, the start from it, the restart by `exec` in the same process under the same name, the vault-first `exec`, a comment-only change, and a silent vault.

The bench's `vault` phase on an empty store, release: p50 17.5–19.2 ms and p95 27.8–42.3 ms over three runs, with every first answer `confirming`.

With Eddie's real note, through a shim `op` that turned the web UI and Discord off:

- A first start with no copy answered in 1,094 ms. Three starts from the copy answered in 18.2, 20.2, and 21.5 ms, and the vault confirmed each 1.02–1.05 s after its spawn.
- A GLM turn sent at a start waited 1.01 s at the gate (`config.wait`), then ran.
- A changed note restarted the daemon in place: 148 ms from `config.changed` to serving again, and confirmed in 992 ms, with the vault's limit.
- A comment-only change confirmed, with no restart.
- An unreachable vault held, said why, refused an acting method after 30 s, and confirmed on its fifth read.

Reviewed (Tabitha, 2026-09-29, 14:03 to 14:13).

- The gate rerun passed: 280 tests, and all four bench phases within budget (from the copy, p95 25.0 ms).
- On the release build, with Eddie's real note through Tabitha's own shim, and an empty scratch state dir:
 - a first start answered in 1,065 ms ("read before serving ... there was no copy yet"), and kept a 0600 copy of 13,781 bytes under its reference line;
 - the next start answered in 31 ms, saying `confirming`. A GLM turn sent at once waited 1.02 s at the gate (`config.wait`), then answered, and the daemon said `confirmed in 1041 ms`;
 - a note changed through the shim (`[kernel] spend_limit_usd = 60.0`) gave `config.changed (["kernel"])` and the restart. The pid stayed the same (2992727), `/proc/<pid>/comm` stayed `theseusd`, and the vault's limit, \$60, took effect.
- Installed at 14:13.
- Found in the review:
 - F1b's real-binary test leaks its daemon when an assertion fails before its explicit `stop()`. One restarted debug daemon from a deliberately failing probe ran for 45 minutes, named `exe`. The review stopped it, and the fix, a guard that kills on drop, goes with the OTel step.
 - `theseusd config | head` panics on the broken pipe (theseus-gi7, with the same step).

Divergence from Parts I and II, and from the brief.

Planned	Actual	Why	Disposition
Resolution happens once at startup, and again on an explicit <code>config.reload</code> (§3.19)	No <code>config.reload</code> ; a restart is the reload, triggered by the vault's answer or by the operator	A config the process holds in memory is the config it started with; a restart is routine (§3.22) and cheap	§3.19 amended
The config note is read before serving (§3.19, F1)	Served from the last-known-good copy; the vault is read behind the socket, and nothing acts until it confirms	FAST (§2), with the vault as the only authority	§3.19 amended
The issue's first proposal: "restart to apply" (a stale copy keeps serving)	The daemon restarts itself onto the vault's version; a restarted one that finds another change holds	Tabitha's refinement: nothing may act on a file an agent can write	Keep
The brief: clients waiting at the gate see their connection close	Each is answered first with <code>config_unconfirmed</code> (state <code>restarting</code> : "send this again once it answers"), then its connection closes	A stdio client would otherwise wait forever, and a socket client learns why	Keep
The brief's list of methods that act (7)	8: <code>session.open</code> too	It writes a session and opens an execution whose spend limit is the config's	Keep
Kernel startup writes depend on no config value that decides policy (the brief's expectation)	One did: the unit-budget migration's dollar limit	Since theseus-0sg	Fixed: refused under an unconfirmed config, and read from the vault first
The Discord DM after a restart	Implemented (2b's approval-DM path, one call), not proven live	A scratch daemon cannot bind while Eddie's holds the bot token	Prove at Eddie's next restart onto a changed note

Known gaps.

- The Observatory's config line was not seen in a browser, because the live checks kept the web UI off while Eddie's daemon held port 7433.
- A job that can write the copy can hold the daemon, but never make it act. That is a denial of service, not an escalation, and L1's sandbox (M4) takes the write away.
- The restart's 100 ms grace is a delay, not a handshake.
- The copy is written without `fsync`, so a copy lost in a crash costs one slow start.
- A start spawned the moment `theseus.shutdown` returns can still fail with "Database already open" (F4).

Step F3. Parallel tool calls (theseus-a60; 2026-09-29, 14:15–15:10; a579f63, d421e1b, 59b3d9f)

Why. Eddie, 2026-09-28 23:57: "the most performant way to run trivially parallelizable tasks is to avoid os threads and use truly async code." A response's tool calls ran one after another, so five reads and two greps took the sum of their times. In-process toollets ran on tokio's blocking pool, bounded only by its 512 threads.

What exists.

- **Gate first, then run in groups** (`ToolRuntime::run_calls`).
 - The calls are gated in order. An unknown tool or invalid input is answered at once.
 - The first call that waits for the operator asks after the calls before it, and the calls after it wait for the continuation.
 - Consecutive reads run as futures in the turn's task (`join_all`, no task per call). A write or a program runs alone.
 - Each call keeps its own two frames, and every kernel call stays in the turn's task, one at a time.
 - A continuation runs its ungated calls the same way.
- **The request does not change.** The compiler already placed each result after the call it answers, whatever its position.
- **The trace shows the overlap.** A group's calls sit under a `tools` span, and `theseus ask --trace` draws them on that span's own time.
- **A fixed CPU pool** (`CpuPool`): a semaphore of `available_parallelism()` permits, held by every in-process toollet while it runs. A call's deadline, and its time, are its run's own.
- `fs.grep` **borrowes free cores** for a big tree. The walk stays sequential. Past the first 256 files, chunks of 64 go to every core that is free at that moment. The merge, in walk order, keeps the output identical.
- **The kernel simulator** dispatches 3 to 6 actions at once in a quarter of its turns, with crashes after the dispatch and between the completions.

How it is proven.

- The gate passed at each commit: 288, 290, then 291 tests, 11 of them new, with the lifecycle bench within its budgets at all three.
- **The output-diff probe**, with each provider request's digest added:
 - its 26 scenarios are byte-identical to the parent's after masking, and so are all their requests;
 - a 27th scenario, five reads and two greps in one response, sends the same requests;
 - its records, notifications, frames, and history reorder only inside the batch.
- **The simulators:** `kernel-sim` (22 seeds, 746 crashes, 83 of them inside a batch, every invariant held), and `crash-test`.
- **A copy of Eddie's store** under the old and the new binaries, with GLM turns. The old binary reads the new binary's session byte for byte.

Measure (release)	Before	After
a GLM response of 5 reads and 2 greps: the calls' wall time	407–411 ms (their sum)	168–184 ms (the slowest, 135–149 ms, plus its start)
one <code>fs.grep</code> over <code>~/projects/openclaw</code> (49,370 files), in the daemon	291–294 ms	123–126 ms
the same, in the benchmark: a full scan / the walk alone	284 ms / 82 ms	114 ms / 82 ms
32 CPU-bound calls in 4 sessions on 16 cores: most at once		15–16, never more

Reviewed (Tabitha, 2026-09-29, 15:17 to 15:25).

- The gate rerun passed: 291 tests, and all four bench phases within budget.
- On the release build, over a fresh copy of Eddie's store, a GLM response of four `fs.reads` and one `fs.grep`:
 - ran its five calls together, under one 69.0 ms `tools` span, each call about 41 ms;
 - answered right: the `stable` channel, and 3 files for the `grep`, as `grep -rl` says;
 - wrote 24 frames, with the session's open.
- The old binary reads the new session byte for byte: 2,568 B of text, and 16,638 B of JSON.
- Installed at 15:24.
- **At review, the kernel's lost update (section 6 of the report) was filed as theseus-id9 (P1),** and put next in the chain, before the OTel step. Concurrency makes the race likelier, and task sessions and wakes will add writers.
- **F3's two decisions for Eddie, taken at review as engineering calls:**
 - one plan frame per group joins theseus-l6y (F4);
 - a tightening pressed mid-batch applies from the next response, which is the right grain for a response that was gated as a whole.

Divergence from Parts I and II, and from the issue and the brief.

Planned	Actual	Why	Disposition
The issue: calls that write a path another call touches run in call order	Every write and every program is a barrier	Ruthlessly simple (the brief); path-level analysis is described in the report, and not built	Keep
The issue: <code>fs.glob</code> uses the parallel walker too	<code>fs.glob</code> unchanged; <code>fs.grep</code> 's walk sequential, its search parallel	The walk must keep its order, and <code>fs.glob</code> is all walk	Keep; a sorted parallel walk would change today's output order
The brief: order results by their <code>ToolCall</code> nodes in the renderer	Nothing changed there	The renderer already placed each result after its call	Keep
Concurrent completions share an <code>fsync</code> through group commit	Not within a turn, whose frames are committed from its one task; across turns, half of them	Commits stay in the turn's task, because the kernel rewrites an execution's record from what it read	One plan frame per group (theseus-l6y); the kernel lock first (theseus-id9)

Planned	Actual	Why	Disposition
A call's <code>duration_ms</code> is its run (implicit)	Timed on its core since 59b3d9f	Timed from its future, a read in a group said 45–50 ms	Fixed

Known gaps.

- A group of k calls writes $2k$ frames, one after another from the turn's task: about 100 ms for seven instant reads on this disk (theseus-l6y).
- A kernel transition read and rewritten on two threads can lose an update. That is old, but likelier during a batch's commit bursts (theseus-id9, next).
- `session.history` lists a group's results in the order they finished.

Step K1. The kernel never loses an update to an execution (theseus-id9; 2026-09-29, 15:26–16:15; 920f732, 5e52a63)

Why. The kernel read an execution record, changed it, and wrote it back, with no lock, and the store indexes a frame only after its `fsync`. So two writers of one execution on two threads could lose an update: a turn's commit, read before a cancel's frame was indexed, put `running` back over `cancelled`. The bug was as old as M2. F3's commit bursts made it likelier, and task sessions (DD7) and wakes (DD8) add writers. F3's report found it, and it was filed at F3's review (P1) and put before the OTel step.

What exists.

- **One writer at a time per execution** (`theseus-kernel/src/locks.rs`).
 - The locks are the set of executions being written, under one mutex, with a condvar for the waiters.
 - An id is in the set exactly while it is held, so nothing needs pruning when an execution ends.
 - A map of mutexes was rejected: it loses the lock when an entry is pruned under a waiter.
 - Stripes were rejected: they make unrelated executions wait.
- **Every transition that reads and writes back holds the lock from its read until its frame is indexed**, the `fsync` included. An action's transition reads the action to find its execution, then locks that and reads the action again. `kernel::view` shares the locks, so a turn's view and the kernel are one.
- **Decisions read outside the lock are read again inside it:**
 - the reconciler's due wake, from its scan;
 - `mark_unknown`'s "still dispatched";
 - startup's step 2 rewrites (`lock_all`, in id order).

A reconcile no longer fails when an overdue action settles under it.

- **A lock taken twice on one thread panics**, instead of waiting on itself.

- **The ordered two-lock helper**, `lock_two`, for DD7's carved budgets. Nothing calls it yet.
- **Nothing else changed**: no record, frame, or row. A plain turn still writes 8 frames.

How it is proven.

- The gate passed at both commits: 300 tests, 9 of them new, with the lifecycle bench within budget. `crash-test`, 20 iterations × 3 restarts, lost zero committed records.
- **Deterministic races**. A test store stops one thread between a transition's read and its write while a second writer runs. Without the lock, a throwaway break, six races lose an update:
 - `running over cancelled`;
 - a dispatched call dropped;
 - `queued over cancelled`, twice;
 - `waiting over cancelled`;
 - a decline undone.

All six pass with the lock, and writers of different executions never wait. `lock_two` in opposite orders never deadlocks. Without its id order, it deadlocks at the test's 20 s timeout.

- `kernel-sim` **with a second OS thread**. Raced turns (`--p-race`) put a cancel, completions arriving, wakes, input, and the heartbeat on the other thread, with crashes inside them.
 - A new invariant: a cancelled execution never runs again, read from the ledger in WAL order.
 - Five runs held: 38 seeds, 145 crashes, and 614 raced turns, with `fsync` on in one run and every turn raced in another.
 - Without the lock, every seed group failed within a few steps.
- **Live, over a copy of Eddie's store**. GLM turns of seven reads, four of them FIFOs that hold the batch open:
 - a cancel while the batch ran ended `cancelled`, stayed so across a restart, and left nothing dispatched;
 - a cancel released together with the FIFOs landed in the burst of completion frames. On the parent build it was lost, 2 of 2: the model answered a second loop, the execution ended `waiting`, and three reads that succeeded were recorded `cancelled`. On K1, 3 of 3 stayed cancelled, with nothing lost.

Measure (release, medians of three interleaved rounds)	Parent	K1
frames: a plain turn / two loops with one read / with five reads at once	8 / 12 / 20	8 / 12 / 20
a plain turn	56.2 ms	58.0 ms
two loops, one read / five reads at once	86.1 / 148.8 ms	89.4 / 145.8 ms

Measure (release, medians of three interleaved rounds)	Parent	K1
lifecycle p50: cold / from the copy / shutdown / kill	17.4 / 17.1 / 35.3 / 34.1 ms	16.9 / 17.0 / 37.0 / 34.4 ms
an uncontended lock and unlock; the extra action read		216 ns; 1.9 μ s (about 3 μ s a plain turn)

Load was 3.6 to 5.5, with other sessions compiling. The differences go both ways, and each is inside the spread of its own rounds.

Reviewed (Tabitha, 2026-09-29, 16:18 to 16:26).

- **The first gate rerun failed on one bench run.** Clean shutdown had p95 148.3 ms against 100 + 4, with p50 a normal 38.3 ms, while 1.3 GB of dirty pages from other sessions' builds were being written back. Four standalone bench runs right after passed at about 50 ms, and the gate passed after a `sync`: 300 tests, with shutdown p95 50.9 ms. So the bench measured the machine's writeback, not K1. The fix, a `sync` before the bench and a second run before a miss fails the gate, goes into the OTel step.
- **Tabitha's own race, on the release build, over a fresh copy of Eddie's store.**
 - Seven reads, four held by FIFOs.
 - The FIFOs were released and `theseus executions cancel` sent at once. The cancel landed in the burst, after the first FIFO completion, and asked 3 calls to stop.
 - The turn's next loop was refused (`kernel`: not running, state cancelled).
 - The execution ended `cancelled` with nothing outstanding, and stayed so across a restart.
 - K1's WAL-order checker found no problems.
- Installed at 16:25.
- **Found by K1, and filed at review:** `theseus-xeo`. The core's `SessionRecord` has the same read-and-write-back shape: a `session.recompile` during a turn can be lost. It is folded into DD7, and DD7 also gives `lock_two` its first caller.

Divergence from the issue and the brief.

Planned	Actual	Why	Disposition
The issue: a lock held from the read to the WAL write, released before the <code>fsync</code>	Held through the <code>fsync</code> and the index update (the brief)	The index moves only after the <code>fsync</code> ; a lock released before it lets the next writer read the old record	Keep; group commit across one execution's frames needs an index that moves before the <code>fsync</code>
The brief: a map of <code>Arc<Mutex<(>>></code> pruned when an execution ends, or striped locks	The set of executions being written, with a <code>condvar</code>	Nothing to prune, no pruning race, and no false sharing	Keep
The brief: a test hook between a transition's read and its write	A test store that stops one thread at one read	No hook in the product	Keep

Planned	Actual	Why	Disposition
kernel-sim reproducible from its seed (M2)	Only up to its first raced turn; <code>--p-race 0</code> is reproducible throughout	The OS picks the interleaving	Keep
The ordered two-lock helper	<code>lock_two</code> , and <code>lock_all</code> for startup's rewrites	Startup rewrites many executions in one frame	Keep

Known gaps.

- A turn's frames, and one execution's frames in general, still take one fsync each. Group commit over a group's frames needs the index to move before the fsync (theseus-l6y).
- The core's `SessionRecord` read-and-write-back (theseus-xeo, with DD7).

Step O1. OTel without the SDK: a native OTLP exporter in every build (theseus-hee, theseus-gi7; 2026-09-29, 16:27–17:20; 6ad89a2, e7d8495, b51252f, 5240563)

Why. Eddie, 2026-09-29, 08:52: "I'd like it built in by default, but 19 crates is surprising"; and at 09:21, "let's do cheaper otel". The exporter was the `otel` cargo feature, off by default, over the OpenTelemetry SDK: 19 crates, among them prost and a second request (0.13) with its own rustls stack and aws-lc.

What exists.

- **A native exporter** (`theseus-core/src/telemetry/`). It speaks OTLP/HTTP with the JSON encoding over the workspace's request 0.12: lowerCamelCase fields, hex ids, integer enums, and 64-bit integers as decimal strings. It is in every build, and `theseusd`'s dependency tree is the default build's own set.
- **The same picture as the SDK's.** The trace walk, attributes, events, status, limits (128), and flags were ported from `otel.rs`. So were the eight instruments, with their units, descriptions, and attributes, aggregated in the process with cumulative temporality.
- **Off the hot path.**
 - A turn records its metrics and queues its trace. One sender task posts the trace, and the metrics every interval.
 - A failure, a 429, or a 5xx gets one retry after a second; then the batch is dropped and counted.
 - A full queue (64) drops its oldest.
- **Health** says `off`, `waiting` (for the vault or the headers secret), `exporting` (with what was sent and dropped, and the last error), or `failed` (the exporter could not be built). `theseus health` prints it.
- **Built after serving**, once the vault confirms the config and the headers secret resolves. A stopping daemon flushes, bounded at 1 s.
- **Removed:** the feature, its dependencies, the warning about a build without it, and the gate's second clippy run.

- **theseus-gi7.** `theseusd`'s printing subcommands end with success on a closed pipe. SIGPIPE stays ignored, so no pipe write can kill the daemon.
- **No leaked daemons.** Every `theseusd` that a real-binary test or the lifecycle bench spawns is held by a guard that kills and reaps it on drop.
- **The gate** runs `sync` before the lifecycle bench, and a second run before a miss fails it.

How it is proven. 322 tests in the gate, 22 of them new.

- **Conformance.** What is posted is read back through `opentelemetry-proto`'s own types: a dev-dependency, built without tonic. Every field comes back unchanged, and every encoding is checked.
- **The old picture.** A golden file dumped from the SDK exporter itself at 964411f matches span for span and metric for metric.
- **Failures:**
 - a 503 then a 200 delivers once;
 - a failing or absent receiver gets one retry, then the batch is dropped and counted in health;
 - a receiver that hangs costs a turn 63–73 μ s.
- **Headers**, in both forms, with no value shown.
- **Throwaway probes** of each test failed as they should: a wrong field case, kind, integer encoding, temporality, or id encoding; SIGPIPE's default; a panic or a `bail!` before a stop. The temporality probe failed only once the tests wrote OTLP's enum values out, instead of reading the code's own constants.
- A 100 ms sleep on the start path still failed the gate, on both runs.
- **Live, on the release build** over a copy of Eddie's store, with his note and a local receiver:
 - a GLM tool turn's spans arrived with the trace's durations to the tenth of a millisecond, and the metrics after one interval;
 - with the receiver stopped, the next turn ran unchanged, and health counted 7 batches dropped, with the last error;
 - with the receiver back, the counters were still cumulative.

Measure	Before	After
crates in <code>theseusd</code> 's build (<code>cargo tree -e normal</code> , name and version)	292, or 311 with the exporter	292 with the exporter
release <code>theseusd</code>	19,702,816 B, or 23,942,128 B with the exporter	19,830,208 B with the exporter
a clean shutdown with telemetry on, after a turn (release)		27.1 ms (24.9 ms off)

Reviewed (Tabitha, 2026-09-29, 17:48 to 17:54).

- The gate rerun passed: 322 tests, and all four bench phases within budget. The gate has no `--features otel` run left.
- `theseusd`'s normal dependency tree was counted by name: 281 names, the same as before this step. None of them is `opentelemetry`, `prost`, or `aws-lc`.

- On the release build (19,830,208 B), over a fresh copy of Eddie's store, with `otlp_endpoint` pointed at a local receiver:
 - a GLM tool turn posted one trace (12,862 B of JSON, 7 spans), whose durations match `-trace` (turn 11,560.8 ms, the tool 14.0 ms);
 - the metrics arrived every 5 s, cumulative;
 - health read sent 2 (1 traces, 1 metrics) · dropped 0.
- `theseusd example-config` and `example-bindings` into a closed pipe exit 0.
- Installed at 17:54.
- **Filed at review** as `theseus-yf1`, all small, with no consumer yet: the histogram bounds past 10 s; tool metrics by family, backend, and outcome, with a duration; the served model in `gen_ai.response.model`; failed continuations counted; and the provider-call histogram's empty attributes, which Tabitha found live and which the SDK exporter had too.

Divergence from Parts I and II, and from the brief.

Planned	Actual	Why	Disposition
OTLP over HTTP/protobuf (§3.20)	OTLP/HTTP with the JSON encoding	JSON needs no protobuf library, and the Collector's receiver takes it	§3.20 amended; a hand-written protobuf encoder is the fallback for a receiver that won't
The exporter is the <code>otel</code> build feature, off by default (§3.20, <code>theseus-0g4</code>)	In every build, with no feature and no added crate	Eddie, 2026-09-29	§3.20 amended
§3.20's table: root attributes <code>theseus.turn_id</code> , <code>theseus.session_id</code> , <code>theseus.profile</code>	The root's attributes as recorded (<code>turn_id</code> , <code>session_id</code> , <code>profile</code> , ...), as the SDK exporter emitted them	The same picture, so a dashboard built on it still works	Table corrected
§3.20's table: <code>theseus.tool.calls</code> by family, tool, backend, and outcome, and <code>theseus.tool.duration_ms</code>	By tool, with the turn's attributes; no duration	Neither exporter had them (A3 recorded it)	Table corrected; <code>theseus-yf1</code>
The SDK's batching (2,048 spans, a 5 s delay) and its blocking <code>force_flush</code> inside <code>shutdown</code>	One post per turn, a queue of 64, and a flush after the serving loop, bounded at 1 s	Simpler, and a stop never blocks a task	Keep
<code>theseus-gi7</code> : restore SIGPIPE's default for the printing subcommands, or end quietly	End quietly, with exit 0	A default SIGPIPE would expose <code>check</code> 's write into <code>op inject</code> 's stdin	Keep
The brief: the lifecycle bench "already kills on its error paths"	Not on all of them: a failed stop, confirmation, copy wait, or health read after a start leaked	Found by reading, and shown by a probe	Fixed

Known gaps.

- Vendors' acceptance of OTLP JSON (Honeycomb, Datadog, Tempo, ADOT) was checked only against a local receiver. Eddie's first real endpoint will tell.
- `opentelemetry-proto` 0.33 reads `asInt` only as a number, so a receiver built on that crate's `serde` would refuse our sums. Ours follow the protobuf JSON mapping.
- No gzip.

Step J1. A job cannot answer approvals (theseus-6qy; 2026-09-29, 17:56–19:00; 4bd8bdc, d567d05, 0b23036)

Why. At L0 a `proc.run` job runs as the operator's own user. So it could reach the CLI socket and the loopback web UI, and answer an approval its own session waited on, whenever `cli` or `web` was a trusted channel, which is the default. This was found in the 2b part 1 review, and Eddie accepted the peer-pid check on 2026-09-29 at 09:21.

What exists.

- **The wrapper** (`theseusd job-wrapper`) is a child subreaper.
 - It reaps every child that exits while its command runs, and reports as before.
 - Then it lingers, marked in `spool/lingering/<id>`, until no descendant remains. It kills nothing.
 - Health's `kernel.lingering_wrappers` counts the lingering wrappers, and `theseus health` and the Observatory show it.
- **A zombie is not alive.** `pid_alive` reads `/proc/<pid>/stat`. Before this, the daemon never reaped its wrappers (theseus-z4b), so a cancel took the wrapper it had just killed for alive, and settled `OutcomeUncertain` after 2.5 s.
- **Who is asking** (`peer.rs`). Each connection keeps its peer:
 - the socket reads `SO_PEERCRED` and the pid's start time at accept;
 - `--stdio` names its parent;
 - the web UI keeps the connection's two addresses, and finds the processes that hold the client's end (`/proc/net/tcp`, then `/proc/*/fd`) only when a judged act arrives;
 - Discord has no process.
- **The check**, in `Core::judge_act`. An answer (approve, decline, the spend reset) and an undo trace the asker's parent chain to pid 1. An asker under a live wrapper, known by its command line, is refused, and so is one that cannot be traced. This holds with or without `[approval]`. A press is not traced.
- **Loud.**
 - `approval.refused` carries `asker` (`pid`, `argv0`, `job`, `wrapper pid`, `trace_us`) and `from_job`.
 - A narrative line.
 - An `approval.refused` notification to every connection. The Discord binding posts it to the approval DM, or else the session's place, and `theseus watch` prints it.
 - The CLI exits 1.
 - An accepted answer records its asker too, and `theseus confirm` takes `--approve`.

How it is proven. 346 tests in the gate, 24 of them new:

- the real wrapper: a double fork stays under it, it reaps while the command runs, it lingers, and a cancel kills its group as before;
- the parent-chain walk and the loopback lookup, on real process trees with a stand-in wrapper (`flock`);
- the acts in the core: approve, decline, and the web UI from a job's process are refused; the spend reset is refused; the undo is refused; a press is accepted;
- end to end, with the real daemon, wrappers, and CLI, and a stand-in Messages API:
 - a job's `theseus confirm --approve` for its own session's call is refused;
 - so is its double-forked grandchild, before and after the job's main process exits;
 - a job's WebSocket answer through the web UI is refused;
 - the operator's answers count;
 - a cancel still kills what it killed.

Throwaway probes: without the subreaper, the grandchild approved the call after the job's main process exited. Without the trace, all six act tests failed.

Measure (release)	p50	p95
each socket connection at accept	3.9 μ s	4.6 μ s
a judged act's trace: the operator's CLI / a job's process	62.5 / 21.6 μ s	105.4 / 24.2 μ s
a judged web act's owner lookup (120 processes, 1,153 fds)	3.08 ms	3.58 ms

Reviewed (Tabitha, 2026-09-29, 19:18 to 19:29).

- The gate rerun passed: 346 tests, and all four bench phases within budget.
- On the release build, over a fresh copy of Eddie's store, with `[approval] channels = ["cli"]`:
 - `theseus policy tighten fs.write` made a GLM turn's write wait.
 - A second GLM session was asked to approve it with `proc.run`. It declined twice on its own judgment, and once investigated the binary's strings. When it ran `theseus confirm --approve`, the daemon refused it: "from a Theseus job's process (job act_..., pid 3452497, theseus)", exit 1, a 46 μ s trace, and `approval.refused` with `from_job` in the ledger.
 - Tabitha's own `theseus confirm --approve` from her shell then counted, with a 126 μ s trace, and the file was written.
- **Eddie restarted his daemon at 18:50** (pid 3431665), on the O1 build. Every check kept to its own socket and state dir, and never touched his.
- Installed at 19:28.
- **Taken at review:**
 - The one hole left before M4 (a job that kills its own wrapper, whose orphans go to init) is closed next, with `theseus-z4b`'s reaping: the daemon becomes a child subreaper, and refuses answers from its own descendants (step Z1).
 - Refusing another local account's web client, and counting any Theseus daemon's job, are right for a one-operator machine.

- A web banner for a refusal isn't needed now: the Observatory's ledger and the narrative show it, and the DM carries it.

Divergence from Parts I and II, and from the brief.

Planned	Actual	Why	Disposition
The brief: refuse when the asker or an ancestor is a live job wrapper	Any Theseus daemon's wrapper, known by its command line	The command line names the job, needs no spool state, and holds for as long as the wrapper lives	Keep
§3.9: the web UI's member is anyone with an account on the machine	Another account's client cannot be traced, so it is refused	The brief: a pid that cannot be read counts as a job's	§3.9 amended
The brief: after the command exits, the wrapper keeps reaping	It also reaps while the command runs	Orphans reparented to a subreaper would otherwise wait as zombies for the whole job	Keep
—	<code>pid_alive</code> treats a zombie as gone	A cancel of a real job settled <code>OutcomeUncertain</code> after 2.5 s	Fixed; the zombie leak is theseus-z4b (Z1)
The brief: <code>theseus confirm --approve <id></code>	The flag added: the default, and it conflicts with <code>--decline</code>	The CLI had only <code>--decline</code>	Keep
The brief: a notice where approvals go (the DM)	The DM when one is trusted, else the session's place, else the log; <code>theseus watch too</code>	The web UI has no banner; its Observatory shows the row, highlighted	Keep

Known gaps.

- A user systemd unit, a tmux server already running, cron, and a process started outside the job are M4's.
- A job that kills its own wrapper is closed next (Z1, theseus-z4b).
- A Windows browser, through a Hyper-V firewall rule for 7433 that does not exist today, would be refused as untraceable.
- The web lookup's `/proc/net/tcp` read takes about 1 ms; `sock_diag` would make it tens of μ s.

Step Z1. The daemon reaps its job wrappers, and adopts a job's orphans (theseus-z4b; 2026-09-29, 20:30–21:28; d0212c0, 0e15190)

Why.

- The daemon spawned each job's wrapper and forgot it. Every wrapper that exited stayed a zombie child of `theseusd` until the daemon exited, and an exec restart (F1b) kept them. Eddie's daemon runs about 160 jobs a day. J1 found this.

- A job could kill its own wrapper (`kill -9 $PPID`). Its processes then went to init, out of the parent-chain walk's reach, and an orphan's `theseus confirm` counted. This was J1's one hole before M4.
- The first run was cancelled at 19:32, two minutes in and before any change, so that Eddie could restart the OpenClaw gateway. It was relaunched at 20:30.

What exists.

- **Who reaps what** (`theseus-kernel/src/children.rs`): a registry of the daemon's children, by pid.
 - A job wrapper is registered with its job as it is spawned, and is reaped by its own pid once it exits.
 - An `op` process is registered as tokio's, with its start time, and is never reaped here, since tokio waits for each of its children by pid.
 - Any other child is an orphan the daemon adopted, and is reaped once it exits.
 - A spawn holds the registry's lock through its registration, and a sweep holds it through its scan and its reaps. So a sweep never takes a new `op` for an orphan. Nothing waits for "any child".
- **The reaper** is a task woken by SIGCHLD, and every 10 s, that sweeps on the blocking pool. It runs in the socket daemon and in `--stdio` alike.
- **The daemon adopts a job's orphans.** The socket daemon is a child subreaper, set in `main` before the runtime starts.
- **An orphan can't answer.** The walk also stops at the daemon.
 - An asker under `theseusd` with no live wrapper between is refused: from a process under `theseusd` itself (`pid <n>`, `<argv0>`), which is a job's orphan, with `under_daemon` in its `asker`. The Discord text says so.
 - The web UI's owner lookup still skips the daemon's own fds.
- **After an exec restart**, the new image sets the flag again and learns its children, recognizing a wrapper by its command line.
- **A reaped wrapper's pid may be reused.** So a wrapper is alive only while its pid's command line names its job (the reconciler, the restart check), and a cancel leaves alone a pid that is now another process.
- **Health's** `children`: wrappers running and lingering, orphans adopted, zombies (0 in steady state), the `op` processes, and what was reaped. A `theseus health` line and an Observatory line show it.

How it is proven. 356 tests in the gate, 10 of them new:

- the registry, in a test binary of its own: a wrapper and an orphan are reaped, an owned child is left for its owner (whose `wait()` still gets its status), and the children are learned again after an `exec`;
- the orphan rule on real process trees, on the socket and through the web UI;
- a cancel leaves alone a process that took a reaped wrapper's pid;
- end to end, with the real daemon:
 - 200 jobs leave no zombie;

- an `op` run through a burst of 50 jobs keeps its exit status;
- a job that kills its wrapper leaves a grandchild that the daemon adopts. Its answer is refused, and the operator's counts;
- after an `exec` restart, the daemon is a subreaper again, and reaps what the old image left.

Throwaway runs:

- the parent's daemon left 200 zombies after 200 jobs;
- without the subreaper, the orphan approved the call;
- with `op` unregistered, the sweep took tokio's children, and every `op read` failed with `ECHILD`.

Live, on a copy of Eddie's store:

- 20 GLM turns with a job each left no zombie.
- A GLM job ran a script that killed its own wrapper. The orphan it left went to the daemon, and its `theseus confirm --approve` was refused with the new reason, after an 82 μ s trace. The operator's own answer then counted.

Measure (release)	p50	p95
a sweep: 2 wrappers / 100 children, 25 threads	47.6 / 264 μ s	68.8 / 404 μ s
a census (health): 2 wrappers / 100 children	54.9 / 462 μ s	77.2 / 682 μ s

The lifecycle bench is unchanged within its noise: cold start p50 18.9 ms, against the parent's 18.7.

Reviewed (Tabitha, 2026-09-29, 21:48 to 21:55).

- The gate rerun passed: 356 tests, and all four bench phases within budget.
- On the release build, over a fresh copy of Eddie's store:
 - five GLM turns each ran a `proc.run` job, and each answered with its own echo;
 - the daemon then had 0 zombie children by a `/proc scan`;
 - health said `subreaper: true`, `reaped_wrappers: 5`, `zombies: 0`.
- Installed at 21:55.
- **Taken at review:**
 - Z1's two follow-ups are `theseus-6uo`, folded into the secret broker's step:
 - refuse the descendants of any serving `theseusd`, not only this daemon's, and make `--stdio` a subreaper;
 - ledger a job's killed wrapper as `job.wrapper_lost`, and mark its action unknown at once.
 - A job's orphans keep running until they exit, as a lingering wrapper's descendants do.

Divergence from Parts I and II, and from the brief.

Planned	Actual	Why	Disposition
The brief: reap from the heartbeat, or from a task woken on SIGCHLD	A SIGCHLD task, with a 10 s tick	The heartbeat is 60 s, starts only once the config gate opens, and a wrapper pokes it before it exits	Keep
The brief: a registry at the one spawn site (OpReader)	One registry of every child: wrappers with their job, op as tokio's with its start time	The sweep reaps wrappers by pid, and must tell tokio's children from orphans; the start time guards a reused pid	Keep
—	wrapper_alive(pid, job), and terminate takes the job	Once reaped, a wrapper's pid can belong to another process	Keep
—	--stdio reaps its own wrappers too, and adopts nothing	The brief sets the flag for the socket daemon only; reaping by pid is safe in every mode	theseus-6uo makes --stdio a subreaper

Known gaps.

- The rule covers this daemon's own descendants. An orphan of another Theseus daemon (a scratch daemon's, or a --stdio daemon's) that answers this one is not caught. That is theseus-6uo.
- A job's orphans run under the daemon until they exit, and nothing kills them. A cancel still reaches the ones that stayed in the job's process group.
- A wrapper killed before it reported leaves its action dispatched until its deadline, when the reconciler marks it unknown. theseus-6uo marks it at once.

Step B1. The secret broker, and any daemon's descendants refused (theseus-dcy, theseus-6uo; 2026-09-30, 00:12–00:57; 096aa10, 5857e21)

Why.

- op ran 125 times in the audit's 30 days (55 in the DM), and the floor makes each one wait for approval. Eddie accepted the broker on 2026-09-29 at 09:39: "we don't want to continuously query op anyway, so this might get broader use."
- Z1's two follow-ups (theseus-6uo) were folded in, since both touch job spawning.
- The first run started at 23:27 and was ended at 23:36 by a gateway stop, while it was still reading. It wrote nothing. This entry is the second run.

What exists.

- **Config** (theseus-core/src/broker.rs, config.rs, the template).
 - [broker.programs.<program>] env = { VAR = "<secret>" }, and [broker.secrets.<name>] posture, notify when absent.
 - Validation names each problem: a program name with a /, a bad variable name, a secret that is not a [secrets] entry, an empty env, and an unknown key or posture.
 - An empty broker is skipped, so a note without [broker] loads and prints as it did.

- **Direct argv** (`Broker::program_for`), and the gate and the spawn: §3.19 and §3.9.
- **Surfaces.** One `decision.granted` on the gate record reaches every surface:
 - Discord's tool line (`gh got GH_TOKEN`), the notice card's Secrets field, and `policy.notified`;
 - the web UI's pill, the Observatory's row, the CLI's notice line, and the confirm's reason.
 - `tool.started` says what the job actually got, so a withheld secret replaces the gate's word.
- **Never written.** The wrapper's arguments carry no value: the environment goes by `env_clear` and `envs`. `WrapperArgs`' Debug prints names only, and the spawner's copies are zeroized after the launch.
- **Toollets.** `ToolCtx::secret(name)` over a broker bound to the call (`Bound`), granted by wiring (`grant_tool`). The M4 hook is a one-line comment above `secret_for_tool`.
- **Health and the Observatory.** `health.broker[]` lists the kind, the target, the variable, the secret, the posture, and the uses since start. `theseus health` prints `broker: gh gets GH_TOKEN (github_token, notify), used 1 time`.
- **theseus-6uo** (`peer.rs`, `job.rs`, `rpc/driver.rs`, the reaper): §3.9's "Any serving daemon's descendants", and §3.16's two bullets. A drift test keeps the core's list of the daemon's value-taking options equal to the CLI's, so `--config check` is not read as a subcommand.
- **Test support.** The in-process wrapper now gives its command the job's environment, as a wrapper process has it. Before, a core test's job saw the whole test process's environment.

How it is proven. 382 tests in the gate, 14 of them new or extended:

- the broker's unit tests: direct argv against `sh -c`, `env`, `bash -lc`, `./gh`, and a call-set PATH; the bounded wait; the withheld secret; the posture at the gate and at the spawn; and a toollet's grant;
- `a_program_run_by_its_own_argv_gets_its_secret_and_nothing_else_does`, against the real daemon with stub programs:
 - the job's environment holds exactly one vault value, and no `AWS_` name;
 - an `approve secret` waits;
 - a failed secret is withheld;
 - no file under the state dir, and not the log, holds the value;
- `an_orphan_of_another_daemon_cannot_answer_this_one`: with the check disabled, a probe shows the answer used to count;
- `a_job_that_kills_its_wrapper_...`: one `job.wrapper_lost`, and the action unknown at once. A cancel's kill gives none.

Reviewed (Tabitha, 2026-09-30, 01:01 to 01:07).

- The gate rerun passed: 382 tests, and all four bench phases within budget (cold start p50 36.3 ms).
- On the release build, over a fresh copy of Eddie's store, with his note and the grant, and a shim `op` that logs each run:
 - a GLM turn's `["gh", "api", "user", "--jq", ".login"]` answered `zeroaltitude`, notified with `gh got GH_TOKEN`, and never waited;

- **the token came from the broker.** With gh's own stored login hidden (`GH_CONFIG_DIR` set to an empty directory), direct `gh` still answered `zeroaltitude.sh -c "gh ..."` got the broker's note and gh's exit 4, "To authenticate, please run `gh auth login`";
- `op` ran once in all, at startup;
- the ledger held 2 `secret.granted` rows and no confirm; health said `used 2 times`;
- the token's value was in none of the 6 files under the copy's state (the WAL segment, `index.redb`, the manifest, and the three jobs' spool outputs), and in neither the log nor the CLI's output. The control matched.
- Eddie's unchanged note loads under the new binary.
- Installed at 01:06.
- **Taken at review:**
 - The raw spool output keeps what a program prints (`gh auth token`). Filed as a follow-up: the wrapper redacts its own granted values from its output file.
 - DD5 adds one comment under the template's `[broker]`: a granted program passes its variable to what it runs (gh's extensions and shell aliases, or a hook), so the posture is the control.
 - Put to Eddie: gh's stored login authenticates any job's gh, broker or not, and whether `github_token` should stay `notify`.

Divergence from Parts I and II, and from the brief.

Planned	Actual	Why	Disposition
<code>broker.secret_for_tool("web.search", name)</code>	That call, and <code>ToolCtx::secret(name)</code> over a <code>Bound</code> broker for the call	A toollet runs on a core and cannot wait, and needs the call's posture; the wiring grants the tool (<code>grant_tool</code>)	Keep
Toollet grants in the config	Granted by wiring code, from the consumer's own <code>..._secret</code> key (DD5)	Config keys are not defined before code honors them, and no toollet needs one yet	DD5 adds the key
"The tool line and the notice say gh got <code>GH_TOKEN</code> "	So do the web UI's pill, the CLI's notice, the Observatory's row, and the confirm's reason	One <code>decision.granted</code> on the gate record reaches every surface	Keep
—	A spawn withholds a secret whose posture is stricter than the call ran at	The program's resolution could change between the gate and the spawn	Keep
—	<code>secret.withheld</code> rows	A withheld grant is as auditable as a given one	Keep

Planned	Actual	Why	Disposition
—	The in-process wrapper applies the job's environment	Core tests saw the test process's whole environment	Keep

Known gaps.

- A granted program that runs other programs passes the variable on: gh's extensions and shell aliases (`gh alias set --shell`), or git's hooks, if git were ever granted.
- The spool's raw output file keeps whatever a program prints. The node, the model, and every surface get the scrubbed text. The follow-up above closes it.
- At L0 a job of the same user can read another job's `/proc/<pid>/environ`, and gh's own stored login (`~/.config/gh/hosts.yml`) authenticates any job's gh, through a shell too. L1 (M4) closes both.

A4. M3.6 Daily Driver (theseus-5jl)

P5d's items, each recorded when its review ends. Items 1 to 4 run before M3.5 (P5d, Order). The milestone closes when its prove passes: Eddie carries his daily Discord work on Theseus end to end.

Item 1. Budgets in dollars (theseus-0sg, theseus-woy; 2026-09-29, 00:55–01:55; cb824c7, fdf4813, d6183d1)

Why it moved up. Eddie's Discord DM ended `budget_exhausted` at about \$0.42 on 2026-09-29 at 00:05; its session's recorded cost is \$0.4216.

- Its execution had opened under the old limit of 1,000,000 units, and a limit was fixed when an execution opened.
- After 15 turns and 28 tool calls it had spent 877,683 units, most of them cache reads counted at full weight. At Sonnet 5's prices a cache read costs \$0.20 per million tokens and an output token \$10, and units weighed them the same.
- The next call needed 172,068 units (Sonnet 5.5's 128 K output ceiling plus about 44 K of input), and 112,317 were left.
- The place rebound Eddie to a fresh session, which started with an empty conversation.

Eddie decided the design at 00:09 (\$3.13, where his words are).

What exists.

- **The catalog in micro-dollars** (cb824c7).
 - A price in dollars per million tokens is micro-dollars per token. `cost_micros` prices a call's input, output, cache reads, and cache writes, each at its own price. `reserve_micros` prices the output cap at the output price, plus the input estimate at the input price. The arithmetic is integer, with one rounding up per call.

- The template lists all 12 built-in models, uncommented, with their four prices.
`Catalog::template_tables()` writes that text, and a test holds it equal to the built-in table.
- At startup, one warning names every built-in model that has no table in the config.
- **The kernel** (fdf4813). A budget is micro-dollars: the limit, the spend, the reservations, and the amounts held unknown. The control reserve is gone.
 - A reservation that does not fit is refused with `overBudget`, and it writes nothing. The turn then asks (`ask_budget`: a planned `budget.reset` action, one open at a time) and parks on the new `Wake::Budget`.
 - `reset_budget` is the only transition that lowers spend, and it is one frame. The question settles as succeeded, and the spend goes to \$0. Reservations and held amounts stay, and `resets` goes up by one. A `budget.reset` row records who approved, the spend before, and the limit. The execution is queued, so the waiting call goes ahead.
 - A decline leaves it waiting, and new input asks again. Terminal stays terminal.
 - A versioned reader (schema 1) serves executions stored with unit budgets. Each gets the configured limit, its spend comes from its session's recorded `cost_usd`, and the unit figures are kept as `units_before`. Startup rewrites each one once, in one frame, with a `budget.migrated` row.
- **The simulator.** Five new random operations: ask, approve, decline, new input on a budget wait, and a cancel of one. Six new invariants:
 - nothing is reserved past the limit;
 - spend goes down only by an approved reset;
 - an execution has at most one open question;
 - no unit budget survives startup;
 - terminal stays terminal;
 - nothing new ends `budget_exhausted`.

40 seeds × 300 steps held them, with 397 questions and 171 resets. The sweep found one real bug, fixed in the commit: a turn that ended its execution left its question open.

- **The core.** `[kernel] spend_limit_usd = 100.0`. `default_budget` and `control_reserve` load, are ignored, and warn once together. A model with no price anywhere is not called (class `unpriced`).
- **The narrative speaks dollars:** the limit when a session opens, each call's reservation, the budget wait ("reached its \$100 limit; waiting for the operator to reset it"), and the reset ("Spend reset to \$0 by X; continuing").
- **theseus-woy.** A `TurnError`'s text no longer embeds its cause, so a failure says it once.
- **Surfaces.**
 - The protocol, the CLI, and Discord count budgets in dollars. `theseus executions` and `theseus confirm` show budgets and questions.

- Discord posts the question with Approve and Decline in the session's place, and only its listed users can press them.
- The web UI (d6183d1) shows dollars, a reset count, an "at its limit" pill, and summaries for `budget. asked`, `budget. reset`, and `budget. migrated` in the Observatory. The transcript shows the question as a card: "Reset to \$0 and continue" or "Keep waiting".

How it is proven.

- **Tests.** 167, up from 151; the gate reran on d6183d1 at 01:48. The new ones cover:
 - in the kernel: a call over the limit waits, and an approved reset continues; a declined question keeps waiting, and new input asks again; a reset approved before the turn parks still continues; cancelling a budget wait closes its question, and terminal stays terminal; an execution that ends closes its open question; executions stored with unit budgets serve in dollars;
 - in the core: a session at its limit asks, and an approved reset makes the waiting call; a declined reset keeps waiting, and the next message asks again; the frame budget still holds at 17;
 - in the catalog: a call's reservation and cost in micro-dollars match the prices, and the template equals the built-in table;
 - in the config: Eddie's vault config, with its unit budget, loads with one warning.
- **Live** (Tabitha, 01:51–01:53; a release build on a scratch daemon over a copy of Eddie's store; `spend_limit_usd = 0.002`, and the glm profile capped at 2,048 output tokens, so one reservation is about \$0.0014):
 - Startup warned twice: once for the retired unit keys, and once for the 12 built-in models without a `[catalog]` table.
 - Eddie's old exhausted DM execution read in dollars and stayed `budget_exhausted`, at \$0.4216, showing "before dollars: 877683 of 1000000 units".
 - Turn 1 (GLM, `fs.list`) ran 2 loops and 1 tool call, for \$0.0008.
 - Turn 2 asked: "This session has spent \$0.0008 of its \$0.002 limit. Reset its spend to \$0 and continue?" Its `budget. asked` row reads spent \$0.000814, needed \$0.001431, limit \$0.002.
 - `theseus confirm` approved it. The `budget. reset` row reads by `sock#7`, spent before \$0.000814, resets 1. The resumed turn ran `fs.list` and answered after 2 loops, for \$0.0003.
 - Afterwards the session's lifetime cost was \$0.0011, the sum of both turns, so the reset lowered nothing. The execution showed \$0.0003 since the reset.
- **The run.** The subagent's run was aborted ("CLI run aborted") at about 01:42. It had pushed cb824c7 and fdf4813 and left the web half staged. Tabitha finished the step in review, from 01:48 to 01:55: she committed the web half (d6183d1), reran the gate, ran the live check, and wrote the report. Installed at 01:53.

Follow-ups.

- theseus-kks. A call whose reservation is larger than the whole limit never fits after a reset, so the question would come back after each approval. It cannot happen at \$100 with today's models, whose largest reservation is about \$3, but a tiny test limit reaches it. The question should say so and name the fixes: raise the limit, or lower `max_output_tokens`.
- An unpriced model is now refused. That is a hard stop where A3 had it run, and Eddie may prefer that it ask.
- `theseus executions` prints limits to two decimals, so a \$0.002 test limit shows as "\$0.00".
- Discord's question was not exercised live, because a second daemon must not bind the bot token while Eddie's runs. `render.rs` tests cover it.
- Eddie's note needs the new template: `narrative = true` at the top, `spend_limit_usd = 100.0` in place of the unit keys, and the twelve `[catalog]` tables.
- A session keeps the limit it opened with. A dollar-era execution stores its limit (`Budget::new` at open), and only a unit-era record takes the configured limit when it is read, so a changed `spend_limit_usd` reaches new sessions only. That includes a long-lived Discord place. Verified in the code at review, 02:50. It is held for Eddie whether an open session should follow the config.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
An unknown model runs with a startup warning and its cost unknown (A3; P5's catalog bullet)	A model priced neither in the config nor in the built-in table is refused as <code>unpriced</code>	Under a dollar limit, nothing may run unpriced	Held for Eddie: he may prefer that it ask (theseus-kks)
A reserved control and cleanup budget, so that reaching a ceiling never prevents a cancel (§3.13; P4)	Retired	Nothing at the limit ends work or refuses a cancel	Part I §3.13 amended
Reaching the limit ends the execution as <code>budget_exhausted</code> (§3.15; the M2 kernel)	The session waits and asks for a reset. Nothing new ends <code>budget_exhausted</code> , and old ones stay ended	Eddie's decision, 00:09	Part I §1, §2, §3.13, and §3.15 amended
Money budgets land together with provider-safe caching (§4.5; theseus-ev1)	Money budgets landed first	Eddie's DM hit its unit limit in real use	Provider-safe caching stays scheduled (theseus-ev1)

Item 2. The workspace: context files and roots (theseus-58a; 2026-09-29, 01:55–02:15; 76e7d35, a29e9f7)

What exists.

- `context_files` on a profile, with `[model].context_files` as the default, compiled into the system block with their digests as §4.4 describes.
 - Loading checks spelling only, and a relative path fails with the key named.

- `ContextFiles` is the daemon's stat cache: one per runner, empty at startup, with reads capped at 64 KB + 1 byte.
- Records:
 - `Manifest.context_files` and the `context.compiled` row's `context_files` (both absent with no files, so stored manifests compare equal);
 - a `context.file_missing` row once per file per run;
 - the narrative's context line ends with ", N context files (M missing)".
- The template sets `context_files = []` at `[model]` with an example, shows the profile line commented, and shows `roots = ["/home/zeroaltitude/reports"]` commented under `[tools]`.
- The Observatory's Context view lists the current compilation's files with their digests.

How it is proven.

- **Tests.** 176, 9 of them new. They cover:
 - the rule in the system block and the digest in the manifest;
 - four turns giving exactly `[new_session, system_changed]`, and an unchanged file that is not read again;
 - a missing file warned once while the turn runs;
 - a cut at the cap;
 - a config without files keeping the same system bytes and manifest;
 - inheritance and validation, and the template;
 - the stat cache's hit, miss, and racy reread.

The frame budget holds: a plain turn writes 17 frames or fewer.

- **Live**, on a scratch daemon over a copy of Eddie's store:
 - GLM ended an answer with "Theseus", which only a context file asked for, with no tool call;
 - after the file was edited to "Ithaca", the next turn recompiled once (`system_changed`) and ended with "Ithaca";
 - the turn after that appended.

Eddie's vault config loads under the new binary with the same single warning as before.

- **Review** (02:26). The gate reran at 176 tests. Tabitha's own check on the release build, over a store copy, got a GLM answer that ended with "Ithaca", a rule only the context file stated, and the manifest's digest equals `sha256sum` of the file. Installed at 02:24.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
An edit recompiles "on the next loop" (the step's brief)	On the next turn's first loop. The system block is fixed for a turn	A recompile inside a tool loop strips the prefix's thinking between a tool call and its result	Kept
<code>context_files = []</code> on a profile in the template	Set at <code>[model]</code> . On the profile the line is commented	A live <code>[]</code> on a profile hides the <code>[model]</code> default	Kept
A stat of mtime and size per compile	Size, mtime, and inode per turn, plus the two-second racy reread	The inode catches an atomic replace, and the racy rule a same-size edit within one timestamp tick	Kept
With <code>~/openclaw/workspace</code> and <code>~/reports</code> in <code>roots</code> , a spec session's reads and edits need no confirm (usage audit §6, item 2; P5d)	Not proven live in this step. Roots are unchanged A3 code, covered by the gate's root tests, and the template now shows the example	Eddie chooses his roots. It belongs to the exit test's config, which needs no code	Held for the exit test

Known gaps.

- An edit to a context file re-caches the whole session. The system block leads the cached prefix, so the next turn writes the session's entire context to the cache, and the prefix loses its thinking blocks. At the DM's measured 391 k tokens per call, one edit costs about \$1 at Sonnet 5.5's cache-write price. If it bites, the files get their own later cache breakpoint, with the provider-safe caching work (theseus-ev1).
- The reads are synchronous file calls under the turn lock. A hung network mount would hang the turn, as it would any fs tool.
- Relative paths are refused at load.

Open, held for Eddie.

- Which files and roots go in his note.
- Whether a mid-turn edit should take effect within the turn.

Item 3. Quiet notices (theseus-w4f; 2026-09-29, 02:28–02:38; b481a0b)

What exists.

- `[discord] notice_embeds`, false by default. With it off, a notified call posts no embed. Its line in the loop's tool message carries the notice and names the setting that made it one, for example `␣ `proc.run` cargo test · ␣ notified (enforcement = notify) · 900 ms`, or `([policy.tools] "proc.run" = notify)` for a per-tool rule. With it on, the embed behaves as before: a card, then its outcome.

- The reason needs no new plumbing: `tool.proposed` already carries the gate's decision and its setting.
- When a loop's calls overflow one message, the line that folds the oldest ones counts their notices: `-# ... 4 earlier call(s) · 4 notified`.
- Unchanged: the `tool.notified` row, the `policy.notified` event, the web UI's notices, the CLI's `! notified:` line, and the narrative.

How it is proven.

- **Tests.** 180. They include:
 - with the setting off, a notified call rides on its tool line and posts no card;
 - with it on, the old card test still holds;
 - thirty notified `proc.run` calls in one loop render as one tool message, one create and 56 edits of 1,883 bytes, with no other message;
 - a core turn of thirty notified commands gives thirty `tool.notified` rows, each carrying the setting the renderer reads;
 - Eddie's `[discord]` shape loads unchanged, with the embeds off.
- **Live.** Discord was not exercised, because one bot token means one daemon, and Eddie's holds it. On a scratch daemon over a copy of his store, one notified `proc.run` wrote its `tool.notified` row with `setting = "enforcement = notify"`. Eddie's vault note loads under the new binary with only the known budget-units warning.
- **Review** (02:47). The gate reran at 180 tests.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
"A turn with 30 notified <code>proc.run</code> calls posts one tool message, edited in place, and no other message" (P5d, item 3)	One tool message per loop. Thirty calls in one loop post one message; calls spread over loops post one per loop, as before	The tool message is per loop by design (A3); the setting removes the per-call embeds	Kept
Every notice is visible in Discord	A loop that overflows one message shows its oldest calls only as a count on the fold line	Discord's 2,000-character limit	Kept; the ledger and the web UI keep every notice

Known gaps. The parenthesis adds about 23 bytes a line (about 36 for a `[policy.tools]` rule), so a loop folds sooner. If it proves too long in use, the fallback is to name the setting once per message.

Item 4. Attachments and images (theseus-9g2; 2026-09-29, 03:18–03:50; 7bcfd9a, 48fd2c8)

Why. Of Eddie's 332 DM messages in 30 days, 7 carried an attachment: 6 `text/plain` (Discord turns a long paste into `message.txt`) and 1 PNG. The binding listed attachments by name and size and never read them, so Theseus silently missed every long paste.

What exists.

- **The input.** `turn.submit` takes `attachments` (serde default), and an empty `input` is accepted when there are any. The user node keeps each file:
 - text, capped at `[tools].max_read_bytes` on a character boundary and marked when cut;
 - an image, as a reference to `<store dir>/blobs/<sha256>`, stored once and never in a WAL frame;
 - or the reason it was not read.
- **Rendering.** Each file is a block before the typed text, under a header that names it and its sender. An image is an image block for a model whose catalog entry has vision, and the "not shown, this model has no vision" line for any other. A message without attachments renders exactly as before.
- `fs.read` returns an image the same way, inside its `tool_result`, through a `Tool::run_with_image` hook. Its description gained one clause.
- **The estimate** leaves out base64 data and adds each image's tiles, so a 1 MB 1920×1080 PNG reserves 2,691 tokens on Sonnet 5.5, not about 333,000.
- **The catalog.** `glm-5.3` and `glm-5.2` are text-only (2026-09-29.1); `glm-5.3-flash` keeps vision.
- **Discord.** `on_message` plans each file and spawns its download. The place's submit task awaits the downloads before `turn.submit`, so the gateway loop never waits and order is kept. A failed download is listed, and the turn runs.
- **The CLI.** `theseus ask --attach <file>`, repeatable.
- **Restore** carries `blobs/`.

How it is proven.

- **Tests.** 204, 24 of them new (9 with the text commit, 15 with the image commit). They cover:
 - a 5,001-character `message.txt` in the scripted provider's request, under its header;
 - a text cut at the cap, and a file listed with its reason;
 - a failed download that still runs its turn;
 - an image as one image block, stored once, with no bytes in the node, rendered byte for byte the same across turns and a cold cache;
 - the "no vision" line;
 - the pixel-based estimate;
 - `fs.read` of a PNG;
 - a 6 MB image refused with its reason;
 - the Discord planner and entry function, with bytes and no network;
 - the sniffer, the blobs, and restore.

The frame budget holds at 17, and Eddie's vault config loads unchanged.

- **Live** (03:47–03:50, debug build on a scratch daemon over a copy of Eddie's store; about \$0.02 in all):
 - Haiku answered a fact that only a 5,000-character attached note held.
 - Haiku read "TEAL HERON 77" from a headless-Chrome PNG, and the one blob is named for the PNG's SHA-256.
 - GLM-5.3 quoted the "no vision" line.

- `glm-5.3-flash` read the PNG as an attachment and through `fs.read`, so `z.ai` takes images in user content and in `tool_results`.
- A 20 MB archive was listed as not read, with the reason.

Discord was not exercised, because one bot token means one daemon, and Eddie's holds it.

- **The runs.** The first run (02:52–03:01) was marked external by the provenance plugin after it used web tools and a probe of `z.ai`, so `exec` and `write` were refused. It stopped without changing anything and left its findings, and run 2 built from them with no web tools.
- **Review** (04:22). The gate reran at 204 tests. Tabitha's own check on the release build, over a store copy: `glm-5.3-flash` answered from a 4,878-character attached note and read "AMBER FALCON 58" from a headless-Chrome screenshot, the one blob is named its SHA-256, and `glm-5.3` got the "no vision" line. Installed at 04:22.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
"The binding downloads each attachment up to <code>[tools].max_read_bytes</code> " (P5d, item 4)	Text up to <code>max_read_bytes</code> (256 KB); images up to 5 MiB	256 KB would refuse most screenshots; 5 MiB is the provider's safe limit on every route	Kept
"text becomes user content ... marked external"	Its own block under a header naming the file and its sender	Theseus has no provenance labels yet (§3.9)	The header is the mark until labels exist
<code>fs.read</code> returns "text, image, PDF as typed nodes" (§3.24)	Text and images; a PDF is still reported as binary	PDFs were not in this item's audit numbers	Held
One <code>vision</code> flag, true for every GLM row (the catalog as it stood)	<code>glm-5.3</code> and <code>glm-5.2</code> false	OpenClaw's table marks them text-only; <code>glm-5.3</code> answered a PNG with empty text	Kept; a <code>[catalog]</code> table can override

Known gaps.

- Blobs are never collected, and redaction (§5.6) does not reach them. At Eddie's rate of one image a month, this costs nothing yet.
- An image the provider rejects fails every later request of its session (theseus-0s4). The sniffer checks the header, the size, and the sides, but not the pixels. `/new` in Discord, or a new message and then `recompile fresh`, recovers.
- A restore from a bare segment directory that is not named `wal` carries no blobs, and its images show as missing.
- The web UI shows an image's header line, not the image.
- The first turn of every session after install recompiles once as `tools_changed`, with its thinking stripped. For Eddie's DM that is about \$1 of cache writes, once.
- A coalesced Discord batch from several authors labels its files "from discord", not per author.

Open, held for Eddie.

- Whether PDFs should be read (as text, or as document blocks).
- Whether the "no vision" line should instead route the turn to a vision profile.

Items 1 and 2, follow-ups: the limit follows the config, and context files by persona (theseus-3pj, theseus-c48; 2026-09-29, 22:10–22:53; 430d291, 68cb127)

Why. Eddie accepted both on 2026-09-29 at 09:21, and the second again at 09:39.

- A dollar-era execution kept the limit it opened with, so a changed `spend_limit_usd` reached only new sessions, and never the long-lived Discord place. Since F1b, a changed note restarts the daemon onto it, so "I changed the limit and restarted" had to mean what it says.
- Item 2's `context_files` hung on profiles. Eddie wants a system level that every session gets, plus a persona's files, with the persona chosen by default until Jev chooses one from an ontology.

What exists.

- **The limit follows the config** (§3.13).
 - `Kernel::follow_spend_limit`, with the same rule in startup's step 2 for a config that may act at once. The core calls it on the vault's word, before the gate opens.
 - One frame, with a `budget.limit_changed` row for each execution.
 - A raise withdraws the waiting question and queues it as the next turn's result, and the continuation treats that as "the waiting call proceeds". A lower limit asks at the next reservation.
 - `Budget.pinned` marks a limit the opener named. It is absent from every record the product writes.
 - A narrative line per session, `confirm.resolved` (withdrawn) to the session's clients, and an Observatory summary of the row.
- **Context files in two levels** (§4.4).
 - `[context]` files, `[personas.<name>]` files, and `[context]` `default_persona`.
 - `[model]` `context_files` and the profile field are removed. Eddie's note used neither, checked by key name.
 - The headers name the level, and the manifest and the row record each file's persona.
 - Health's `context`, a `theseus health` line, and an Observatory line with a level column.
 - The template documents `[context]`, and a commented `[personas.theseus]`.

How it is proven. 369 tests in the gate, against 356 before:

- a raise across a restart lets a waiting session continue;
- a lowered limit makes the next turn ask;
- under an unconfirmed copy, startup writes no limit;
- a declined wait proceeds on a raise, and a pinned or ended execution keeps its limit;
- end to end, a limit raised in the vault: the old copy, the restart onto the changed note, and the vault's word, after which the waiting call runs, with the row before `config.confirmed`;

- system then persona files, each labeled; a persona with no files; an edited persona file recompiling once; an unknown `default_persona` refused, naming the known ones; and the template.

`kernel-sim` holds its budget invariants with limits that change across restarts, a third of the starts served from an unconfirmed copy. Over 40 seeds × 300 steps: 497 changes, 327 rewrites, and 15 waits let proceed. Four throwaway breaks were each caught, by the kernel tests and by the simulator.

Live, over a copy of Eddie's store and his real note through a shim `op`:

- a GLM turn's manifest listed the scratch system file, then the persona draft;
- a lower limit made the session's next turn ask;
- a raise restarted the daemon onto the note, and the session went on without a reset.

Reviewed (Tabitha, 2026-09-29, 23:18 to 23:25).

- The gate rerun passed: 369 tests, and all four bench phases within budget.
- On the release build, over a fresh copy of Eddie's store, with a file config:
 - Health said `context: 1 file at the system level · persona theseus (1 file)`.
 - A GLM turn's compilation listed the system file, then the persona draft, with the report's digests (`3ede0c7e...`, `7e3c050e...`). The answer knew Eddie from the persona draft. It did not end with the system file's "Ithaca" this time, which is the model's instruction-following: the file was in the block.
 - A limit lowered from \$1 to \$0.50 across a restart gave five `budget.limit_changed` rows, \$1.0 → \$0.5, for the open executions.
- Installed at 23:24.
- **Taken at review:**
 - The stale Discord question after a raise (a card whose buttons outlive its withdrawn question) goes into DD6's outbox brief: on reconnect, edit the cards of questions that closed while the binding was away.
 - Personas without a default stay a warning.
 - A carved task budget stays `pinned` (DD7).

Divergence from the brief and the issues.

Planned	Actual	Why	Disposition
The brief: rewrite the limits once, after the vault confirms the config at startup	Also in startup's step 2, for a config that may act at once	A file, or a vault read before serving, has no confirmation; step 2 already reads every execution	Keep
"Make an open execution's limit the config's"	Unless the opener named its own (<code>pinned</code>)	The kernel tests and the simulator open executions with limits of their own, and DD7's carved budgets will	Keep

Planned	Actual	Why	Disposition
A raise wakes a session parked at its limit	Also one whose question was declined; and a raise still too small withdraws the question, and the call asks again in the new figures	A declined wait is still parked on its budget; a stale question would show the old limit	Keep
The 58a header # Context file: <path>	# Context file (system): <path> and # Context file (persona <name>): <path>	Each file labeled by its level	Keep

Known gaps.

- A Discord question posted before a restart keeps its buttons after a raise withdraws it (to DD6). *(Closed 2026-09-30 by theseus-q4v for every card posted since. A card posted before then has no post to settle it.)*
- A lower limit makes theseus-kks likelier: a limit below one call's reservation asks again after every reset.
- A persona switch by Jev will rewrite the whole cached prefix, until the files get their own breakpoint (theseus-ev1).

Item 5. `http.fetch` and `web.search` (theseus-yd6; 2026-09-30, 01:10–01:46 and 02:14–02:36; acb16f4, 28ac9d3)

Why. In 30 days the DM made 59 `web_fetch`, 36 `web_search`, and 29 `curl` calls (the usage audit, section 6). Eddie chose the Brave Search API on 2026-09-29.

What exists.

- **Async tools.**
 - `Backend::Async` and `Tool::run_async`: a call is `tokio::spawn` under the in-process deadline (120 s), and holds no core while it waits. A panic is the call's error, not the turn's.
 - It goes through the one in-process path: `tool.started` (`backend: async`), the bound broker, the result node, and the completion.
 - Both tools are class `Read`, so F3's `run_calls` polls the fetches of one response together.
- `http.fetch` (theseus-core/src/web/).
 - GET only. The client follows no redirect itself; the tool follows up to 5, judging each hop first.
 - `[tools.web] timeout_secs` (30) over the whole call, and `max_bytes` (2 MiB) over the body.
 - HTML becomes text: headings, paragraphs, list items, links as `text (url)`, and `pre` as it is. Script, style, noscript, svg, and the like are dropped, and entities are decoded. The converter is hand-written, about 510 lines, and reads tags one at a time, so broken or cut markup still gives its text.
 - A page of 16 KiB or more converts on the pool. The converter runs at about 210 MB/s, and a hop to the pool costs 24 μ s, so a smaller page converts on the call's own task in under 80 μ s.

- Text, JSON, and XML come back as they are. Any other type is named with its size, and is not downloaded when its size is given.
- Any response is a result, a 404 included. Only a call that got no response fails.
- **Private addresses** wait at the gate, or are refused at connect (§3.9). Both clients use `no_proxy()`. A test-only resolver (`net::Dns`, with a host map and addresses taken as public) lets tests reach 127.0.0.1; no config key reaches it.
- `web.search`.
 - The Brave Search API gives rank, title, URL, and snippet, with the markup stripped.
 - Its key is `brave_api_key`, granted with `grant_tool` and read through `ToolCtx::secret`, so each search is at least `notify`, and health counts its uses.
 - No key, a key held stricter than the call ran at, a 401, or a 429 is a result the model reads. Only Brave's error code comes through, never its free text.
- **External text.** A result node's `external { url }` has a `serde` default, so old records read unchanged. An error result is never marked, since its text is Theseus's own. A redirect whose `Location` is not a URL names the parse error, never the header (`28ac9d3`).
- **Surfaces.** The URL or the quoted query shows on the Discord tool line, the web UI, and the notices.
- **Config.** `[tools.web]` has defaults, so a note without it loads unchanged. The template has the Brave `[secrets]` line, the table, and B1's comment under `[broker]`.

How it is proven.

- **Tests.** 402 in the gate, 20 of them new: the converter; every kind of non-public address, in every spelling; the fetches against a local server (content, redirects, the caps, and the gate's five URLs); Brave's fixture and its refusals; two fetches of one response at once; a declined loopback fetch that never connects; and a search held to its key's posture.
- **The step's live check** (02:23–02:27): the Mutex page's `lock`, the three right URLs for `ignore's WalkParallel`, `http://127.0.0.1:7433/` waiting and never starting once declined, and a name that `/etc/hosts` maps to 127.0.0.1 refused at connect.
- **The runs.** Run 1 (01:10–01:46) wrote the code. The provenance plugin then tainted its session: the Bash heredoc that wrote its report held the word "links" followed by a space, and a placeholder URL, and the plugin's exec rule for the `links` browser matched them (openclaw-provenance-fqu). Every later exec, edit, and write was refused. Run 2 continued from the tree, and wrote every file with the Write and Edit tools.

Reviewed (Tabitha, 2026-09-30, 02:40 to 02:47).

- The gate rerun passed: 402 tests, and all four bench phases within budget (cold start p95 46.5 ms).
- On the release build, over a fresh copy of Eddie's store, with `proc.run` and the write tools pinned to `approve`:
 - one GLM response made two fetches, of the `HashMap` and `vec` pages (196 KB and 953 KB). They were dispatched 7 ms apart and succeeded at 262 and 311 ms, so they ran together. Both answers were right: `entry` returns the `Entry` enum, and `with_capacity` makes an empty vector with at least that capacity;

- a search found tokio's `JoinSet` page on docs.rs, from 5 results, and health said `used once`;
- `http://169.254.169.254/latest/meta-data/` waited ("169.254.169.254 is a link-local address, and a private address waits for approval"). Declined, it was never dispatched, and the model tried nothing else;
- the Brave key's value was in none of the copy's state files, the log, or the CLI's output. The control matched.
- Eddie's unchanged note loads under the new binary.
- Installed at 02:46.
- **Taken at review:**
 - **Web text can now steer a session that acts at `notify`.** A page's text reaches the model, and in Eddie's config `proc.run` and the write tools run at `notify`. Until provenance labels (§3.9) and Jev exist, a deterministic rule closes the gap: once a session has read external text, a call that acts waits for approval, until the operator clears it. Filed as theseus-9bp, and added to the chain before Eddie's end-to-end test.
 - The runtime's "the full output is stored" is wrong for an in-process result, which stores nothing. It predates DD5. Filed as theseus-46v.

Divergence from Parts I and II.

Planned	Actual	Why	Disposition
"native toollets" (P5d, item 5)	Async tools on the runtime, not toollets on a core	They wait on the network: async for waiting, the pool for compute (F3)	Keep. §3.23's "a toollet computes on a core" still holds for toollets
<code>http.* fetch, post</code> (§3.23)	<code>fetch only</code>	The audit's calls were GETs	<code>post</code> when a need shows
External text is marked (§5.2)	<code>external { url }</code> on the result node	There are no provenance labels yet (§3.9). A field reads old records unchanged	The field is the mark until labels exist
"a loopback or private address waits for approval" (P5d)	It waits when the URL names it, and is refused at connect when a name resolves to it	The gate sees the URL, and only the resolver sees the answer	Keep
"text, image, PDF" (§3.24)	A PDF is named with its size, not read	Decision 10	Held

Known gaps.

- An approval can reach a private name only if it is `localhost`. A name like `printer.1an` looks public to the gate, and its private answer is refused at connect. Asking by address breaks TLS and virtual hosts. The fixes are to resolve at the gate and pin the answer, or an operator's list of private names.
- A page's text is cut to `[tools] result_max_chars` (30,000) from its head, with no offset to read further. §3.16's node reference with a range is the likely home.
- Pages are read as UTF-8 only, with no gzip or brotli.

- Not exercised live: the Discord and web UI tool lines, since Discord and the web UI stay off in a scratch daemon. The Discord summary is unit-tested.

Item 6. Durable delivery (theseus-q4v; 2026-09-30, 02:50–04:19; f028083, 9dd4cef, 765111d)

Why. 64 DM turns ran past 5 minutes. A reply that ended while Discord was away was lost, since the binding posted with a plain HTTP call from a live watcher of its session. For the same reason every continuation waited at startup, up to 20 s, for the binding to watch its sessions (A3). That wait was on FAST's start path.

What exists.

- **Posts** (`theseus-kernel/src/outbox.rs`, `theseus-core/src/outbox.rs`). The core writes an outbox post when something must reach Discord, whether or not the binding is there:
 - a turn's reply: its loops' text by node, and its footer;
 - a confirm card, with its question;
 - a card's settle at every close: an answer, a supersede, or a raise's withdrawal;
 - a failed turn, a job's refused answer, and a restart onto a changed config note;
 - the binding's own notes.

A post is a kernel action of the record kind `OUTBOX`: planned and authorized, dispatched, and settled with its message ids. It is no execution's work.
- **Frames.** A reply rides in the frame that ends its turn, and a card in its question's, so a plain turn still writes 8 frames. Delivering a post writes 2 more, off the turn path.
- **The lanes** (`theseus-discord/src/courier.rs`). One per place, and one for the operator, each the only writer of its messages.
 - A lane sends its posts in order, then the live progress, which keeps only each message's latest state and is dropped while Discord is away.
 - A card is routed when it is delivered (2b's rule), and its completion keeps what its settle edits.
 - The lanes need only REST, and start before Discord answers anything.
- **Exactly once.** A create carries a nonce from its message's key, with `enforce_nonce`. A create that the nonce returns with an earlier send's content is edited to the post's state. Past a 120 s window, a create that may have landed says it may be a copy.
- **S1's stale card.** Every close writes its card's settle. A level-triggered pass, on connect and every heartbeat, catches the closes that no event said.
- **No startup wait.** `BindingBoard::expect`, `started`, and `wait` are gone, with the driver's wait. The driver's start is a startup phase.

- **Health and the Observatory** show each binding's outbox: pending, sent, refused, the oldest pending post's age, and the last error.
- **For tests and scratch daemons**, `[discord] rest_proxy` and `gateway_proxy` point a binding at local stand-ins. `theseus-sim fake-discord` is one for REST: it honors a nonce as Discord does, can be down, hang creates, or fail, and never records a header.

How it is proven.

- The gate at 765111d: 420 tests. Among them are 6 binding scenarios against the fake: away during a turn; three replies in order; a crash between send and settle; a card's settle after its create, to its id; live edits coalescing; and 2b's DM route.
- Three real-daemon tests: a `kill -9` between send and settle leaves one message; a continuation at restart, with Discord away, posts once Discord is back; and the REST goes to the fake, with no header kept.
- The lifecycle bench now runs the binding, with its token resolving after 1000 ms. The driver starts at p50 41 ms, before the token.
- The step's live check (04:02–04:18), on a copy of Eddie's store: a reply posted once after the fake came back, and one message after a `kill -9` (tries `hung, hung, deduped`).

Reviewed (Tabitha, 2026-09-30, 04:40 to 04:47).

- The gate rerun passed: 420 tests, and every bench phase within budget (cold start p95 38.3 ms; the driver started at p50 41.1 ms, before the binding's token resolved).
- On the release build, over a fresh copy of Eddie's store, with the binding on a fake REST (port 9472) and a gateway that never connected, and a scratch bindings file that binds a DM with a user who does not exist:
 - the bind notice went out through the outbox;
 - **away during a turn:** with the fake down, a GLM turn ended. Health said `1 pending ... last error: parsing or receiving the response failed`, and the fake had no message. With the fake back at 04:44:41, the reply was posted once, footer included, at 04:44:55;
 - **a kill between send and settle:** with the fake hanging creates, the reply's create reached it twice with one nonce (the stream's and the post's). I killed the daemon with `kill -9` at 04:45:39, brought the fake back, and restarted. The retry at 04:45:49 was deduped (`hung, hung, deduped`), and the message was edited once to its final form. The channel held the bind notice and the two replies, each once, and nothing was pending;
 - **nothing reached Discord.** The daemon's connections went to 127.0.0.1 (24), `api.github.com` (2, the startup token check), and `api.z.ai` (2, the turns). No Discord host name was in the log.
- Eddie's unchanged note loads under the new binary.
- Installed at 04:46.
- **Taken at review:**
 - The labeled possible copy after a long outage stays, rather than a lookup of recent messages first, which would need another permission and a call per retry.

- A post for a place no longer in the bindings file stays pending forever. Filed as theseus-13m: settle it as refused, "not bound here any more", when the binding starts without that place.

Divergence from the brief and the issue.

Planned	Actual	Why	Disposition
"Every post and edit the binding makes becomes an outbox action" (the issue's title)	Posts that must be seen are durable; live progress (streamed text, tool messages, typing, notice embeds) stays best-effort	Only a live message's latest state matters, and a replay is noise	Keep
The nonce "derived from the outbox action's id"	Derived from the message's key, which holds the post's or question's id, or the turn's for a reply	The stream's create of a part and the reply's create of it share a nonce, so a stream create whose answer was lost cannot become a copy	Keep
"Use the kernel's actions"	Kernel actions of their own record kind, <code>OUTBOX</code> , with their own transitions; the <code>Action</code> type, states, <code>Completion</code> , and ledger rows are the kernel's	An execution's machinery would queue a post's result on its execution, cancel a reply when its execution ends, and let the reconciler mark a post unknown	Keep
The card's route decided when the live renderer saw the question	Decided when the card is delivered, with a fresh check of who can view the channel	Routing needs REST, which is the lane's	Keep
—	A card whose question closed before it could be posted is posted, then settled at once	Discord keeps a record of what was asked while it was away	Keep
—	A turn that parks on a confirm: its card is written with the question, and its reply at the turn's end	A card must exist whenever its question does. After an outage the card is posted above the text that led to it	Revisit if it reads badly
—	The lifecycle bench runs the Discord binding	To show no driver wait, the bench's daemon needs a binding to wait for	Keep

Known gaps.

- The index reads every outbox record once per process, at first use: $O(\text{posts ever})$. Compaction, or a settled-below watermark, when it matters.
- A lane's maps (key to message, sent contents, sealed keys) grow for the process's life.
- A card posted before DD6 has no post: only a press settles it.
- The gateway side cannot be faked, so no test sends a Discord message in; turns enter through the protocol.
- `kernel-sim` does not yet inject crashes around the outbox's transitions.
- A task's report (DD7) will be a post of its own kind, through `outbox::post`. (*Done in item 7.*)

Item 7. Task sessions (theseus-qn2, with theseus-xeo; 2026-09-30, 04:49–06:03; 795ed91, 625dde3, 5b44ff5, 9610b2e, 3b0b20d)

Why. 43% of all tool calls in the audit's 30 days ran in delegated sessions. Eddie's daily pattern is "go do this long thing and tell me when it's done", while he keeps talking.

What exists.

- `task.create { brief, budget_usd? } (theseus-core/src/task.rs)`, a harness tool (`Backend::Harness, new`): it needs the turn's kernel and store, so the runtime runs it in the calling turn's task. It opens the child in one kernel frame (`Kernel::open_task`, under `lock_two(parent, child)`) and returns at once. The child's ids come from the call's correlation id (`act_X opens exe_X in ses_X`), so a call run again after a crash finds its task instead of opening a second.
- **What the child inherits** (§3.2a's note): authority, persona and context files, postures, model, and where its approvals go. Its cards, its budget question, and its notices name it (`task a1b2c3:`).
- **The carve** (`theseus-kernel/src/tasks.rs`). `budget_usd`, or a quarter of what the parent has left, capped at all of it, reserved in the parent as `task:<child>`, with the child's limit pinned. Each settle of the child's cost moves it into the parent's spend and shrinks the carve, under both locks (`lock_family`). The child's end releases the carve, but for what it still has in flight. At its own limit the child asks, as any session does.
- **Depth one.** A task's `task.create` is refused, and the kernel refuses too (`TaskDepth`).
- **The report.** A task turn that would wait on input ends the task, and its last message is its report. The frame that ends it carries the report's outbox post (`report:<task>`, one message ever) and puts the child on the parent's `reports`. The parent's next turn writes one node per report into its own session (`origin: harness, author task:<short>`), in one frame that clears the list. Nothing starts a parent turn. A failed task reports `failed`, and a cancel reports `stopped`, in the cancel's own frame.
- **Seeing and stopping.** `task.list` and `task.cancel` in the protocol, `theseus tasks` and `theseus cancel <task>`, Discord's `/tasks` and `/cancel task:<id>`, and the web UI's session tree with each task's state and spend. A cancel terminates the task's jobs and walks each action's cancel, as `execution.cancel` does. A second cancel writes nothing.
- **Crash safety.** A task is an execution and a session, so startup requeues it, and the driver continues it. A job that outlives a `kill -9` is found still running, and its result finishes the task.
- **theseus-xeo.** `Store::update_session` holds a per-session lock from the read to the indexed write, for one frame. A turn writes only its own fields, and takes a pending recompile when it starts. So a `session.recompile` asked during a turn is kept for the next.
- **lock_two's first callers:** `open_task`, and `lock_family` for every transition of a task, which takes the task and its parent together.
- **A kernel fix found on the way.** A late completion after a cancel that held its reservation as unknown books its real cost and releases the hold. Before, a cancelled task would have kept its carve.

How it is proven.

- The gate at 3b0b20d: 445 tests, 25 of them new:
 - 12 in the kernel: the carve, the cap, one task per call, depth one, the spend carried, the end, the cancel, a late completion after a cancel, a crash, and three `lock_two` races that lose an update with one lock (a throwaway probe, reverted);
 - 5 in the core, through the real driver;
 - 2 for theseus-xeo, the filed race and the lock;
 - 4 in Discord, and 2 against the real daemon (a `kill -9` while a task's job runs, and a cancel that kills a real job).
- The step's live check (05:53–06:00), on a copy of Eddie's store:
 - a GLM turn started a task that ran `scripts/gate.sh`, and a second question was answered meanwhile;
 - the report posted once, and the parent quoted it;
 - a `kill -9` during a second task's gate run: the task finished after the restart, in 3 turns, and reported once.
 - The first gate run inside a task found that the broker test's own approval, from inside a job, is refused by J1's guard, correctly. 3b0b20d makes that test skip its operator's part inside a job, as `job_approval.rs` does.
- **The run.** A clean WSL shutdown at 06:03:36 (a WSL update) ended the run while it wrote the report's last four sections. Its commits were all pushed, and Tabitha wrote those sections at review.

Reviewed (Tabitha, 2026-09-30, 08:28 to 08:40).

- **The first gate rerun failed one test, `theseus-tools`**
`git::tests::diff_and_log_against_a_real_repository`, in 60 s: "gpg failed to sign the data". Its fixture builds a repository with the git CLI and inherited the operator's global `commit.gpgsign = true`, and the reboot had left the gpg-agent locked. This is not DD7's fault, and it predates DD7. Fixed at review in 94d184d: the fixture sets `GIT_CONFIG_GLOBAL=/dev/null` and `GIT_CONFIG_NOSYSTEM=1`, and passes in 49 ms with the agent still locked. The gate then passed: 445 tests, and every bench phase within budget.
- On the release build of 94d184d, over a fresh copy of Eddie's store, with the binding on a fake REST (port 9473) and a gateway that never connected:
 - **A task that reports.** A GLM turn in the bound session called `task.create` with a \$0.50 budget and a brief to run `git log --oneline -3`, and returned in 22.5 s. The task ran one turn, spent \$0.0007 of its \$0.50, and completed. Its report was one message at the fake, `Task 4bed87 finished`, with the three subjects;
 - **the parent's next turn quoted it** exactly, in one loop with no tool call, and the parent's session held exactly one report node, authored `task:4bed87`;
 - **a cancel.** A second task ran `sleep 120` in a job wrapper. `theseus cancel 164775` stopped it: the wrapper and the `sleep` were gone, and one message, `Task 164775 stopped ... cancelled by sock#13`, reached the fake. A second cancel asked 0 actions to stop and posted nothing;
 - **the parent's spend holds its tasks'.** The parent's own three turns cost \$0.001444, and the tasks \$0.000995, which makes \$0.002439. The parent's execution said \$0.002442 spent, and \$0 reserved once both tasks had ended;

- nothing reached Discord: the connections went to 127.0.0.1, api.z.ai, and api.github.com.
- Eddie's unchanged note loads under the new binary.
- Installed at 08:38.
- **Taken at review:**
 - A cancel's author on Discord reads as the connection's label (`sock#13`). DD8, which touches `/cancel` for wakes, makes it name the surface or the person.
 - Whether a finished task should start a parent turn is put to Eddie as a later `wake_parent` option (the report's section 13).

Divergence from the brief and the issue. The report's section 14 has the table:

- `task.create` is a harness tool, not a toollet;
- the report reaches the parent at its next turn;
- the default carve is a quarter;
- `/cancel` now takes a required task;
- the kernel fix for a late completion after a cancel;
- the broker test's skip inside a job;
- and, at review, the hermetic git fixture.

Known gaps.

- Discord's `/tasks` and `/cancel` were not run live, since the gateway is not faked. The binding's tests cover them.
- A task's live progress shows only in the web UI's session, not in the place.
- `/stop` does not stop a session's tasks, which `/cancel <task>` does.
- §3.2a's arrangement, `autonomous: true`, sub-tasks, and §3.5's task graph are not built.

Item 8. `wake.at` : one-shot wakes into the current session (theseus-cff; 2026-09-30, 08:41–09:50; 84ab96d, 07c0bb8, ba49df2, 4c6b72c)

Why. 6 of the DM's 8 schedule adds in the audit's 30 days were one-shot wakes into the current conversation: "check the build in 10 minutes".

What exists.

- `wake.at { at | after, note }` (`theseus-core/src/wake.rs`), a harness tool. `after` is a duration (`90s`, `10m`, `2h`, `1h30m`, `1d`), and `at` an RFC 3339 time with its offset, 1 s to 30 days ahead. It writes the wake onto the session's execution in one kernel frame (`kernel::set_wake`), and returns at once with the wake's id, its time, and the line its turn will read. It is refused in a task, and at 5 pending, with the five listed.
- **Pending wakes** (`theseus-kernel/src/wakes.rs`): a list on the execution beside its one `wake` (§3.15). A due wake fires when its execution is free; the driver's tick queues it within half a second, and the reconciler is the backstop. A busy execution is queued again by the frame that ends its turn. A due `wake::DueAt` now sets `resume_pending`, so the driver takes it: before, nothing did.

- **The wake's turn.** Its catch-up takes the due wakes in one frame, each a node (`wake:<short>`, `wake (set 13:05): <note>`), with a `wake.fired` row (`late_ms`, `while_down`). The reply's outbox post carries the wake's line, which the binding shows above the reply.
- **Late, and never twice.** Startup writes nothing for a due wake: the driver's first tick queues it, once the vault has confirmed the config. A wake more than 5 s late says so, when it was due, and why.
- **Seeing and stopping.** `wake.list`, `wake.cancel`, and health's `wakes`; `theseus wakes` and `theseus cancel <id>`; Discord's `/wakes`, and `/cancel id:` for a task or a wake; and the Observatory's Wakes section, with a cancel. A cancel's author names the surface or the person (DD7's review).

How it is proven.

- The gate at 4c6b72c: 473 tests, 28 of them new: 9 in the kernel, under the virtual clock; 6 in the core through the real driver, and 8 unit tests of the tool's parsing and text; 2 against the real daemon with the fake Discord REST (a wake's reply posted once, and a `kill -9` with 9 s down, then a late wake, once); 2 in Discord; and 1 in the CLI. The frame budget test holds 8.
- The step's live check (09:26–09:45), on a copy of Eddie's store:
 - "Remind me in one minute to check the build" set a 60 s wake, and the daemon was restarted 11 s later. The wake's turn started 60.0 s after the wake was set, ran a real `cargo check`, and its reply posted once under its wake line;
 - a 30 s wake with the daemon down for 61 s ran after startup, `42 s late: the daemon was not running then`, once. That start queued it before serving, which ba49df2 fixed. On ba49df2's build, the same check wrote nothing before serving;
 - `theseus cancel` cleared a pending wake, `by the CLI`, and a task's cancel read `cancelled by the CLI`.
- The step found and fixed three things live: startup's write before serving, a doubled full stop in the tool's result, and a due time printed without its seconds.

Reviewed (Tabitha, 2026-09-30, 10:00 to 10:07).

- The gate rerun passed: 473 tests, and every bench phase within budget (cold start p95 37.9 ms). The step's own last gate had passed cold start at 56.7 ms against 57, on a busy machine.
- On the release build of 4c6b72c, over a fresh copy of Eddie's store, with the binding on a fake REST (port 9475):
 - a GLM turn in the bound session, "Remind me in 20 seconds to stretch", set a wake due 10:05:08;
 - **down across the due time.** I stopped the daemon at 10:05:00 and restarted it at 10:05:53. The wake's turn ran at once, and its node read `wake (set 10:04, due 10:05:08, 46 s late: the daemon was not running then): ...`. Its `wake.fired` row had `late_ms 45877` and `while_down true`, and its reply reached the fake once, under the wake line;
 - **startup wrote nothing for it before serving.** The restart's frames were the startup steps, then `server.started+server.serving`, then `driver.started`, and only then the driver's `execution+execution.queued`;

- **listing and cancelling.** A two-hour wake showed in `theseus wakes`. `theseus cancel` `beeee4` cleared it, the list was empty, and the `wake.cancelled` row said the CLI;
- nothing reached Discord: the connections went to 127.0.0.1, api.z.ai, and api.github.com.
- Eddie's unchanged note loads under the new binary.
- Installed at 10:06.
- **Taken at review:**
 - `/cancel`'s option stays `id`, since it names a task or a wake, and a wake keeps waiting while an approval or the budget question is open, since only the operator answers those.
 - A task still cannot set a wake, and a session may hold 5.
 - A clean shutdown just after a turn's end can stop the daemon between a reply's create and its settle. The next start resends it, deduped by its nonce. Filed as theseus-pfv: the lanes settle what they already sent, within the shutdown budget.

Divergence from the brief and the issue.

Planned	Actual	Why	Disposition
"sets the kernel's existing Wake : :DueAt on the current session" (issue, P5d)	A list of pending wakes on the execution, beside its one wake ; the due scan that fired Wake : :DueAt fires them too	One field cannot hold input and a due time, and a session parked on its own job must still be woken	Keep
The reconciler runs due wakes every heartbeat_secs (the audit)	The driver's 500 ms tick fires a due wake; the reconciler is the backstop	A 60 s heartbeat would run "after 2 s" up to a minute late	Keep
—	A due Wake : :DueAt now sets resume_pending	Before, the reconciler queued it and nothing took the turn	Keep
—	Startup's reconcile no longer fires due wakes (ba49df2)	It wrote two frames before serving, which FAST forbids	Keep
" /cancel clears a session's pending wakes ... or /cancel wake <id>"	/cancel id:<id> names a task or a wake, and /wakes lists them	One verb for "stop that one"	Keep
—	A wake's reply goes to where it was set, after /new	The reminder should reach the place that asked for it	Keep
—	A task cannot set a wake	A task ends when it has nothing to wait on	Keep; a later option
"a small number of pending wakes"	5	The DM set 6 in 30 days	Keep

Known gaps.

- Discord's /wakes and /cancel were not run live, since the gateway is not faked. The binding's tests cover them.
- at needs an RFC 3339 offset, and times print in the daemon's zone.
- The kernel-sim's random operations do not include wakes yet.

- A wake is not moved when its session is `/new`ed away. It runs in its own session, and its reply reaches the place.
- An older binary reads a store with pending wakes but ignores them, and would drop them the next time it wrote the execution. F4's versioned readers close this.

Item 9. W1: a task can wake its parent, and `/stop` only halts (theseus-lji; 2026-09-30, 10:09–11:02; 2975183, 426e2b7)

Why. Eddie, 2026-09-30, answering DD7's two questions. First, a finished task should be able to start the conversation's next turn, as an opt-in: "Yes!" Second, `/stop` should only halt the work and keep the conversation, while `/new` alone starts fresh: "Yes!" His rule for controls is one command, one effect.

What exists.

- `task.create { brief, budget_usd?, wake_parent? }`. With `wake_parent`, a task that finishes or fails asks for its parent's next turn. The frame that ends it puts it on the parent's `report_wakes`, with a `task.report_wake` row, and queues a free parent for the driver (why: `report`), as a due wake does. A busy parent keeps the ask until its turn ends. The turn's catch-up reads every report and clears both lists, and its reply posts where the parent posts, under `task a1b2c3 reported`. A cancelled task wakes nothing. The start's notice says the task will wake the conversation, and `task.list` and `theseus tasks` show the option.
- **A soft stop** (`theseus-kernel/src/stops.rs`, `Kernel::stop_execution`), §3.15's Stopping.
- **Surfaces.** `execution.stop` (an acting method), `theseus stop <session>`, and Discord's `/stop`, which no longer rebinds. The bind notice, the help text, and the commands' descriptions name each control with its one effect. The place freezes the stopped turn's stream, and a declined card settles as `stopped`. `/new` is unchanged.

How it is proven.

- The gate at 426e2b7: 496 tests, 23 of them new: 11 in the kernel; 7 in the core (6 through the real driver, and the notice's unit test); 2 in Discord (a place's `/stop` and `/new` over a real core); 2 against the real daemon with the fake Discord REST (a report's turn, and a stop that kills a real job); and 1 in the CLI. The frame budget test holds 8.
- The step's live check (10:54–10:58), on a copy of Eddie's store with the binding on a fake REST: a task with `wake_parent` ran `git log`, and the parent's turn started by itself 14 ms after the task's end frame; a task without it reported, and no turn followed in 45 s; `theseus stop` during a `sleep 45` job killed it, and the next message continued the same execution.
- Discord's `/stop` itself was not run live, since the gateway is not faked. The binding's test drives it.

Reviewed (Tabitha, 2026-09-30, 11:30 to 11:37).

- The gate rerun passed: 496 tests, and every bench phase within budget (cold start p95 42.2 ms).

- On the release build of 426e2b7, over a fresh copy of Eddie's store, with the binding on a fake REST (port 9477):
 - the bind notice names `/stop`'s new meaning: "halts what I am doing and keeps the conversation";
 - **a task that wakes its parent.** A GLM turn started task `8ec704`, with `wake_parent` and \$0.30, to run `hostname`. Its report reached the fake. Its end wrote `task.report_wake` (`queued: true`) and the parent's `execution.queued` (`why: report`). The parent's turn then started by itself, read the report (`task.reports_read`, `woke`), and posted "The hostname is `zeroradeons`." under `## task 8ec704 reported`;
 - **a stop.** A turn ran `sleep 40` in a job. `theseus stop` answered "1 action(s) told to stop ... the conversation goes on", and the `sleep` was gone. The ledger had `execution.stopped` by the CLI, the turn's end with `stop_reason: stopped`, and `execution.waiting`, `why: stopped`. The fake got only the call's tool line, and no reply;
 - **the conversation went on.** The next question ran in the same session and the same execution as the first turn, and GLM answered from its history that the sleep "was stopped by the CLI";
 - nothing reached Discord: the connections went to 127.0.0.1, `api.z.ai`, and `api.github.com`.
- Eddie's unchanged note loads under the new binary.
- Installed at 11:35.
- **Taken at review:**
 - A stop keeps the session's pending wakes, as it keeps its tasks: one command, one effect.
 - The web UI's stop control is filed (`theseus-nkt`), and so is aborting a stopped turn's in-flight model stream (`theseus-yey`).
 - A call that a stop killed shows `... cancelled` on its Discord tool line, which reads as a failure, though it was asked for. It goes into fix batch 2 on the roadmap: `stopped`.

Divergence from the brief and the issue.

Planned	Actual	Why	Disposition
" <code>/stop</code> ... cancels the session's current work, as today"	A new transition, <code>Kernel::stop_execution</code> , not <code>execution.cancel</code>	A cancel is terminal, so a session it ends cannot take a turn	Keep
"the running turn, its jobs, and the queued messages"	Also declines planned calls, approvals, and the budget question	An approval answered after a stop would resume the stopped work	Keep
—	The running turn's model call runs to its end; its answer is kept, but not acted on or posted	No abort exists for a provider stream today	Keep; <code>theseus-yey</code>
— (DD8: a cancel, <code>/stop</code> then, dropped the wakes)	A stop keeps the session's wakes	One command, one effect	Keep

Planned	Actual	Why	Disposition
"the frame that ends the task also queues a continuation turn"	For a free parent in that frame; for a busy one, in the frame that frees it	A turn cannot start over another, or over an open question	Keep
—	A failed task wakes its parent too	A failure is a result a chain should review	Keep
—	<code>theseus stop <session></code> in the CLI	So the stop runs live, through the method Discord's <code>/stop</code> calls	Keep
—	A job a stop or a cancel killed ends the turn's in-turn wait at once	<code>job_settled</code> did not count cancelled before	Keep

Known gaps.

- Discord's `/stop` was not run live, since the gateway is not faked.
- A stopped turn's in-flight model call runs to its end, and its cost is paid (theseus-yey).
- An older binary would drop `stopped`, `report_wakes`, and `wake_parent` on a rewrite. F4a closes this.
- The kernel-sim's random operations include neither stops nor report wakes yet.

Item 10. T1: after a session reads external text, a call that acts waits (theseus-9bp; 2026-09-30, 11:39–12:27; 822f127, 41e8fa5)

Why. This came from DD5's review. Since DD5, a page's text reaches the model, and in Eddie's config `proc.run` and the writers run at `notify`. So an injection in a page could steer the model into running a command, with only a notice after the fact. OpenClaw's provenance plugin closes this path for OpenClaw sessions. Theseus has no provenance labels yet, and Jev's `security.v1` is M5. T1 is the interim, deterministic floor, and it landed before Eddie's end-to-end test.

What exists (§3.9, External text).

- **The hold** (`theseus-core/src/external.rs`): `SessionRecord.external` (since when, the tool, the URL, the node, and, when it came from another session, which one and how), with one `session.external_read` row. It is written in the frame that brings the text in, under the record's lock (`Store::with_session`, holding it until the frame is indexed; the lock order is always the session, then the execution). There are three such frames: a result's completion, a task's `open_task` (for a task that a holding session starts), and the parent's `take_reports` (for a report from a holding task). A turn never writes it (`take_turns_fields`).
- **The gate** (`external::gate`, after the broker's posture): a call whose class is not `Read` waits, with the reason, the allow list's calls included. A `Read` call keeps its posture and reads no record. A record that cannot be read fails closed.
- **Trusting it again:** `policy.trust` and `action.confirm { trust }`, judged by `judge_act` (`Act::Trust`), and ledgered as `session.trusted`.

- **Surfaces:** `theseus policy trust <session>`, `theseus confirm --trust`, and a hint on a waiting call's lines; health's `external_text` and the CLI's `external_text:` line; Discord's third button, "Approve + trust session", and its settle; the web UI card's third button and marker, and the Observatory's External text section.
- **Config:** `[policy] external_text = "ask" | "notify"`, ask by default, in the template.

How it is proven.

- The gate at 41e8fa5 ran 509 tests, 13 of them new: 9 through the whole core, 2 unit tests of the rule, a config test, and a CLI test. The core tests cover the same turn and the next, trust, a read keeping its posture, a job's process refused, a restart, a clean session unchanged, approve with trust, a task, a report's turn, and a wake's turn. The frame budget test holds 8.
- The step's live check (12:19–12:22) ran on a copy of Eddie's store with his note, with Discord and the web UI off:
 - a GLM turn fetched the `option` page (200, 248,029 bytes), and `proc.run echo hi` waited, with the reason word for word;
 - the hold rode in the result's frame (`...node+session.external_read+session`), and the run was declined;
 - `theseus policy trust` cleared it (`session.trusted`, by the CLI), and the next `proc.run echo hi` in that session was a notice and ran;
 - beyond the brief, a hold survived a restart, and `theseus confirm --trust` approved a waiting run and trusted the session in one answer.

Reviewed (Tabitha, 2026-09-30, 12:47 to 13:05).

- **The gate rerun.** The first rerun's lifecycle bench missed twice, right after the test run's writeback: cold start p95 was 62.2 ms, then 74.6 ms, against 57, and the store phase's p95 was about 50 ms. On a quiet disk, the bench alone passed (cold start p95 41.6 ms). The full gate then passed with 509 tests and every phase within budget (cold start p95 41.2 ms). T1 does not touch the start path.
- **Reading the code.**
 - Every path that takes a session's lock takes it before an execution's, and no kernel transition takes a session's.
 - `run_harness (task.create, wake.at)` and the error path complete without the hold, and neither result can be marked external.
 - `task::create` reads the parent's hold without the lock. The one race is a fetch that completes in the same response as the `task.create`, and that call was written before the page was seen, so a clean child is correct.
- **A live check on the release build of 41e8fa5**, over a fresh copy of Eddie's store, with his note. Discord and the web UI were off, and the Brave key's reference was added, because his note has none.
 - **A search gives the hold, and the allow list waits.** A GLM turn ran `web.search` (a notice), and then `1s`, which his allow list runs `open`. `1s` waited: "this session read external text (web.search ...?q=tokio+JoinSet+documentation..., at 12:55), and a call that acts waits for approval after that (§3.9)". Health listed the session. The call was declined.

- **A job's process cannot trust its own session.** GLM was told plainly that this was the operator's test. It asked `proc.run` to run the scratch CLI's `policy trust` on its own session. The call waited, and was approved without trust. The job exited 1: "trusting the session again from the CLI does not count: from a Theseus job's process (job act_..., pid 366807, theseus). It still holds external text". The refusal was ledgered as `approval.refused (act: policy.trust, from_job: true)`, and the hold stayed.
- **Approve and trust, then the allow list is back.** The next `ls` waited. `theseus confirm -trust` ran it and cleared the hold (`session.trusted, how: action.confirm`). The `ls` after that ran at `open`, from the allow list, with no notice.
- Outbound connections went to `api.github.com`, `api.z.ai`, and the Brave search API. Nothing reached Discord.
- Eddie's unchanged note loads under the new binary.
- Installed at 12:57.
- **Taken at review:**
 - **A job the operator approves can open a clean session.** It can run `theseus ask` over the socket, and that session holds nothing. This needs an approved acting call in the holding session first, and the card shows the command. Filed as theseus-d64 (P3): J1's trace, applied at `session.open` and `turn.submit`, would pass the job session's hold on.
 - **Cosmetic** (theseus-qiy, P3, fix batch 2): a search's hold names the search API's address rather than the query; health gives the hold's time in UTC while the reason uses local time; and a trust through an approval names the connection (`sock#32`), not the surface.
 - **Eddie's open questions, with the defaults the chain keeps.**
 - Reads keep their posture, so a fetch's URL can still carry data out, and its notice names the URL.
 - `wake.at` waits in a holding session.
 - There is no Discord `/trust`: the card's button, the CLI, and the Observatory clear a hold.

Divergence from the brief and the issue.

Planned	Actual	Why	Disposition
"every call whose class is not <code>Read</code> waits ... It is a tightening, so the stricter posture wins"	Applied after the whole order, the allow list included, as a granted secret's posture is	An allow-list prefix is the easiest way through for a page that steers the model	Keep
"A fetch, then <code>proc.run</code> in the same turn: it waits"	From the next model call on; a call in the fetch's own response keeps its posture	That call was written before the model saw the page (F3 gates a response's calls first)	Keep
—	<code>wake.at</code> and <code>task.create</code> wait too	Both act (a write, a run), and the rule covers every class but <code>Read</code>	Keep; Eddie's question 2
Discord: "a button on the first such confirm, or a slash command"	A third button, Approve + trust session , on every confirm the rule raises	The operator learns of the rule on that card, and each later one offers the same	Keep; no <code>/trust</code>

Planned	Actual	Why	Disposition
The CLI: <code>theseus policy trust <session></code>	That, and <code>theseus confirm --trust <id></code>	The card's button, for symmetry	Keep
"a <code>session.external_read</code> ledger row" per session	One per hold: a session that is trusted and then reads again writes another	"A later external read taints the session again"	Keep
—	A trust through an approval names the approval's author (the connection's label)	Approvals record the connection's label	Keep; theseus-qiy

Known gaps.

- A job's output that carries outside text (a download, `gh issue view`, a pulled README, and files it leaves that `fs.read` reads later) is not marked external, so it gives no hold (theseus-20f).
- A job the operator approves can open a clean session through the socket (theseus-d64).
- A page can still send data out through a fetch's URL: a read keeps its posture, by design, and each fetch is a notice with its URL.
- Discord's third button was not pressed live, since the gateway is not faked. The binding's tests cover its id and its parse, and the core's test covers the answer.
- An older binary would drop a session's hold the next time it wrote the record. F4a closes this.
- The model is not told that its session holds external text. It learns only when a call waits.